

Modern .NET Web Development

A Practical Guide for Experienced C# Developers

July 2026

Contents

Part I — How the Web Works	2
1. From URL to Screen	2
2. HTTP and Browser–Server Communication	11
Part II — ASP.NET Core	24
3. Inside an ASP.NET Core Application	24
4. From Request to Application Code	34
5. Application Services	48
Part III — Blazor	61
6. Components and Razor	61
7. Rendering and State	77
8. Building Complete Interfaces	86
Part IV — Data	100
9. Relational Data and SQL	100
10. Entity Framework Core	112
Part V — Secure, Maintainable Applications	126
11. Authentication and Authorization	126
12. Application Structure	140
Part VI — Running a Real Application	158
13. Features Beyond Request and Response	158
14. Deployment and Operation	172
15. The Complete Request Revisited	184
Appendices	197
Appendix A. Practical Command and Diagnostic Reference	197

This book assumes solid C# knowledge, including object-oriented design, interfaces, generics, asynchronous programming, dependency injection, testing, and application architecture. It focuses on the mechanisms specific to web development and on how the browser, HTTP, ASP.NET Core, Blazor, Entity Framework Core, security, background work, and deployment fit together.

The examples target .NET 10 and follow one Product Catalog from a typed URL to a production service. Code listings show the relevant slice of each step rather than repeating unchanged surrounding files, while Appendix A collects the commands and diagnostic paths used throughout the book. The Product Catalog evolves with the chapters, so later listings extend or replace earlier chapter-local versions of the same types rather than forming one cumulative source file.

Part I — How the Web Works

1. From URL to Screen

A web application starts with an action that looks almost trivial: the user types an address into a browser and presses Enter. A short moment later, a page appears. Text is laid out, images arrive, buttons become interactive, and the application begins responding to input. That short delay hides a surprising amount of work. The browser must understand the address, find the correct server, create a secure connection, request the right resource, receive a response, interpret several document formats, download additional files, build an internal representation of the page, calculate its layout, and finally draw pixels on the screen. None of those steps is unique to .NET. ASP.NET Core and Blazor sit on top of this process. Understanding the path from a URL to a rendered page makes the later framework behaviour feel much less arbitrary.

This chapter follows that path from beginning to end. It introduces the browser, addresses, DNS, networks, ports, secure connections, HTML, CSS, JavaScript, the DOM, and browser developer tools. HTTP messages themselves are introduced only briefly here; Chapter 2 examines them properly.

1.1 The Browser as a Web Platform

It is tempting to think of a browser as a program that displays pages. That is true, but incomplete. A modern browser is also a network client, a document parser, a layout engine, a graphics engine, a JavaScript runtime, a storage system, a security boundary, and a debugging environment. When the browser opens a page, it performs several different jobs:

- it communicates with remote servers;
- it interprets HTML, CSS, and JavaScript;
- it builds an internal tree of page elements;
- it calculates the size and position of those elements;
- it draws the result;
- it processes mouse, keyboard, touch, and other input;
- it enforces security rules between different websites;
- it stores selected data locally;
- it exposes tools that let developers inspect all of this work.

Blazor does not replace the browser. It works inside the browser's rules. Even interactive server rendering, where much of the component logic runs on the server, still depends on the browser to display HTML, apply CSS, process events, and maintain the connection to the application. A useful mental model is:

The browser provides the platform.

HTML describes the content.

CSS controls presentation.

JavaScript and Blazor provide behaviour.

The server provides data and application logic.

Later chapters will refine this model, but it is already enough to explain why web development feels different from desktop development. A desktop application usually owns its window and runs most of its logic in one process. A web application crosses a network boundary and must cooperate with a browser that the application does not control.

1.2 URLs, DNS, Addresses, Ports, and Latency

Consider this address: `https://catalog.example.com:443/products/42?currency=eur#details`. A browser does not treat it as one opaque string. It separates it into meaningful parts:

<code>https://</code>	<code>catalog.example.com</code>	<code>:443</code>	<code>/products/42</code>	<code>?currency=eur</code>	<code>#details</code>
scheme	host	port	path	query	fragment

The **scheme** selects the communication method, normally `https`; plain `http` does not provide transport encryption and should generally be avoided for public production traffic. The **host**, such as `catalog.example.com`, names the machine or service, while the **port** selects a network service on it. Browsers omit the usual values because they already know that HTTP defaults to 80 and HTTPS to 443. The **path** identifies a resource or route, so `/products/42` might select product 42, while the **query string** adds optional values such as `currency=eur`.

Finally, the **fragment** points to a location within the current document. A fragment such as **#details** is normally handled by the browser and is not sent to the server. The terminology matters because ASP.NET Core exposes these parts separately. Routing usually works with the path. Model binding may read the query string. Middleware can inspect the scheme and host. Blazor's navigation services work with the full URL.

A browser cannot send packets to the words **catalog.example.com**. It needs a numeric network address. The system that translates names into addresses is the **Domain Name System**, usually called DNS. The browser first checks whether it already knows the answer. It may have a cached DNS result from an earlier request. The operating system may also have a cached result. If neither does, the computer asks a DNS resolver, usually supplied by the local network, internet provider, or a public DNS service. A simplified lookup looks like this:

Browser

```
-> Operating system
-> DNS resolver
-> "What is the address of catalog.example.com?"
<- "203.0.113.25"
```

The real DNS hierarchy is more elaborate. A resolver may contact root name servers, top-level-domain servers, and the authoritative server responsible for **example.com**. For application development, the result is simpler: the host name becomes one or more IP addresses. DNS records can represent more than a direct name-to-address mapping. A name may point to another name, email servers have their own records, and services may publish additional metadata. Large systems frequently return different addresses depending on the user's region or current server load. DNS results are cached for a limited time. That improves speed, but it also explains why changing a domain's configuration is not always visible immediately. Some clients may continue using the previous result until its cache entry expires.

An **IP address** identifies a network interface that can receive Internet Protocol traffic. An IPv4 address looks like 203.0.113.25, while an IPv6 address is longer, for example 2001:db8:85a3::8a2e:370:7334. The browser does not need to understand where the server physically lives. It gives packets to the operating system, and the network infrastructure forwards them toward the destination. Along the way, the packets may pass through the local router, the internet provider, regional networks, and the hosting provider's network before reaching the server. The route is not usually one permanent wire between the browser and server. Data is divided into packets. Each packet carries addressing information and may pass through several routers. The receiving side combines the delivered data into the original stream. A greatly simplified journey looks like this:

Browser

```
-> Local network
-> Router
-> Internet provider
-> Several internet routers
-> Hosting provider
-> Server
```

This is why web applications must assume that communication is neither immediate nor perfectly reliable. Packets can be delayed, arrive out of order, or be lost and retransmitted. The protocols used above IP hide much of this complexity, but the effects remain visible as latency and connection failures.

Two network properties are commonly confused: **bandwidth** and **latency**. Bandwidth describes how much data can be transferred over a period of time. A faster connection can download a large image or video more quickly. Latency describes how long it takes for data to travel to the destination and for a response to begin returning. Even a high-bandwidth connection may have noticeable latency if the server is far away or the network path is busy. This distinction matters because a web page often requires many small exchanges. A browser may first resolve DNS, then establish a connection, then negotiate encryption, then request HTML, then discover that it also needs CSS, JavaScript, fonts, and images. If each step waits for the previous one, latency accumulates.

Modern protocols and browsers reduce this cost by reusing connections, downloading resources in parallel, compressing content, caching earlier results, and placing servers closer to users. As an application developer, you still feel the consequences. Excessive API calls, unnecessary redirects, and very large dependency chains can make an application feel slow even when the server code itself is fast. An IP address identifies a machine or network interface, but one machine can run many network services at once. Ports distinguish them. For example:

```
203.0.113.25:443    -> HTTPS web service
203.0.113.25:22     -> SSH administration service
203.0.113.25:5432   -> PostgreSQL database service
```

A port is not a physical socket. It is a number used by the operating system to deliver incoming traffic to the correct process. In development, an ASP.NET Core application often runs on an address such as `https://localhost:7184`. Here, `localhost` refers to the current computer and 7184 is the port on which the development server is listening. Another application can use a different port on the same machine without conflict. In production, users usually do not see the port because HTTPS uses port 443 by default, as in `https://catalog.example.com`. A reverse proxy or load balancer may listen on port 443 and forward the request internally to an ASP.NET Core process listening on another port. The deployment chapter will explain that arrangement in detail.

1.3 Connections and HTTPS

Once the browser knows the server's address, it must establish communication. Traditionally, web traffic has used TCP, which provides a reliable ordered stream of bytes. Modern HTTP/3 uses QUIC, which runs over UDP while still providing reliability and secure transport features suited to web traffic. The browser selects a supported protocol automatically. Application code normally works with HTTP without manually managing TCP or QUIC. For now, the useful idea is that a logical connection is established between the browser and server:

```
Browser                               Server
|                                   |
|----- establish connection -->|
|<----- connection ready -----|
```

A reliable connection does not guarantee that the whole application will always succeed. The server might be unavailable, a proxy might close the connection, a mobile device might switch networks, or the user might lose connectivity. Web applications therefore need sensible timeouts, cancellation, retries only where appropriate, and clear error states.

With HTTPS, the browser and server do not immediately exchange ordinary HTTP data. They first establish a secure channel using TLS. TLS provides two essential properties. First, it encrypts the traffic. Someone observing the network should not be able to read credentials, personal data, or application content. Second, it helps the browser verify that it is communicating with the intended server. The server presents a certificate issued for its domain. The browser checks whether the certificate is trusted, valid for the requested host, and within its validity period. The high-level sequence is:

```
Browser
-> Connect to catalog.example.com
<- Server presents certificate
-> Browser validates certificate
<-> Browser and server establish encryption keys
-> Encrypted HTTP communication begins
```

The cryptographic details are intentionally outside the scope of this tutorial. HTTPS protects the connection between two endpoints. It does not automatically make application code secure. A server can still contain authorization errors, SQL injection vulnerabilities, leaked secrets, or unsafe file handling. Transport security is necessary, but it is only one layer. During local development, .NET commonly uses a development certificate so that applications can run at `https://localhost:...`. The browser may warn about an untrusted certificate if the local certificate has not been installed or trusted correctly.

1.4 Requesting the Document and Its Resources

After DNS resolution and connection setup, the browser can ask for a resource. If the user entered `https://catalog.example.com/products/42`, the browser would send an HTTP request representing that action. Conceptually, it says:

```
Get the resource at /products/42
for the host catalog.example.com.
Here is information about my browser,
the formats I understand,
my cookies,
and other request metadata.
```

The request reaches a web server. In a .NET application, that may ultimately be Kestrel, ASP.NET Core's web server, either directly or behind a reverse proxy. The server processes the request and returns a response. The response contains a status, headers, and usually a body. The body might contain HTML, JSON, an image, a file, or another format. A minimal view of the exchange is:

Browser

-> Request: GET /products/42

Server

-> Response: 200 OK

-> Body: HTML document

Chapter 2 will open these messages and examine methods, headers, status codes, request bodies, responses, caching, cookies, REST, JSON, and CORS. Here, the important point is that the browser does not remotely open a server-side page. It sends a message, receives bytes, and decides what those bytes mean. Suppose the server returns an HTML document. The browser begins parsing it and discovers references to other resources:

```
<link rel="stylesheet" href="/css/site.css">
<script src="/js/site.js"></script>

```

The original HTML does not necessarily contain the CSS, JavaScript, or image data. It contains addresses that point to those resources. The browser sends additional requests to obtain them.

GET /products/42

<- HTML

GET /css/site.css

<- CSS

GET /js/site.js

<- JavaScript

GET /images/product.jpg

<- Image

This explains several common observations in browser developer tools:

- loading one page can produce dozens of requests;
- a missing stylesheet can break the appearance without preventing the HTML from loading;
- JavaScript may fail while the static page remains visible;
- fonts and images may appear slightly later than the initial text;
- caching one resource does not necessarily cache the whole page.

Blazor applications also load framework resources. Depending on the hosting model and render mode, the browser may download JavaScript boot files, .NET assemblies, WebAssembly files, CSS, or establish a SignalR connection. These details will make more sense after the Blazor architecture is introduced.

1.5 HTML, CSS, JavaScript, the DOM, and Events

HTML describes the structure and meaning of page content. It uses elements such as headings, paragraphs, buttons, forms, links, tables, and sections.

```
<article class="product-card">
  <h2>Mechanical Keyboard</h2>
  <p>Compact wireless keyboard.</p>
  <button type="button">Add to cart</button>
</article>
```

This document says that there is an article containing a heading, a paragraph, and a button. It does not specify every visual detail. That is CSS's job. HTML is not merely a drawing language. Elements have meaning. A `<button>` is expected to behave like a button. A `<nav>` represents navigation. A `<label>` is associated with a form control. Using meaningful elements improves accessibility, keyboard navigation, browser behaviour, search indexing, and maintainability. Blazor components ultimately produce HTML. Razor syntax and C# determine which elements should exist, but the browser still receives and displays ordinary web elements. CSS describes how elements should appear and how they should be arranged.

```
.product-card {
  display: grid;
```

```

    gap: 0.75rem;
    padding: 1rem;
    border: 1px solid #ccc;
    border-radius: 0.5rem;
}

```

CSS rules select elements and assign presentation properties. These properties control spacing, colours, fonts, borders, alignment, layout, responsiveness, animation, and much more. For an experienced desktop developer, the most important shift is that web layout is usually declarative and responsive. Instead of placing a control at fixed coordinates, you describe relationships:

- arrange these items in a row or grid;
- allow them to wrap when space becomes limited;
- give this column the remaining width;
- hide or rearrange content on narrow screens;
- inherit typography from the surrounding document.

The browser calculates the final sizes and positions from those rules, the available viewport, the content, and the styles inherited from parent elements. The tutorial will not turn into a full CSS course. It will cover enough CSS to build clear, responsive Blazor interfaces and to understand why a layout behaves as it does. JavaScript runs inside the browser. It can react to events, modify page elements, call APIs, use browser features, and coordinate complex client-side behaviour. A simple script might attach behaviour to a button:

```

document.querySelector("button")
    .addEventListener("click", () => console.log("Clicked"));

```

Blazor reduces how much JavaScript a C# developer must write, but it does not remove JavaScript from the platform. Blazor itself depends on browser integration, and some browser APIs are available only through JavaScript. Existing charting libraries, maps, editors, media APIs, clipboard access, and specialised UI widgets may require JavaScript interoperability. The practical rule is not “never use JavaScript.” It is “use C# for application behaviour where Blazor supports it cleanly, and use JavaScript when the browser or a JavaScript library is the natural owner of the feature.” As the browser parses HTML, it builds an in-memory tree called the **Document Object Model**, or DOM. For this HTML:

```

<main>
  <h1>Products</h1>
  <button>Refresh</button>
</main>

```

The DOM can be pictured as:

```

Document
  -> main
    -> h1
      -> "Products"
    -> button
      -> "Refresh"

```

The DOM is not the HTML text itself. It is the browser’s object representation of that text. Scripts can inspect and change it. Browser developer tools display it. CSS selectors match elements within it. When code changes an element, the browser may need to recalculate style, layout, and painting. UI frameworks try to make those updates efficient. Blazor introduces another representation: the **render tree**. A component produces a render tree describing the UI it wants. Blazor compares the new tree with the previous one, calculates the required changes, and applies those changes to the DOM. This is why understanding the DOM first is useful: Blazor’s rendering system ultimately updates the browser’s document model.

Building the DOM is not enough to display a page. The browser must also understand styles and calculate the visible result. A simplified rendering process is:

```

HTML
  -> DOM
CSS
  -> style rules
DOM + style rules

```

- > visible element tree
- > layout calculation
- > painting
- > compositing
- > pixels on screen

The browser parses CSS and determines which rules apply to each element. It then calculates layout: widths, heights, positions, line breaks, and spacing. After that, it paints text, backgrounds, borders, images, and other visual details. Modern browsers may place some elements on separate layers and combine them efficiently during compositing. Not every change requires the same amount of work. Changing text may affect layout. Changing a colour may require repainting but not recalculating positions. Animating certain transforms can often be handled efficiently by the compositor. You do not need to become a browser-engine specialist to build Blazor applications. Still, this model explains why excessive DOM changes, huge element trees, large images, and expensive CSS can affect performance.

Once the page is visible, the browser waits for events. Events include mouse clicks, keyboard input, touch gestures, scrolling, window resizing, network completion, timers, and many others. A click travels through the browser's event system. JavaScript or a UI framework can register a handler. In Blazor, event directives such as `@onclick` connect browser events to C# methods. Conceptually:

User clicks button

- > Browser creates click event
- > Blazor receives the event
- > C# handler runs
- > Component state changes
- > Component renders again
- > Blazor updates the DOM
- > Browser redraws the affected area

The exact transport differs between Blazor hosting models. With interactive server rendering, the browser sends event information over a connection to the server, where the component executes. With WebAssembly, the .NET runtime and component code execute in the browser. From the component author's perspective, the programming model is intentionally similar.

1.6 Browser Storage and Security Boundaries

Browsers can retain selected data between requests and page visits. Different mechanisms serve different purposes. Cookies are small values associated with a domain and are commonly sent with HTTP requests, often to represent an authentication session or other server-related state. `localStorage` keeps string values until the application or user removes them, while `sessionStorage` normally keeps them only for the lifetime of a browser tab.

IndexedDB provides a larger structured client-side database for substantial offline or local data, and the browser cache reuses previously downloaded resources when the server permits it. These mechanisms are not interchangeable. Sensitive authentication data should not be placed casually into browser storage, and large application state should not be forced into cookies. Chapter 2 introduces cookies and caching from the communication side, while the security chapter explains the consequences of different authentication storage choices.

A browser may have several unrelated websites open at once. It must prevent one site from freely reading another site's data. The **same-origin policy** is one of the central browser security rules. An origin is based on the scheme, host, and port. These are different origins:

```
https://catalog.example.com
https://admin.example.com
http://catalog.example.com
https://catalog.example.com:8443
```

Because they differ by host, scheme, or port, the browser treats them as separate security contexts. When browser code tries to call a different origin, the server may need to permit the operation using Cross-Origin Resource Sharing, or CORS. CORS is often misunderstood as a general server security mechanism. It is primarily a browser rule controlling whether frontend code may read a cross-origin response. It does not stop non-browser clients from sending requests. The browser also restricts access to local files, devices, the clipboard, camera, microphone, and other capabilities. Many operations require a secure connection, explicit user permission, or a direct user gesture. These rules sometimes feel inconvenient during development, but they are essential.

Without them, opening one malicious tab could expose data from every other application the user is signed into.

1.7 Developer Tools

Every major browser includes developer tools. For someone new to web development, these tools are as important as the debugger in an IDE. They are usually opened with **F12**, **Ctrl+Shift+I**, or the browser's context menu. The **Elements** panel shows the current DOM and the styles applied to the selected element. It is the fastest way to discover why spacing, sizing, visibility, or CSS selection is not behaving as expected. The displayed DOM may differ from the original HTML because scripts and frameworks can modify it after loading.

The **Network** panel shows requests and responses. It reveals the requested URL, method, status, duration, request headers, response headers, cookies, transferred size, and returned body. When an API call fails, this panel is often the first place to look. The **Console** shows JavaScript errors, browser warnings, and application messages. Even in a mostly C# Blazor application, boot failures and JavaScript interoperability errors often appear there.

The **Application** or **Storage** panel shows cookies, local storage, session storage, IndexedDB, cache data, and service workers. The **Sources** panel supports JavaScript debugging and displays downloaded files. The **Performance** panel helps analyse slow rendering, scripting, and interaction. It is useful later, after the application's basic behaviour is correct. A productive debugging habit is to ask which boundary failed:

```
Did the browser send the request?
Did the server return a response?
Was the response successful?
Did the browser receive the expected data?
Did the component update its state?
Did the DOM change?
Did CSS hide or misplace the result?
```

Browser tools let you answer those questions rather than guessing.

1.8 One Complete Page Load

We can now connect the pieces. Assume the user enters `https://catalog.example.com/products`. The browser separates the URL into its scheme, host, default port, and path. It checks caches for the host's address, then uses DNS if necessary. The resulting IP address tells the network where to send traffic. The browser establishes a connection to the server. Because the scheme is HTTPS, it validates the server certificate and creates an encrypted channel. It sends an HTTP request for `/products`.

The server processes that request and returns an HTML response. The browser begins parsing the HTML and builds the DOM. It encounters references to CSS, JavaScript, fonts, images, and possibly framework boot resources, so it requests those files as well. CSS rules are matched to DOM elements. The browser calculates layout, paints the page, and composites the final image. Text and basic structure may appear before slower resources finish loading.

JavaScript or Blazor starts the interactive parts of the application. Event handlers are connected. The page can now respond to clicks, typing, navigation, and data changes. The complete journey can be summarised as:

```
User enters URL
-> Browser parses URL
-> DNS resolves host name
-> Network connection is established
-> TLS secures the connection
-> Browser sends HTTP request
-> Server returns HTML
-> Browser requests CSS, JavaScript, images, and other resources
-> Browser builds DOM and applies styles
-> Browser calculates layout and paints pixels
-> Application becomes interactive
```

Real applications add caching, redirects, proxies, authentication, APIs, streaming, and other behaviour, but they still build on this sequence.

1.9 Responsibilities and the Product Catalog

It is useful to separate browser responsibilities from server responsibilities. The browser owns:

- URL navigation;
- network requests from the client side;
- HTML parsing;
- CSS layout and painting;
- the DOM;
- user input;
- browser storage;
- enforcement of browser security rules.

ASP.NET Core owns server-side concerns such as:

- listening for incoming requests;
- executing middleware;
- selecting endpoints;
- reading request data;
- running application services;
- authenticating and authorising users;
- creating responses.

Blazor bridges application code and browser UI:

- components describe interface structure;
- events connect browser actions to C# handlers;
- component state determines what should be rendered;
- the render system updates the DOM;
- JavaScript interoperability reaches browser APIs when required.

Entity Framework Core appears further behind the server boundary. It translates application queries and changes into database operations.

Browser

```
-> HTTP
-> ASP.NET Core
-> Application services
-> Entity Framework Core
-> Database
```

The response travels back through the same layers in the opposite direction. This separation will become one of the tutorial's recurring ideas. Many design mistakes come from placing a responsibility on the wrong side of a boundary: trusting browser input, keeping server-only secrets in client code, using UI components as business services, or treating a database entity as if it were an API contract. The tutorial will gradually build a small Product Catalog. It is intentionally ordinary. The application will display products, allow authorised users to edit them, store them in a relational database, notify connected clients about changes, and eventually run on a Linux server. At the beginning, the user sees a page such as:

Products

Mechanical Keyboard	EUR 129.00
Wireless Mouse	EUR 59.00
USB-C Dock	EUR 189.00

That simple screen will eventually involve every major part of the tutorial:

Browser

```
-> Blazor product list
-> ASP.NET Core endpoint
-> Product service
-> Entity Framework Core
-> Products table
```

For now, only the browser side matters. The visible list is represented by HTML elements, styled by CSS, and displayed from the browser's DOM. Later chapters will explain where the product data comes from and how a C# component causes the DOM to change.

1.10 What to Keep in Mind

A browser is a network client, document parser, runtime, storage system, security boundary, and renderer. A URL identifies how and where to request a resource; DNS resolves its host, ports select the service, and TLS protects an HTTPS connection. The first HTML response often triggers further requests for CSS, JavaScript, fonts, images, and Blazor resources. HTML becomes the DOM, CSS determines layout and appearance, and JavaScript or Blazor connects browser events to application behaviour. Browser storage mechanisms serve different purposes, while same-origin rules restrict how sites interact. Developer tools expose each stage. ASP.NET Core owns server-side request handling, Blazor bridges *C#* components to browser UI, and EF Core later connects application operations to durable data.

2. HTTP and Browser–Server Communication

The previous chapter followed a page from a typed URL to pixels on the screen. The browser resolved a host name, established a secure connection, requested a resource, received HTML, downloaded supporting files, built the DOM, applied CSS, and made the page interactive.

This chapter opens the messages that crossed the network. Those messages are HTTP requests and responses. Nearly everything an ASP.NET Core application does begins with a request and ends with a response, whether the result is a complete HTML page, a JSON document, an image, a file download, or a small update used by a Blazor application. HTTP is simple enough to understand in one sitting, but its consequences reach through the entire web stack. Its stateless nature explains cookies and sessions. Its methods and status codes shape APIs. Its headers control caching, authentication, content formats, and security. Its request–response model explains why long-running work needs special handling and why real-time communication uses technologies such as WebSockets and SignalR.

2.1 HTTP Messages and Request Anatomy

HTTP stands for **Hypertext Transfer Protocol**. The name comes from the early web, where its main purpose was transferring linked documents. Modern applications use it for much more: API calls, uploads, authentication, streaming, telemetry, and communication between services. The basic model has remained the same:

```
Client
  -> HTTP request
Server
  -> HTTP response
```

The client initiates the exchange. A browser is the most familiar HTTP client, but command-line tools, mobile applications, desktop programs, test runners, and other servers can send the same kinds of requests. The server listens for requests, processes them, and returns responses. In an ASP.NET Core application, Kestrel accepts the incoming connection and the framework represents the request and response through an `HttpContext`. A useful detail is that HTTP describes messages, not application architecture. It does not require controllers, Minimal APIs, Blazor, Entity Framework Core, or any particular language. Those are ways of implementing server behaviour around the protocol. A request contains four main pieces:

```
Method + target + protocol version
Headers
Blank line
Optional body
```

A simplified request might look like this:

```
GET /api/products/42 HTTP/1.1
Host: catalog.example.com
Accept: application/json
User-Agent: Mozilla/5.0
Cookie: session=abc123
```

The first line says that the client wants to perform a `GET` operation on `/api/products/42` using `HTTP/1.1`. The headers add metadata. This request has no body, so it ends after the blank line. A request that creates a product might contain a JSON body:

```
POST /api/products HTTP/1.1
Host: catalog.example.com
Content-Type: application/json
Accept: application/json
Content-Length: 84
```

```
{
  "name": "Mechanical Keyboard",
  "price": 129.00,
  "currency": "EUR"
}
```

The method expresses the intended operation. The target identifies the resource. Headers describe the request and its body. The body carries data when the operation needs it. Browsers construct these messages automati-

cally when navigating, submitting forms, loading resources, or running client-side code. ASP.NET Core parses them before your endpoint executes, so normal application code rarely handles raw HTTP text.

2.2 Methods and Request Targets

HTTP methods are sometimes called verbs because they describe what the client wants to do. The most common methods are:

Method	Typical purpose
GET	Read a resource or collection
POST	Create something or trigger an operation
PUT	Replace a resource at a known address
PATCH	Partially update a resource
DELETE	Remove a resource
HEAD	Read response metadata without the response body
OPTIONS	Ask which operations or communication options are supported

A Product Catalog API might expose these operations:

GET	/api/products	-> list products
GET	/api/products/42	-> read product 42
POST	/api/products	-> create a product
PUT	/api/products/42	-> replace product 42
PATCH	/api/products/42	-> change part of product 42
DELETE	/api/products/42	-> delete product 42

These conventions are useful because tools, developers, browsers, proxies, and infrastructure can reason about the request before knowing the application-specific code.

Safe and idempotent methods

Two method properties appear often in API design. A **safe** method is intended not to change server state. GET, HEAD, and OPTIONS are considered safe. Reading a product may still create logs or update metrics, but it should not perform a business operation such as deleting the product. An **idempotent** method can be repeated without changing the final result beyond the first successful execution. GET, PUT, and DELETE are intended to be idempotent. For example, deleting product 42 twice should leave the same final state as deleting it once: the product does not exist.

The second request may return a different status, but it should not delete an unrelated item or perform another business action. POST is not generally idempotent. Sending the same create request twice may create two products. This matters when clients retry requests after timeouts. A timeout does not always mean the server failed; the server may have completed the operation while the response was lost.

The method does not enforce behaviour by itself. Nothing prevents a careless endpoint from deleting data during GET. The convention is valuable only when the server follows it. The request target usually contains a path and an optional query string, as in /api/products/42?includeReviews=true. ASP.NET Core routing can map the path template /api/products/{id} so that a request for /api/products/42 produces the route value id = 42. Query values are better suited to optional controls over a read operation, for example /api/products?category=keyboards&page=2&pageSize=20. A practical distinction is:

Path	-> which resource?
Query	-> which view, filter, sort, or page of that resource?
Body	-> what data is being submitted?

This is not an absolute law, but it produces predictable APIs. An identifier generally belongs in the path, filtering and pagination in the query, and create or update data in the body. Query values are visible in browser history, logs, analytics, and copied links. They should not contain passwords, access tokens, or other secrets.

2.3 Headers, Bodies, and Content Types

Headers are name-value pairs that describe the request or response. They are case-insensitive, although standard casing improves readability. Common request headers include:

Header	Purpose
Host	Identifies the requested host
Accept	Lists response formats the client understands
Content-Type	Describes the format of the request body
Authorization	Carries authentication credentials
Cookie	Sends cookies associated with the request
User-Agent	Identifies the client software
If-None-Match	Supports cache validation with an entity tag
Origin	Identifies the origin of certain browser requests

Headers do not all have the same security properties. Some are generated and controlled by the browser, while application code can set others. On the server, every client-provided value must still be treated as untrusted input. Custom headers are possible, but standard mechanisms should be preferred when they already express the requirement. For example, authentication belongs in the standard **Authorization** header rather than an invented header such as **UserPassword**. The request body carries submitted data. The **Content-Type** header tells the server how to interpret it. Common content types include:

```
application/json
application/x-www-form-urlencoded
multipart/form-data
text/plain
application/octet-stream
```

JSON is common for APIs. Traditional HTML forms often use **application/x-www-form-urlencoded**. Forms that upload files usually use **multipart/form-data**, which divides the body into separate named sections. **application/octet-stream** represents arbitrary binary data. The body is a stream of bytes. ASP.NET Core uses the content type and endpoint parameter definitions to convert those bytes into .NET values. This conversion is part of model binding and input formatting, which Chapter 4 will examine.

A content type is not merely a file extension. It is an agreement between sender and receiver about how to interpret the body. If the client sends JSON but declares **text/plain**, automatic deserialization may fail even though the visible text looks correct. Large bodies require care. Upload limits, memory usage, streaming, timeouts, and cancellation become important. Reading a multi-gigabyte upload fully into memory is very different from processing it as a stream.

2.4 Responses and Status Codes

A response has a structure similar to a request:

```
Protocol version + status code + reason phrase
Headers
Blank line
Optional body
```

A JSON response might look like this:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 78
Cache-Control: no-store
```

```
{
  "id": 42,
  "name": "Mechanical Keyboard",
  "price": 129.00
}
```

The status code summarises the outcome. Headers describe the response and body. The body contains the representation returned to the client. The server may return no body. A deletion might respond with 204 **No Content**. A redirect may rely primarily on a **Location** header. A cached response may be validated with 304 **Not Modified**, telling the browser to reuse data it already has. HTTP status codes are grouped by their first digit:

Range	Meaning
1xx	Informational
2xx	Successful
3xx	Redirection or cache-related
4xx	The client request cannot be fulfilled as sent
5xx	The server failed while handling a valid-looking request

The most common codes are enough for most applications:

Code	Meaning	Typical use
200 OK	Request succeeded	Reading or updating a resource
201 Created	A resource was created	Successful POST
204 No Content	Request succeeded without a body	Delete or update with no representation returned
304 Not Modified	Cached representation is still valid	Conditional GET
400 Bad Request	Request data is malformed or invalid	Invalid JSON or validation failure
401 Unauthorized	Authentication is missing or invalid	User must sign in or provide valid credentials
403 Forbidden	Identity is known but lacks permission	Signed-in user cannot perform the action
404 Not Found	Resource or route does not exist	Unknown product identifier
409 Conflict	Request conflicts with current state	Duplicate key or concurrency conflict
415 Unsupported Media Type	Body format is not supported	JSON expected but another type was sent
429 Too Many Requests	Client exceeded a limit	Rate limiting
500 Internal Server Error	Unexpected server failure	Unhandled exception
503 Service Unavailable	Service is temporarily unable to respond	Maintenance or unavailable dependency

The distinction between 401 and 403 is especially important. Despite its historical name, **401 Unauthorized** means the request is not successfully authenticated. **403 Forbidden** means authentication may have succeeded, but the user is not allowed to perform the operation. Status codes should be meaningful but not used as a substitute for a useful response body. An API can return structured validation details alongside 400, or a stable error code alongside 409. The status gives the broad category; the body gives application-specific context. Returning 200 OK for every outcome and placing an error only inside JSON makes clients harder to build. Infrastructure, browser tools, logging systems, and HTTP libraries already understand status codes, so the application should use them.

2.5 Representations, API Contracts, and REST

A resource is the application concept identified by an address. A representation is the data format used to describe it. The product at `/api/products/42` is the resource. JSON is one possible representation:

```
{
  "id": 42,
  "name": "Mechanical Keyboard",
  "price": 129.00,
  "currency": "EUR"
}
```

The same resource could theoretically be represented as HTML, XML, CSV, or another format. The **Accept** request header and **Content-Type** response header help client and server agree on the representation. JSON maps naturally to common programming structures: objects, arrays, strings, numbers, booleans, and `null`. It is compact, readable, and supported throughout the web ecosystem. A JSON object is not a C# object travelling across the network. The sender serializes values into text bytes. The receiver parses those bytes and creates its own in-memory object.

```
C# ProductResponse
-> JSON serialization
-> HTTP response body
-> JSON parsing
-> Client-side object
```

That boundary matters. Methods, interfaces, reference identity, private fields, and runtime behaviour are not transferred. Only represented data crosses the network. It is convenient to serialize database entities directly, but doing so makes the network contract depend on persistence details. A new database property may accidentally appear in the API. A navigation property may create a large object graph or serialization cycle. Internal identifiers or sensitive data may be exposed. A dedicated response model makes the contract explicit:

```
public sealed record ProductResponse(int Id, string Name, decimal Price, string Currency);
```

A create request may use a different shape:

```
public sealed record CreateProductRequest(string Name, decimal Price, string Currency);
```

The client does not choose the new product identifier, and server-only fields are not accepted. Request and response models describe what may cross the boundary rather than mirroring every property of an internal class. This distinction becomes more valuable as an application grows. Database entities, domain models, API contracts, and Blazor view state may overlap, but they do not necessarily serve the same purpose. REST stands for **Representational State Transfer**. It describes an architectural style for networked systems rather than a strict API specification. In ordinary business applications, a REST-style API usually means:

- resources have stable addresses;
- standard HTTP methods express common operations;
- requests contain the information needed to process them;
- responses use meaningful status codes;
- representations such as JSON carry resource data;
- HTTP caching and other protocol features are used where appropriate.

A practical Product Catalog design might look like this:

```
GET    /api/products
GET    /api/products/{id}
POST   /api/products
PUT    /api/products/{id}
DELETE /api/products/{id}
```

This is clearer than operation-heavy routes such as:

```
POST /api/getAllProducts
POST /api/createNewProduct
POST /api/deleteProduct
```

The REST-style version uses the path to identify the resource and the method to express the operation. Not every useful HTTP endpoint must be forced into simple CRUD. Some operations represent commands rather than resource replacement:

```
POST /api/orders/42/cancel
POST /api/reports/monthly/generate
```

That is acceptable. A clear, stable contract is more important than pretending every business operation is a textbook resource update. The tutorial will use REST-style conventions because they make request design predictable, not because every application must satisfy a purity test.

2.6 Statelessness, Cookies, Sessions, and Browser Storage

HTTP is described as stateless because each request is processed as an independent message. The protocol does not automatically remember that two requests came from the same user or belong to the same workflow. Consider two requests:

```
GET /products
POST /orders
```

The second request does not inherently carry the meaning, “this is the same person who viewed the products a moment ago.” If the server needs that information, the requests must include an identifier or credential that allows the application to associate them. Statelessness makes systems easier to scale. Any suitable server instance can process a request if the necessary information is available in the request or shared storage. The application does not need one permanent network conversation for every user action. It also creates a problem: useful applications do need continuity. A shopping cart, signed-in identity, chosen language, and multi-step form all require some form of state. Cookies, tokens, server-side sessions, browser storage, and database records provide that continuity above HTTP.

A cookie is a small name–value item associated with a website. The server creates one with a **Set-Cookie** response header:

```
Set-Cookie: session=abc123; Path=/; Secure; HttpOnly; SameSite=Lax
```

The browser stores the cookie and sends it with later matching requests:

```
Cookie: session=abc123
```

The exchange becomes:

First response

```
<- Set-Cookie: session=abc123
```

Later request

```
-> Cookie: session=abc123
```

The cookie value often identifies server-side state rather than containing the whole state itself. For authentication, it may contain an encrypted and signed ticket or a session identifier that the server can validate. Important cookie attributes include:

Attribute	Effect
Secure	Send the cookie only over HTTPS
HttpOnly	Prevent normal JavaScript from reading it
SameSite	Restrict cross-site sending to reduce certain attacks
Path	Limit the URL paths for which it is sent
Domain	Control the hosts to which it applies
Expires / Max-Age	Control persistence

Cookies are automatically attached by the browser when their rules match. This is convenient for authentication, but it also means the server must defend against cross-site request forgery for state-changing operations. The security chapter will return to this. Cookies are small and are sent repeatedly, so they should not become a general-purpose storage system. Large cookies increase every matching request and response. The word **session** is used in several related ways. Broadly, it means a period of interaction associated with one user or client. A traditional server-side session works like this:

```
Browser cookie: sessionId=abc123
```

|

v

Server session store:

```
abc123 -> cart, preferences, temporary workflow state
```

The browser sends the identifier. The server loads the associated state from memory, a distributed cache, or another store. This can be useful for short-lived, user-specific data, but it introduces operational concerns. If session data exists only in one server’s memory, another server instance cannot continue the session unless requests are pinned to the original server. A restart may erase the data. Distributed storage solves some of this but adds complexity.

ASP.NET Core supports session middleware, but many applications use explicit database records, authentication tickets, claims, request data, or client-side state instead. Session state is a tool, not a requirement for every web application. In Blazor, the term session may also refer informally to an interactive connection or signed-in period. Those concepts should not be confused automatically with ASP.NET Core’s session middleware.

Browser storage mechanisms do not all participate in HTTP communication. **localStorage** and **sessionStorage** are accessed by code running in the browser. Their values are not automatically sent with every request. A Blazor application can use them through JavaScript interoperability or a library built on top of it.

Cookie

- > automatically included in matching HTTP requests
- `localStorage` / `sessionStorage`
- > read explicitly by browser-side code
 - > included in a request only if the application adds the value

This difference has security consequences. A cookie marked `HttpOnly` cannot be read by ordinary JavaScript, which helps protect it from some script-based token theft. A token stored in `localStorage` is available to scripts running in that origin. If the application has an XSS vulnerability, malicious script may read it. The correct storage mechanism depends on the hosting model, authentication design, and threat model. The security chapter will compare cookie and bearer-token approaches in detail.

2.7 Caching, Redirects, and Content Negotiation

Web applications repeatedly request the same resources: CSS files, JavaScript bundles, images, fonts, product lists, and API responses. Caching allows a browser or intermediary to reuse a previous response when it is still valid. The server controls much of this behaviour through response headers. A static versioned file may be cached for a long time:

`Cache-Control: public, max-age=31536000, immutable`

A sensitive response may forbid storage:

`Cache-Control: no-store`

A response that may be stored but must be revalidated can use an entity tag:

`ETag: "product-42-v7"`

The browser later asks whether its cached copy is still current:

`If-None-Match: "product-42-v7"`

If nothing changed, the server can return:

`HTTP/1.1 304 Not Modified`

No response body is needed; the browser reuses its cached representation. The basic decision looks like this:

Browser has cached response

- > Is it still fresh?
 - > yes: use it immediately
 - > no: revalidate with server
 - > unchanged: 304, reuse cached body
 - > changed: 200, store new body

Caching improves speed and reduces server load, but stale data can confuse users. Static assets with content hashes are easy to cache aggressively because a changed file receives a new URL. User-specific or rapidly changing API data needs more careful rules. A browser cache is different from an application cache on the server. ASP.NET Core may also cache expensive results in memory or in a distributed service. One prevents network transfer; the other prevents repeated computation or database access. A redirect response tells the client that another address should be used. It usually includes a `Location` header:

`HTTP/1.1 302 Found`

`Location: /account/login`

The browser receives the response and sends another request to the new address.

`GET /admin/products`

`<- 302 Location: /account/login`

`GET /account/login`

`<- 200 HTML`

Common redirect codes include:

Code	General meaning
301 Moved Permanently	Resource has a permanent new address
302 Found	Temporary redirect, historically common
303 See Other	Retrieve another resource with <code>GET</code> after an operation

Code	General meaning
307 Temporary Redirect	Temporary redirect preserving method and body
308 Permanent Redirect	Permanent redirect preserving method and body

The method-preservation distinction matters. Redirecting a `POST` with `307` or `308` tells the client to repeat the `POST` at the new address. A `303` tells it to follow with `GET`. ASP.NET Core commonly redirects HTTP traffic to HTTPS and unauthenticated browser users to a login page. APIs often return `401` instead of redirecting because an API client expects a machine-readable failure, not login-page HTML. A client can express which response formats it accepts:

```
Accept: application/json
```

The server chooses an available representation and identifies it with `Content-Type`:

```
Content-Type: application/json; charset=utf-8
```

This process is called content negotiation. Many APIs return JSON exclusively, which keeps the contract simple. ASP.NET Core controllers can support multiple output formatters, but supporting formats that no client needs creates extra testing and maintenance. Language and compression can also be negotiated. A browser may send `Accept-Language`, and it commonly advertises supported compression through `Accept-Encoding`. The server can return compressed content with a matching `Content-Encoding` such as `gzip` or `br`. Compression reduces transferred bytes, especially for HTML, CSS, JavaScript, and JSON. Images and videos are often already compressed in their own formats.

2.8 CORS and Authentication Information

Suppose a Blazor application is loaded from `https://app.example.com` and calls an API at `https://api.example.com`. These are different origins because their hosts differ. The browser's same-origin policy restricts frontend code from freely reading cross-origin responses. The API can grant permission through CORS response headers:

```
Access-Control-Allow-Origin: https://app.example.com
```

For some requests, the browser sends a **preflight** request first:

```
OPTIONS /api/products HTTP/1.1
Origin: https://app.example.com
Access-Control-Request-Method: POST
Access-Control-Request-Headers: content-type
```

The server responds with the allowed origin, methods, and headers. Only then does the browser send the actual request.

```
Browser
-> OPTIONS preflight
API
-> CORS permission
Browser
-> actual POST
API
-> response
```

Simple requests may not require a preflight, but they are still subject to CORS rules when frontend code tries to read the response. CORS does not authenticate the user, authorise an operation, or prevent direct requests from tools such as `curl`. It tells browsers which origins may access responses from frontend code. The API must still validate credentials and permissions normally. A common development error is to “fix CORS” by allowing every origin, method, header, and credential. That may hide the immediate error while weakening the security boundary. The policy should allow only the clients that genuinely need access. HTTP itself does not know who a user is. Authentication information is carried through requests. Two common approaches are:

```
Cookie authentication
-> browser automatically sends authentication cookie
Bearer authentication
-> client explicitly sends token in Authorization header
```

A bearer-token request looks like this:

Authorization: Bearer eyJhbGciOi...

The server validates the cookie or token and creates an in-memory identity for the current request. ASP.NET Core then exposes that identity through `HttpContext.User`. The credential is not the user object itself. It is evidence that the server validates and turns into claims such as user identifier, name, role, or permission. Authentication and authorization are large enough to deserve their own chapter. For now, the important connection is that user identity must be reconstructed or validated from information arriving with each request.

2.9 Errors, Timeouts, Cancellation, and Retries

When server code throws an unexpected exception, the client cannot receive the exception object directly. The server turns the failure into an HTTP response. During development, ASP.NET Core may return a detailed error page to a local browser. In production, exposing stack traces, SQL details, file paths, or secrets would be dangerous. Production responses should contain safe error information while the full exception is written to server logs. An API error might use a structured format:

```
{
  "type": "https://example.com/problems/product-not-found",
  "title": "Product not found",
  "status": 404,
  "detail": "No product with identifier 42 exists.",
  "traceId": "00-a1b2c3..."
}
```

The client receives enough information to react, while the trace identifier helps operators correlate the response with logs. Validation errors are expected application outcomes, not server crashes. A malformed price should usually result in **400 Bad Request** with validation details, not **500 Internal Server Error**. The network boundary forces errors to become explicit contracts. A Blazor component cannot assume every call succeeds; it needs loading, success, validation, unauthenticated, forbidden, not-found, and unexpected-failure states where appropriate. Networks fail in ambiguous ways. A client may wait too long and give up, but the server may still complete the operation. A connection may close after the database update but before the response reaches the browser. This creates an important distinction:

The client did not receive success
is not always the same as
The server did not complete the operation

Retries are usually safe for reads. They require more thought for state-changing operations. Repeating a **POST** that creates an order may create a duplicate. Some systems use idempotency keys. The client sends a unique operation identifier, and the server stores the result of the first execution. Repeating the same key returns the original outcome instead of performing the operation again. ASP.NET Core exposes request cancellation through `HttpContext.RequestAborted`.

Endpoint and service methods should normally pass the resulting `CancellationToken` into asynchronous database and network calls. This allows work to stop when the client disconnects, although application rules may sometimes require an operation to continue independently. Timeouts, cancellation, and retries are not decorative resilience features. They are ways of handling the fact that distributed communication cannot provide the same certainty as a local method call.

2.10 HTTP Versions and Real-Time Communication

You may encounter HTTP/1.1, HTTP/2, and HTTP/3. They preserve the same broad request-response semantics while changing how messages are transported. HTTP/1.1 made persistent connections standard but can suffer when many requests compete over limited connections. HTTP/2 multiplexes many streams over one connection and compresses headers, reducing several sources of delay. HTTP/3 runs over QUIC and improves behaviour on unreliable or changing networks, especially when packet loss occurs or a device switches between connections. Application code normally does not choose a different endpoint design for each version. ASP.NET Core and the hosting infrastructure negotiate supported protocols. The important lesson is that HTTP semantics such as methods, headers, status codes, and bodies remain useful even as the transport evolves.

Ordinary HTTP works well when the client asks and the server answers. Some features need the server to send information as soon as something changes:

- chat messages;

- live dashboards;
- collaborative editing;
- notification counters;
- progress updates;
- product availability changes.

Polling is the simplest approach:

Every 5 seconds:

Browser -> "Anything new?"

Server -> "No."

Polling is easy but may waste requests and introduce delay. Long polling keeps a request open until data is available, then the client immediately creates another request. Server-Sent Events allow a server to stream one-way events over an HTTP connection. WebSockets provide a long-lived, two-way communication channel.

Ordinary HTTP

Browser -> request

Browser <- response

connection may be reused, but exchange is request-led

WebSocket

Browser <=====> Server

both sides can send messages while connection remains open

WebSockets begin with an HTTP handshake and then upgrade the connection to the WebSocket protocol. SignalR is ASP.NET Core's real-time communication library. It gives applications a hub-based programming model and handles many connection details. Instead of manually designing low-level WebSocket messages, the server can expose hub methods and send messages to connected clients. SignalR selects an available transport and manages connection identifiers, groups, serialization, reconnection support, and integration with ASP.NET Core authentication. For the Product Catalog, authorised users might edit a product while other users are viewing the list. After saving, the server could broadcast a notification:

Admin browser

-> PUT /api/products/42

Server

-> updates database

Server SignalR hub

-> ProductUpdated(42)

Viewer browsers

-> refresh or update product 42

SignalR does not replace normal APIs or database storage. HTTP endpoints still perform commands and queries. SignalR tells connected clients that something changed or delivers time-sensitive messages. Chapter 13 will return to SignalR after the tutorial has covered ASP.NET Core, Blazor, authentication, and application structure.

2.11 Inspecting HTTP

The Network panel turns HTTP from an abstract protocol into observable behaviour. Open the panel, reload a page, and select a request. The browser usually shows:

- the full URL;
- request method;
- status code;
- remote address;
- request and response headers;
- query parameters;
- cookies;
- request payload;
- response body;
- timing information;
- whether the response came from cache.

For an API call, check the request in this order:

1. Was a request sent?
2. Is the URL correct?
3. Is the method correct?
4. Is the request body what the endpoint expects?
5. Is Content-Type correct?
6. Did authentication information accompany the request?
7. What status code came back?
8. What does the response body say?
9. Did CORS or the browser block access?
10. How long did each phase take?

A frontend error such as “Failed to fetch” may represent several different problems: DNS failure, refused connection, invalid certificate, CORS rejection, aborted request, or network disconnection. The Network and Console panels usually reveal more than the application message alone. You can also reproduce requests outside the browser with tools such as `curl`, HTTP files in an editor, or API clients. This helps separate server behaviour from browser restrictions. A `curl` request for the product list might be:

```
curl -i https://catalog.example.com/api/products
```

The `-i` option includes response headers. A JSON POST might be:

```
curl -i \
  -X POST \
  -H "Content-Type: application/json" \
  -d '{"name":"Mechanical Keyboard","price":129,"currency":"EUR"}' \
  https://catalog.example.com/api/products
```

The same HTTP contract can be exercised without the UI, which is useful for debugging and automated testing.

2.12 The Product Catalog HTTP Contract

Before writing ASP.NET Core code, we can describe part of the Product Catalog as a protocol contract. List products:

```
GET /api/products?page=1&pageSize=20 HTTP/1.1
Accept: application/json
```

Successful response:

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{
  "items": [
    {
      "id": 42,
      "name": "Mechanical Keyboard",
      "price": 129.00,
      "currency": "EUR"
    }
  ],
  "page": 1,
  "pageSize": 20,
  "totalCount": 1
}
```

Create a product:

```
POST /api/products HTTP/1.1
Content-Type: application/json
Accept: application/json
```

```
{
  "name": "Wireless Mouse",
  "price": 59.00,
```

```
    "currency": "EUR"
}
```

Successful creation:

```
HTTP/1.1 201 Created
Location: /api/products/43
Content-Type: application/json
```

```
{
  "id": 43,
  "name": "Wireless Mouse",
  "price": 59.00,
  "currency": "EUR"
}
```

Invalid request:

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json
```

```
{
  "title": "Validation failed",
  "status": 400,
  "errors": {
    "price": ["Price must be greater than zero."]
  }
}
```

Unknown product:

```
HTTP/1.1 404 Not Found
Content-Type: application/problem+json
```

```
{
  "title": "Product not found",
  "status": 404
}
```

This contract already answers important questions before the framework appears. We know the addresses, methods, data formats, successful outcomes, and failure categories. ASP.NET Core's job will be to receive these requests, select the correct code, bind the data, run application logic, and create these responses. Assume a signed-in administrator edits the price of product 42 in a Blazor form and clicks Save. The component validates the obvious client-side rules and sends an HTTP request:

```
PUT /api/products/42 HTTP/1.1
Content-Type: application/json
Cookie: auth=encrypted-ticket
```

```
{
  "name": "Mechanical Keyboard",
  "price": 119.00,
  "currency": "EUR"
}
```

The request crosses the secure connection and reaches ASP.NET Core. The framework reads the authentication cookie, creates the current identity, checks authorization, selects the endpoint, and deserializes the JSON body. Application code loads product 42, applies the change, and saves it through Entity Framework Core. The endpoint returns:

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{
  "id": 42,
  "name": "Mechanical Keyboard",
```

```
    "price": 119.00,  
    "currency": "EUR"  
}
```

The Blazor component receives the response, updates its state, renders again, and changes the relevant DOM text. The browser paints the new price. If the product was changed by someone else in the meantime, the server might instead return 409 **Conflict**. If the authentication cookie is missing, it may return 401. If the user is signed in but lacks permission, it returns 403. If the body is malformed, it returns 400. If the server encounters an unexpected failure, it returns 500 and records the exception. The complete interaction is:

User clicks Save

- > Blazor creates request data
- > Browser sends HTTPS request
- > ASP.NET Core authenticates and routes
- > Application updates product
- > EF Core updates database
- > ASP.NET Core creates HTTP response
- > Browser receives response
- > Blazor updates component state
- > Blazor changes DOM
- > Browser repaints price

Every later part of the tutorial fits somewhere in this path.

2.13 What to Keep in Mind

HTTP is a message contract: the client sends a method, target, headers, and optional body; the server returns a status, headers, and optional body. Methods express intent, status codes classify outcomes, and representations such as JSON carry data rather than live C# objects. HTTP is stateless, so applications add continuity through cookies, tokens, sessions, browser storage, and persistent records. Caching, redirects, content negotiation, CORS, authentication, and structured errors all operate through the same message boundary. Timeouts are ambiguous, cancellation should flow into I/O, and retries of writes require idempotency. SignalR and related transports supplement request-response when the server must notify connected clients.

Part II — ASP.NET Core

3. Inside an ASP.NET Core Application

The previous chapters followed a request across the network. The browser resolved a host name, opened a secure connection, sent an HTTP message, and received another message in return. We can now step across the server boundary and ask a more practical question: what is actually running on the machine that receives that request? An ASP.NET Core application is an ordinary .NET process with a web server and a request-processing pipeline. It is not a special kind of operating-system service, nor does it require a large framework-generated structure.

At its smallest, the entire application can fit into a few lines of `Program.cs`. That small beginning is intentional. ASP.NET Core supplies the hosting, configuration, logging, dependency injection, networking, and request abstractions, but it does not force every application to use the same architecture. A tiny API, a Blazor application, a background-processing service, and a large business system can all start from the same host and add only what they need.

This chapter opens that host. We will create an application, inspect its files, separate development settings from runtime behaviour, understand Kestrel, and follow the process from `dotnet run` to a listening server. Middleware, routing, and application services are introduced only far enough to make the application understandable; the next two chapters examine them in depth.

3.1 The Smallest Application and Its Project

Create an empty ASP.NET Core project from a terminal:

```
dotnet new web -n ProductCatalog
cd ProductCatalog
```

The web template produces a deliberately small application. Its central file is `Program.cs`:

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
app.MapGet("/", () => "Hello World!");
app.Run();
```

Run it:

```
dotnet run
```

The terminal prints one or more listening addresses. Open one of them in a browser and the text **Hello World!** appears. Only four statements are visible, but each marks a separate phase:

Create builder

- > Configure services and hosting
- > Build application
- > Configure request handling
- > Run until shutdown

`WebApplication.CreateBuilder(args)` prepares the host and its defaults. `builder.Build()` turns the collected configuration into a runnable application. `MapGet` declares what should happen when a GET request arrives for `/`. `Run` starts the server and keeps the process alive. The code is small because the template uses sensible defaults, not because nothing is happening. The builder prepares configuration, logging, dependency injection, and Kestrel. The built application represents both the running host and the place where the HTTP pipeline is assembled. The project file is usually named `ProductCatalog.csproj`:

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
</Project>
```

The important difference from a normal class library or console application is the SDK at the top:

```
<Project Sdk="Microsoft.NET.Sdk.Web">
```

The Web SDK adds the build behaviour and framework references expected by ASP.NET Core applications. Most core ASP.NET Core APIs come from the shared framework installed with the .NET runtime, so a new project does not need a long list of package references. Additional packages are added only when the application needs features that are not already included. `TargetFramework` selects the .NET version used to compile the application. This tutorial uses `net10.0`.

`Nullable` enables nullable reference-type analysis, and `ImplicitUsings` supplies common namespace imports automatically. Neither setting is specific to the web, but both keep the source focused on application code. A project created from a larger template may contain more properties and package references. The underlying rule remains the same: the project file defines how the application is built, while `Program.cs` defines how it is assembled and run.

The template does not show a `Program` class or a `Main` method because it uses C# top-level statements. The compiler still generates an entry point. This:

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
app.Run();
```

The same code is conceptually equivalent to:

```
public static class Program
{
    public static void Main(string[] args)
    {
        var builder = WebApplication.CreateBuilder(args);
        var app = builder.Build();
        app.Run();
    }
}
```

Top-level statements remove ceremony; they do not create a different hosting mechanism. Command-line arguments still arrive through `args`, exceptions can still fail startup, and the process still begins from one generated entry point. Types can be declared below the top-level statements in the same file, although larger applications normally move them into their own files. The template keeps everything together because it is showing the minimum, not recommending that an entire application remain in `Program.cs`.

3.2 Program.cs, the Builder, and the Build Boundary

For an experienced C# developer, the most useful way to think about `Program.cs` is as the application's composition root. It is where infrastructure is selected and connected. A typical file eventually has three visible regions:

```
var builder = WebApplication.CreateBuilder(args);
// Register services and configure hosting.
builder.Services.AddProblemDetails();
var app = builder.Build();
// Configure middleware and endpoints.
app.UseExceptionHandler();
app.MapGet("/", () => "Product Catalog");
app.Run();
```

Before `Build`, the application is being described. Services are registered, configuration sources can be added, logging can be adjusted, and server options can be changed. After `Build`, the application exists. Middleware and endpoints are arranged to define how requests are handled. `Run` begins execution of the host. In a normal web application, it does not return until the process is shutting down. This gives `Program.cs` a clear responsibility: assemble the application. Business rules do not belong there. A price calculation, order workflow, or product validation rule should live in an application or domain service. `Program.cs` decides that such a service is available and connects it to the web host. A useful boundary is:

Program.cs decides what is connected.
Application classes decide what the system does.

Small demonstrations often place logic directly inside endpoint lambdas because that keeps the example visible. As the Product Catalog grows, those lambdas will delegate to services rather than becoming a second business layer hidden inside startup code. `WebApplication.CreateBuilder(args)` creates a `WebApplicationBuilder` with common defaults already configured. The exact providers and settings can evolve between framework versions, but the important categories remain stable. The builder exposes several central areas:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services;           // Dependency Injection registrations
builder.Configuration;      // Configuration values and providers
builder.Logging;            // Logging providers and filters
builder.Environment;        // Environment name and application paths
builder.WebHost;            // Web-server-specific configuration
builder.Host;               // General host configuration
```

The builder is not the running application. It is a mutable setup object. Each area collects decisions that will later be used to construct the host. The defaults are why this minimal code can read `appsettings.json`, use environment variables, log to the console, create services through dependency injection, and listen through Kestrel without manually wiring those systems together. Defaults are not hidden magic. They are a prepared starting point. You can replace or extend them when the application has a reason to do so. The best approach is normally to keep the defaults until a concrete requirement proves that they are insufficient. This line is the boundary between setup and execution:

```
var app = builder.Build();
```

During `Build`, the framework creates the host, constructs the dependency injection container, validates selected configuration, and prepares the application components needed at runtime. Service registrations belong before this line:

```
builder.Services.AddSingleton<ProductClock>();
var app = builder.Build();
```

Request-pipeline configuration belongs after it:

```
var app = builder.Build();
app.UseExceptionHandler();
app.MapGet("/api/products", () => Results.Ok());
```

Trying to treat the builder and application as interchangeable leads to confusion. `builder.Services` describes how objects should be created. `app` describes how the running server should process requests. Later chapters will show that these two phases are related. Middleware and endpoints can request services registered through the builder. The host creates those services according to their configured lifetime and supplies them when a request needs them.

3.3 The Host and Kestrel

The word **host** appears frequently in ASP.NET Core documentation. It refers to the runtime environment that manages the application process and its shared infrastructure. The host is responsible for concerns such as:

- starting and stopping the web server;
- loading configuration;
- creating the dependency injection container;
- providing logging;
- exposing the current environment;
- starting hosted background services;
- responding to shutdown signals.

Kestrel is one part of the host, not the host itself. Kestrel accepts network connections and turns incoming HTTP data into framework request objects. The host manages Kestrel together with the rest of the application. The relationship can be pictured as:

Operating-system process
-> .NET application host

- > Configuration
- > Logging
- > Dependency Injection
- > Background services
- > Kestrel web server
- > ASP.NET Core request pipeline

This separation is useful because a .NET host can manage work that is not tied directly to an HTTP request. A background service may process a queue while Kestrel handles web traffic. Both share configuration, logging, dependency injection, startup, and shutdown. Kestrel is the cross-platform web server included with ASP.NET Core and enabled by the standard web templates. It listens on configured network endpoints, accepts connections, understands supported HTTP protocols, and passes each request into ASP.NET Core. When the terminal prints something like `Now listening on: http://localhost:5098`, Kestrel has opened a listening socket on that address and port. The flow is:

Browser or API client

- > Network connection
- > Kestrel
- > ASP.NET Core pipeline
- > Endpoint
- > ASP.NET Core response
- > Kestrel
- > Network connection
- > Client

Kestrel is designed for direct use and can also run behind another server or proxy. In production, a reverse proxy such as Nginx, IIS, Apache, a cloud load balancer, or an ingress service may accept public traffic first and forward it to Kestrel.

Internet

- > Reverse proxy or load balancer
- > Kestrel
- > ASP.NET Core application

That arrangement can centralise TLS certificates, host several applications on one machine, apply forwarding rules, or integrate with platform-specific process management. It is common, but it does not mean Kestrel is merely a development server. Kestrel remains the server that directly hosts the ASP.NET Core application. The deployment chapter will return to this boundary. During development, the important point is simply that `dotnet run` starts a real web server inside the application process.

3.4 Addresses, Launch Profiles, and Development HTTPS

A listening address combines a scheme, host binding, and port. The value is `http://localhost:5098`. `localhost` means the current machine. A browser on the same computer can connect, but another device on the network normally cannot use that binding. A server can instead listen on all IPv4 network interfaces at `http://0.0.0.0:5098`, or use the equivalent IPv6 any-address. This makes the process reachable through the machine's network addresses, assuming firewalls and routing allow it.

The distinction is important during testing. An application can work perfectly in the desktop browser and still be unavailable from a phone because Kestrel is bound only to `localhost`. Listening publicly also changes the security situation. Development endpoints, detailed errors, and unprotected test data should not be exposed merely to make another device connect. A development machine should use an intentional binding and appropriate firewall rules.

ASP.NET Core can obtain listening addresses from launch profiles, environment variables, command-line arguments, configuration, or explicit Kestrel setup. The chosen source depends on how the application is launched and deployed. A generated project usually contains `Properties/launchSettings.json`. It may look similar to this:

```
{
  "$schema": "https://json.schemastore.org/launchsettings.json",
  "profiles": {
    "http": {
      "commandName": "Project",
      "dotnetRunMessages": true,
```

```

    "launchBrowser": true,
    "applicationUrl": "http://localhost:5098",
    "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
    }
},
"https": {
    "commandName": "Project",
    "dotnetRunMessages": true,
    "launchBrowser": true,
    "applicationUrl": "https://localhost:7184;http://localhost:5098",
    "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
    }
}
}
}
}

```

A launch profile tells development tools how to start the application. It can select URLs, set environment variables, choose whether to open a browser, and provide different profiles for HTTP and HTTPS. It is not production configuration. `launchSettings.json` is intended for local launch behaviour and is not the mechanism by which a deployed server should be configured. You can choose a profile explicitly:

```
dotnet run --launch-profile https
```

You can also avoid launch profiles:

```
dotnet run --no-launch-profile
```

Visual Studio, VS Code launch configurations, Rider, and `dotnet run` may each participate in how the process is started. When an application listens on an unexpected port or uses the wrong environment, inspect the launch profile and the actual command before changing application code.

The HTTPS launch profile requires a certificate. In development, .NET can use a local development certificate rather than a publicly issued domain certificate. If the browser does not trust that certificate, it displays a warning even though the application itself is running correctly. The certificate must be trusted by the operating system or browser for the warning to disappear. The exact trust setup differs by operating system and browser, especially on Linux.

This is only a development convenience. A production domain uses a certificate valid for that domain, normally issued and renewed through a trusted certificate authority or hosting platform. HTTPS should not be postponed until the final deployment step. Authentication cookies, browser capabilities, and several security features behave differently on secure and insecure origins. Developing with HTTPS exposes those differences earlier.

3.5 Environments and Configuration

ASP.NET Core applications have a named runtime environment. The conventional names are:

```

Development
Staging
Production

```

Custom names are possible, but these three cover most applications. The current environment is available from the builder and the built application:

```

var builder = WebApplication.CreateBuilder(args);
Console.WriteLine(builder.Environment.EnvironmentName);
var app = builder.Build();
if (app.Environment.IsDevelopment())
{
    // Development-only behaviour.
}

```

The environment name is selected before normal application execution. Local launch profiles commonly set it to **Development**. A deployed application should normally run as **Production** unless the hosting environment explicitly sets another value. Environments let infrastructure behaviour vary without changing the compiled application. Development can show detailed error pages and use local integrations. Production can use stricter error handling, external services, and production configuration.

The environment should not become a substitute for business configuration. Code such as `if (Production) charge customer else do not` is usually a sign that the application's operational mode and business rules have been mixed together. Environments describe where the application is running; explicit configuration describes what features and integrations it should use. The default builder assembles configuration from several providers. Common sources include:

- `appsettings.json`;
- an environment-specific file such as `appsettings.Development.json`;
- user secrets during development when configured;
- environment variables;
- command-line arguments.

Later providers can override values loaded earlier. This allows a stable base file to be committed while a deployment supplies its own connection strings, service addresses, or secrets. A simple `appsettings.json` might contain:

```
{
  "Catalog": {
    "DefaultCurrency": "EUR",
    "PageSize": 20
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  }
}
```

The environment-specific file can override only the values that differ:

```
{
  "Catalog": {
    "PageSize": 5
  }
}
```

Configuration is available through `builder.Configuration`:

```
var currency = builder.Configuration["Catalog:DefaultCurrency"];
```

Direct string lookup is useful for a quick demonstration, but larger applications bind related values to typed options classes. Chapter 5 will cover that approach and explain provider precedence, validation, environment variables, and secrets properly. Do not place production passwords, API keys, or signing secrets into committed `appsettings.json` files. Configuration files are convenient, but they are not secret stores.

3.6 Content, Public Assets, and Project Structure

The **content root** is the application's base path for content files. It is commonly the directory containing the project during development and the published application after deployment. The builder exposes it through:

```
builder.Environment.ContentRootPath
```

Files such as `appsettings.json` are resolved relative to the content root by default. The **web root** is the directory intended to contain public static assets. By convention, it is `wwwroot`:

```
ProductCatalog/
-> wwwroot/
    -> css/
```

```
-> images/  
-> js/
```

The corresponding path is available through:

```
builder.Environment.WebRootPath
```

The distinction is a security boundary. Source files, configuration, database files, and arbitrary content-root files must not become publicly downloadable merely because they exist beside the application. Only configured static assets should be served. A request for `/images/logo.png` may map to `wwwroot/images/logo.png` when static-asset handling is enabled. A request for `/appsettings.json` should not expose the application's configuration file. Modern ASP.NET Core applications can map static assets efficiently through framework endpoint conventions, while some scenarios still use static-file middleware. Blazor templates configure the appropriate static assets for their hosting model. We will revisit this when the UI project is introduced. After the application has been run, the project may contain:

```
ProductCatalog/  
-> ProductCatalog.csproj  
-> Program.cs  
-> appsettings.json  
-> appsettings.Development.json  
-> Properties/  
    -> launchSettings.json  
-> bin/  
-> obj/
```

`Program.cs` contains the composition root. The project file controls building. The `appsettings` files provide configuration. `launchSettings.json` controls local launch profiles. `bin` contains build output. `obj` contains intermediate files used by the compiler and SDK. They are generated artifacts, not source code, and are normally excluded from Git. Larger templates add directories such as `Components`, `Pages`, `Controllers`, or `wwwroot`. Those names reflect selected features, not mandatory ASP.NET Core architecture. The framework does not require every application to have `Controllers`, `Services`, `Repositories`, and `Models` folders. Start with the structure that expresses the application. Add projects and folders when they separate meaningful responsibilities, not because a template or diagram suggests that every possible layer must exist from the beginning.

3.7 Running, Startup, Lifetime, and Shutdown

When you execute:

```
dotnet run
```

The .NET CLI restores missing dependencies if necessary, builds the project, and starts the produced application. The console output comes from both the CLI and the application's configured logging. A release build can be selected explicitly:

```
dotnet run --configuration Release
```

During normal development, `dotnet watch` is often more convenient:

```
dotnet watch
```

It watches source files and applies supported hot-reload changes or restarts the application when necessary. Hot reload changes the development loop, not the architecture of the running application. Requests still reach Kestrel and travel through the same pipeline. An IDE debugger starts the same kind of process but attaches debugging services and may use its own launch configuration. If behaviour differs between the terminal and debugger, compare the selected launch profile, environment variables, working directory, and command-line arguments. Not every failure occurs while processing a request. The application can fail during startup. Examples include:

- invalid configuration;
- a port already used by another process;
- an unavailable certificate;
- an exception while registering or constructing startup services;
- invalid dependency injection registrations detected during validation;
- missing files required during composition.

If startup fails before Kestrel begins listening, the browser cannot receive an application error response because there is no functioning server to answer. The useful information is in the process output and logs. This distinction prevents a common debugging mistake. A browser message such as “connection refused” is not an HTTP 500 response. It normally means the connection could not be established at all. The process may not be running, may be listening on another address, or may have failed during startup.

No connection

- > Check process and listening address

HTTP error response

- > Server was reached; inspect request handling

Developer tools can confirm which case occurred, but the terminal output is often the fastest source for startup failures. The final line in the minimal template is:

```
app.Run();
```

It starts the host and waits until shutdown. Without this call, the application would be assembled and then the process would reach the end of `Program.cs` without serving normal traffic. There is also an asynchronous form:

```
await app.RunAsync();
```

For most applications, either form expresses the same lifetime: start the host and remain active until shutdown. `RunAsync` becomes convenient when startup code is already asynchronous or when the application needs to compose its lifetime with other asynchronous work. Do not place ordinary sequential code after `Run` and expect it to execute while the server is active:

```
app.Run();
```

```
Console.WriteLine("This runs only after the host stops.");
```

Work that must run concurrently with the server belongs in hosted services or another lifecycle-aware component. Work that must happen before serving requests belongs before `Run`, ideally in an explicit startup step rather than an arbitrary endpoint or constructor.

A web process eventually receives a shutdown signal. During development, this often happens when you press `Ctrl+C`. In production, a service manager, container platform, or operating system may request termination. The host begins graceful shutdown rather than immediately abandoning all managed work. It signals hosted services, stops accepting normal new work, and allows active operations a limited period to finish before the process exits. This does not make every operation automatically safe. Application code should still observe cancellation where appropriate and avoid starting work that cannot be tracked. A request may be cancelled because the client disconnected. A background service receives a stopping token. Database and HTTP operations can often accept those tokens. The broader rule is that a server is a long-running process with an explicit lifetime:

Configure

- > Build
- > Start
- > Serve many requests
- > Receive shutdown signal
- > Stop gracefully

That model differs from a command-line utility that performs one calculation and exits. It also explains why per-request data must not be placed casually into static fields: the same process serves many users and many requests over a long period.

3.8 One Process Serves Many Requests

Kestrel does not start a new application for each browser request. One running process normally handles many concurrent requests.

ASP.NET Core process

- > Request A from user 1
- > Request B from user 2
- > Request C from user 1
- > Background work

Requests can overlap. One may be waiting for the database while another is serializing a response. This is why asynchronous I/O matters in web applications: waiting for network or database operations should not unnecessarily occupy a thread. It is also why shared mutable state requires care. A singleton object may be accessed by many requests at once. A static collection used as an in-memory store must be thread-safe. A request-specific object must not leak into another user's operation. ASP.NET Core's dependency injection lifetimes help express these boundaries. Singleton services belong to the whole process, scoped services normally belong to one request, and transient services are created whenever requested. Chapter 5 will explain those lifetimes using the Product Catalog. For now, remember that the process is shared while request context is not.

3.9 The First Product Catalog Host

Replace the initial `Program.cs` with a small but complete Product Catalog host:

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
ProductSummary[] products =
[
    new(1, "Mechanical Keyboard", 129.00m, "EUR"),
    new(2, "Wireless Mouse", 59.00m, "EUR"),
    new(3, "USB-C Dock", 189.00m, "EUR")
];
app.MapGet("/", () => Results.Ok(new
{
    application = "Product Catalog",
    environment = app.Environment.EnvironmentName,
    status = "Running"
}));
app.MapGet("/api/products", () => Results.Ok(products));
app.Run();
public sealed record ProductSummary(int Id, string Name, decimal Price, string Currency);
```

Run the application and request the root address. The response is JSON describing the process and current environment. Request `/api/products` and the array is serialized to JSON automatically:

```
[
  {
    "id": 1,
    "name": "Mechanical Keyboard",
    "price": 129.00,
    "currency": "EUR"
  },
  {
    "id": 2,
    "name": "Wireless Mouse",
    "price": 59.00,
    "currency": "EUR"
  },
  {
    "id": 3,
    "name": "USB-C Dock",
    "price": 189.00,
    "currency": "EUR"
  }
]
```

Several framework systems are already cooperating:

```
Kestrel receives request
-> ASP.NET Core selects mapped endpoint
-> Endpoint returns object result
-> Framework serializes object as JSON
-> Kestrel sends HTTP response
```

The product data is intentionally held in a local array. This is not the final design. It keeps the current example focused on the host. Chapter 5 will move data access behind services, and the Entity Framework Core chapter will replace the in-memory data with a database. A browser is convenient for GET requests, but command-line tools expose the HTTP exchange more clearly. Use `curl` with the listening address printed by the application:

```
curl -i http://localhost:5098/api/products
```

The `-i` option includes response headers. The result shows the status line, content type, and body rather than only rendering the JSON. You can also use the Network panel in browser developer tools, an HTTP file in VS Code, or a dedicated API client. The tool is less important than the habit of inspecting the actual request and response. At this stage, debugging should move through layers:

Is the process running?

- > Is Kestrel listening on the expected address?
- > Does the request reach the server?
- > Which status and body return?
- > Does the client interpret them correctly?

That sequence is faster than changing several parts of the application at once.

3.10 The Host Mental Model

The Product Catalog host is small, but it already has the main shape of a production ASP.NET Core process. The .NET runtime starts the generated entry point. **CreateBuilder** prepares configuration, logging, dependency injection, the environment, and web-host defaults. **Build** creates the host and application. Kestrel opens listening endpoints. The endpoint mappings describe two possible request destinations. **Run** keeps the process alive and participates in graceful shutdown. What remains deliberately unexplained is how an incoming request moves from Kestrel to a particular endpoint. That is the job of the request pipeline and routing. The boundary currently looks like this:

Client

- > HTTP connection
- > Kestrel
- > ?
- > Mapped endpoint
- > Response

Chapter 4 fills in that question mark. We will follow a request through middleware, understand why order matters, see how routing selects an endpoint, compare Minimal APIs and controllers, and examine how request data becomes `C#` parameters.

An ASP.NET Core application is a long-running .NET process assembled in `Program.cs`. **CreateBuilder** prepares configuration, logging, dependency injection, the environment, and Kestrel; **Build** creates the application; mapped endpoints define request destinations; and **Run** owns the process lifetime. Launch profiles and development certificates affect local startup, while production configuration comes from the deployment environment. The content root and web root serve different purposes, and only configured public assets should be exposed. One process serves many concurrent requests, so service lifetimes, cancellation, shared state, and graceful shutdown matter. The host is the outer shell; the request pipeline determines which code handles each message.

4. From Request to Application Code

Kestrel can accept a connection and turn incoming bytes into an HTTP request, but that still does not answer the most useful question: how does ASP.NET Core decide which C# code should run? The answer is not one mechanism. A request moves through a pipeline. Middleware can inspect it, change it, reject it, or perform work around the rest of the pipeline. Routing then matches the request to an endpoint. The endpoint reads values from the request, calls application code, and creates a response. That response travels back through the middleware before Kestrel sends it to the client. The complete path looks like this:

```
Client
-> Kestrel
-> Middleware
-> Routing
-> Endpoint
-> Application code
-> Endpoint result
-> Middleware
-> Kestrel
-> Client
```

This chapter follows that path in detail. It introduces `HttpContext`, middleware order, short-circuiting, routing, route templates, Minimal APIs, controllers, model binding, results, errors, and request cancellation. The Product Catalog will grow from two fixed endpoints into a small HTTP API, but its data will remain in memory until the next chapter moves responsibility into services.

4.1 `HttpContext` and the Middleware Pipeline

ASP.NET Core represents the current HTTP exchange with an `HttpContext`. One context is created for each request and is available while that request moves through the pipeline. The most important parts are:

```
HttpContext
-> Request
-> Response
-> User
-> Items
-> RequestServices
-> RequestAborted
-> TraceIdentifier
```

`Request` contains the method, path, query string, headers, cookies, body, content type, and other incoming information. `Response` contains the status code, headers, body, and features used to construct the outgoing message. `User` holds the authenticated principal after authentication has run. `Items` is a per-request dictionary for limited communication between middleware and later request handling. `RequestServices` exposes the dependency injection scope for this request. `RequestAborted` is cancelled when the client disconnects or the server aborts the request. `TraceIdentifier` helps correlate logs belonging to one exchange. You can inspect the context directly:

```
app.MapGet("/request-info", (HttpContext context) => Results.Ok(new
{
    method = context.Request.Method,
    path = context.Request.Path.Value,
    scheme = context.Request.Scheme,
    host = context.Request.Host.Value,
    trace = context.TraceIdentifier
}));
```

Most endpoint code should not depend on the entire `HttpContext`. Strongly typed parameters are clearer and easier to test. Still, understanding the context is useful because middleware, routing, authentication, model binding, and response generation all work around it. The context belongs to one request. It should not be stored in a singleton, kept after the request completes, or used from unrelated background work. Anything that must outlive the request should be copied into an ordinary value with a deliberate lifetime. Middleware is code arranged into an ordered pipeline. Each middleware component receives the current `HttpContext` and usually a delegate representing the next component. A simplified middleware component looks like this:

```
app.Use(async (context, next) =>
{
    Console.WriteLine($"Before: {context.Request.Method} {context.Request.Path}");
    await next(context);
    Console.WriteLine($"After: {context.Response.StatusCode}");
});
```

The first part runs while the request travels inward. Calling `next` transfers control to the remaining pipeline. When the downstream work completes, execution returns and the second part runs while the response travels outward.

Request

```
-> Middleware A: before
    -> Middleware B: before
        -> Endpoint
    <- Middleware B: after
<- Middleware A: after
```

Response

This shape makes middleware useful for cross-cutting work. Logging can record a request before and after it executes. Exception handling can wrap everything below it. Response compression can inspect the response produced downstream. Authentication can establish the user before an endpoint needs that identity. Middleware is not the right place for ordinary business operations such as calculating a product price or checking inventory rules. Those operations belong in application services. Middleware is best for concerns that apply to many endpoints or to the HTTP pipeline itself.

4.2 Pipeline Composition, Short-Circuiting, and Order

ASP.NET Core exposes several ways to add pipeline behaviour. `Use` adds middleware that can call the next component:

```
app.Use(async (context, next) =>
{
    context.Response.Headers["X-Application"] = "Product Catalog";
    await next(context);
});
```

`Run` adds a terminal delegate. It handles the request and does not call anything after it:

```
app.Run(async context =>
{
    context.Response.ContentType = "text/plain";
    await context.Response.WriteAsync("No endpoint handled this request.");
});
```

`Map` creates a branch for a matching path prefix:

```
app.Map("/diagnostics", diagnostics =>
{
    diagnostics.Run(async context =>
    {
        await context.Response.WriteAsync("Diagnostics are available.");
    });
});
```

A branch created with `Map("/diagnostics", ...)` handles paths beginning with `/diagnostics`. The matching prefix is removed from `Request.Path` while the branch executes and is available through `Request.PathBase`. `MapWhen` and `UseWhen` branch based on a predicate rather than a path. They are useful occasionally, but too many branches make the pipeline difficult to understand. Endpoint routing is usually the clearer tool for application URLs. The `MapGet`, `MapPost`, and similar methods seen earlier are different from pipeline branching. They register endpoints with routing. Their handlers do not execute during startup; they execute later when a matching request arrives. Middleware does not have to call `next`. It can create a response immediately and end processing. This is called short-circuiting.

```

app.Use(async (context, next) =>
{
    if (context.Request.Path.StartsWithSegments("/maintenance"))
    {
        context.Response.StatusCode = StatusCodes.Status503ServiceUnavailable;
        await context.Response.WriteAsJsonAsync(new
        {
            error = "The application is temporarily unavailable."
        });
        return;
    }
    await next(context);
});

```

Short-circuiting is appropriate when later work should not run. Static-file middleware can return a file without invoking an endpoint. Authentication-related middleware may reject a request. Rate limiting may stop excessive traffic. A cache may return a stored response. The behaviour can be pictured as:

```

Request
-> Middleware A
-> Middleware B decides to handle request
<- Response
Middleware C and endpoint never run.

```

After a response has started, headers and the status code can no longer be changed reliably. Middleware should therefore decide early whether it will handle the request, and code that runs after `next` should not assume it can replace an already-written response. Middleware executes in registration order on the way in and reverse order on the way out. Reordering two lines in `Program.cs` can therefore change security, error handling, performance, and routing behaviour. Consider exception handling:

```

app.UseExceptionHandler("/error");
app.UseAuthentication();
app.UseAuthorization();

```

Because the exception handler is earlier, it can catch exceptions from authentication, authorization, endpoints, and other downstream middleware. If it were placed after the endpoint execution, it could not catch exceptions that had already escaped. Authentication must run before authorization. Authentication determines who the user is. Authorization decides whether that user may perform an operation.

```

Request
-> Authentication: establish identity
-> Authorization: check permission
-> Endpoint

```

Static files are often served early because a request for a public image or stylesheet usually does not need to pass through application endpoints. HTTPS redirection must run before traffic is processed as ordinary HTTP. Response compression must wrap the components whose responses it should compress. A typical API-oriented order is conceptually similar to:

```

var app = builder.Build();
app.UseExceptionHandler();
app.UseHttpsRedirection();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
app.MapGet("/health", () => Results.Ok());
app.Run();

```

The exact order depends on the application and enabled features. The important habit is to read the pipeline from top to bottom and ask what state each component requires from the components before it.

4.3 Routing, Templates, and Endpoint Metadata

Routing matches an incoming request to an executable endpoint. An endpoint is a unit of request-handling code plus metadata describing it. When the application starts, methods such as `MapGet`, `MapPost`, `MapControllers`, and `MapRazorComponents` add endpoint definitions to a collection. When a request arrives, routing compares its method, path, host, and other relevant information against those definitions.

Registered endpoints

```
-> GET /api/products
-> GET /api/products/{id}
-> POST /api/products
-> PUT /api/products/{id}
-> DELETE /api/products/{id}
```

Incoming request

```
-> PUT /api/products/3
-> matches PUT /api/products/{id}
-> id = 3
```

Routing does not normally scan application source code at request time. Endpoint definitions are prepared during startup. The runtime uses that prepared information to select a candidate efficiently. With the minimal `WebApplication` model, routing middleware is added automatically when endpoints are mapped and explicit `UseRouting` is not called. You can still call `UseRouting` when you need precise placement between middleware that runs before routing and middleware that depends on the selected endpoint. The useful mental split is:

Routing answers: Which endpoint matches?

Endpoint execution answers: What code should run?

A route template contains literal path segments and parameters.

```
app.MapGet("/api/products/{id}", (int id) => Results.Ok(id));
```

The template `/api/products/{id}` matches paths such as `/api/products/1` and `/api/products/42`. The value from the final segment is made available as the route value named `id`. Constraints narrow the accepted values:

```
app.MapGet("/api/products/{id:int}", (int id) => Results.Ok(id));
```

Now the route matches an integer segment but not `/api/products/keyboard`. Constraints help routing distinguish URL shapes. They are not a substitute for business validation. The fact that `id` is an integer does not prove that a product with that identifier exists. Parameters can be optional:

```
app.MapGet("/api/products/category/{category?}", (string? category) =>
{
    return Results.Ok(new { category });
});
```

Default values can be included:

```
app.MapGet("/api/products/page/{page:int=1}", (int page) => Results.Ok(page));
```

A catch-all parameter captures the remaining path:

```
app.MapGet("/files/{**path}", (string path) => Results.Ok(path));
```

Routes should remain readable. A path is part of the public contract of an application, so it should describe resources and operations consistently rather than mirror internal class names. For the Product Catalog, these URLs are easy to understand:

```
GET    /api/products
GET    /api/products/42
POST   /api/products
PUT    /api/products/42
DELETE /api/products/42
```

An endpoint contains more than a handler. It can also carry metadata used by other parts of the framework. Metadata can describe:

- the endpoint name;
- authorization requirements;
- accepted and produced content types;
- OpenAPI information;
- rate-limiting policies;
- antiforgery requirements;
- filters;
- custom application information.

A Minimal API endpoint can be enriched fluently:

```
app.MapGet("/api/products/{id:int}", (int id) =>
Results.Ok(id)).WithName("GetProduct").WithTags("Products")

.Produces<ProductResponse>(StatusCodes.Status200Ok).Produces(StatusCodes.Status404NotFound);
```

Middleware that runs after routing can inspect the selected endpoint:

```
app.Use(async (context, next) =>
{
    var endpoint = context.GetEndpoint();
    if (endpoint is not null)
        app.Logger.LogInformation("Executing {Endpoint}", endpoint.DisplayName);
    await next(context);
});
```

This is one reason routing is more than a switch statement over paths. It creates a shared description of the selected operation that authorization, diagnostics, link generation, OpenAPI generation, and other systems can use.

4.4 Minimal APIs and Route Groups

Minimal APIs connect a route pattern to a delegate with little surrounding ceremony.

```
app.MapGet("/api/products", () => Results.Ok(products));
```

The route handler can be a lambda, local function, static method, or method on another type. A handler may be synchronous or asynchronous.

```
app.MapGet("/api/products/{id:int}", GetProduct);
IResult GetProduct(int id)
{
    var product = products.FirstOrDefault(product => product.Id == id);
    return product is null
        ? Results.NotFound()
        : Results.Ok(product);
}
```

Minimal does not mean simplistic or unsuitable for production. It means the endpoint model is exposed directly rather than through a controller class. A small API can remain readable with route groups, handler methods, filters, validation, and application services. The danger is not the Minimal API model itself. The danger is allowing every handler to grow into a mixture of HTTP parsing, business rules, persistence, logging, and response formatting. Endpoint code should translate between HTTP and application operations, not become the whole application. As an API grows, repeating a path prefix and metadata becomes noisy. A route group collects related endpoints.

```
var productEndpoints = app.MapGroup("/api/products").WithTags("Products");
productEndpoints.MapGet("/", GetProducts);
productEndpoints.MapGet("/{id:int}", GetProduct);
productEndpoints.MapPost("/", CreateProduct);
productEndpoints.MapPut("/{id:int}", UpdateProduct);
productEndpoints.MapDelete("/{id:int}", DeleteProduct);
```

Metadata applied to the group flows to its endpoints. Later, the whole group could require authorization:

```
productEndpoints.RequireAuthorization();
```

A common organisation is to place mapping in an extension method:

```
public static class ProductEndpoints
{
    public static IEndpointRouteBuilder MapProductEndpoints(this IEndpointRouteBuilder
        endpoints)
    {
        var products = endpoints.MapGroup("/api/products").WithTags("Products");
        products.MapGet("/", GetProducts);
        products.MapGet("/{id:int}", GetProduct);
        return endpoints;
    }
    private static IActionResult GetProducts()
    {
        // Handler implementation.
        return Results.Ok();
    }
    private static IActionResult GetProduct(int id)
    {
        // Handler implementation.
        return Results.Ok(id);
    }
}
```

Program.cs then states the composition without containing every implementation detail:

```
app.MapProductEndpoints();
```

This is organisation, not a new architectural layer. The handlers should still delegate real application work to services introduced in Chapter 5.

4.5 Binding Request Data and Services

A route handler declares the values it needs. ASP.NET Core obtains those values from the request or from registered services. This process is called parameter binding.

```
app.MapGet(
    "/api/products/{id:int}",
    (int id, string? currency, HttpContext context) =>
    {
        return Results.Ok(new
        {
            id,
            currency,
            trace = context.TraceIdentifier
        });
    }
);
```

For a request such as GET /api/products/42?currency=EUR, ASP.NET Core can bind:

```
id      <- route value "42" converted to int
currency <- query string value "EUR"
context <- current HttpContext
```

Minimal API parameters can come from route values, the query string, headers, the body, forms, special framework types, or dependency injection. Binding may be inferred or made explicit with attributes.

```
using Microsoft.AspNetCore.Mvc;
app.MapGet(
    "/api/products/{id:int}",
```

```
(
    [FromRoute] int id,
    [FromQuery] string? currency,
    [FromHeader(Name = "X-Correlation-ID")] string? correlationId
) => Results.Ok(new { id, currency, correlationId });
```

Explicit attributes are useful when inference would be unclear or when the source is part of the endpoint contract. They should not be added mechanically to every obvious parameter. Binding converts transport data into .NET values. It does not prove that the values make sense. `page=0` may bind successfully to an integer and still violate the application's paging rules. Create and update operations commonly send JSON in the request body.

```
{
  "name": "Ergonomic Keyboard",
  "price": 149.00,
  "currency": "EUR"
}
```

Define a request type that represents the expected contract:

```
public sealed record CreateProductRequest(string Name, decimal Price, string Currency);
```

A Minimal API handler can receive it directly:

```
app.MapPost(
    "/api/products",
    (CreateProductRequest request) =>
    {
        return Results.Created(
            "/api/products/4",
            new ProductResponse(4, request.Name, request.Price, request.Currency)
        );
    }
);
```

ASP.NET Core reads the body and asks the configured JSON serializer to create `CreateProductRequest`. If the JSON is malformed or cannot be converted, the framework returns a client error rather than calling the handler with an arbitrary partially parsed object. A request contract should represent data accepted from the client, not expose an internal database entity by convenience. The distinction becomes important when entities gain persistence-only properties, relationships, concurrency values, or fields that clients must not control. The body is usually a forward-only stream. Framework binding handles normal JSON bodies efficiently. Reading the same body manually in several middleware components requires buffering and careful stream positioning, so it should not be done casually. A registered dependency can appear directly in a Minimal API handler:

```
app.MapGet("/api/products", (ProductService service) => Results.Ok(service.GetProducts()));
```

ASP.NET Core distinguishes service parameters from request data using their types and binding rules. You can make the source explicit with `[FromServices]`, although it is often unnecessary:

```
app.MapGet("/api/products", ([FromServices] ProductService service) =>
Results.Ok(service.GetProducts()));
```

This is constructor injection's endpoint equivalent. The handler declares what it needs and the framework supplies it from the current request's service scope. Chapter 5 will introduce registrations, service lifetimes, scopes, options, logging, HTTP clients, and background services. For now, notice how parameter binding and dependency injection meet at the endpoint boundary: some parameters come from the request, while others come from the host.

4.6 Results, Status Codes, and Validation

An endpoint result turns application output into an HTTP response. Minimal APIs can return plain values, `ActionResult`, or typed result types. Returning a string normally creates a text response:

```
app.MapGet("/version", () => "1.0");
```

Returning an object serializes it as JSON:

```
app.MapGet("/api/products", () => products);
```

The `Results` helpers make status and response intent explicit:

```
return Results.Ok(product);
return Results.NotFound();
return Results.BadRequest(new { error = "Invalid product." });
return Results.Created($"/api/products/{product.Id}", product);
return Results.NoContent();
```

Typed results express the concrete result types to the compiler and improve endpoint metadata:

```
using Microsoft.AspNetCore.Http.HttpResults;
static Results<Ok<ProductSummary>, NotFound> GetProduct(int id, ProductSummary[] products)
{
    var product = products.FirstOrDefault(product => product.Id == id);
    return product is null
        ? TypedResults.NotFound()
        : TypedResults.Ok(product);
}
```

The important design choice is not whether every endpoint uses `Results` or `TypedResults`. It is whether the status code accurately represents the outcome and whether the response contract is stable and clear. For a typical create operation:

Valid request and product created	-> 201 Created
Malformed or invalid client input	-> 400 Bad Request
Unauthenticated request	-> 401 Unauthorized
Authenticated but not permitted	-> 403 Forbidden
Product does not exist	-> 404 Not Found
Unexpected server failure	-> 500 Internal Server Error

A common mistake is to return 200 OK for every response and place success or failure only in a JSON property.

```
{
  "success": false,
  "message": "Product was not found."
}
```

That body may be useful, but the HTTP status should also say 404 Not Found. Browsers, API clients, proxies, monitoring systems, tests, and framework helpers already understand status codes. Another mistake is to use 500 Internal Server Error for invalid client input. A server error means the server failed unexpectedly. A missing required value, unsupported format, or invalid route value belongs to the 4xx range. Status codes should communicate the category of the outcome, while the body communicates application-specific detail. A useful error response might contain:

```
{
  "type": "https://example.com/problems/product-not-found",
  "title": "Product not found",
  "status": 404,
  "detail": "No product exists with identifier 42.",
  "traceId": "00-..."
}
```

ASP.NET Core supports Problem Details, a standard JSON shape for HTTP API errors. A consistent error format makes clients simpler because they do not need to understand a different failure object for every endpoint. Binding and validation are related but different. Binding asks whether the request can be converted into the declared .NET types. Validation asks whether the resulting values satisfy the endpoint's input rules. Business rules then ask whether the operation is allowed in the current domain state. Consider a create request:

```
public sealed record CreateProductRequest(string Name, decimal Price, string Currency);
```

Possible failures belong to different layers:

Body is not valid JSON

-> binding failure

Price is -20

-> input validation failure

Currency is valid text but unsupported by the catalog

-> application rule failure

User is not allowed to create products

-> authorization failure

Keeping these distinctions clear improves status codes and error messages. It also prevents endpoint code from becoming a long sequence of unrelated checks. Chapters 8 and 12 examine data annotations, Blazor form validation, custom validation, and application-level rules. At this stage, remember that successful deserialization does not make a request valid.

4.7 Controllers and Minimal APIs

Controller-based APIs use classes and action methods rather than mapping every handler directly. First register controller services and map controller endpoints:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllers();
var app = builder.Build();
app.MapControllers();
app.Run();
```

A controller might look like this:

```
using Microsoft.AspNetCore.Mvc;
[ApiController]
[Route("api/products")]
public sealed class ProductsController : ControllerBase
{
    private static readonly ProductSummary[] Products =
    [
        new(1, "Mechanical Keyboard", 129.00m, "EUR"),
        new(2, "Wireless Mouse", 59.00m, "EUR")
    ];
    [HttpGet]
    public ActionResult<IReadOnlyList<ProductSummary>> GetProducts() => Ok(Products);
    [HttpGet("{id:int}")]
    public ActionResult<ProductSummary> GetProduct(int id)
    {
        var product = Products.FirstOrDefault(product => product.Id == id);
        return product is null ? NotFound() : Ok(product);
    }
}
```

[Route] supplies the shared route template. [HttpGet], [HttpPost], and related attributes identify methods and additional path segments. ControllerBase provides helpers such as Ok, NotFound, CreatedAtAction, access to HttpContext, and model-state information. The [ApiController] attribute enables API-oriented conventions, including binding-source inference and automatic 400 Bad Request responses for invalid model state. Controllers remain normal endpoint definitions. They use the same routing, middleware, dependency injection, authentication, authorization, and response infrastructure as Minimal APIs. Minimal APIs and controllers are two ways to express HTTP endpoints. Neither changes the underlying request pipeline. Minimal APIs are attractive when:

- the API is small or focused;
- direct route-to-handler mapping remains clear;
- little controller-specific convention is needed;

- route groups and handler methods provide enough organisation.

Controllers are attractive when:

- the application has many related actions;
- attributes and class-based grouping improve navigation;
- filters and MVC conventions are already central;
- the team prefers an established controller style;
- shared action behaviour fits naturally at controller level.

The wrong comparison is “Minimal APIs are for prototypes and controllers are for real applications.” Both can support production systems. The better question is which representation keeps the HTTP boundary readable for the size and style of the application. This tutorial will use Minimal APIs for the Product Catalog’s focused HTTP API because they expose routing and binding directly. Blazor endpoints later use their own mapping model. Controller examples remain important because many .NET applications and libraries use them. Controller actions declare parameters, and model binding obtains values from route data, query strings, forms, headers, and request bodies.

```
[HttpGet("{id:int}")]
public ActionResult<ProductSummary> GetProduct([FromRoute] int id, [FromQuery] string?
currency)
{
    // ...
}
```

A JSON body can bind to a request model:

```
[HttpPost]
public ActionResult<ProductSummary> CreateProduct([FromBody] CreateProductRequest request)
{
    // ...
}
```

With `[ApiController]`, complex types are commonly inferred from the body and route-compatible simple values from the route or query string. Explicit attributes are still useful when the source matters to readability. Controller model binding also populates model state with conversion and validation errors. The automatic API behaviour can return a structured 400 response before the action executes. The same rule applies in either endpoint model: request DTOs describe transport input. They should not silently become domain entities, persistence entities, and response models all at once.

4.8 Endpoint Filters

Endpoint filters provide behaviour around Minimal API handlers. They are narrower than middleware because they apply only to selected endpoints or groups and run after routing has selected an endpoint.

```
productEndpoints.AddEndpointFilter(async (context, next) =>
{
    app.Logger.LogInformation(
        "Calling product endpoint {Endpoint}",
        context.HttpContext.GetEndpoint()?.DisplayName
    );
    var result = await next(context);
    app.Logger.LogInformation("Product endpoint completed");
    return result;
});
```

A filter can inspect bound arguments, stop execution, or transform the result. This makes filters useful for endpoint-specific validation, logging, or conventions that should not affect unrelated routes. The scopes are different:

Middleware

- > broad HTTP pipeline concern
- > may run before an endpoint is selected

Endpoint filter

- > selected Minimal API endpoint or route group
- > sees bound handler arguments

Application service

- > business operation independent of HTTP mechanism

Choosing the narrowest appropriate mechanism keeps responsibilities clearer.

4.9 Errors and Request Cancellation

An unexpected exception should not expose a stack trace or implementation details to a production client. It should be logged on the server and translated into a controlled error response. During development, detailed exception pages are useful because the developer needs the stack trace. In production, the response should remain safe and stable. A basic configuration is:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddProblemDetails();
var app = builder.Build();
if (app.Environment.IsDevelopment())
    app.UseDeveloperExceptionPage();
else
    app.UseExceptionHandler();
app.MapProductEndpoints();
app.Run();
```

Exception handling belongs early in the pipeline so that it wraps later middleware and endpoints. Expected application outcomes should not all be represented by exceptions. “Product 42 does not exist” is a normal result for a lookup and can become 404 **Not Found**. An unavailable database or violated invariant may be exceptional. The application layer can express structured outcomes, while the HTTP boundary translates them into status codes and response bodies. The client should receive enough information to react, but not internal SQL, file paths, secrets, stack traces, or implementation type names.

A client can close the connection before the server completes. The user may navigate away, cancel a request, lose connectivity, or exceed a timeout. Continuing expensive work after the result can no longer be delivered wastes resources. ASP.NET Core exposes request cancellation through `HttpContext.RequestAborted`. Minimal API handlers can receive it directly as a `CancellationToken`:

```
app.MapGet(
    "/api/products",
    async (CancellationToken token) =>
    {
        await Task.Delay(TimeSpan.FromMilliseconds(100), token);
        return Results.Ok(Array.Empty<ProductSummary>());
    }
);
```

Later, the token should flow into database queries, outgoing HTTP calls, and other cancellable I/O:

```
var products = await database.Products.AsNoTracking().ToListAsync(token);
```

Cancellation is not a failure that should always be logged as an application error. It often means the requester no longer needs the work. Code should allow the cancellation to propagate unless it has a specific reason to translate or clean up. Not every operation should be cancelled halfway through. Once a critical external side effect has begun, the application may need transactional or idempotent behaviour rather than simply abandoning it. The token is a signal, not a substitute for operation design.

4.10 The Product Catalog API and Diagnosis

The following example expands the in-memory Product Catalog. It is intentionally contained in one file so that the request boundary remains visible. Chapter 5 will separate state and operations into services.

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddProblemDetails();
var app = builder.Build();
if (app.Environment.IsDevelopment())
```

```

    app.UseDeveloperExceptionPage();
else
    app.UseExceptionHandler();
app.UseHttpsRedirection();
List<ProductSummary> products =
[
    new(1, "Mechanical Keyboard", 129.00m, "EUR"),
    new(2, "Wireless Mouse", 59.00m, "EUR"),
    new(3, "USB-C Dock", 189.00m, "EUR")
];
var productEndpoints = app.MapGroup("/api/products").WithTags("Products");
productEndpoints.MapGet("/", () => TypedResults.Ok(products));
productEndpoints.MapGet(
   ("/{id:int}",
    Results<Ok<ProductSummary>, NotFound> (int id) =>
    {
        var product = products.FirstOrDefault(product => product.Id == id);
        return product is null
            ? TypedResults.NotFound()
            : TypedResults.Ok(product);
    }
);
productEndpoints.MapPost(
    "/",
    Results<Created<ProductSummary>, BadRequest<ProblemDetails>> (CreateProductRequest
request) =>
    {
        if (string.IsNullOrEmpty(request.Name))
        {
            return TypedResults.BadRequest(new ProblemDetails
            {
                Title = "Invalid product",
                Detail = "A product name is required.",
                Status = StatusCodes.Status400BadRequest
            });
        }
        if (request.Price < 0)
        {
            return TypedResults.BadRequest(new ProblemDetails
            {
                Title = "Invalid product",
                Detail = "The price cannot be negative.",
                Status = StatusCodes.Status400BadRequest
            });
        }
        var nextId = products.Count == 0
            ? 1
            : products.Max(product => product.Id) + 1;
        var product = new ProductSummary(
            nextId,
            request.Name.Trim(),
            request.Price,
            request.Currency.ToUpperInvariant()
        );
        products.Add(product);
        return TypedResults.Created($"{api/products/{product.Id}", product);
    }
);
productEndpoints.MapPut(
   ("/{id:int}",

```

```

    Results<Ok<ProductSummary>, NotFound, BadRequest<ProblemDetails>> (int id,
UpdateProductRequest request) =>
{
    var index = products.FindIndex(product => product.Id == id);
    if (index < 0)
        return TypedResults.NotFound();
    if (string.IsNullOrEmpty(request.Name) || request.Price < 0)
    {
        return TypedResults.BadRequest(new ProblemDetails
        {
            Title = "Invalid product",
            Detail = "Name is required and price cannot be negative.",
            Status = StatusCodes.Status400BadRequest
        });
    }
    var product = new ProductSummary(id, request.Name.Trim(), request.Price,
request.Currency.ToUpperInvariant());
    products[index] = product;
    return TypedResults.Ok(product);
}
);
productEndpoints.MapDelete(
   ("/{id:int}",
    Results<NoContent, NotFound> (int id) =>
    {
        var removed = products.RemoveAll(product => product.Id == id) > 0;
        return removed
            ? TypedResults.NoContent()
            : TypedResults.NotFound();
    }
);
app.Run();
public sealed record ProductSummary(int Id, string Name, decimal Price, string Currency);
public sealed record CreateProductRequest(string Name, decimal Price, string Currency);
public sealed record UpdateProductRequest(string Name, decimal Price, string Currency);

```

The sample uses namespaces that may need explicit using directives depending on project settings:

```

using Microsoft.AspNetCore.Http.HttpResults;
using Microsoft.AspNetCore.Mvc;

```

The HTTP path for a create request is now visible:

```

POST /api/products
-> Kestrel creates HttpContext
-> Exception middleware begins
-> HTTPS policy is applied
-> Routing selects POST /api/products
-> JSON body binds to CreateProductRequest
-> Handler validates request
-> Handler creates in-memory product
-> Created<ProductSummary> becomes 201 response
-> Exception middleware completes
-> Kestrel sends response

```

The example is useful for learning, but its design limitations are intentional. The list is shared mutable state and is not protected for concurrent writes. Validation is duplicated between handlers. Identifier generation is unsafe under concurrent requests. Business operations live inside the HTTP boundary. Restarting the process loses all changes. Those problems lead directly to Chapter 5. The next step is not to add more endpoint code. It is to move responsibility into services with deliberate lifetimes and dependencies.

When an endpoint does not behave as expected, identify the stage that failed. A **404 Not Found** may mean that no route matched, or it may be an intentional result returned by a matched handler. A **405 Method Not Allowed** usually means the path exists but not for the supplied HTTP method. A **400 Bad Request** can come from malformed JSON, failed type conversion, model validation, or an explicit handler result. A **401 Unauthorized** means the request did not establish an acceptable identity, while **403 Forbidden** means an identity exists but lacks permission. A **500 Internal Server Error** means an unexpected failure escaped application handling; the server logs and trace identifier are normally more useful than the generic client response. A practical diagnostic sequence is:

Check request method and URL

- > Check returned status and body
- > Check whether routing selected an endpoint
- > Check parameter binding and validation
- > Check endpoint logs
- > Check application service and infrastructure logs

Add logging where it reveals a boundary, not at every line. ASP.NET Core already logs request and routing information at useful categories. Increasing log verbosity temporarily is often better than adding permanent ad hoc output.

4.11 What to Keep in Mind

Each request receives an **HttpContext** and moves through ordered middleware before routing selects an endpoint. Middleware can wrap later work or short-circuit it, so registration order affects security, error handling, caching, and response behaviour. Routes describe URL shapes and carry metadata used by authorization, diagnostics, and OpenAPI. Minimal APIs and controllers are two forms of the same HTTP boundary. Binding converts route, query, header, form, body, and service inputs into typed parameters; validation and business rules remain separate concerns. Results translate application outcomes into status codes and bodies, while exception handling protects production details and request cancellation prevents abandoned I/O from consuming resources. Endpoints should adapt HTTP to application operations rather than own the application.

5. Application Services

Chapter 4 left the Product Catalog in an intentionally awkward state. The endpoints owned a shared list, repeated validation, generated identifiers, and decided how products should change. That kept the request path visible, but it also mixed HTTP concerns with application behaviour. A request handler should usually coordinate work rather than own it. It reads HTTP input, calls an application service, and turns the result into an HTTP response. The service owns the operation itself.

Once that boundary exists, the same operation can be called from a Minimal API, controller, Blazor component, background worker, test, or another host without copying its rules. ASP.NET Core's built-in dependency injection container connects those pieces. It knows which services exist, how to create them, how long their instances should live, and when disposable instances should be released. The same container also provides configuration, logging, HTTP clients, framework services, and hosted background work.

This chapter moves the Product Catalog's behaviour out of its endpoints and into services. It then uses that example to explain service registration, request scopes, lifetimes, options, structured logging, outgoing HTTP calls, and background services.

5.1 Endpoints Coordinate; Services Perform the Operation

Consider an endpoint that creates a product. At the HTTP boundary it must read a request body, choose a status code, and return a response. Those are web concerns. Deciding whether the name is acceptable, normalising the currency, assigning an identifier, storing the product, and reporting the outcome are application concerns. When all of that code remains in the endpoint, the endpoint grows for reasons unrelated to HTTP. A second endpoint or background process must either call through the HTTP layer or copy the same rules. Tests also become unnecessarily tied to routing and serialization. A cleaner request path is:

HTTP request

- > Endpoint reads web input
- > Product service performs operation
- > Endpoint converts result to HTTP response

The distinction does not require a large architecture. One service with a clear responsibility is enough. The aim is not to create interfaces and layers mechanically, but to keep the web boundary from becoming the owner of the application. For the Product Catalog, an endpoint can become this small:

```
productEndpoints.MapPost(
    "/",
    async Task<Results<Created<ProductSummary>, BadRequest<ProblemDetails>>> (
        CreateProductRequest request,
        IProductCatalog products,
        CancellationToken token
    ) =>
    {
        var result = await products.CreateAsync(request, token);
        return result.IsSuccess
            ? TypedResults.Created($" /api/products/{result.Value!.Id}", result.Value)
            : TypedResults.BadRequest(new ProblemDetails
                {
                    Title = "Invalid product",
                    Detail = result.Error,
                    Status = StatusCodes.Status400BadRequest
                });
    }
);
```

The handler still understands HTTP, but it no longer owns product storage or validation. Its work is translation: JSON becomes a typed request, the request goes to the service, and the service result becomes an HTTP response. This pattern will appear repeatedly. Blazor components should not become repositories, controllers should not calculate business rules, and background workers should not duplicate endpoint logic. Each outer adapter should call the same application operation.

5.2 The Service Container Connects Objects

A normal C# object receives its dependencies through its constructor:

```
public sealed class ProductCatalog(IProductStore store, ILogger<ProductCatalog> logger) :
    IProductCatalog
{
    // Operations use store and logger.
}
```

`ProductCatalog` states what it needs without deciding how those objects are created. It does not call `new SqlProductStore(...)`, open configuration files, or construct a logger. Object creation belongs to the application's composition root, which in a small ASP.NET Core application is usually `Program.cs` and its registration extension methods. The required types are registered before `builder.Build()`:

```
builder.Services.AddSingleton<IProductStore, MemoryProductStore>();
builder.Services.AddScoped<IProductCatalog, ProductCatalog>();
```

The registration means:

```
When something requests IProductStore
    -> provide MemoryProductStore
When something requests IProductCatalog
    -> create ProductCatalog
    -> supply its IProductStore
    -> supply its ILogger<ProductCatalog>
```

The container already knows how to create `ILogger<T>` because logging services are registered by the host. It recursively resolves constructor dependencies until the complete object graph can be created. Minimal API handlers can request services as parameters:

```
app.MapGet(
    "/api/products",
    async (IProductCatalog products, CancellationToken token) =>
        TypedResults.Ok(await products.ListAsync(token))
);
```

Controllers usually receive them through constructor injection:

```
[ApiController]
[Route("api/products")]
public sealed class ProductsController(IProductCatalog products) : ControllerBase
{
    [HttpGet]
    public async Task<ActionResult<IReadOnlyList<ProductSummary>>> GetAll(
        CancellationToken token
    ) => Ok(await products.ListAsync(token));
}
```

Later, Razor components will use `@inject` or `[Inject]`. These are different entry points into the same service container. The container is not a global dictionary that application code should query whenever it needs something. Calling `IServiceProvider.GetRequiredService<T>()` throughout the application hides dependencies and turns resolution into runtime behaviour. Constructor or parameter injection keeps dependencies visible in the type's public shape. Direct resolution is still useful at infrastructure boundaries where a scope must be created explicitly, such as application startup or a background service. It should be the exception, not the normal way a service finds its collaborators.

5.3 Lifetimes, Request Scopes, and Captive Dependencies

A registration also defines how long an instance lives. The built-in container supports three main lifetimes: transient, scoped, and singleton. A **transient** service is created each time it is requested. It suits lightweight, stateless objects whose identity does not need to be shared. A **scoped** service is created once within a dependency injection scope. In an ordinary ASP.NET Core HTTP request, the framework creates one scope for the request, so all resolutions of a scoped service within that request receive the same instance. A later request gets

another instance. A **singleton** service is shared for the lifetime of the application process. Every request and component that resolves it receives the same instance.

Application process

-> Singleton instance

Request A scope

-> Scoped instance A

-> Transient instance 1

-> Transient instance 2

Request B scope

-> Scoped instance B

-> Transient instance 3

Registration uses corresponding methods:

```
builder.Services.AddTransient<IPriceFormatter, PriceFormatter>();
builder.Services.AddScoped<IProductCatalog, ProductCatalog>();
builder.Services.AddSingleton<IClock, SystemClock>();
```

The lifetime is not a statement about how important a service is. It describes instance ownership and sharing. A singleton is not automatically faster or better, and a transient is not automatically safer. Choose the lifetime from the state and dependencies the service owns. A transient service should normally be cheap to create and should not expect its fields to survive between resolutions. A scoped service can coordinate work within one request and can depend on other scoped or singleton services. A singleton must be safe for concurrent use because many requests can call it at the same time. The following table is a useful starting point:

Lifetime	Instance sharing	Typical use
Transient	New instance for each resolution	Small stateless operation or formatter
Scoped	One instance per request scope	Request-oriented application service, EF Core <code>DbContext</code>
Singleton	One instance for the process	Thread-safe shared cache, clock, immutable provider

These are defaults, not laws. The correct lifetime depends on what the service owns. A service that wraps per-request transaction state should not be singleton. A large thread-safe cache should not be recreated every time it is requested.

The request scope

The request scope is one reason scoped services are common in web applications. Kestrel receives a request, ASP.NET Core creates a scope, the endpoint and its dependencies are resolved from that scope, and the scope is disposed after the request finishes.

Request begins

-> Create request scope

-> Resolve endpoint dependencies

-> Execute request

-> Dispose scoped services

Request ends

Entity Framework Core registers `DbContext` as scoped by default because a unit of database work commonly belongs to one request. Chapter 10 will explain the important Blazor exceptions and when `IDbContextFactory<TContext>` is a better fit.

Captive dependencies

A longer-lived service must not hold a shorter-lived service. The clearest invalid combination is a singleton depending on a scoped service:

Singleton ProductCache

-> Scoped ProductDbContext X

The singleton would capture one scoped instance and keep it beyond its intended scope. In development, scope validation commonly detects this and fails early. Even when an invalid graph is not detected, it can produce

stale state, concurrency errors, and use-after-disposal failures. The safe direction is: `Transient` -> `Scoped` -> `Singleton`. A shorter-lived object can depend on a longer-lived one because the dependency remains valid for the shorter object's entire life. The reverse direction requires a deliberate scope created only for the duration of one operation.

5.4 Moving the Product Catalog into Services

The in-memory list from Chapter 4 must survive between requests, so its owner needs a lifetime longer than one request. A temporary in-memory store can be a singleton, but it must also be thread-safe because requests execute concurrently. First define the product model and operation contracts:

```
public sealed record ProductSummary(int Id, string Name, decimal Price, string Currency);
public sealed record CreateProductRequest(string Name, decimal Price, string Currency);
public sealed record UpdateProductRequest(string Name, decimal Price, string Currency);
public sealed record ProductResult(ProductSummary? Value, string? Error)
{
    public bool IsSuccess => Value is not null;
    public static ProductResult Success(ProductSummary value) => new(value, null);
    public static ProductResult Failure(string error) => new(null, error);
}
```

The application service exposes operations without mentioning HTTP status codes or `HttpContext`:

```
public interface IProductCatalog
{
    Task<IReadOnlyList<ProductSummary>> ListAsync(CancellationToken token = default);
    Task<ProductSummary?> FindAsync(int id, CancellationToken token = default);
    Task<ProductResult> CreateAsync(CreateProductRequest request, CancellationToken token = default);
    Task<ProductResult?> UpdateAsync(int id, UpdateProductRequest request, CancellationToken token = default);
    Task<bool> DeleteAsync(int id, CancellationToken token = default);
}
```

The interface is useful because the service has more than one plausible implementation and because endpoints should depend on the operation contract rather than storage details. An interface is not required for every class. A small stateless helper with one obvious implementation can be registered by its concrete type. The store owns temporary process memory:

```
public interface IProductStore
{
    IReadOnlyList<ProductSummary> List();
    ProductSummary? Find(int id);
    ProductSummary Add(string name, decimal price, string currency);
    ProductSummary? Update(int id, string name, decimal price, string currency);
    bool Delete(int id);
}
```

A thread-safe implementation can use a lock around its mutable collection:

```
public sealed class MemoryProductStore : IProductStore
{
    private readonly object _lock = new();
    private readonly List<ProductSummary> _products =
    [
        new(1, "Mechanical Keyboard", 129.00m, "EUR"),
        new(2, "Wireless Mouse", 59.00m, "EUR"),
        new(3, "USB-C Dock", 189.00m, "EUR")
    ];
    private int _nextId = 4;
    public IReadOnlyList<ProductSummary> List()
    {
        lock (_lock) return _products.ToArray();
    }
}
```

```

    }
    public ProductSummary? Find(int id)
    {
        lock (_lock) return _products.FirstOrDefault(product => product.Id == id);
    }
    public ProductSummary Add(string name, decimal price, string currency)
    {
        lock (_lock)
        {
            var product = new ProductSummary(_nextId++, name, price, currency);
            _products.Add(product);
            return product;
        }
    }
    public ProductSummary? Update(int id, string name, decimal price, string currency)
    {
        lock (_lock)
        {
            var index = _products.FindIndex(product => product.Id == id);
            if (index < 0) return null;
            var product = new ProductSummary(id, name, price, currency);
            _products[index] = product;
            return product;
        }
    }
    public bool Delete(int id)
    {
        lock (_lock) return _products.RemoveAll(product => product.Id == id) > 0;
    }
}

```

Returning an array snapshot from List prevents callers from modifying or enumerating the internal list while another request changes it. This store is suitable only for learning. It loses data on restart and cannot coordinate across multiple server processes. Entity Framework Core will later replace it without forcing the HTTP endpoints to own database code. The application service validates and normalises input before calling the store:

```

public sealed class ProductCatalog(IProductStore store, ILogger<ProductCatalog> logger) :
    IProductCatalog
{
    public Task<IReadOnlyList<ProductSummary>> ListAsync(
        CancellationToken token = default
    ) => Task.FromResult(store.List());
    public Task<ProductSummary?> FindAsync(
        int id,
        CancellationToken token = default
    ) => Task.FromResult(store.Find(id));
    public Task<ProductResult> CreateAsync(CreateProductRequest request, CancellationToken
        token = default)
    {
        var error = Validate(request.Name, request.Price, request.Currency);
        if (error is not null)
            return Task.FromResult(ProductResult.Failure(error));
        var product = store.Add(request.Name.Trim(), request.Price,
            request.Currency.Trim().ToUpperInvariant());
        logger.LogInformation(
            "Created product {ProductId} named {ProductName}",
            product.Id,
            product.Name
        );
        return Task.FromResult(ProductResult.Success(product));
    }
}

```

```

}
public Task<ProductResult?> UpdateAsync(int id, UpdateProductRequest request,
Cancellation token = default)
{
    var error = Validate(request.Name, request.Price, request.Currency);
    if (error is not null)
        return Task.FromResult<ProductResult?>(ProductResult.Failure(error));
    var product = store.Update(id, request.Name.Trim(), request.Price,
request.Currency.Trim().ToUpperInvariant());
    if (product is null) return Task.FromResult<ProductResult?>(null);
    logger.LogInformation("Updated product {ProductId}", id);
    return Task.FromResult<ProductResult?>(ProductResult.Success(product));
}
public Task<bool> DeleteAsync(int id, Cancellation token = default)
{
    var deleted = store.Delete(id);
    if (deleted) logger.LogInformation("Deleted product {ProductId}", id);
    return Task.FromResult(deleted);
}
private static string? Validate(string name, decimal price, string currency)
{
    if (string.IsNullOrEmpty(name)) return "A product name is required.";
    if (price < 0) return "The price cannot be negative.";
    if (string.IsNullOrEmpty(currency)) return "A currency is required.";
    if (currency.Trim().Length != 3) return "Currency must contain three letters.";
    return null;
}
}

```

The methods are asynchronous because the eventual store will perform database I/O. The current memory implementation completes synchronously, so `Task.FromResult` keeps the contract ready for later replacement without pretending that in-memory work itself needs await. Register the singleton store and scoped application service:

```

builder.Services.AddSingleton<IProductStore, MemoryProductStore>();
builder.Services.AddScoped<IProductCatalog, ProductCatalog>();

```

The singleton store preserves data across requests. The scoped service can later use a scoped `DbContext`, so its lifetime already matches the likely production design.

5.5 Ownership, Disposal, and Multiple Implementations

The container owns the instances it creates. If a registered service implements `IDisposable` or `IAsyncDisposable`, the container disposes it when the owning scope ends. A scoped disposable is released with the scope; a singleton is released when the application shuts down. Application code should not dispose an injected service. It did not create the instance and does not know whether another consumer shares it. The same ownership rule applies outside dependency injection: the code that creates a resource should normally own its disposal. Avoid registering a pre-created disposable unless the external owner is intentional:

```

var connection = new ExternalConnection();
builder.Services.AddSingleton(connection);

```

Because the container did not construct that instance through a type or factory registration, ownership can be unclear. Prefer a registration that lets the container create and dispose the object:

```

builder.Services.AddSingleton<ExternalConnection>();

```

Services should also be designed for constructor injection. They should not depend directly on static global state, read environment variables in the middle of operations, or construct infrastructure clients internally. Visible dependencies make the object easier to understand and test. Do not inject a service merely because it exists in the container. A class with ten unrelated dependencies is usually carrying too many responsibilities. Dependency injection makes an object graph possible; it does not make every object graph well designed.

Multiple implementations

Sometimes one contract has several implementations. Registering the same service type more than once allows `IEnumerable<T>` resolution, while keyed services allow selecting an implementation by key. Both are useful, but neither should replace a clear application concept. For example, several export formats can be represented as a collection:

```
builder.Services.AddSingleton<IProductExporter, CsvProductExporter>();
builder.Services.AddSingleton<IProductExporter, JsonProductExporter>();
```

A service can receive them together:

```
public sealed class ProductExportService(IEnumerable<IProductExporter> exporters)
{
    private readonly IReadOnlyList<IProductExporter> _exporters = exporters.ToArray();
}
```

Keyed services are appropriate when a known key selects one implementation, but frequent runtime service lookup can make dependencies harder to follow. Prefer an explicit strategy service when selection contains real business logic.

5.6 Typed Options and Structured Logging

Chapter 3 introduced configuration as a combined set of key-value sources. Reading raw strings directly is useful at startup, but application services are easier to use when related settings are represented by a typed object. Suppose the Product Catalog has a default currency and a maximum page size:

```
{
    "ProductCatalog": {
        "DefaultCurrency": "EUR",
        "MaximumPageSize": 100
    }
}
```

Define an options type:

```
public sealed class ProductCatalogOptions
{
    public const string SectionName = "ProductCatalog";
    public string DefaultCurrency { get; init; } = "EUR";
    public int MaximumPageSize { get; init; } = 100;
}
```

Register binding and validation:

```
builder.Services.AddOptions<ProductCatalogOptions>().BindConfiguration(
    ProductCatalogOptions.SectionName)
    .Validate(options => options.MaximumPageSize is > 0 and <= 500, "MaximumPageSize must be
between 1 and 500.")
    .Validate(options => options.DefaultCurrency.Length == 3, "DefaultCurrency must contain
three letters.")
    .ValidateOnStart();
```

`ValidateOnStart` turns invalid configuration into a startup failure instead of allowing the application to start and fail only when a particular request reaches the bad setting. This is especially useful for required addresses, limits, integration settings, and other values that the application cannot operate without. A service can receive `IOptions<ProductCatalogOptions>`:

```
public sealed class ProductQueryService(IOptions<ProductCatalogOptions> options)
{
    private readonly ProductCatalogOptions _options = options.Value;
}
```

The main options interfaces serve different refresh needs:

Interface	Behaviour
<code>IOptions<T></code>	Reads one options instance and is suitable for stable settings
<code>IOptionsSnapshot<T></code>	Re-evaluates options once per scope and is scoped
<code>IOptionsMonitor<T></code>	Supports current values and change notifications, including in singletons

Most application configuration should be treated as stable during one process run unless live reload is a real requirement. Automatically changing critical behaviour halfway through a request can be more confusing than restarting with a known configuration. Options are configuration, not application state. A setting such as `MaximumPageSize` belongs in options. The user's selected page, the current product count, and an order's status do not. Secrets should enter through secure configuration providers or deployment facilities rather than source-controlled JSON. Binding a secret into an options type does not make storage secure; it only gives the application typed access after the secret has been supplied.

ASP.NET Core uses `ILogger<T>` throughout the framework and makes it available through dependency injection. The generic type becomes the logging category, which lets configuration control verbosity by subsystem. A log entry should describe an event with named values:

```
logger.LogInformation(
    "Updated product {ProductId} from {OldPrice} to {NewPrice}",
    product.Id,
    oldPrice,
    product.Price
);
```

The placeholders are not only string-formatting positions. Logging providers can preserve `ProductId`, `OldPrice`, and `NewPrice` as structured fields that can later be searched, grouped, or analysed. Prefer this over interpolation:

```
logger.LogInformation($"Updated product {product.Id}");
```

Interpolation creates the final string before the logger decides whether the message is enabled and discards the field names as structure. For low-volume development logs the difference may be small, but structured templates are the better default. Log levels express severity and expected audience:

Level	Typical meaning
Trace	Very detailed internal flow, normally disabled
Debug	Diagnostic detail useful during development
Information	Normal significant application event
Warning	Unexpected condition that the application handled
Error	Operation failed because of an exception or serious condition
Critical	Application or subsystem cannot continue safely

Do not log every method entry and exit. Logs should help reconstruct meaningful behaviour: a product changed, an external provider failed, a background job completed, or a security-relevant action was rejected. Framework request logs already describe much of the HTTP pipeline. Avoid logging secrets, access tokens, passwords, complete payment details, or unnecessary personal data. Logging a request body by default is dangerous because bodies often contain exactly the data that should not be copied into a long-lived log store. Exceptions should be passed to the logger as exceptions:

```
try
{
    await provider.RefreshAsync(token);
}
catch (HttpRequestException exception)
{
    logger.LogError(exception, "Failed to refresh exchange rates");
    throw;
}
```

This preserves the exception type, message, and stack trace. Logging only `exception.Message` throws away most diagnostic context. The application writes through `ILogger`; providers decide where entries go. The default host includes console and debug-oriented providers. Production systems often add a central telemetry or structured logging provider so entries from multiple processes can be searched together. Application services should not know which provider is installed.

5.7 Outgoing HTTP and Background Services

A server frequently becomes an HTTP client itself. The Product Catalog might call a currency service, image processor, payment provider, or another internal API. These calls should not be scattered across endpoints or constructed with `new HttpClient()` for every request. `HttpClientFactory` centralises logical client configuration and manages the lifetime of underlying handlers. A typed client keeps the external contract in one place:

```
public sealed class CurrencyClient(HttpClient httpClient, ILogger<CurrencyClient> logger)
{
    public async Task<decimal?> GetRateAsync(string from, string to, CancellationToken token
    = default)
    {
        var path = $"rates/{from.ToUpperInvariant()}/{to.ToUpperInvariant()}";
        using var response = await httpClient.GetAsync(path, token);
        if (response.StatusCode == HttpStatusCode.NotFound) return null;
        response.EnsureSuccessStatusCode();
        var result = await response.Content.ReadFromJsonAsync<RateResponse>(token);
        logger.LogDebug(
            "Received exchange rate from {SourceCurrency} to {TargetCurrency}",
            from,
            to
        );
        return result?.Rate;
    }
    private sealed record RateResponse(decimal Rate);
}
```

Register and configure it once:

```
builder.Services.AddHttpClient<CurrencyClient>(client =>
{
    client.BaseAddress = new Uri("https://rates.example.com/api/");
    client.Timeout = TimeSpan.FromSeconds(10);
});
```

The client can now be injected into a service. The external base address should normally come from validated options rather than a literal, but the important pattern is already visible: the endpoint calls an application service, the application service uses a dedicated integration client, and the integration client owns the remote HTTP contract.

Endpoint

```
-> ProductPricingService
    -> CurrencyClient
        -> External HTTP API
```

Timeouts, retries, and circuit breaking need deliberate policies. Retrying a safe rate lookup may be reasonable. Retrying a non-idempotent payment request without an idempotency design can charge a customer twice. Resilience is not achieved by adding retries everywhere. A cancellation token from the incoming request should flow into the outgoing call when the remote work is useful only for that request. This allows the server to stop waiting when the client disconnects or the request times out.

Some work should not keep an HTTP request open. Examples include periodically refreshing exchange rates, processing queued emails, cleaning expired data, generating reports, and consuming messages. ASP.NET Core can run this work as a hosted service. `IHostedService` defines startup and shutdown methods, while `BackgroundService` provides an `ExecuteAsync` method for a long-running loop. A simple rate refresh worker can look like this:

```

public sealed class ExchangeRateWorker(
    IServiceScopeFactory scopeFactory,
    ILogger<ExchangeRateWorker> logger
) : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        using var timer = new PeriodicTimer(TimeSpan.FromMinutes(30));

        try
        {
            await RefreshAsync(stoppingToken);

            while (await timer.WaitForNextTickAsync(stoppingToken))
                await RefreshAsync(stoppingToken);
        }
        catch (OperationCanceledException) when (stoppingToken.IsCancellationRequested)
        {
            // Normal application shutdown.
        }
    }

    private async Task RefreshAsync(CancellationToken token)
    {
        try
        {
            await using var scope = scopeFactory.CreateAsyncScope();
            var refresher =
                scope.ServiceProvider.GetRequiredService<IExchangeRateRefresher>();
            await refresher.RefreshAsync(token);
        }
        catch (Exception exception) when (exception is not OperationCanceledException ||
            !token.IsCancellationRequested)
        {
            logger.LogError(exception, "Exchange-rate refresh failed");
        }
    }
}

```

Register it with:

```
builder.Services.AddHostedService<ExchangeRateWorker>();
```

Hosted services are registered as singletons. The worker therefore must not constructor-inject a scoped service such as a `DbContext`. Instead, it injects `IServiceScopeFactory`, creates a scope for one iteration, resolves the scoped operation, completes the operation, and disposes the scope.

Singleton background worker

- > Create scope for one iteration
- > Resolve scoped refresher
- > Perform work
- > Dispose scope

A background service shares the web application's process. If the process stops, the worker stops. This is sufficient for periodic maintenance and modest in-process queues, but it does not guarantee that important work survives a crash or deployment. Durable jobs need persistent state or an external queue so another process can resume them. Do not start unobserved work from an endpoint with `Task.Run` and immediately return.

That task has no durable ownership, may outlive request-scoped dependencies, and can disappear when the process stops. A request should either await the operation or place a durable work item into a queue that a controlled background worker consumes. Shutdown also matters. `stoppingToken` tells the worker that the host

is stopping. Loops and I/O should observe it and finish promptly. A deployment should not have to kill the process because a worker ignored cancellation indefinitely.

5.8 Registration and the Final Endpoints

As the application grows, `Program.cs` should remain a readable composition root rather than a long list of unrelated registrations. ASP.NET Core libraries commonly expose `Add...` extension methods that register one coherent feature. The Product Catalog can follow the same convention:

```
public static class ProductCatalogServiceCollectionExtensions
{
    public static IServiceCollection AddProductCatalog(this IServiceCollection services,
        IConfiguration configuration)
    {
        services.AddOptions<ProductCatalogOptions>().Bind(configuration.GetSection(
            ProductCatalogOptions.SectionName))
            .Validate(
                options => options.MaximumPageSize is > 0 and <= 500,
                "MaximumPageSize must be between 1 and 500."
            ).ValidateOnStart();
        services.AddSingleton<IProductStore, MemoryProductStore>();
        services.AddScoped<IProductCatalog, ProductCatalog>();
        return services;
    }
}
```

Then startup stays compact:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddProblemDetails();
builder.Services.AddProductCatalog(builder.Configuration);
var app = builder.Build();
app.UseExceptionHandler();
app.UseHttpsRedirection();
app.MapProductEndpoints();
app.Run();
```

The extension method does not hide application behaviour. It groups object-construction details that belong together. A reader can see that the application uses Product Catalog services and open the extension when the precise registrations matter. Avoid giant registration methods named `AddEverything`. Group by capability: `AddProductCatalog`, `AddIdentity`, `AddNotifications`, or `AddPersistence`. The composition root should reveal the application's major pieces. The endpoint mapping can now focus on HTTP translation. It no longer owns a mutable list or product rules:

```
public static class ProductEndpointRouteBuilderExtensions
{
    public static IEndpointRouteBuilder MapProductEndpoints(this IEndpointRouteBuilder
        endpoints)
    {
        var products = endpoints.MapGroup("/api/products").WithTags("Products");
        products.MapGet(
            "/",
            async (IProductCatalog catalog, CancellationToken token) =>
                TypedResults.Ok(await catalog.ListAsync(token))
        );
        products.MapGet(
           ("/{id:int}",
            async Task<Results<Ok<ProductSummary>, NotFound>> (
                int id,
                IProductCatalog catalog,
                CancellationToken token
            ) =>
```

```

        {
            var product = await catalog.FindAsync(id, token);
            return product is null
                ? TypedResults.NotFound()
                : TypedResults.Ok(product);
        }
    );
    products.MapPost(
        "/",
        async Task<Results<Created<ProductSummary>, BadRequest<ProblemDetails>>>> (
            CreateProductRequest request,
            IProductCatalog catalog,
            CancellationToken token
        ) =>
        {
            var result = await catalog.CreateAsync(request, token);
            return result.IsSuccess
                ? TypedResults.Created(
                    $"/api/products/{result.Value!.Id}",
                    result.Value
                )
                : InvalidProduct(result.Error!);
        }
    );
    products.MapPut(
        $"/{id:int}",
        async Task<Results<Ok<ProductSummary>, NotFound, BadRequest<ProblemDetails>>>> (
            int id,
            UpdateProductRequest request,
            IProductCatalog catalog,
            CancellationToken token
        ) =>
        {
            var result = await catalog.UpdateAsync(id, request, token);
            if (result is null) return TypedResults.NotFound();
            return result.IsSuccess
                ? TypedResults.Ok(result.Value!)
                : InvalidProduct(result.Error!);
        }
    );
    products.MapDelete(
        $"/{id:int}",
        async Task<Results<NoContent, NotFound>>> (
            int id,
            IProductCatalog catalog,
            CancellationToken token
        ) => await catalog.DeleteAsync(id, token)
            ? TypedResults.NoContent()
            : TypedResults.NotFound()
    );
    return endpoints;
}

private static BadRequest<ProblemDetails> InvalidProduct(string detail) =>
    TypedResults.BadRequest(new ProblemDetails
    {
        Title = "Invalid product",
        Detail = detail,
        Status = StatusCodes.Status400BadRequest
    });
}

```

The endpoint code is still not tiny, and it does not need to be. HTTP has real concerns: route values, request models, result unions, status codes, resource locations, and problem details. The improvement is that each concern now belongs to the correct side of the boundary. The create request follows this path:

```
POST /api/products
-> Routing selects endpoint
-> JSON binds to CreateProductRequest
-> Container provides scoped IProductCatalog
-> Container builds ProductCatalog
    -> singleton IProductStore
    -> ILogger<ProductCatalog>
-> ProductCatalog validates and creates product
-> Endpoint converts ProductResult to 201 or 400
-> Request scope is disposed
```

When Chapter 10 replaces `MemoryProductStore` with EF Core, the endpoint contract can remain almost unchanged. The new store will be scoped rather than singleton, and its methods will perform real asynchronous I/O, but the web boundary still calls `IProductCatalog`.

5.9 What to Keep in Mind

Dependency injection separates object construction from application behaviour. Registrations define implementations and lifetimes: transient instances are created per resolution, scoped instances normally live for one request, and singletons live for the process and must be concurrency-safe. Shorter-lived services must not be captured by longer-lived ones. Typed options group configuration, structured logs preserve searchable fields, `HttpClientFactory` manages outgoing HTTP clients, and hosted services own work outside requests. In the Product Catalog, a thread-safe in-memory store and scoped application service move state and rules out of endpoints, leaving the HTTP layer responsible only for translation. Production storage can later replace the memory implementation without rewriting the callers.

Part III — Blazor

6. Components and Razor

The Product Catalog now has an HTTP API and a set of application services, but it still has no user interface. A browser can call `GET /api/products`, receive JSON, and display the raw response, yet that is not how users expect to work with an application. They need a page that presents products clearly, reacts to clicks and form input, and updates when application state changes. Blazor provides that interface using **Razor components**. A component is a reusable piece of UI written with HTML-like markup and C#. It can receive values from a parent, raise events, hold state, render child components, use injected services, and decide what markup should appear. A complete page is a component, but so is a product card, navigation menu, dialog, form field, or status message.

This chapter focuses on how components are written and connected. It covers Razor syntax, component classes, parameters, callbacks, binding, templates, cascading values, dependency injection, arbitrary HTML attributes, and CSS isolation. The next chapter will examine what happens after a component changes: render trees, rerendering, lifecycle methods, state ownership, and disposal.

6.1 Blazor and the Component Model

Traditional server-rendered applications generate HTML for each request. The user follows a link or submits a form, the server produces a new page, and the browser replaces the old document. JavaScript can make such pages more interactive, but the server remains responsible for most page generation. Client-side JavaScript frameworks take a different approach. The browser loads an application that maintains UI state, handles events, calls APIs, and updates selected parts of the DOM without replacing the whole page. Blazor uses a component model similar to other modern UI frameworks, but the components and most application behaviour are written in C#. Depending on the chosen render mode, a component can render on the server, become interactive through a server connection, execute in WebAssembly inside the browser, or move from server-rendered HTML to client-side execution after the application loads. The syntax used to build the component remains largely the same:

```
<h1>Products</h1>
<button type="button" @onclick="RefreshAsync">Refresh</button>
@code {
    private async Task RefreshAsync()
    {
        // Load products and update component state.
    }
}
```

The browser still receives HTML and displays ordinary web elements. Blazor does not create a new kind of button or heading. It gives C# code a structured way to describe those elements and connect them to application state and browser events. A useful separation is:

```
Razor component
-> describes the desired UI
Blazor renderer
-> compares the desired UI with the previous UI
Browser DOM
-> contains the actual elements shown to the user
```

This chapter concentrates on the first line. Chapter 7 explains the renderer and its relationship with the DOM. A component normally lives in a file whose name ends with `.razor`:

```
Components/
  Products/
    ProductList.razor
    ProductCard.razor
    ProductEditor.razor
```

The file combines markup and C#. During compilation, Razor generates a C# class. The generated type derives from `ComponentBase` unless another base type is specified. Markup becomes instructions that build the component's render tree, while members inside `@code` become members of the generated class. Consider a small component:

```

<h2>@Title</h2>
<p>@Description</p>
@code {
    private string Title { get; } = "Product Catalog";
    private string Description { get; } = "Browse the available products.";
}

```

You can think of it as a class whose rendering logic produces an `h2` and a `p` element using the current property values. Razor hides the low-level render-tree construction because writing the UI as markup is easier to read. Component names must begin with an uppercase letter. Lowercase tags are treated as HTML elements, while tags that match component types are treated as components:

```

<section>...</section>
<ProductCard />

```

`section` is an HTML element. `ProductCard` is a component type. When Razor encounters `<ProductCard />`, it creates that child component as part of the parent component's render tree. A `.razor` file can contain only markup, only directives and code, or a mixture of both. Small components often keep everything together. Larger components can move the C# portion into a code-behind file:

```

ProductList.razor
ProductList.razor.cs

```

The code-behind type is declared as a partial class:

```

public partial class ProductList
{
    private IReadOnlyList<ProductSummary> _products = [];
}

```

Both styles compile to the same component type. Keeping markup and code together is often clearer while the component is small. A code-behind file becomes useful when the logic grows large enough to distract from the UI structure, though a very large code-behind class can also indicate that application logic belongs in a service rather than in the component.

6.2 Razor Syntax and Directives

Razor treats ordinary text as markup and uses `@` to enter C#. A property can be inserted directly into an element:

```

<h2>@product.Name</h2>
<p>@product.Price.ToString("C")</p>

```

An explicit expression uses parentheses:

```

<p>Including tax: @(product.Price * 1.25m)</p>

```

Explicit expressions are useful when Razor cannot determine where the C# expression ends or when a more complex expression appears inside text. Control statements also begin with `@`:

```

@if (_products.Count == 0)
{
    <p>No products are available.</p>
}
else
{
    <ul>
        @foreach (var product in _products)
        {
            <li>@product.Name</li>
        }
    </ul>
}

```

The braces contain a mixture of C# control flow and markup. Razor understands that `<p>`, `<ul>`, and `<li>` are rendered content rather than C# statements. A local variable can be declared in a code block:

```
@{
    var available = _products.Count(product => product.IsAvailable);
}
<p>@available products are currently available.</p>
```

Use such blocks for short presentation calculations. If the calculation contains important application rules, performs I/O, or becomes hard to read, move it into a property, method, or application service. Razor encodes strings inserted into markup. If a product name contains `<script>`, the browser receives text rather than an executable script. This automatic HTML encoding is an important defence against cross-site scripting. `MarkupString` can deliberately render a string as raw markup, but it should only be used with content that is known to be safe. Comments use Razor's comment syntax:

```
@* This comment is not included in the rendered HTML. *@
```

An HTML comment, by contrast, can appear in the browser's DOM:

```
<!-- This comment may be sent to the browser. -->
```

Do not place secrets, private implementation details, or sensitive user information in either kind of client-visible content. Razor directives usually appear near the top of a component. They add metadata, imports, base types, interfaces, injected services, or routing information. A routable page component uses `@page`:

```
@page "/products"
<h1>Products</h1>
```

This does not create a separate controller. It gives the generated component type a route template so that Blazor's router can select it for `/products`. A component can have multiple routes:

```
@page "/products"
@page "/catalog"
```

Route parameters are written inside braces:

```
@page "/products/{Id:int}"
<h1>Product @Id</h1>
@code {
    [Parameter]
    public int Id { get; set; }
}
```

The `:int` constraint prevents the route from matching values that are not integers. The route value is assigned to the parameter whose name matches `Id`. `@using` imports a namespace for the current component:

```
@using ProductCatalog.Application
```

Common imports belong in `_Imports.razor`, where they apply to components in the same folder and its descendants:

```
@using ProductCatalog.Application
@using ProductCatalog.Contracts
@using Microsoft.AspNetCore.Components
@using Microsoft.AspNetCore.Components.Web
```

`_Imports.razor` is a Razor convention, so the name matters. It reduces repeated directives but should not become a dumping ground for every namespace in the solution. Import the namespaces that are genuinely common in that subtree. `@inject` requests a service from dependency injection:

```
@inject IProductCatalog ProductCatalog
@inject ILogger<ProductList> Logger
```

Razor generates properties for these services and assigns them before the component uses them. Constructor injection is common in ordinary C# classes, but Razor components normally use `@inject` or `[Inject]` because the framework creates the component and manages its parameterless construction. `@implements` adds an interface:

```
@implements IDisposable
```

This is commonly used when a component subscribes to events and must detach when it is removed. `@inherits` changes the base class:

`@inherits ProductComponentBase`

A shared base class can centralise a small amount of common component behaviour, but composition and injected services are usually more flexible than deep UI inheritance. `@typeparam` declares a generic component:

`@typeparam TItem`

`@attribute` places an attribute on the generated class:

`@attribute [Authorize]`

`@rendermode` selects an interactive render mode for a component when the application does not provide one globally:

`@rendermode InteractiveServer`

Render modes affect where and how interaction executes. They are introduced later in this chapter and examined more closely in Chapter 7. `@code` opens the component's C# member block. It is not an event and does not execute at a particular time. It simply contributes fields, properties, methods, and lifecycle overrides to the generated class.

6.3 Choosing Component Boundaries

A component is not merely a way to split a long `.razor` file. It creates a boundary with its own inputs, events, state, rendering work, and lifecycle. Good component boundaries normally follow meaningful UI responsibilities. For the Product Catalog, a useful decomposition might be:

```
ProductsPage
-> ProductToolbar
-> ProductFilter
-> ProductGrid
    -> ProductCard
-> ProductEditorDialog
-> StatusMessage
```

`ProductsPage` coordinates the screen. `ProductToolbar` exposes actions. `ProductFilter` edits search criteria. `ProductGrid` arranges products. Each `ProductCard` displays one product. `ProductEditorDialog` handles editing, while `StatusMessage` presents operation results. The split is useful because the parts have different responsibilities and may change independently. It is less useful to create a separate component for every heading, wrapper `div`, or two-line fragment. Thousands of tiny components make navigation harder and add rendering overhead without improving the design. A practical question is whether the proposed component has at least one of these qualities:

- a clear input and output boundary;
- reusable presentation;
- independent state or lifecycle;
- enough markup to deserve a name;
- behaviour that should be tested separately;
- a responsibility that makes the parent easier to understand.

If none apply, ordinary markup or a small helper method may be enough.

6.4 Parameters, Callbacks, Events, and Binding

A child component receives input through properties marked with `[Parameter]`. `ProductCard.razor`:

```
<article class="product-card">
  <h2>@Product.Name</h2>
  <p>@Product.Description</p>
  <strong>@Product.Price.ToString("C")</strong>
</article>
@code {
  [Parameter, EditorRequired]
  public required ProductSummary Product { get; set; }
}
```

The parent supplies the parameter as an attribute:

```
@foreach (var product in _products)
{
    <ProductCard Product="product" />
}
```

The value is a C# expression because `Product` is not an HTML string attribute. For simple literal values, Razor can usually infer the conversion:

```
<ProductGrid PageSize="20" ShowPrices="true" />
```

A parameter describes what the parent may provide. It is not private component state. Blazor can assign it again whenever the parent renders, so a child should not treat an ordinary parameter as permanent storage or overwrite it casually. For example, this is fragile:

```
[Parameter]
public ProductSummary Product { get; set; } = default!;
private void Rename()
{
    Product = Product with { Name = "Changed" };
}
```

The next parent render may assign the original value again. A better design depends on the intent. The child can raise an event asking the parent to change the product, or it can copy the parameter into internal edit state when parameters are received. Use parameters for values that define the child from the parent's perspective: the product to display, whether editing is enabled, a title, a selected identifier, or a formatting option. Keep internal details such as loading flags, temporary form values, and element references private unless the parent genuinely needs to control them. `[EditorRequired]` tells tooling that a parameter should be supplied, but it does not create runtime validation. The component must still handle invalid values appropriately when misuse is possible.

Data usually flows down through parameters. User actions flow back up through callbacks. Suppose `ProductCard` should notify its parent when the Edit button is clicked:

```
<article class="product-card">
    <h2>@Product.Name</h2>
    <p>@Product.Price.ToString("C")</p>
    <button type="button" @onclick="EditAsync">Edit</button>
</article>
@code {
    [Parameter, EditorRequired]
    public required ProductSummary Product { get; set; }
    [Parameter]
    public EventCallback<ProductSummary> EditRequested { get; set; }
    private Task EditAsync() => EditRequested.InvokeAsync(Product);
}
```

The parent handles the callback:

```
@foreach (var product in _products)
{
    <ProductCard
        Product="product"
        EditRequested="BeginEdit"
    />
}
@code {
    private ProductSummary? _editing;
    private void BeginEdit(ProductSummary product)
    {
        _editing = product;
    }
}
```

`EventCallback<T>` is preferable to exposing a raw delegate for normal component events. It integrates with Blazor's event and rendering flow, supports asynchronous handlers, and gives the parent a strongly typed event

value. The event name should describe what happened or what the child requests, not expose the child's implementation. `EditRequested` communicates intent better than `ButtonClicked`. A reusable child should not reach into its parent or mutate parent fields directly. When a callback may perform asynchronous work, await it:

```
private async Task SaveAsync()
{
    await SaveRequested.InvokeAsync(_model);
}
```

Avoid `async void` event handlers. Blazor supports handlers that return `Task`, allowing exceptions and completion to remain observable. The parent remains the owner of shared state:

Parent owns selected product
-> passes Product to child
<- receives `EditRequested` from child
-> updates selected product
-> renders children with new values

This unidirectional flow is simple to trace and prevents two components from silently fighting over the same value. HTML elements expose browser events. Blazor connects them to C# with event directives such as `@onclick`, `@onchange`, `@oninput`, `@onsubmit`, `@onkeydown`, and `@onfocus`.

```
<button type="button" @onclick="RefreshAsync">Refresh</button>
```

The handler can return `void`, `Task`, or `ValueTask`, though `Task` is the normal choice for asynchronous work:

```
private async Task RefreshAsync()
{
    _products = await ProductCatalog.ListAsync(CancellationToken.None);
}
```

Some events provide arguments:

```
<input @oninput="SearchChanged">
@code {
    private string _search = "";
    private void SearchChanged(ChangeEventArgs args)
    {
        _search = args.Value?.ToString() ?? "";
    }
}
```

Mouse, keyboard, clipboard, drag, focus, and other browser events have their own argument types. Use them when the event information is genuinely needed. A handler that only needs to call `RefreshAsync` does not need a `MouseEventArgs` parameter. Event modifiers can prevent default browser behaviour or stop propagation:

```
<a href="/products" @onclick="OpenProducts" @onclick:preventDefault>
    Open products
</a>
```

`preventDefault` stops the browser from following the link automatically. `stopPropagation` stops the event from continuing through Blazor's event-handling hierarchy. These modifiers should solve specific interaction requirements, not be added habitually. After a normal event handler completes, Blazor usually schedules the component to render again. You do not normally call `StateHasChanged` after changing a field in an `@onclick` handler. Chapter 7 explains exactly when rendering is triggered and when manual notification is needed. Manual event handling is sometimes useful, but forms often need a simpler connection between an input and a C# value. The `@bind` directive combines reading the current value and updating it when the user changes the input.

```
<input @bind="_search">
<p>Searching for: @_search</p>
@code {
    private string _search = "";
}
```

Conceptually, this combines a value attribute with an event handler:

```
C# field
    -> rendered input value
Browser change event
    -> converted value
    -> C# field
```

For a text input, the default update event is normally **change**, which occurs when the user commits the edit, often by leaving the field. To update on every input event:

```
<input @bind="_search" @bind:event="oninput">
```

Typed binding performs conversion:

```
<input type="number" @bind="_minimumPrice">
<input type="checkbox" @bind="_availableOnly">
@code {
    private decimal? _minimumPrice;
    private bool _availableOnly;
}
```

Blazor also supports binding between components. A child exposes a parameter and a matching callback named with the **Changed** suffix:

```
@code {
    [Parameter]
    public string Value { get; set; } = "";
    [Parameter]
    public EventCallback<string> ValueChanged { get; set; }
}
```

The parent can then write:

```
<SearchBox @bind-Value="_search" />
```

This expands conceptually to:

```
<SearchBox
    Value="_search"
    ValueChanged="value => _search = value"
/>
```

The convention is useful, but it does not mean both components own the state. The parent still owns **_search**; the child receives the current value and requests changes through **ValueChanged**. For more control around an update, **@bind:get**, **@bind:set**, **@bind:after**, and related forms can separate reading, assigning, and post-update work. Use the simplest form that expresses the behaviour clearly. If binding starts hiding substantial validation, network calls, or business logic, an explicit event handler may be easier to understand.

6.5 Fragments and Generic Components

A component parameter does not have to contain data. It can contain markup. **RenderFragment** represents a block of UI that can be rendered later. The conventional parameter name for unnamed child content is **ChildContent**. **Panel.razor**:

```
<section class="panel">
    <header>
        <h2>@Title</h2>
    </header>
    <div class="panel-content">
        @ChildContent
    </div>
</section>
@code {
    [Parameter]
    public string Title { get; set; } = "";
    [Parameter]
    public RenderFragment? ChildContent { get; set; }
}
```

The parent places markup between the component tags:

```
<Panel Title="Available products">
    <p>Select a product to view its details.</p>
    <ProductGrid Products="_products" />
</Panel>
```

The nested markup becomes the `ChildContent` fragment. `Panel` controls the surrounding structure but leaves the inner content to its caller. A named fragment allows several content areas:

```
<Dialog>
    <Header>
        <h2>Edit product</h2>
    </Header>
    <Body>
        <ProductEditor Product="_editing" />
    </Body>
    <Footer>
        <button type="button" @onclick="Close">Cancel</button>
    </Footer>
</Dialog>
```

The corresponding component declares `RenderFragment?` `Header`, `Body`, and `Footer` parameters. A typed fragment is written as `RenderFragment<T>`. It receives a value that the parent can use while defining the markup. `ProductGrid.razor`:

```
@typeparam TItem
<div class="product-grid">
    @foreach (var item in Items)
    {
        @ItemTemplate(item)
    }
</div>
@code {
    [Parameter, EditorRequired]
    public required IReadOnlyList<TItem> Items { get; set; }
    [Parameter, EditorRequired]
    public required RenderFragment<TItem> ItemTemplate { get; set; }
}
```

Usage:

```
<ProductGrid Items="_products">
    <ItemTemplate Context="product">
        <ProductCard Product="product" EditRequested="BeginEdit" />
    </ItemTemplate>
</ProductGrid>
```

`ProductGrid` controls iteration and layout. The parent controls how each item appears. This is the component equivalent of passing a formatting function, but the result is UI rather than a string. Templates are powerful when a reusable component owns structure while callers need control over content. They are unnecessary when the component has only one sensible representation. A component API with many fragments, parameters, callbacks, and switches can become harder to use than repeating a small amount of markup. A component can be generic in the same way as an ordinary C# type:

```
@typeparam TItem
```

Parameters and templates can use that type:

```
[Parameter, EditorRequired]
public required IReadOnlyList<TItem> Items { get; set; }
[Parameter, EditorRequired]
public required RenderFragment<TItem> ItemTemplate { get; set; }
```

Razor often infers `TItem` from the `Items` argument. It can also be specified explicitly:

```
<ProductGrid TItem="ProductSummary" Items="_products">
  <ItemTemplate Context="product">
    <strong>@product.Name</strong>
  </ItemTemplate>
</ProductGrid>
```

Generic components are useful for reusable lists, tables, selectors, forms, and wrappers that preserve the caller's model type. They should not be introduced merely to make a component appear flexible. A strongly typed `ProductGrid` can be clearer than a generic grid if the component contains product-specific behaviour. The same design rule used for ordinary generic code applies: create the abstraction when several real cases share the same structure, not when only one case exists.

6.6 Cascading Values and Application Services

Passing a value through several layers can become noisy when intermediate components do not use it.

Page

```
-> passes Theme to LayoutSection
    -> passes Theme to Toolbar
        -> passes Theme to Button
```

A cascading value allows an ancestor to provide a value to descendants without assigning it at every intermediate level.

```
<CascadingValue Value="_displaySettings">
  <ProductWorkspace />
</CascadingValue>
@code {
  private readonly DisplaySettings _displaySettings = new()
  {
    CompactCards = true
  };
}
```

A descendant receives it with `[CascadingParameter]`:

```
[CascadingParameter]
public DisplaySettings DisplaySettings { get; set; } = default!;
```

This is useful for values that naturally belong to a component subtree: a form editing context, a tab coordinator, a dialog host, display settings, or a parent component that coordinates descendants. Cascading values are not a general replacement for parameters or dependency injection. They make dependencies less visible at the call site, so overuse can make a component difficult to understand. Application-wide services normally belong in dependency injection. Direct parent-to-child input normally belongs in parameters. Cascading values fit contextual values that are owned by an ancestor UI boundary. If several values share the same type, name them:

```
<CascadingValue Name="DisplaySettings" Value="_displaySettings">
  <ProductWorkspace />
</CascadingValue>
```

```
[CascadingParameter(Name = "DisplaySettings")]
public DisplaySettings DisplaySettings { get; set; } = default!;
```

A cascading value can be marked fixed when its reference will not change. This allows Blazor to avoid maintaining change subscriptions for descendants. That optimisation matters only after correctness and clear ownership are established. Components should coordinate UI behaviour, not become a second application layer. They can inject the application services built in Chapter 5 and translate user interaction into service calls. A page might load products through `IProductCatalog`:

```
@page "/products"
@Inject IProductCatalog ProductCatalog
<h1>Products</h1>
@if (!_loading)
{
```

```

        <p>Loading products...</p>
    }
    else if (_error is not null)
    {
        <p role="alert">@_error</p>
    }
    else
    {
        <ProductGrid Items="_products">
            <ItemTemplate Context="product">
                <ProductCard Product="product" />
            </ItemTemplate>
        </ProductGrid>
    }
}
@code {
    private IReadOnlyList<ProductSummary> _products = [];
    private bool _loading = true;
    private string? _error;
    protected override async Task OnInitializedAsync()
    {
        try
        {
            _products = await ProductCatalog.ListAsync(CancellationTokens.None);
        }
        catch (Exception)
        {
            _error = "Products could not be loaded.";
        }
        finally
        {
            _loading = false;
        }
    }
}

```

This component owns presentation state: products currently shown, whether loading is in progress, and the message displayed after a failure. It does not implement product validation, storage, pricing rules, or logging policy. Those responsibilities remain in application and infrastructure services. The example catches **Exception** only to keep the first component simple. A real application should log unexpected failures and usually centralise repeated error handling. Chapter 8 will refine loading, validation, cancellation, and user feedback. The way a service is called depends on where the component executes. A server-interactive component can use server-registered application services directly because its C# code runs on the server. A WebAssembly component runs in the browser and cannot directly use server-only services or database access. It normally calls the server through **HttpClient**.

Interactive server component

- > IProductCatalog
- > Product service
- > Store

Interactive WebAssembly component

- > HttpClient
- > HTTP API
- > Product service
- > Store

The UI can look similar in both cases, but the process boundary is different. This distinction affects service registration, security, latency, deployment, and which assemblies may contain a component.

6.7 Render Modes

A Blazor Web App can render components in several ways. **Static server rendering** runs the component on the server for an HTTP request and sends HTML to the browser. The rendered output is not automatically

interactive after it arrives. **Interactive Server** keeps component instances on the server and sends browser events over a persistent connection. C# handlers run on the server, and DOM updates are sent back to the browser. **Interactive WebAssembly** downloads the .NET runtime and client assemblies so that component code runs in the browser. **Interactive Auto** initially uses server interactivity and can use WebAssembly on later visits after the client resources have downloaded. The same component model is used across these modes, but the execution environment matters:

```
Static server rendering
    request -> server renders HTML -> browser displays HTML
Interactive Server
    browser event -> connection -> server component
    server render update -> connection -> browser DOM
Interactive WebAssembly
    browser event -> component running in browser
    render update -> browser DOM
```

A render mode can be selected globally or for a component. For example:

```
@rendermode InteractiveServer
```

Components using interactive WebAssembly or Auto need client-compatible code and normally live in the `.Client` project of a Blazor Web App. They cannot depend on server-only implementation details, local files, server secrets, or a server database connection. Do not choose a render mode by habit. Server interactivity has a small initial download and direct access to server services, but it depends on a live connection and uses server resources for each active circuit. WebAssembly can continue executing client-side and reduces per-user server UI state, but it requires a larger download and reaches protected data through APIs. Static rendering is simple and efficient for content that does not need client-side interaction. Chapter 7 explains prerendering, interactive startup, circuits, and the rendering consequences in more detail.

6.8 Attributes and References

Component attributes can receive literals or C# expressions:

```
<ProductCard
    Product="_selectedProduct"
    ShowPrice="@_preferences.ShowPrices"
    CssClass="featured"
/>
```

For many non-string parameters, the leading `@` is optional because Razor already interprets the attribute value as C#:

```
<ProductCard Product="_selectedProduct" ShowPrice="_preferences.ShowPrices" />
```

Use the form that is clearest and consistent with the surrounding code. A component that wraps an HTML element may need to accept attributes it does not know in advance. Attribute splatting uses a parameter with `CaptureUnmatchedValues = true`:

```
<button
    type="button"
    class="@CssClass"
    @attributes="AdditionalAttributes"
>
    @ChildContent
</button>
@code {
    [Parameter]
    public string? CssClass { get; set; }
    [Parameter]
    public RenderFragment? ChildContent { get; set; }
    [Parameter(CaptureUnmatchedValues = true)]
    public IReadOnlyDictionary<string, object>? AdditionalAttributes { get; set; }
}
```

The caller can now supply normal HTML attributes:

```

<ActionButton
    CssClass="primary"
    aria-label="Save product"
    disabled="_saving"
>
    Save
</ActionButton>

```

This is especially useful for accessibility attributes, data attributes, identifiers, and browser features that a reusable component should not need to model individually. Attribute order can matter when the same attribute appears both explicitly and inside the dictionary because later values win during rendering. A component should define a clear policy for whether callers may override built-in attributes. Avoid creating parameters for every possible HTML attribute. Preserve the underlying web platform where it helps. A wrapper around a button should still allow meaningful button attributes instead of hiding them behind a large custom API. Razor can capture a reference to an element or component with `@ref`. An element reference:

```

<input @ref="_searchInput">
@code {
    private ElementReference _searchInput;
}

```

An `ElementReference` identifies a DOM element for framework features or JavaScript interoperability. It is not a normal DOM object that C# can inspect directly. A component reference gives access to a child component instance:

```

<ProductEditor @ref="_editor" />
@code {
    private ProductEditor? _editor;
}

```

The parent can call public members after the child has rendered, but this creates tighter coupling than parameters and callbacks. A reference is appropriate for imperative operations such as focusing an input, opening a specialised editor, or invoking a method on a third-party component. It should not become the normal way parent and child components exchange state. References are assigned only after rendering creates the target. Accessing them during initialisation is too early. The lifecycle chapter will show when they become available.

6.9 CSS Isolation, Imports, and Discovery

A Razor component can have a matching `.razor.css` file:

```

ProductCard.razor
ProductCard.razor.css
ProductCard.razor.css:

```

```

.product-card {
    display: grid;
    gap: 0.5rem;
    padding: 1rem;
    border: 1px solid #d4d4d4;
    border-radius: 0.5rem;
}
.product-card h2 {
    margin: 0;
}

```

During the build, Blazor rewrites the selectors with a generated scope attribute and bundles the isolated styles. The rendered component receives the corresponding scope marker. The result is that `.product-card` in this file normally applies only to elements rendered directly by `ProductCard`. Conceptually:

```

.product-card
-> .product-card[b-abc123]
<article class="product-card">
-> <article class="product-card" b-abc123>

```

The exact generated identifier is an implementation detail. Common class names can be reused across components without every rule becoming global. CSS isolation is not complete shadow-DOM isolation. Normal CSS inheritance still applies, global styles can still affect elements, and styling markup produced inside child components may require a deep combinator or a different styling boundary. Use global CSS for application-wide typography, design tokens, resets, and layout conventions. Use isolated CSS for styles owned by one component. The component still needs meaningful class names. CSS isolation prevents many collisions, but it does not make unclear markup easier to maintain. A component is available where its namespace is imported. If `ProductCard` lives under `Components/Products/ProductCard.razor`, its namespace is usually based on the project root namespace and folder path. A parent can import it directly:

```
@using ProductCatalog.Components.Products
```

Alternatively, place the import in `_Imports.razor`. Blazor also supports fully qualified component names, but ordinary imports are easier to read:

```
<ProductCatalog.Components.Products.ProductCard Product="product" />
```

Folder structure is not merely cosmetic because it influences generated namespaces and the scope of `_Imports.razor`. Keep related components together, but avoid deeply nested folders whose only purpose is to classify one file. A practical Product Catalog layout could be:

```
Components/
  Layout/
    MainLayout.razor
    NavMenu.razor
  Pages/
    Home.razor
    Products.razor
    ProductDetails.razor
  Products/
    ProductCard.razor
    ProductGrid.razor
    ProductEditor.razor
  Shared/
    ActionButton.razor
    StatusMessage.razor
```

Page components represent routes. Feature components represent reusable pieces of a feature. Shared components are genuinely cross-feature UI elements. A component should not be moved to `Shared` simply because it is used twice inside the same feature.

6.10 The First Product Page

The pieces can now be combined into a small page. `Products.razor`:

```
@page "/products"
@inject IProductCatalog ProductCatalog
@inject ILogger<Products> Logger
<PageTitle>Products</PageTitle>
<h1>Products</h1>
<div class="product-toolbar">
  <input
    type="search"
    placeholder="Search products"
    @bind="_search"
    @bind:event="oninput"
  />
  <button type="button" @onclick="LoadAsync" disabled="_loading">
    Refresh
  </button>
</div>
@if (_loading)
{
  <p>Loading products...</p>
}
```

```

}
else if (_error is not null)
{
    <StatusMessage Level="MessageLevel.Error">
        @_error
    </StatusMessage>
}
else
{
    <ProductGrid Items="FilteredProducts">
        <ItemTemplate Context="product">
            <ProductCard
                Product="product"
                EditRequested="BeginEdit"
            />
        </ItemTemplate>
    </ProductGrid>
}
@if (_editing is not null)
{
    <ProductEditor
        Product="_editing"
        SaveRequested="SaveAsync"
        CancelRequested="CancelEdit"
    />
}
@code {
    private IReadOnlyList<ProductSummary> _products = [];
    private string _search = "";
    private bool _loading;
    private string? _error;
    private ProductSummary? _editing;
    private IEnumerable<ProductSummary> FilteredProducts =>
        string.IsNullOrEmpty(_search)
        ? _products
        : _products.Where(product =>
            product.Name.Contains(
                _search,
                StringComparison.OrdinalIgnoreCase
            )
        );
    protected override Task OnInitializedAsync() => LoadAsync();
    private async Task LoadAsync()
    {
        _loading = true;
        _error = null;
        try
        {
            _products = await ProductCatalog.ListAsync(
                CancellationToken.None
            );
        }
        catch (Exception exception)
        {
            Logger.LogError(exception, "Failed to load products");
            _error = "Products could not be loaded.";
        }
        finally
        {
            _loading = false;
        }
    }
}

```

```

    }
    private void BeginEdit(ProductSummary product)
    {
        _editing = product;
    }
    private void CancelEdit()
    {
        _editing = null;
    }
    private async Task SaveAsync(UpdateProductRequest request)
    {
        if (_editing is null) return;
        var result = await ProductCatalog.UpdateAsync(
            _editing.Id,
            request,
            CancellationToken.None
        );
        if (result is null)
        {
            _error = "The product no longer exists.";
            return;
        }
        if (!result.IsSuccess)
        {
            _error = result.Error;
            return;
        }
        _editing = null;
        await LoadAsync();
    }
}

```

This page already demonstrates the central component model:

Products page owns screen state

- > passes Product to ProductCard
- <- receives EditRequested
- > passes selected product to ProductEditor
- <- receives SaveRequested or CancelRequested
- > calls IProductCatalog
- > replaces its product collection
- > renders children with the new state

The page is not complete. It uses `CancellationToken.None`, reloads the whole product list after saving, and has only basic error handling. The editor and form validation have not yet been implemented. Those choices are deliberate. The purpose here is to see component composition and communication before adding rendering details and full interface behaviour. The markup remains readable because application operations live in `IProductCatalog`, repeated presentation lives in child components, and state ownership stays in the page. If the page grows, its loading and editing behaviour may move into a dedicated presentation-state service, but that extra abstraction is not yet necessary.

6.11 Common Mistakes and Key Ideas

Component problems usually come from unclear ownership rather than Razor syntax. Keep database access, business rules, HTTP translation, and unrelated workflow state outside the component; a page should coordinate presentation and call application services. Treat parameters as parent-owned input, report requested changes through callbacks, and use an internal edit model when temporary mutation is necessary. Prefer a clear value owner over webs of two-way binding, cascading values, or component references.

Blazor already rerenders after its normal events and lifecycle methods, so manual `StateHasChanged` calls should be reserved for external notifications or deliberate intermediate states. Introduce generic components only after several real uses justify the abstraction, and remember that moving code into a `.razor.cs` file changes layout rather than responsibility. Interactive WebAssembly code runs in the browser and cannot contain secrets or

server-only dependencies; all browser input remains untrusted regardless of render mode. A component is a C# type that describes UI. Parameters move data down, **EventCallback** reports actions up, events and binding connect browser input to state, fragments represent reusable UI, and services connect presentation to application operations. The browser still owns HTML, the DOM, CSS, accessibility, and final pixels.

7. Rendering and State

A component describes what the interface should look like for its current state. When that state changes, Blazor runs the component again, compares the new result with the previous one, and updates only the affected parts of the browser DOM. That sounds similar to redrawing a desktop window, but Blazor does not paint pixels directly and it does not normally replace the whole page. It works with a component tree, a render tree, and a browser document that may live in another process or even on another machine.

This chapter explains that mechanism and the lifecycle around it. It answers the questions that usually matter most when a Blazor application begins behaving unexpectedly: who requests a rerender, when lifecycle methods run, why changing a field sometimes updates the page and sometimes does not, where state should live, what survives navigation or refresh, why event subscriptions must be removed, and how server-side circuits change dependency-injection scopes.

7.1 State, Render Trees, and the DOM

Consider a product page with three fields:

```
private IReadOnlyList<ProductSummary> _products = [];  
private bool _loading;  
private string? _error;
```

The markup chooses what to show from those values:

```
@if (_loading)  
{  
    <p>Loading products...</p>  
}  
else if (_error is not null)  
{  
    <p role="alert">@_error</p>  
}  
else  
{  
    <ProductGrid Items="_products" />  
}
```

There is no separate command that says “hide the loading message, show the grid, and insert these rows.” The component changes its state and renders again. The markup then describes the correct result for that state. If `_loading` is `true`, the loading paragraph belongs in the output; if it becomes `false`, that paragraph disappears and the grid can appear. This is the central Blazor model:

Current component state
-> component renders
-> new render tree
-> differences are applied to the DOM

Thinking in state is usually simpler than manually coordinating individual controls. The important design question becomes not “which element should I manipulate?” but “what state should produce this interface?” Razor markup is compiled into instructions that build a **render tree**. The tree contains frames describing elements, text, attributes, child components, event handlers, and regions. It is Blazor’s compact representation of the UI a component wants. The browser has its own tree, the DOM. Blazor compares the newly produced render tree with the component’s previous render tree and creates a batch of changes. Those changes might add an element, remove an attribute, replace text, update a child parameter, or remove a component. The renderer then applies the batch to the DOM directly in WebAssembly or sends it to the browser for an interactive server component.

Razor component state
-> new render tree
-> compare with previous render tree
-> minimal edit batch
-> browser DOM
-> browser layout and paint

Rendering therefore does not mean that Blazor rebuilds the whole page. A component may execute its rendering logic again, but only the calculated differences are applied. A page with one changed product price can produce a small text update rather than a new document. The comparison relies on the sequence and identity of render-tree entries. This is why manually building render trees requires stable sequence numbers and why normal Razor markup is preferable for almost all application code: the compiler can generate efficient, predictable instructions from the source structure.

7.2 Render Triggers and External Notifications

A component renders when it is first added to the component hierarchy. After that, `ComponentBase` requests rendering in several common situations: when the parent supplies parameters, when a Blazor event callback completes, and when the component explicitly calls `StateHasChanged`. Framework features such as cascading values can also cause parameter updates that lead to rendering. This explains why a button handler normally needs no manual notification:

```
<p>Count: @_count</p>
<button type="button" @onclick="Increment">Add</button>
@code {
    private int _count;
    private void Increment() => _count++;
}
```

Blazor dispatches the click to `Increment`, then requests a render after the handler completes. The new render tree contains the new count, so the text node is updated. A plain field assignment is not observable by itself. Blazor does not monitor every object and property. If code changes `_count` from a timer, an event raised by another service, or an arbitrary callback outside Blazor's event flow, the component may need to request rendering explicitly. The distinction is not whether the value changed, but whether Blazor already knows that work affecting the component has completed.

`StateHasChanged` tells the renderer that the component may have different output. It queues rendering; it does not synchronously rebuild the DOM before the method returns. If several calls occur before the renderer processes the queue, Blazor can coalesce them into one render. Normal component events and lifecycle methods already request rendering, so this is usually unnecessary:

```
private void Increment()
{
    _count++;
    StateHasChanged();
}
```

The explicit call adds no value because Blazor rerenders after the click handler. Manual notification is useful when state changes outside the normal flow, or when an asynchronous operation should display an intermediate stage before it finishes. Suppose a long operation first changes a status message, then awaits work:

```
private async Task ImportAsync()
{
    _status = "Importing...";
    StateHasChanged();
    await ImportService.ImportAsync();
    _status = "Import complete.";
}
```

The first call allows the intermediate status to render before the awaited operation completes. When the event handler finishes, Blazor requests the final render automatically. The example should still avoid CPU-heavy synchronous work on the UI context; long computation belongs in an appropriate background or server operation. A state service, timer, SignalR callback, or background notification can change data while no component event is running. A subscribed component should marshal the update through `InvokeAsync` and then request rendering:

```
@implements IDisposable
@Inject ProductState ProductState
<p>@ProductState.Status</p>
@code {
    protected override void OnInitialized()
```

```

    {
        ProductState.Changed += OnStateChanged;
    }
    private void OnStateChanged()
    {
        _ = InvokeAsync(StateHasChanged);
    }
    public void Dispose()
    {
        ProductState.Changed -= OnStateChanged;
    }
}

```

`InvokeAsync` schedules the callback on Blazor's synchronization context. This matters particularly for interactive server components, where each circuit provides a logical single-threaded environment even though the server handles many circuits concurrently. Calling component rendering methods directly from an unrelated thread can violate that context. The discarded task in this compact example is acceptable only because the event delegate cannot return a `Task`; production code should ensure that exceptions are observed, commonly by using an asynchronous notification abstraction or by routing failures to an error-handling service. The essential pattern is subscription, `InvokeAsync`, `StateHasChanged`, and unsubscription.

7.3 Parent Rendering, `ShouldRender`, and `@key`

When a parent renders, Blazor evaluates the parameters supplied to each child. If a child receives new parameter values, its parameter lifecycle runs and it may render again. The child should therefore treat parameters as current input, not as values assigned only once.

```
<ProductCard Product="_selectedProduct" Compact="_compact" />
```

If `_selectedProduct` or `_compact` changes and the parent renders, the child sees the new values. For primitive and other known immutable values, Blazor can often avoid child rendering when values have not changed. Complex objects are more conservative because the framework cannot know whether their internal data was mutated. This is another reason to avoid modifying a parameter inside its child. The parent remains the source of truth and may overwrite the local assignment on the next render. When a child needs editable state, copy the relevant values into an edit model during parameter processing and raise a callback when the user commits the change. A parent rendering does not mean that every descendant must produce DOM changes. Children can render and produce identical trees, after which the diff contains nothing meaningful to apply. Optimising that work is rarely necessary until measurements show a real problem.

A component can override `ShouldRender` to suppress a noninitial render:

```
protected override bool ShouldRender() => _shouldRender;
```

The first render still occurs. On later updates, returning `false` tells Blazor to keep the existing output for that component subtree. This can help when a frequently updated parent supplies complex parameters to an expensive child whose visible output rarely changes. The child can compare a stable identifier or version and render only when the meaningful input changes. It is not a general best practice. An incorrect `false` leaves stale UI on screen, and the added state-tracking logic can cost more than the render it avoids. Most pages, forms, dialogs, and ordinary feature components do not need `ShouldRender`. Start with normal rendering, use browser and application measurements to find a hotspot, simplify the component or its data flow first, and override `ShouldRender` only when the condition is easy to prove.

Blazor normally matches repeated render-tree entries by position. That is efficient, but position is not always the same as identity. Suppose products are inserted at the beginning of a list containing stateful child components:

```

@foreach (var product in _products)
{
    <ProductEditor Product="product" />
}

```

If the list changes order, Blazor may reuse an existing `ProductEditor` instance for a different product because it occupies the same position. Focus, temporary input, or other local state can then appear attached to the wrong item. `@key` tells Blazor which value defines identity:

```
@foreach (var product in _products)
{
    <ProductEditor @key="product.Id" Product="product" />
}
```

When product 42 moves, its component identity follows product 42. When a product disappears, its component is removed and disposed. A key can also force replacement when identity changes:

```
<ProductDetails @key="_selectedProductId" ProductId="_selectedProductId" />
```

Use `@key` when items can be inserted, deleted, or reordered and child identity matters. It is not needed on every loop, and an unstable key such as a newly created object defeats the purpose.

7.4 The Component Lifecycle

A component passes through a predictable set of stages. The exact framework implementation contains more detail, but the practical order is:

Component is created

- > parameters are assigned
- > `OnInitialized` / `OnInitializedAsync`
- > `OnParametersSet` / `OnParametersSetAsync`
- > render
- > DOM update
- > `OnAfterRender` / `OnAfterRenderAsync`
- > later parameter or state updates repeat relevant stages
- > `Dispose` or `DisposeAsync` when removed

The synchronous method in each pair runs before its asynchronous counterpart. Override only the methods the component needs. Lifecycle methods are not a checklist that every component should implement. The component must also remain valid whenever an asynchronous lifecycle method yields. Other work may continue, parameters may change, navigation may remove the component, or disposal may begin before the awaited operation returns. Assign a sensible loading state before the first `await`, and do not assume the component is still active afterward without considering cancellation or disposal. `OnInitialized` and `OnInitializedAsync` are intended for work that depends on the component instance but not on changing parameters. Typical examples include creating local state, subscribing to a stable service, or loading data that remains the same for the lifetime of the instance.

```
protected override async Task OnInitializedAsync()
{
    _loading = true;
    _products = await ProductCatalog.ListAsync(Cancellation.Token.None);
    _loading = false;
}
```

Initialization is not the right place to react to a route parameter that can change while the same component instance remains active. For that, use parameter processing. Calling the base implementation is usually unnecessary when directly deriving from `ComponentBase` because its lifecycle methods do no work. It becomes necessary if a custom base component implements behaviour that must run. Prerendering adds an important qualification: a component can be created once to produce initial server HTML and again when interactive rendering starts. Code that performs external work during initialization may therefore run twice unless state is persisted across the transition or prerendering is disabled for that component. This is discussed in Section 7.18.

`OnParametersSet` and `OnParametersSetAsync` run after parameters have been assigned, including the initial assignment. They are the correct place to derive internal state or load data from a route or parent parameter.

```
@page "/products/{Id:int}"
@code {
    [Parameter]
    public int Id { get; set; }
    private ProductDetails? _product;
    protected override async Task OnParametersSetAsync()
    {
        _product = await ProductCatalog.GetAsync(Id, Cancellation.Token.None);
    }
}
```

```
    }
}
```

Navigating from `/products/41` to `/products/42` may reuse the same page component. `OnInitializedAsync` would not run again, but `OnParametersSetAsync` does, allowing the page to load the new product. Avoid launching duplicate work when unrelated parent renders supply the same effective input. Store the previous identifier or let a state service cache the result when loading is expensive. Also consider cancellation: a slow request for product 41 should not overwrite the result for product 42 if the user navigates quickly. `OnAfterRender` and `OnAfterRenderAsync` run after Blazor has updated the UI. Their `firstRender` argument is `true` only after the first completed render of that component instance. This stage is appropriate for work that requires an existing DOM element or component reference, especially JavaScript interoperability:

```
<input @ref="_searchInput">
@code {
    private ElementReference _searchInput;
    protected override async Task OnAfterRenderAsync(bool firstRender)
    {
        if (firstRender)
            await _searchInput.FocusAsync();
    }
}
```

Element references are not usable during initialization because the corresponding element does not exist yet. `firstRender` prevents repeating one-time setup after every update. Unlike ordinary event callbacks and other lifecycle methods, completion of `OnAfterRenderAsync` does not automatically request another render. Automatically doing so would create an endless loop: render, after-render task, render again. If after-render work changes state that must be displayed, request rendering explicitly and ensure the condition eventually stops. Server prerendering does not call the after-render methods because there is no interactive browser DOM at that stage. They run after the component becomes interactive.

7.5 Async Work, Cancellation, and Disposal

An asynchronous lifecycle method or event handler can yield before it completes. Blazor can render the synchronous state established before the first incomplete `await`, then render again when the task completes.

```
private async Task SaveAsync()
{
    _saving = true;
    _error = null;
    await ProductCatalog.SaveAsync(_editModel, CancellationToken.None);
    _saving = false;
}
```

A button bound to `_saving` can become disabled while the request is in progress and return to normal afterward. The component does not need to block a thread while waiting. The state before each `await` should be internally consistent because rendering or another callback may occur. Do not leave a required object temporarily null unless the markup handles that state. In interactive server rendering, components in one circuit appear logically single-threaded, but asynchronous reentrancy is still possible whenever a method yields. Never use `.Result`, `.Wait()`, `Thread.Sleep`, or similar blocking calls in component code. They block progress and can deadlock or make the UI unresponsive. Use asynchronous APIs end to end.

A component can disappear while an operation is still running. The user may navigate away, change a parameter, close a dialog, or disconnect. Cancellation does not guarantee that every dependency stops immediately, but it gives cooperative operations a way to avoid unnecessary work and stale updates.

```
@implements IAsyncDisposable
@code {
    private readonly CancellationTokenSource _lifetime = new();
    protected override async Task OnInitializedAsync()
    {
        _products = await ProductCatalog.ListAsync(_lifetime.Token);
    }
    public ValueTask DisposeAsync()
    {

```

```

        _lifetime.Cancel();
        _lifetime.Dispose();
        return ValueTask.CompletedTask;
    }
}

```

Production code should handle the expected cancellation path without showing it as an unexpected failure. A component may also use a separate token source for each parameter-dependent request, cancelling the previous request before starting the next one. Do not start fire-and-forget tasks that continue using the component after disposal. If work must outlive the UI, move it to an application or background service and let the component observe its status. Blazor calls `Dispose` or `DisposeAsync` when a component is removed from the UI. Disposal is needed for resources the component owns: event subscriptions, timers, cancellation sources, JavaScript object references, streams, or other disposable objects created for that instance.

```

@implements IDisposable
@Inject ProductState ProductState
@code {
    protected override void OnInitialized()
    {
        ProductState.Changed += OnChanged;
    }
    private void OnChanged()
    {
        _ = InvokeAsync(StateHasChanged);
    }
    public void Dispose()
    {
        ProductState.Changed -= OnChanged;
    }
}

```

Failing to unsubscribe lets the service keep a reference to the component. The removed component can continue receiving events and may remain in memory. Disposal timing should not be used as application logic. It may occur after navigation, conditional removal, circuit loss, or other framework activity, and an asynchronous initialization task may still be incomplete. Disposal code should tolerate partially created resources and should not call `StateHasChanged`, because the component is leaving the render tree. If both `IDisposable` and `IAsyncDisposable` are implemented, Blazor uses the asynchronous path. Prefer one interface that matches the resources being released.

7.6 State Lifetimes and Ownership

“State management” is not one feature. It is the decision of where a value belongs and how long it must survive. The Product Catalog may contain several kinds of state at once:

- whether one button is busy;
- the current filter and selected product;
- an edit model shared by a page and dialog;
- the signed-in user’s preferences;
- products loaded from the server;
- durable product records in the database.

These values should not all use the same mechanism. A practical hierarchy is:

```

Private component field
-> state used only by one component instance
Parent-owned parameter state
-> state shared with direct descendants
Cascading UI context
-> state shared through one component subtree
Scoped state service
-> state shared by unrelated components in one user session or circuit
Browser storage
-> state that should survive navigation or refresh on one browser

```

Server store or database

-> authoritative or durable application state

Keep state as local as its required lifetime permits. Global state is easy to reach but harder to reason about, reset, test, and protect between users. A field is the simplest form of component state:

```
private bool _editorOpen;
private ProductSummary? _selectedProduct;
```

It survives rerenders because rerendering reuses the component instance. It disappears when that instance is removed, which can happen during navigation, conditional rendering, a changed `@key`, or a full browser refresh. When siblings need the same value, move ownership to their nearest common parent:

ProductsPage owns SelectedProduct

-> ProductGrid receives selected identifier
-> ProductEditor receives selected product
<- both raise callbacks to ProductsPage

This is often called lifting state. It keeps one source of truth and makes data flow visible in component markup. A state service becomes worthwhile only when the nearest common parent is too high, unrelated routes need the same state, or the state has behaviour that deserves an ordinary C# abstraction. A simple state container can be an ordinary service:

```
public sealed class ProductWorkspaceState
{
    private ProductSummary? _selectedProduct;
    public event Action? Changed;
    public ProductSummary? SelectedProduct => _selectedProduct;
    public void Select(ProductSummary? product)
    {
        if (_selectedProduct?.Id == product?.Id) return;
        _selectedProduct = product;
        Changed?.Invoke();
    }
}
```

Register it with a lifetime that matches the intended ownership:

```
builder.Services.AddScoped<ProductWorkspaceState>();
```

Components subscribe, use `InvokeAsync(StateHasChanged)` when notified externally, and unsubscribe during disposal. The service should expose meaningful operations such as `Select`, `BeginEdit`, or `Clear`, rather than public setters that allow arbitrary mutation. The meaning of `Scoped` depends on the Blazor execution model. In an ordinary HTTP endpoint, scoped means one instance per request. In interactive server rendering, a scoped service normally lives for the user's circuit, which can span many events and navigations. In `WebAssembly`, scoped services behave much like client-side singletons because there is no normal HTTP request scope inside the browser application. Never place per-user mutable state in a server singleton. A singleton is shared by all users in the process and can leak one user's data into another session. Use singleton services only for thread-safe application-wide data and behaviour.

7.7 Navigation, Reconnection, and Prerendering

Internal Blazor navigation often preserves the interactive application and its scoped services. Page components may be replaced, but a circuit-level or client-level state service can remain alive. A full browser refresh starts a new document. Component instances disappear, and ordinary in-memory UI state is lost. In interactive server rendering, the old circuit may remain temporarily available for reconnection, but the application should not treat that as durable storage. A server restart, expired circuit, long disconnection, or deployment can remove it. State that must survive a refresh belongs in browser storage or on the server. Preferences such as compact layout may fit `localStorage`. A temporary workflow may fit `sessionStorage`. Business records, permissions, orders, and other authoritative data belong in a protected server store or database.

The URL is also a useful state store. Filters, page numbers, selected tabs, and identifiers placed in route or query values can survive refresh, support bookmarking, and make links shareable. Do not hide navigational state in memory when it naturally belongs in the address. A Blazor Web App can render a component on the

server to send useful HTML immediately, then make it interactive after the browser establishes the selected render mode. These are two distinct phases and can involve separate component instances.

Initial HTTP request

- > server prerenders component
- > HTML reaches browser
- > interactive runtime starts
- > interactive component is created

Initialization code may therefore run during prerendering and again during interactive startup. If both phases call a database or remote API, the user may see duplicate work or a brief change from loaded content back to a loading message. Persistent component state can serialize selected prerendered values into the response and restore them for the interactive instance. The interactive component then continues from the result instead of loading it again. This is appropriate for serializable state that was already obtained during prerendering; it is not a substitute for durable storage. Prerendered components also lack a usable browser DOM. JavaScript interoperability and element references must wait until interactive rendering and `OnAfterRenderAsync`.

7.8 Interactive Server and WebAssembly State

With interactive server rendering, each connected user has a **circuit**. The circuit contains component instances, their fields, the current render tree, scoped services, and enough framework state to process browser events and send UI updates.

Browser

- <-> persistent connection
- <-> server circuit
 - > component tree
 - > component state
 - > scoped services
 - > render state

This model provides direct access to server services and a small client download, but it changes resource and failure considerations. Every active circuit consumes server memory. Network latency affects interactions. A disconnection can interrupt the UI, and long-running synchronous work in one circuit delays that user's events. Different circuits can run concurrently, but Blazor presents each circuit with a logical single-threaded synchronization context. Code must still protect shared singleton data because other circuits and server work can access it at the same time. Do not store large datasets in every circuit merely because fields are convenient. Keep authoritative data in shared infrastructure and hold only the view state needed for the current interaction.

With interactive WebAssembly, component instances and scoped state services live in the browser's .NET runtime. They survive component rerenders and internal navigation while the client application remains loaded, but a refresh starts a new runtime and removes ordinary memory state. The client cannot directly access server memory, local server files, server secrets, or a database connection. Durable or protected state is reached through HTTP APIs. Values in the downloaded application must be considered visible to the user, even when they are not displayed in the UI. WebAssembly reduces server-held UI state and can continue some interaction after a temporary network interruption, but API calls still need loading, failure, and retry behaviour. Browser storage can preserve selected client preferences, while security-sensitive authority remains on the server.

7.9 Product Catalog State Design

The Product Catalog can now assign each value to an appropriate owner:

ProductCard

- > hover and local visual state

ProductsPage

- > current filter
- > loading and error state
- > selected product identifier

ProductWorkspaceState

- > selection shared by unrelated product components

URL

- > search text, category, page number when shareable

Browser storage

- > compact/list display preference

Server and database

-> products, prices, permissions, saved changes

The page can keep its search in the URL and its display preference in browser storage later. Product records remain authoritative on the server. A shared workspace service is only needed if components outside the page hierarchy must coordinate selection. When a product changes, the application should replace or update the relevant product in state, then let rendering update the visible card. Reloading the entire page is unnecessary. SignalR can eventually notify other connected users, but the notification should lead to a clear state update rather than direct DOM manipulation. A compact rendering flow is:

User saves product

-> event handler sets `_saving`
-> intermediate render disables Save
-> application service updates product
-> page replaces product in `_products`
-> final render updates `ProductCard`

The UI remains predictable because state ownership is explicit and every visible change follows from new state.

7.10 Common Mistakes and Key Ideas

Rendering problems usually indicate unclear state ownership or lifecycle assumptions. Let Blazor rerender after its own events, call `StateHasChanged` only when changes originate outside that flow, and avoid `ShouldRender` until measurement shows a real hot spot. Prefer explicit or immutable state changes when shared mutable objects make updates difficult to trace, use `@key` when list identity matters, and never store per-user mutable state in a singleton. Parameter-dependent loading belongs in parameter lifecycle methods, element references become usable only after rendering, and subscriptions or JavaScript references must be released during disposal. Async methods should leave valid state before every `await`, cancel stale work, and avoid blocking or uncontrolled fire-and-forget operations. Component fields do not survive refresh, deployment, or a lost server circuit unless the state is copied to the URL, browser storage, or durable server storage.

Blazor compares render trees and applies the required DOM changes. The lifecycle separates initialization, parameter changes, and post-render work, while Interactive Server and WebAssembly place component state in different processes. Choose the owner and lifetime of each value before choosing the mechanism used to share it.

8. Building Complete Interfaces

The previous two chapters explained how Blazor components communicate, render, and hold state. Those mechanisms are enough to build an isolated widget, but an application needs a larger structure around them. Users move between addresses, pages share navigation and common chrome, forms collect and validate input, layouts respond to different screen sizes, and some browser features still require JavaScript. A complete interface also needs keyboard support, useful focus behaviour, clear loading and error states, and enough performance discipline to remain pleasant as the amount of data grows.

This chapter combines those concerns into one Product Catalog interface. It begins with routing and layouts, then builds an editor with `EditForm`, validation, and server-side errors. It finishes with styling, accessibility, JavaScript interoperability, and practical performance choices. The goal is not to catalogue every Blazor component. It is to show how the pieces cooperate in an application that feels like one coherent interface rather than a collection of demonstrations.

8.1 Pages, Routes, Query Values, and Navigation

A page is still a Razor component. The difference is that it has one or more route templates:

```
@page "/products"
<PageTitle>Products</PageTitle>
<h1>Products</h1>
```

During compilation, `@page` adds route metadata to the generated component type. The router uses that metadata to select the component whose template matches the current address. The file's folder does not define its URL. `Components/Pages/Products.razor` can map to `/products`, while a component in another folder can map to any route it declares. Route parameters carry values from the path:

```
@page "/products/{Id:int}"
<PageTitle>Product @Id</PageTitle>
<h1>Product @Id</h1>
@code {
    [Parameter]
    public int Id { get; set; }
}
```

The `:int` constraint prevents this route from matching `/products/new`, leaving that address available for a different page. Constraints help routing choose a shape; they do not prove that product 42 exists. The page still loads the product and handles a not-found result. A component may declare several routes when two addresses intentionally represent the same page:

```
@page "/products"
@page "/catalog"
```

Do this sparingly. Duplicate public addresses complicate links, analytics, caching, and search indexing. A redirect is often clearer when one address is the canonical one. The route should represent navigational state. Product identifiers, page numbers, and selected categories often belong in the URL because users can refresh, bookmark, and share them. Temporary visual state such as whether a menu is open usually belongs in component memory. Path segments identify the main resource, while query values are well suited to optional filters and paging:

```
/products?search=keyboard&category=accessories&page=2
```

A page can receive query values with `[SupplyParameterFromQuery]`:

```
@page "/products"
@code {
    [SupplyParameterFromQuery]
    private string? Search { get; set; }
    [SupplyParameterFromQuery]
    private string? Category { get; set; }
    [SupplyParameterFromQuery]
    private int Page { get; set; } = 1;
}
```

These properties are parameters, so query changes can cause `OnParametersSet` or `OnParametersSetAsync` to run again. That makes the URL the owner of the navigational filter state. The component can load the matching

page whenever the parameters change rather than keeping a separate unsynchronised copy. Use query values for information that should survive refresh or appear in a copied link. Do not place secrets or sensitive personal data in them; URLs commonly appear in browser history, logs, screenshots, analytics, and referrer information. A component may keep edit fields separate from supplied query parameters. For example, a search box can hold `_searchText` while the user types, then update the URL when the user submits or pauses:

```
private void ApplySearch()
{
    var uri = Navigation.GetUriWithQueryParameter("search", _searchText);
    Navigation.NavigateTo(uri);
}
```

This distinction avoids changing the route on every keystroke unless that behaviour is genuinely useful. A Blazor Web App normally contains a `Routes.razor` component with a `Router`. A simplified version looks like this:

```
<Router AppAssembly="typeof(Program).Assembly">
    <Found Context="routeData">
        <RouteView
            RouteData="routeData"
            DefaultLayout="typeof(Layout.MainLayout)"
        />
        <FocusOnNavigate
            RouteData="routeData"
            Selector="h1"
        />
    </Found>
</Router>
```

The router searches the supplied assemblies for routable components. `RouteView` renders the selected page inside its layout. `FocusOnNavigate` moves focus to the first matching element after navigation, usually the page's main heading. This is important for keyboard and screen-reader users because changing component content does not automatically move browser focus as a traditional full-page load would. In a Blazor Web App, initial static rendering and later interactive routing can use different parts of the platform. The first request may pass through ASP.NET Core endpoint routing and render HTML on the server. Once the router is interactive, internal navigation can replace the selected component without requesting a completely new document. The visible result is similar to a single-page application, but the route still remains a real URL.

A missing route should produce a deliberate not-found experience. Depending on the application's rendering setup, this may be handled through ASP.NET Core status-code handling, a not-found page, or router configuration. The important behaviour is that an unknown address should not silently show an empty layout or redirect every mistake to the home page. For ordinary navigation, use an anchor element:

```
<a href="/products">Products</a>
```

Blazor can enhance internal navigation so that the browser keeps the current document and replaces the relevant content. The link remains valid even before interactivity starts and retains normal browser behaviour such as opening in a new tab, copying the address, or using the context menu. `NavLink` is useful in navigation menus because it adds an active CSS class when its target matches the current address:

```
<nav aria-label="Primary">
    <NavLink href="/" Match="NavLinkMatch.All">Home</NavLink>
    <NavLink href="/products">Products</NavLink>
    <NavLink href="/admin/products">Administration</NavLink>
</nav>
```

Use a button for an action and a link for navigation. A clickable `div` or button that merely changes the address loses native browser semantics and keyboard behaviour. Programmatic navigation is still useful after an operation:

`@inject NavigationManager` Navigation

```
private void OpenProduct(int id)
{
```

```

    Navigation.NavigateTo($"products/{id}");
}

```

`NavigationManager` also exposes the current absolute URI, base URI, navigation events, and helpers for constructing query strings. `NavigateTo` should not replace an ordinary link when no decision or application operation is involved. Declarative links are simpler and more accessible. A forced full load can be requested when navigating outside the interactive application model or when the server must reconstruct the document:

```

Navigation.NavigateTo("/account/login", forceLoad: true);

```

Use it only when required. Internal enhanced navigation usually preserves more state and transfers less data.

8.2 Layouts and Responsive Structure

A layout is a component that inherits from `LayoutComponentBase` and renders the selected page through `@Body`. `MainLayout.razor`:

```

@inherits LayoutComponentBase
<div class="app-shell">
    <header class="app-header">
        <a class="brand" href="/">Product Catalog</a>
    </header>
    <aside class="app-navigation">
        <NavMenu />
    </aside>
    <main class="app-content" id="main-content">
        @Body
    </main>
</div>

```

The page supplies the body, while the layout owns repeated structure such as navigation, headers, status areas, and footers. A default layout can be assigned by `RouteView` or through an `@layout` directive in `_Imports.razor`. A specific page can override it:

```
@layout AdminLayout
```

Layouts can be nested. An administration layout may itself use `MainLayout` while adding a secondary menu:

```

@inherits LayoutComponentBase
@layout MainLayout
<div class="admin-shell">
    <nav aria-label="Administration">
        <NavLink href="/admin/products">Products</NavLink>
        <NavLink href="/admin/audit">Audit</NavLink>
    </nav>
    <section>
        @Body
    </section>
</div>

```

Keep layouts structural. They should not become global controllers containing page-specific loading, editing, and validation state. Shared user information or notification hosts may belong there, but feature behaviour should remain in the relevant page or service. `PageTitle` changes the browser tab title, while `HeadContent` contributes page-specific metadata:

```

<PageTitle>@Product.Name</PageTitle>
<HeadContent>
    <meta
        name="description"
        content="@($"View details for {Product.Name}.")"
    />
</HeadContent>

```

The document title should identify the current page rather than remaining the same throughout the application. The Product Catalog should work on a desktop monitor and a narrow phone without maintaining two separate UIs. The page structure can remain the same while CSS changes the arrangement.

```
.app-shell {
    display: grid;
    grid-template:
        "header header" auto
        "navigation content" 1fr
        / 16rem minmax(0, 1fr);
    min-height: 100vh;
}
.app-header {
    grid-area: header;
}
.app-navigation {
    grid-area: navigation;
}
.app-content {
    grid-area: content;
    min-width: 0;
    padding: 1.5rem;
}
@media (max-width: 48rem) {
    .app-shell {
        grid-template:
            "header" auto
            "navigation" auto
            "content" 1fr
            / 1fr;
    }
}
```

Responsive design is not mainly about adding many breakpoints. Start with content that can shrink and wrap, use relative units, avoid unnecessary fixed widths, and add a breakpoint only where the arrangement no longer works. `minmax(0, 1fr)` and `min-width: 0` often matter in grid and flex layouts because they allow content to shrink rather than overflow. A product grid can adapt naturally:

```
.product-grid {
    display: grid;
    grid-template-columns: repeat(
        auto-fill,
        minmax(min(100%, 16rem), 1fr)
    );
    gap: 1rem;
}
```

The browser chooses as many columns as fit. No C# code needs to measure the window and decide how many cards to render. Use CSS for layout and visual adaptation. Use component state only when the application's behaviour genuinely changes, such as opening a modal navigation drawer or selecting a compact data representation.

8.3 Forms and Typed Inputs

A plain HTML form can post named values to a server, but Blazor's `EditForm` adds a model-aware editing context, input components, validation tracking, and submission callbacks. A Product Catalog editor needs a dedicated form model:

```
public sealed class ProductEditModel
{
    public int Id { get; set; }
    [Required]
```

```

[StringLength(120)]
public string Name { get; set; } = "";
[StringLength(1000)]
public string Description { get; set; } = "";
[Range(0, 1_000_000)]
public decimal Price { get; set; }
[Required]
[StringLength(3, MinimumLength = 3)]
public string Currency { get; set; } = "EUR";
public bool IsAvailable { get; set; }
}

```

The editor binds the model to `EditForm`:

```

<EditForm
    Model="_model"
    OnValidSubmit="SaveAsync"
    FormName="product-editor"
>
    <DataAnnotationsValidator />
    <div class="field">
        <label for="product-name">Name</label>
        <InputText
            id="product-name"
            @bind-Value="_model.Name"
        />
        <ValidationMessage For="() => _model.Name" />
    </div>
    <div class="field">
        <label for="product-price">Price</label>
        <InputNumber
            id="product-price"
            @bind-Value="_model.Price"
        />
        <ValidationMessage For="() => _model.Price" />
    </div>
    <div class="field">
        <label for="product-currency">Currency</label>
        <InputText
            id="product-currency"
            @bind-Value="_model.Currency"
        />
        <ValidationMessage For="() => _model.Currency" />
    </div>
    <label class="checkbox-field">
        <InputCheckbox @bind-Value="_model.IsAvailable" />
        Available
    </label>
    <button type="submit" disabled="_saving">Save</button>
</EditForm>

```

`EditForm` creates an `EditContext` when it receives a model. The context tracks field changes, validation messages, and form state. Built-in inputs integrate with that context, while ordinary HTML inputs do not automatically provide the same field tracking. Give forms unique names. Form names allow Blazor to identify which form was submitted, especially when static server rendering participates in form handling. They also make the form boundary explicit when several forms appear on one page. Do not bind the form directly to an entity already displayed elsewhere and mutate it while the user types. Create an edit model or copy. The existing product remains unchanged until Save succeeds, and Cancel can discard the temporary values without reconstructing the original object. Blazor provides input components for common types:

```

InputText      -> string
InputTextArea  -> multiline string

```

```

InputNumber      -> numeric values
InputDate        -> date values
InputCheckbox    -> bool
InputSelect      -> selection
InputFile        -> browser-selected files

```

They bind through `@bind-Value` and report field changes to the surrounding `EditContext`. `InputSelect` can render options from application data:

```

<InputSelect
    id="product-category"
    @bind-Value="_model.CategoryId"
>
    <option value="">Choose a category</option>
    @foreach (var category in _categories)
    {
        <option value="@category.Id">@category.Name</option>
    }
</InputSelect>

```

An input's HTML type still affects browser behaviour. `InputText` renders a text input by default, but ordinary attributes can improve mobile keyboards and autofill:

```

<InputText
    id="product-code"
    @bind-Value="_model.Code"
    autocomplete="off"
    inputmode="text"
    spellcheck="false"
/>

```

Use the native element or built-in component that best matches the value. A custom JavaScript date picker is not automatically better than a normal date input. Native controls often provide good keyboard, touch, and accessibility behaviour with no additional code. Custom input components can derive from `InputBase<TValue>` when an application repeatedly needs a domain-specific editor that participates in validation. Do not create a wrapper around every built-in input merely to standardise one CSS class; ordinary attributes or a small field component may be clearer.

8.4 Validation, Saving, and Unsaved Changes

`DataAnnotationsValidator` reads validation attributes from the form model. `ValidationMessage` displays messages for one field, while `ValidationSummary` displays all current messages:

```

<DataAnnotationsValidator />
<ValidationSummary />

```

Data annotations are appropriate for rules that can be evaluated from the submitted model: required text, length, numeric ranges, and simple cross-property conditions. They are not sufficient for rules that depend on current server state, permissions, or database contents. The client may validate that a product name is not empty, but only the server can reliably decide whether the user may edit the product or whether a unique code is already in use. A useful division is:

```

Input conversion
    -> Can text become the target .NET value?
Form validation
    -> Does the edit model satisfy local input rules?
Application validation
    -> Is the operation valid in current business state?
Authorization
    -> May this user perform it?

```

All relevant rules must still run on the server. Browser-side validation improves feedback but is not a trust boundary. A client can disable scripts, alter a request, or call the API directly. `OnValidSubmit` runs only when the current form validation succeeds. `OnInvalidSubmit` runs when it fails. `OnSubmit` runs for every submission and gives the handler an `EditContext`; it is useful when the component wants to call `Validate()` explicitly or

coordinate several validation stages. Use `OnSubmit` or the `valid/invalid` pair, not both styles on the same form. The `Model` parameter is convenient, but an explicit `EditContext` provides more control:

```
<EditForm
    EditContext="_editContext"
    OnValidSubmit="SaveAsync"
    FormName="product-editor"
>
    <DataAnnotationsValidator />
    ...
</EditForm>

private readonly ProductEditModel _model = new();
private EditContext _editContext = default!;
protected override void OnInitialized()
{
    _editContext = new EditContext(_model);
}
```

The context exposes events such as `OnFieldChanged`, `OnValidationRequested`, and `OnValidationStateChanged`. It can answer whether a field or form has been modified and provides field identifiers for validation. Subscribe only when the interface needs the behaviour. A basic editor does not need to inspect every keystroke. A page that warns about unsaved changes can check modification state, while a custom validator can respond to validation requests. Subscriptions must be removed when the component is disposed. `ValidationMessageStore` adds and removes messages that do not come from data annotations. Suppose the API rejects a product code because another product already uses it. The editor can attach that server error to the correct field:

```
private ValidationMessageStore _messages = default!;
protected override void OnInitialized()
{
    _editContext = new EditContext(_model);
    _messages = new ValidationMessageStore(_editContext);
}

private void ApplyServerErrors(IReadOnlyDictionary<string, string[]> errors)
{
    _messages.Clear();
    foreach (var (fieldName, messages) in errors)
    {
        var field = _editContext.Field(fieldName);
        _messages.Add(field, messages);
    }
    _editContext.NotifyValidationStateChanged();
}
```

The field names returned by the server should be part of a deliberate contract. Do not expose internal entity paths accidentally and expect the component to understand them. A form submission has more than two states. It may be idle, validating, saving, successful, or failed. The component should make those states visible and prevent accidental duplicate submission.

```
private bool _saving;
private string? _saveError;
private async Task SaveAsync()
{
    if (_saving) return;
    _saving = true;
    _saveError = null;
    _messages.Clear();
    try
    {
        var result = await ProductCatalog.UpdateAsync(
            _model.Id,
```

```

        new UpdateProductRequest(
            _model.Name,
            _model.Description,
            _model.Price,
            _model.Currency,
            _model.IsAvailable
        ),
        _componentCancellation.Token
    );
    if (result.ValidationErrors is not null)
    {
        ApplyServerErrors(result.ValidationErrors);
        return;
    }
    if (!result.IsSuccess)
    {
        _saveError = result.Error;
        return;
    }
    await SaveCompleted.InvokeAsync(result.Value);
}
catch (OperationCanceledException)
    when (_componentCancellation.IsCancellationRequested)
{
}
catch (Exception exception)
{
    Logger.LogError(
        exception,
        "Failed to save product {ProductId}",
        _model.Id
    );
    _saveError = "The product could not be saved.";
}
finally
{
    _saving = false;
}
}

```

The Save button can show the current operation:

```

<button type="submit" disabled="_saving">
    @(_saving ? "Saving..." : "Save")
</button>

```

Disabling the button is useful feedback, but the server must still handle repeated requests safely where duplication matters. Client UI is never the final enforcement layer. After a successful save, update the page's state with the returned product rather than assuming that the submitted model is the complete result. The server may normalise values, assign identifiers, calculate fields, or apply concurrency rules. A Cancel button should normally discard the edit model and return to the previous state:

```

<button
    type="button"
    disabled="_saving"
    @onclick="Cancel"
>
    Cancel
</button>

```

The explicit `type="button"` prevents the browser from treating it as a submit button. If navigation would discard meaningful changes, the application can ask for confirmation. Blazor provides navigation interception

mechanisms, but they should be used selectively. Interrupting every route change after any touched field quickly becomes irritating. Warn only when the user would lose work that cannot be reconstructed easily. The browser's own unload behaviour is different from internal navigation. A full refresh, closing a tab, or leaving the site may require browser-level handling. Such prompts are deliberately restricted by browsers and should not be relied on as the only protection. Autosave, drafts, and small reversible operations often provide a better experience than frequent modal warnings.

The form model should have one clear owner. When the editor closes, that owner discards it. A shared product instance should not remain half-edited because several components held references to the same mutable object. A common mistake is to treat client validation as complete and server validation as a rare fallback. In a web application, the server is authoritative. The useful flow is:

User edits form

- > Blazor gives immediate local feedback
- > Valid form is submitted
- > Server validates again
- > Server applies current business rules
- > Server returns success or structured errors
- > Component displays those errors

Some rules may run in both places because they improve responsiveness. Duplication is acceptable when the two copies serve different trust boundaries, but the shared rule should be represented carefully so that messages and limits do not drift. A shared contract assembly can contain data annotations used by both interactive clients and the server, while the server still performs authorization and database-dependent validation. Do not display raw exception messages from the server. Return stable, user-safe validation and problem details. Log the internal exception with a trace identifier and show the user a message that helps them recover.

8.5 Loading, Empty, Error, and Content States

A page should not render one blank area while data loads and then another blank area when no products exist. Those states mean different things:

```
@if (_loading)
{
    <ProductListSkeleton Count="6" />
}
else if (_loadError is not null)
{
    <StatusMessage Level="MessageLevel.Error">
        <p>@_loadError</p>
        <button type="button" @onclick="LoadAsync">Try again</button>
    </StatusMessage>
}
else if (_products.Count == 0)
{
    <EmptyState
        Title="No products found"
        Description="Change the filters or create the first product."
    />
}
else
{
    <ProductGrid Products="_products" />
}
```

A loading state tells the user to wait. An empty state explains that the request succeeded but returned no items. An error state explains that the operation failed and may offer a recovery action. Content is the successful result. Keep existing content visible during small refreshes when possible. Replacing the entire page with a spinner after every filter change causes unnecessary movement and removes context. A subtle busy indicator or disabled control can show that an update is in progress while the old result remains visible. Skeletons should roughly match the eventual layout and should not animate aggressively. A plain text loading message is better than a decorative skeleton that makes the page harder to read.

8.6 Accessibility, Keyboard Use, and Focus

Accessible interfaces are easier to build when the markup uses elements for their intended purpose:

Navigation	-> <code>nav</code> and <code>links</code>
Main content	-> <code>main</code>
Action	-> <code>button</code>
Navigation	-> <code>anchor</code>
Field name	-> <code>label</code>
Status	-> <code>role="status"</code>
Error	-> <code>role="alert"</code> when immediate interruption is justified
Grouped fields	-> <code>fieldset</code> and <code>legend</code>
Table data	-> <code>table</code> , <code>thead</code> , <code>tbody</code> , <code>th</code> , <code>td</code>

Native elements already provide keyboard behaviour, semantics, focus rules, and browser integration. Recreating a button with a `div` requires the application to rebuild all of that and usually misses part of it. Every input needs an accessible name. The most reliable form is an explicit label:

```
<label for="product-name">Name</label>
<InputText
  id="product-name"
  @bind-Value="_model.Name"
/>
```

Placeholder text is not a replacement for a label. It disappears while typing, often has weak contrast, and does not consistently explain the field. Use headings in a logical hierarchy. Each page should normally have one clear `h1`, with sections below it using `h2` and `h3`. Do not choose heading levels for font size; use CSS for appearance. `aria-*` attributes are useful when native HTML cannot express the relationship, but they do not repair incorrect semantics automatically. Prefer a real button over `role="button"` and a real label over an `aria-label` added to an unrelated wrapper. Every interactive control should be reachable and usable with a keyboard. Native links, buttons, inputs, and selects already support this. Custom menus, dialogs, grids, and composite widgets require deliberate focus behaviour and are easy to get wrong.

Do not remove the focus outline without providing a visible replacement:

```
:focus-visible {
  outline: 0.2rem solid currentColor;
  outline-offset: 0.2rem;
}
```

After navigation, focus should move to the new page's main heading or content region. `FocusOnNavigate` handles this for normal routed pages. When a dialog opens, focus should move into it, keyboard interaction should remain within the modal context, and closing it should restore focus to the control that opened it. A robust dialog implementation is more than a `div` positioned in the centre of the screen. Prefer the native `<dialog>` element where it fits or a well-tested component rather than inventing modal behaviour repeatedly. After a failed form submission, users should be able to discover the errors without searching the page. A validation summary near the start of the form, field messages beside inputs, and sensible focus movement all help. Avoid automatically moving focus on every minor validation change; that can make typing difficult.

Dynamic status updates can use a live region:

```
<p role="status" aria-live="polite">
  @_statusMessage
</p>
```

Use `polite` for ordinary progress and success messages. Reserve `role="alert"` or assertive announcements for failures that need immediate attention. Information should not depend on colour alone. An invalid field can use colour together with an icon, message, or text label. A selected product can use shape, position, text, or border treatment in addition to a background colour. Text and controls need sufficient contrast. Touch targets should be large enough to select comfortably. Content should remain usable at increased browser zoom and with larger system text. Fixed pixel heights often break when text wraps, while padding and minimum sizes adapt better. Respect reduced-motion preferences for nonessential animation:

```
@media (prefers-reduced-motion: reduce) {
  * {
    scroll-behavior: auto !important;
  }
}
```

```

        animation-duration: 0.01ms !important;
        animation-iteration-count: 1 !important;
        transition-duration: 0.01ms !important;
    }
}

```

Do not hide essential progress behind animation that disappears entirely; provide text or another persistent cue. Accessibility should be tested, not inferred from markup alone. Navigate the application with only a keyboard, inspect the accessibility tree in developer tools, run an automated checker, and test at least one screen reader for important workflows. Automated tools find only part of the problem, but they catch many basic mistakes quickly.

8.7 JavaScript Interoperability and DOM Ownership

Blazor covers component rendering and common events, but the browser exposes APIs and libraries that remain JavaScript-based. JavaScript interoperability lets C# call JavaScript and JavaScript call .NET. Use it for capabilities such as:

- focusing or measuring a DOM element when Blazor has no direct abstraction;
- clipboard access;
- charts, maps, editors, and other established JavaScript libraries;
- browser observers and specialised media APIs;
- integrating an existing web component.

Do not use JavaScript to edit DOM elements that Blazor owns unless the integration is specifically designed for it. Blazor maintains its own representation of the rendered UI. External changes can make the DOM disagree with that representation, causing lost changes or undefined behaviour. Prefer JavaScript modules over global functions. A colocated module can sit beside the component:

ProductEditor.razor
ProductEditor.razor.js

ProductEditor.razor.js:

```

export function focus(element) {
    element.focus();
}
export async function copy(text) {
    await navigator.clipboard.writeText(text);
}

```

The component imports the module after the element exists:

```

@implements IAsyncDisposable
@Inject IJSRuntime JS
<InputText
    @ref="_nameInput"
    @bind-Value="_model.Name"
/>
@code {
    private ElementReference _nameInput;
    private IJSObjectReference? _module;
    protected override async Task OnAfterRenderAsync(bool firstRender)
    {
        if (!firstRender) return;
        _module = await JS.InvokeAsync<IJSObjectReference>(
            "import",
            "../Components/Products/ProductEditor.razor.js"
        );
        await _module.InvokeVoidAsync("focus", _nameInput);
    }
    public async ValueTask DisposeAsync()
    {

```

```

        if (_module is null) return;
        try
        {
            await _module.DisposeAsync();
        }
        catch (JSDisconnectedException)
        {
        }
    }
}

```

Interop calls are asynchronous so that the same component model works when calls must cross a server connection. In interactive server rendering, the connection may disappear before disposal, so releasing a module can throw `JSDisconnectedException`. The component does not need to report that expected disconnection as an application failure. Keep the interop boundary small and typed. A component should call `copy(text)` rather than send a large object graph to a generic global dispatcher. Repeated tiny interop calls can also be expensive, particularly when each one crosses the server connection. Combine related work into one JavaScript operation when it naturally belongs together. A JavaScript widget often owns a section of DOM after initialization. Give it an element boundary that Blazor does not independently rewrite:

```
<div @ref="_chartHost"></div>
```

The component can pass data to the library and dispose the widget when the component disappears. Avoid rendering Blazor children into the same subtree that the JavaScript library mutates freely. The ownership model should be explicit:

```

Blazor owns wrapper and component lifecycle
-> JavaScript library owns internal chart DOM
-> Component sends data and commands
-> JavaScript reports selected events

```

If a library exposes many low-level events and methods, wrap it in one focused component rather than spreading interop calls across pages. That wrapper can translate JavaScript data into application-specific parameters and callbacks. DOM cleanup sometimes needs to happen in JavaScript because the element may already be removed when the .NET component is disposing. Browser observers or the library's own lifecycle hooks can handle that case. The exact cleanup belongs to the side that owns the DOM object.

8.8 Performance and Virtualization

Most business interfaces do not need manual rendering optimisations. Network calls, database queries, oversized images, unnecessary reloads, and large data sets usually matter before the cost of one ordinary component render. A practical order is:

```

Measure
-> Avoid unnecessary network and database work
-> Send only needed data
-> Keep state updates focused
-> Render only useful UI
-> Optimise proven component hot spots

```

Do not add `ShouldRender`, manual render fragments, or custom diff logic based on intuition alone. Browser performance tools, application metrics, and traces can show whether the delay comes from event handling, rendering, network transfer, or server work. The UI should not fetch the same data independently in several child components when the page can load it once and pass it down. It should not reload the full product collection after changing one product when the returned product can replace one item in state. It should not send thousands of records when the page shows twenty.

In interactive server rendering, large render batches and frequent events also cross the network. A text box bound on every `oninput` event can send a server event for every keystroke. That may be correct for local filtering, but a debounced search or normal `change` event may provide a better trade-off when every input triggers remote work. Paging is often the best design for server data because it limits the query, response, state, and rendered elements together. When the interface needs continuous scrolling through a large collection, `Virtualize<TItem>` renders only the currently visible region and a buffer around it. For an in-memory collection:

```
<Virtualize Items="_products" Context="product">
  <ProductRow Product="product" />
</Virtualize>
```

For server-backed data, an items provider can load ranges on demand:

```
<Virtualize
  Context="product"
  ItemsProvider="LoadProductsAsync"
  ItemSize="72"
>
  <ProductRow Product="product" />
</Virtualize>

private async ValueTask<ItemsProviderResult<ProductSummary>>
  LoadProductsAsync(ItemsProviderRequest request)
{
  var page = await ProductCatalog.ListAsync(request.StartIndex, request.Count, _filter,
  request.CancellationToken);
  return new ItemsProviderResult<ProductSummary>(page.Items, page.TotalCount);
}
```

Virtualization improves rendering by limiting visible elements, but it does not automatically make a poor data query efficient. The provider still needs indexed filtering, stable ordering, cancellation, and a bounded result. Item heights should be reasonably predictable so that scroll calculations remain accurate. A grid of a few dozen products does not need virtualization. Add it when the collection is large enough that rendering and DOM size are measurable problems.

8.9 Reusable Components Without Premature Design Systems

Consistency matters, but wrapping every HTML element in a custom component can make the interface harder to understand. A useful shared component owns meaningful behaviour or repeated structure:

```
StatusMessage
  -> consistent severity and accessibility
ConfirmDialog
  -> focus management and result
PagedTable
  -> sorting, paging, empty and loading states
ProductEditor
  -> product-specific form workflow
```

A component that only turns `<button class="primary">` into `<PrimaryButton>` may add little unless it also standardises states, icons, accessibility, and application conventions. CSS classes and native elements are often enough. The same restraint applies to third-party UI libraries. They can save time for complex grids, charts, date pickers, and accessible dialogs, but they also shape markup, styling, bundle size, and upgrade work. Adopt one because it solves recurring requirements, not because an empty application feels unfinished without a component suite.

8.10 A Complete Product Workspace

The Product Catalog can now combine routing, layout, filters, form editing, and state updates.

```
MainLayout
  -> NavMenu
  -> ProductsPage
    -> ProductToolbar
    -> ProductGrid
      -> ProductCard
    -> ProductEditor
    -> StatusMessage
```

`ProductsPage` owns the current collection, filter state derived from the URL, loading state, and selected product. `ProductCard` raises edit requests. `ProductEditor` owns a temporary form model, local validation, server

validation messages, and save progress. The application service remains responsible for authoritative validation and persistence. A successful edit follows this path:

```
User opens /products/42/edit
-> Router selects ProductEditPage
-> Page loads product 42
-> ProductEditor copies product into edit model
-> User changes fields
-> EditContext tracks modifications
-> Data annotations provide immediate validation
-> User submits
-> Server validates and updates product
-> Page receives returned ProductSummary
-> Navigation moves to /products/42
-> Product details render with updated state
```

The user can refresh or share the address because the selected product is represented by the route. The edit model is temporary because unfinished form input belongs to the current editor instance. The product itself remains authoritative on the server. An alternative workspace can keep the list and editor on one page. That is equally valid when the workflow benefits from side-by-side editing. The choice should come from user interaction, not from a rule that every entity requires separate list, details, create, and edit pages.

8.11 Common Mistakes and Key Ideas

A complete interface should preserve navigational state in routes or query values when refresh, bookmarking, or sharing matters; ordinary links remain preferable to programmatic navigation for ordinary destinations. Keep layouts structural, use temporary form models instead of mutating shared objects while the user types, and let server validation remain authoritative. Loading, empty, error, forbidden, conflict, and success are different states and should not collapse into one spinner or generic message. Native HTML supplies the best starting point for labels, keyboard behaviour, focus, and semantics. Do not substitute placeholders for labels, colour for meaning, or ARIA attributes for correct elements. JavaScript interop should expose a narrow browser capability or library boundary, with explicit DOM ownership and disposal, rather than becoming a second rendering system inside Blazor markup.

Performance work should follow evidence: bound queries and page sizes, avoid unnecessary reloads and interop calls, and virtualize only collections large enough to justify it. **EditForm**, typed inputs, **EditContext**, validation stores, responsive CSS, and focused reusable components are tools for a coherent workflow, not reasons to build a design system before the application has repeated needs.

Part IV — Data

9. Relational Data and SQL

The Product Catalog now has routes, components, forms, validation, and application services, but its data still lives in process memory. Restarting the server removes every change, a second server instance sees a different collection, and concurrent operations depend on locks inside one process. A real application needs durable shared storage. For ordinary business systems, the default choice is usually a relational database.

A relational database stores data in tables, but that description is too shallow to be useful by itself. Its real value comes from a combination of structure, constraints, queries, transactions, and controlled concurrency. The database does not merely remember objects. It enforces relationships between pieces of data, searches large collections efficiently, coordinates simultaneous users, and preserves committed changes even when the application process stops. Entity Framework Core will later translate C# operations into database commands, but it cannot remove the need to understand what those commands mean. Without a basic SQL and relational model, EF Core behaviour can look magical when it works and arbitrary when it does not.

This chapter introduces the relational model directly. It uses SQL examples close to what EF Core will eventually generate, but it does not try to teach every feature of a specific database product. PostgreSQL, SQL Server, SQLite, MySQL, and other relational systems differ in syntax and capabilities, yet they share the same central ideas.

9.1 Tables, Types, and Nullability

A table represents one kind of stored fact. A **Products** table might contain one row per product and one column per stored attribute:

Id	Name	Price	Currency	IsAvailable
1	Mechanical Keyboard	129.00	EUR	true
2	Wireless Mouse	59.00	EUR	true
3	USB-C Dock	189.00	EUR	false

The table is not a visual spreadsheet even though tools often display it as one. Its columns have declared names and types, rows follow that structure, and the database can enforce rules that ordinary spreadsheet cells do not. **Id** may be an integer that cannot be null. **Name** may have a maximum length. **Price** may use an exact decimal type. **Currency** may require three characters. **IsAvailable** may be a Boolean with a default value. A simplified table definition is:

```
CREATE TABLE Products (  
    Id INTEGER PRIMARY KEY,  
    Name VARCHAR(120) NOT NULL,  
    Description VARCHAR(1000) NOT NULL,  
    Price DECIMAL(12, 2) NOT NULL,  
    Currency CHAR(3) NOT NULL,  
    IsAvailable BOOLEAN NOT NULL  
);
```

The exact type names vary by provider, but the design questions do not. How large can a value become? Is it optional? Does it need exact arithmetic? Must it be unique? What should happen when related data is deleted? These are data-model decisions, not merely persistence syntax. A row usually represents current stored state, while an application object represents data and behaviour inside one process. They may look similar, but they are not the same thing. A C# property can calculate a value, call a service, or hold an in-memory reference. A database column stores a value that survives independently of the process. Methods, delegates, interfaces, and runtime object identity do not become table columns.

The database type controls storage, comparison, valid values, and the operations available to queries. Choosing a type only because it can hold the current example often creates problems later. Identifiers are commonly integers, larger integers, GUIDs, or database-generated values. Text types differ between fixed and variable length. Numbers may be exact, such as **DECIMAL**, or approximate, such as floating-point types. Dates and times may store only a date, a local timestamp, or an instant with offset information. Binary columns store bytes. Some databases also support JSON, arrays, spatial data, and domain-specific types. Prices should normally use an exact decimal type:

```
Price DECIMAL(12, 2) NOT NULL
```

A floating-point type can represent a large range efficiently, but many decimal fractions cannot be represented exactly in binary. That is acceptable for scientific measurements and graphics, but financial values usually need predictable decimal rounding. The corresponding C# type is normally `decimal`. Text length also matters. `VARCHAR(120)` documents and enforces a limit. An unlimited text type may be suitable for descriptions, but it should not be chosen automatically for every string. Indexing, memory, validation, and provider behaviour can differ for very large columns. Date and time values require a clear meaning. A product creation time that represents one global instant should not be confused with a date-only value such as a release date. The database and application should agree whether a stored timestamp is UTC, includes an offset, or intentionally represents local wall-clock time.

A nullable column can contain `NULL`, which means that no value is present. It is not the same as an empty string, zero, `false`, or a default date.

```
Description VARCHAR(1000) NULL
```

A query cannot compare null with ordinary equality:

```
SELECT *  
FROM Products  
WHERE Description IS NULL;
```

This does not work as intended:

```
WHERE Description = NULL
```

SQL uses three-valued logic: a comparison may be true, false, or unknown. Most comparisons involving null are unknown, and rows for which a `WHERE` condition is not true are filtered out. This matters when translating LINQ, designing optional relationships, and interpreting aggregate results. Use null when the absence of a value has a real meaning. Do not make every column nullable merely to simplify inserts. Required data should be `NOT NULL`, which lets the database reject invalid rows even when a bug, script, or another application bypasses normal C# validation. An empty description and an unknown description may or may not mean the same thing in the Product Catalog. The model should decide rather than leaving every caller to guess.

9.2 Keys and Constraints

A primary key uniquely identifies each row. In `Products`, `Id` serves that purpose:

```
Id INTEGER PRIMARY KEY
```

The key must be unique and cannot be null. Application code can use it to retrieve, update, delete, or reference one product without depending on mutable values such as its name. A database may generate integer keys:

```
INSERT INTO Products (Name, Description, Price, Currency, IsAvailable)  
VALUES ('Mechanical Keyboard', '', 129.00, 'EUR', true);
```

The generated syntax differs by database. SQL Server uses identity columns, PostgreSQL commonly uses identity or sequences, and SQLite can generate integer row identifiers. EF Core hides some provider differences, but the database still owns the generation mechanism. GUIDs can be generated before insertion and are useful when identifiers must be created across disconnected systems, yet they are larger and can produce less index-friendly insertion patterns. Integer keys are compact and efficient but usually require database coordination. Neither is universally better.

A natural key uses an existing business value, such as a country code or immutable product code. It can be appropriate when the value is truly stable, short, and unique. Product names are poor keys because they change and may not be unique. Many systems therefore use a surrogate primary key and add a separate unique constraint for the business identifier:

```
CREATE UNIQUE INDEX UX_Products_Code ON Products(Code);
```

The primary key provides technical identity; the unique code enforces a business rule. Application validation improves messages and user experience, but database constraints are the final defence for stored data. They

apply regardless of whether the write comes from ASP.NET Core, a background job, an administrative script, or another service. Important constraints include:

PRIMARY KEY -> row identity is unique
FOREIGN KEY -> referenced row must exist
NOT NULL -> value is required
UNIQUE -> duplicate values are rejected
CHECK -> value must satisfy a condition
DEFAULT -> omitted value receives a database default

For example:

```
CREATE TABLE Products (  
    Id INTEGER PRIMARY KEY,  
    Code VARCHAR(40) NOT NULL,  
    Name VARCHAR(120) NOT NULL,  
    Price DECIMAL(12, 2) NOT NULL,  
    Currency CHAR(3) NOT NULL,  
    IsAvailable BOOLEAN NOT NULL DEFAULT true,  
    CONSTRAINT UX_Products_Code UNIQUE (Code),  
    CONSTRAINT CK_Products_Price CHECK (Price >= 0),  
    CONSTRAINT CK_Products_Currency CHECK (CHAR_LENGTH(Currency) = 3)  
);
```

A check constraint does not replace application rules, but it prevents impossible persisted states. The application may explain that price must not be negative; the database ensures that no writer stores -20. Database errors should be translated carefully. A unique constraint failure is expected in some operations, but exposing the raw provider message to the user leaks implementation details and often produces poor text. The application should detect the relevant failure and return a stable result such as “A product with this code already exists.”

9.3 Relationships, Delete Behaviour, and Normalisation

Data becomes relational when one table refers to another. Suppose products belong to categories:

```
CREATE TABLE Categories (  
    Id INTEGER PRIMARY KEY,  
    Name VARCHAR(100) NOT NULL UNIQUE  
);  
ALTER TABLE Products  
ADD CategoryId INTEGER NULL;
```

CategoryId becomes a foreign key:

```
ALTER TABLE Products  
ADD CONSTRAINT FK_Products_Categories  
FOREIGN KEY (CategoryId)  
REFERENCES Categories(Id);
```

The database now prevents a product from referring to a category that does not exist. This is a one-to-many relationship: one category can contain many products, while each product has at most one category.

Category 1

-> Product A
-> Product B
-> Product C

The foreign key is stored on the many side because each product needs one category identifier. If every product must have a category, `CategoryId` should be `NOT NULL`. If uncategorised products are valid, it remains nullable. A one-to-one relationship uses a foreign key that is also unique. For example, if each product has at most one extended technical specification row:

```
CREATE TABLE ProductSpecifications (  
    ProductId INTEGER PRIMARY KEY,  
    WeightGrams INTEGER NULL,  
    WidthMillimetres DECIMAL(8, 2) NULL,
```

```

    CONSTRAINT FK_ProductSpecifications_Products
    FOREIGN KEY (ProductId) REFERENCES Products(Id)
);

```

The primary key both identifies the specification and ensures that no product receives two specification rows. Many-to-many relationships require a joining table. A product may have many tags, and each tag may belong to many products:

```

CREATE TABLE Tags (
    Id INTEGER PRIMARY KEY,
    Name VARCHAR(60) NOT NULL UNIQUE
);
CREATE TABLE ProductTags (
    ProductId INTEGER NOT NULL,
    TagId INTEGER NOT NULL,
    PRIMARY KEY (ProductId, TagId),
    FOREIGN KEY (ProductId) REFERENCES Products(Id),
    FOREIGN KEY (TagId) REFERENCES Tags(Id)
);

```

The combined primary key prevents the same tag from being attached to one product twice. The joining table can also contain relationship data. If products and suppliers have a many-to-many relationship, the join row might store supplier product code, lead time, or purchase price. At that point it is better understood as its own entity than as an invisible link. A foreign key raises a practical question: what should happen to products when a category is deleted? Common choices are:

```

RESTRICT / NO ACTION -> reject deletion while products still reference it
CASCADE               -> delete dependent rows automatically
SET NULL              -> keep products but clear CategoryId

```

None is universally correct. Deleting a category may reasonably leave products uncategorised, so **SET NULL** could fit. Deleting a product should perhaps delete its **ProductTags** rows, making cascade behaviour useful there. Deleting a customer should not automatically erase invoices required for accounting. The rule must follow the business meaning of the relationship. Cascades can simplify cleanup but can also delete far more data than the caller expects. They should be visible in the model and covered by tests. Application code should not assume that deleting one row affects only one table.

Soft deletion is another option: instead of removing the row, the application sets **DeletedAt** or **IsDeleted**. This preserves history but complicates every query, unique constraint, relationship, and index. Soft deletion should solve a real recovery or audit requirement, not be added to every table by habit. A poorly designed table often repeats related data:

ProductId	ProductName	CategoryName	CategoryDescription
1	Keyboard	Accessories	Computer accessories
2	Mouse	Accessories	Computer accessories
3	Dock	Accessories	Computer accessories

If the category description changes, every matching product row must change. If one row is missed, the database contains conflicting versions of the same category. A normalised design stores each category once and places its key on products:

```

Categories
  Id | Name | Description
Products
  Id | Name | CategoryId

```

Normalisation aims to store each independent fact in one appropriate place. It reduces duplication and update anomalies, but it is not a goal of splitting data into the maximum possible number of tables. A product name and price belong naturally to the product. A currency code might remain a short value on the product, or it might reference a **Currencies** table if the system needs metadata, validation, exchange rules, or relationships around currencies. A useful question is: if this value changes, should one row change or many unrelated rows?

Another is: can this concept exist independently and be referenced by several rows? These questions often reveal whether the value deserves its own table.

Denormalisation deliberately duplicates or precomputes data for performance or reporting. It can be valid, but it introduces synchronisation work. Begin with a clear normalised model, measure the real query problem, and denormalise only when the benefit is worth the extra consistency burden.

9.4 Reading, Filtering, and Parameters

SQL is declarative. A query describes the result, and the database chooses an execution plan. A basic read is:

```
SELECT Id, Name, Price, Currency
FROM Products;
```

Selecting explicit columns is usually better than `SELECT *` in application queries. It documents the required data, reduces transfer, and avoids silently changing the result shape when the table gains columns. Filtering uses `WHERE`:

```
SELECT Id, Name, Price, Currency
FROM Products
WHERE IsAvailable = true
    AND Price <= 150.00;
```

Ordering uses `ORDER BY`:

```
SELECT Id, Name, Price
FROM Products
WHERE IsAvailable = true
ORDER BY Price ASC, Name ASC;
```

Without an `ORDER BY`, row order is not guaranteed. A query may appear stable during development and change after an index, provider update, or different execution plan. Any operation that depends on order must state it. Limiting and paging syntax differs between databases. A common form is:

```
SELECT Id, Name, Price
FROM Products
ORDER BY Id
LIMIT 20 OFFSET 40;
```

This asks for twenty rows after skipping forty. SQL Server typically expresses the same idea with `OFFSET` and `FETCH`. EF Core translates LINQ `Skip` and `Take` into provider-specific syntax. Large offsets can become expensive because the database still has to locate and skip earlier rows. Keyset pagination is often better for deep or continuous paging:

```
SELECT Id, Name, Price
FROM Products
WHERE Id > 240
ORDER BY Id
LIMIT 20;
```

The client remembers the last key rather than a page number. It is fast and stable for forward navigation when ordering uses a suitable unique key, though it does not support arbitrary page jumps as naturally. A query predicate should compare columns with values rather than build SQL text from user input. This is unsafe:

```
var sql = $"SELECT * FROM Products WHERE Name = '{search}'";
```

A value such as `' OR 1=1 --` can change the command's meaning. This is SQL injection. Correct code uses parameters:

```
SELECT Id, Name, Price
FROM Products
WHERE Name = @name;
```

The application sends the SQL command and parameter value separately. The database treats the value as data, not executable SQL. EF Core parameterises ordinary LINQ queries automatically, and raw SQL APIs

provide parameter-safe forms. String interpolation is safe only when the API explicitly converts interpolated values into parameters. Constructing raw command text remains unsafe. Parameters also improve plan reuse and type handling. They are not only a security feature. Dynamic query structure, such as a selectable sort column, cannot always be parameterised in the same way and must be chosen from a controlled allow-list rather than copied directly from request text. Search predicates have performance consequences. An exact comparison on an indexed code can be efficient:

```
WHERE Code = @code
```

A leading-wildcard search usually cannot use a normal index effectively:

```
WHERE Name LIKE '%keyboard%'
```

Full-text search, specialised indexes, or an external search service may be more appropriate when search requirements become complex.

9.5 Joins and Aggregates

A foreign key stores the relationship, while a join retrieves related rows. To list products with category names:

```
SELECT
    p.Id,
    p.Name,
    p.Price,
    c.Name AS CategoryName
FROM Products AS p
INNER JOIN Categories AS c ON c.Id = p.CategoryId;
```

An inner join returns only rows with matches on both sides. If products may have no category, use a left join:

```
SELECT
    p.Id,
    p.Name,
    c.Name AS CategoryName
FROM Products AS p
LEFT JOIN Categories AS c ON c.Id = p.CategoryId;
```

Every product remains in the result; `CategoryName` is null for uncategorised products. A many-to-many query crosses the joining table:

```
SELECT t.Name
FROM Tags AS t
INNER JOIN ProductTags AS pt ON pt.TagId = t.Id
WHERE pt.ProductId = @productId
ORDER BY t.Name;
```

Joins do not copy data into one table. They construct a result from related rows at query time. The database optimiser decides how to perform the join using indexes, statistics, available memory, and estimated row counts. A common application mistake is to load all products, then issue one query per product for its category or tags. This is the N+1 query problem:

```
1 query for products
+ 100 queries for categories
= 101 database round trips
```

A join, projection, or deliberately loaded relationship can retrieve the required data with far fewer round trips. EF Core can generate these shapes, but the developer still needs to recognise the difference. Aggregate functions compute values over sets:

```
SELECT COUNT(*) FROM Products;
SELECT AVG(Price)
FROM Products
WHERE IsAvailable = true;
SELECT MIN(Price), MAX(Price)
FROM Products;
```

Grouping calculates one aggregate result per group:

```
SELECT
    CategoryId,
    COUNT(*) AS ProductCount,
    AVG(Price) AS AveragePrice
FROM Products
GROUP BY CategoryId;
```

WHERE filters rows before grouping, while HAVING filters groups after aggregation:

```
SELECT CategoryId, COUNT(*) AS ProductCount
FROM Products
WHERE IsAvailable = true
GROUP BY CategoryId
HAVING COUNT(*) >= 5;
```

Aggregates may return null when no rows contribute, depending on the function. COUNT(*) returns zero, but AVG, MIN, and MAX commonly return null for an empty set. C# projections must account for this. The database is usually better at filtering and aggregation than application code that loads every row and processes it in memory. A dashboard count should normally become COUNT(*), not ToListAsync() followed by .Count.

9.6 Writes and Transactions

An insert adds a row:

```
INSERT INTO Products (
    Code,
    Name,
    Description,
    Price,
    Currency,
    IsAvailable,
    CategoryId
)
VALUES (
    @code,
    @name,
    @description,
    @price,
    @currency,
    @isAvailable,
    @categoryId
);
```

An update changes every row matching its predicate:

```
UPDATE Products
SET Price = @price,
    IsAvailable = @isAvailable
WHERE Id = @id;
```

A delete removes every matching row:

```
DELETE FROM Products
WHERE Id = @id;
```

The WHERE clause is therefore critical. Omitting it from an update or delete affects the whole table. Database tools often offer safeguards, but application commands must still be constructed correctly. The affected-row count is useful. An update by primary key that affects zero rows may mean the product does not exist or a concurrency condition failed. Affecting one row is the expected result. More than one row indicates that the predicate did not express unique identity. Set-based commands can update many rows efficiently:

```
UPDATE Products
SET IsAvailable = false
WHERE CategoryId = @categoryId;
```

Loading those rows into C#, changing each object, and issuing many separate updates is usually slower and holds more memory. EF Core supports set-based update and delete APIs for suitable operations, though ordinary tracked entity updates remain useful when domain logic must inspect each row. A transaction groups database changes so that they commit together or none of them become permanent. Suppose creating a product also writes an audit row and reserves its unique code. If the second operation fails, storing only the first may leave inconsistent data.

```
BEGIN;
INSERT INTO Products (...);
INSERT INTO ProductAudit (...);
COMMIT;
```

If an error occurs:

```
ROLLBACK;
```

The classic transaction properties are abbreviated as ACID:

```
Atomicity    -> all changes commit or all roll back
Consistency  -> constraints remain satisfied
Isolation    -> concurrent transactions are controlled
Durability   -> committed changes survive failure
```

These words describe goals, while exact guarantees depend on the database, isolation level, configuration, and failure scenario. For normal application design, the key idea is that related writes should cross the commit boundary together. Transactions should be as short as practical. Keeping one open while waiting for user input, calling a slow external API, or performing unrelated work holds locks and increases contention. A database transaction cannot automatically roll back an email, payment request, or message already sent to another system. Cross-system consistency requires other patterns such as outbox records, idempotent operations, or compensating actions, which later chapters will introduce briefly.

EF Core wraps a typical `SaveChanges` call in a transaction when needed. Explicit transactions become useful when several saves or raw commands must commit together. Understanding the transaction boundary remains important even when the framework opens it automatically.

9.7 Concurrency and Isolation

Consider two administrators opening product 42 at the same time:

```
Admin A reads price 129
Admin B reads price 129
Admin A saves price 119
Admin B saves name change using stale row
```

If Admin B's update writes every column, it may restore the old price 129 and silently erase Admin A's change. This is the lost-update problem. Pessimistic concurrency locks data before editing so that other transactions must wait. It can be appropriate for short, tightly controlled operations but is rarely suitable for a user form that may remain open for minutes. Optimistic concurrency assumes conflicts are uncommon and checks whether the row changed before saving. The table stores a version value:

Id	Name	Price	Version
42	Keyboard	129	7

The update includes the version read earlier:

```
UPDATE Products
SET Name = @name,
    Price = @price,
    Version = Version + 1
WHERE Id = @id
AND Version = @originalVersion;
```

If another writer has already changed the row, the version no longer matches and the update affects zero rows. The application can then reload, show a conflict, or merge changes deliberately. EF Core supports concurrency tokens and converts the zero-row outcome into a concurrency exception. Optimistic concurrency is not the same as a database transaction. A transaction protects the consistency of one operation while it runs; a concurrency token detects that data changed between reading and later writing, often across separate requests. Concurrent transactions can interact in several undesirable ways. One transaction might read uncommitted changes from another, read the same row twice and receive different values, or repeat a range query and see newly inserted rows. Databases define isolation levels that trade stronger guarantees for concurrency and cost. Common levels include:

```
Read uncommitted -> weakest isolation, may observe uncommitted data
Read committed  -> reads only committed data, common default
Repeatable read  -> repeated row reads remain stable within transaction
Serializable     -> strongest illusion of serial execution
Snapshot        -> reads from a consistent versioned snapshot
```

The exact behaviour differs between providers. Some use locks, some use row versions, and some combine both. Stronger isolation can reduce anomalies but increase blocking, retries, or storage overhead. It should solve a specific consistency requirement rather than be raised globally without measurement. Deadlocks can still occur when transactions acquire locks in conflicting orders:

```
Transaction A locks Product 1, waits for Product 2
Transaction B locks Product 2, waits for Product 1
```

The database detects the cycle and aborts one transaction. Applications should keep transactions short, access shared resources in consistent order where possible, and retry only operations that are safe to repeat.

9.8 Indexes and Query Plans

Without a useful index, the database may scan every row to find a match. An index stores selected column values in a structure that helps locate rows quickly:

```
CREATE INDEX IX_Products_CategoryId
ON Products(CategoryId);
```

A query filtering by `CategoryId` can now use the index rather than scanning the entire table. Primary keys and unique constraints usually create indexes automatically. Composite indexes cover several columns:

```
CREATE INDEX IX_Products_CategoryId_IsAvailable_Name
ON Products(CategoryId, IsAvailable, Name);
```

Column order matters. This index may support queries beginning with `CategoryId`, then `IsAvailable`, and then `Name`, but it does not automatically optimise every query involving any of those columns in any order. Indexes are not free. Each insert, delete, and indexed-column update must maintain them. They consume disk and memory, and too many indexes can slow writes substantially. Create indexes for real access patterns: foreign keys used in joins, unique business identifiers, frequent filters, and common ordering combinations. Do not index every column by habit. An index can sometimes cover a query by containing all required values, avoiding a lookup back to the table. Provider-specific included columns and filtered indexes can improve targeted workloads, but they should follow measurement and query-plan inspection.

SQL describes the desired result, while the database optimiser chooses how to obtain it. The plan may scan a table, seek through an index, sort rows, use nested loops, build a hash table for a join, or process groups in several stages. Database tools can show the estimated or actual execution plan. Useful questions include:

```
Did the query scan far more rows than it returned?
Was the expected index used?
Did a join multiply rows unexpectedly?
Was a large sort required?
Were row-count estimates far from reality?
Did the query execute once or hundreds of times?
```

A slow query is not fixed reliably by guessing. Measure duration, inspect generated SQL, examine the plan, and consider data volume. A query that is instant with ten development rows may be unusable with ten million production rows. Application-level timing also matters. A fast SQL command repeated 500 times over the

network may be slower than one moderately complex query. Returning every column and relationship can waste serialization and transfer even if the database executes quickly. Performance is the whole path:

```
LINQ construction
-> SQL generation
-> Network round trip
-> Database execution
-> Row transfer
-> Object materialisation
-> JSON serialization
-> Browser rendering
```

Chapter 10 will show how to inspect EF Core's generated SQL and choose projections, tracking behaviour, and loading strategies deliberately.

9.9 Versioning the Schema

The database schema evolves with the application. A new version may add a product code, create a categories table, or make a field required. Editing production tables manually without a repeatable record makes deployments difficult to reproduce and diagnose. A migration is a versioned set of schema changes:

```
001_CreateProducts
002_AddCategories
003_AddProductCode
004_AddConcurrencyVersion
```

Each migration describes how to move the schema forward, and sometimes backward. EF Core can generate migrations from model changes, but generated code still needs review. Renaming a column may be misinterpreted as dropping the old column and adding a new one, which would lose data. Adding a required column to a populated table needs a default, staged rollout, or data backfill. Safe production evolution often requires more than one deployment:

1. Add nullable column
2. Deploy code that writes old and new forms
3. Backfill existing rows
4. Make column required
5. Remove old column in a later release

This is especially important when several application instances run during deployment. The new application and old application may briefly share the same database, so schema changes should remain compatible across that window. A migration changes structure; it is not automatically a backup. Backups, restore testing, retention, and disaster recovery are separate operational responsibilities.

9.10 The Product Catalog Schema and SQL Path

A practical first relational model for the Product Catalog could contain these tables:

```
Categories
  Id
  Name
Products
  Id
  Code
  Name
  Description
  Price
  Currency
  IsAvailable
  CategoryId
  Version
  CreatedAt
  UpdatedAt
Tags
  Id
  Name
```

```
ProductTags
  ProductId
  TagId
```

The relationships are:

```
Category 1 ---- * Product
Product  * ---- * Tag through ProductTag
```

A compact SQL version is:

```
CREATE TABLE Categories (
  Id INTEGER PRIMARY KEY,
  Name VARCHAR(100) NOT NULL UNIQUE
);
CREATE TABLE Products (
  Id INTEGER PRIMARY KEY,
  Code VARCHAR(40) NOT NULL UNIQUE,
  Name VARCHAR(120) NOT NULL,
  Description VARCHAR(1000) NOT NULL,
  Price DECIMAL(12, 2) NOT NULL,
  Currency CHAR(3) NOT NULL,
  IsAvailable BOOLEAN NOT NULL DEFAULT true,
  CategoryId INTEGER NULL,
  Version INTEGER NOT NULL DEFAULT 1,
  CreatedAt TIMESTAMP NOT NULL,
  UpdatedAt TIMESTAMP NOT NULL,
  FOREIGN KEY (CategoryId) REFERENCES Categories(Id),
  CHECK (Price >= 0)
);
CREATE INDEX IX_Products_CategoryId
ON Products(CategoryId);
CREATE TABLE Tags (
  Id INTEGER PRIMARY KEY,
  Name VARCHAR(60) NOT NULL UNIQUE
);
CREATE TABLE ProductTags (
  ProductId INTEGER NOT NULL,
  TagId INTEGER NOT NULL,
  PRIMARY KEY (ProductId, TagId),
  FOREIGN KEY (ProductId)
    REFERENCES Products(Id)
    ON DELETE CASCADE,
  FOREIGN KEY (TagId)
    REFERENCES Tags(Id)
    ON DELETE CASCADE
);
```

This is a starting point, not a universal catalog schema. A real system may need stock records, price histories, translations, tax categories, suppliers, media files, audit records, and tenant ownership. Each additional table should represent a clear fact or relationship. The schema also reveals application decisions. Price is stored directly on a product, so the model assumes one current price and currency per product. If prices vary by market or have history, they should become separate rows. `IsAvailable` is a simple flag; inventory quantities would require another model. The database structure is therefore part of the domain design, not an implementation detail hidden entirely behind EF Core. Suppose the browser requests `/api/products/42`. The application service asks the data layer for one product. At the database boundary, the operation may become:

```
SELECT
  p.Id,
  p.Code,
  p.Name,
  p.Description,
  p.Price,
```

```

    p.Currency,
    p.IsAvailable,
    p.Version,
    c.Id AS CategoryId,
    c.Name AS CategoryName
FROM Products AS p
LEFT JOIN Categories AS c ON c.Id = p.CategoryId
WHERE p.Id = @id;

```

The parameter @id carries 42. The database uses the primary-key index to locate the product and joins the category if present. The returned row is materialised into a .NET result, translated into an API or presentation model, serialized, and sent to the browser. An update may become:

```

UPDATE Products
SET Name = @name,
    Description = @description,
    Price = @price,
    Currency = @currency,
    IsAvailable = @isAvailable,
    CategoryId = @categoryId,
    Version = Version + 1,
    UpdatedAt = @updatedAt
WHERE Id = @id
AND Version = @originalVersion;

```

One affected row means the update succeeded. Zero means the product disappeared or changed since it was loaded. The application distinguishes those cases and returns either not found or conflict. This concrete SQL is what Entity Framework Core will help produce. EF Core removes repetitive mapping and command construction, but it does not change the database's work. A LINQ expression that describes a poor query still becomes poor SQL.

9.11 Common Mistakes and Key Ideas

Relational design should keep independently queried and constrained facts in ordinary columns and tables. Comma-separated identifiers and opaque JSON blobs hide relationships from foreign keys, joins, indexes, and targeted updates; JSON remains useful for genuinely embedded flexible data. Normalisation is not a demand to turn every scalar into a lookup table: a currency code can remain a value until the application needs currency metadata or relationships. C# validation provides useful feedback, while database constraints protect stored state from every writer. Translate known uniqueness, check, and foreign-key failures into stable application outcomes instead of exposing provider messages. Use explicit ordering, filter and aggregate in the database, keep result sizes bounded, avoid N+1 access, and add indexes from measured query patterns rather than habit.

Transactions should remain short, optimistic concurrency should detect edits spanning requests, and migration SQL and execution plans should be reviewed directly. The database is an independent system with its own guarantees and costs; an ORM can improve access to it but cannot replace relational design or operational understanding.

10. Entity Framework Core

Chapter 9 described the database directly: tables store facts, keys provide identity, foreign keys represent relationships, indexes support access patterns, transactions protect groups of changes, and SQL describes set-based reads and writes. Entity Framework Core sits between that relational system and C#. It maps rows to objects, translates LINQ expression trees into provider-specific SQL, tracks selected objects, and turns state changes into `INSERT`, `UPDATE`, and `DELETE` commands. It removes a large amount of repetitive data-access code, but it does not replace the database model or make every C# expression efficient.

The most useful mental model is that EF Core has three connected responsibilities. The **model** describes how entity types map to database structure. The **query pipeline** translates a C# query into a database command and materialises the returned rows. The **change tracker** records entity state so that `SaveChanges` can generate writes. `DbContext` coordinates all three for one short unit of work.

```
Entity classes + mapping
  -> EF Core model
LINQ expression
  -> database provider
  -> SQL command
  -> rows
  -> C# result
Tracked entity changes
  -> SaveChanges
  -> INSERT / UPDATE / DELETE
```

This chapter maps the Product Catalog schema into EF Core, creates migrations, reads and writes data, handles relationships and optimistic concurrency, and examines the generated SQL. SQLite is used for local examples because it needs no separate server, but the same model can target PostgreSQL, SQL Server, or another supported relational provider. Provider differences still matter, especially for types, generated values, migrations, locking, indexes, and SQL features.

10.1 The ORM Model and Provider Setup

EF Core is an object-relational mapper. It translates between two models that do not naturally have the same shape: an object graph with classes, references, and collections, and a relational model with tables, keys, rows, and joins. The mapping reduces manual work, but it cannot make the mismatch disappear. A navigation property is not a permanently loaded object reference; it represents a relationship that may or may not have been queried. A LINQ predicate is not executed by ordinary C# when it remains inside `IQueryable`; EF Core analyses its expression tree and asks the database provider to translate it. `SaveChanges` is not object serialization; it examines tracked state and creates relational commands.

The database remains the source of durable truth. Constraints can reject a write that passed C# validation, another process can change a row after it was loaded, a query can be slow because of a missing index, and provider-specific SQL can behave differently. Good EF Core code therefore remains aware of SQL shape, transaction boundaries, row counts, and context lifetime. The base package supplies EF Core abstractions, while a provider connects them to a particular database. For the SQLite version of the Product Catalog:

```
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
dotnet add package Microsoft.EntityFrameworkCore.Design
dotnet tool install --global dotnet-ef
```

The package and `dotnet-ef` major versions should match the EF Core major version used by the application. `Microsoft.EntityFrameworkCore.Sqlite` translates queries and commands to SQLite and supplies provider-specific type mapping. `Microsoft.EntityFrameworkCore.Design` supports design-time work such as migration generation. `dotnet-ef` is the CLI tool that invokes those design-time services. Changing providers is more than replacing one package name. The common LINQ and change-tracking model remains, but migrations and database capabilities are provider-specific. SQLite has limited schema-alteration behaviour and no SQL Server-style `rowversion`; PostgreSQL and SQL Server differ in identity generation, date types, case sensitivity, indexes, JSON support, and locking. A production switch should be tested against the real provider rather than assumed from an in-memory or SQLite test.

10.2 Entities, DbContext, and Model Configuration

The first Product Catalog entities can be ordinary C# classes:

```

public sealed class Product
{
    public int Id { get; set; }
    public string Code { get; set; } = "";
    public string Name { get; set; } = "";
    public string Description { get; set; } = "";
    public decimal Price { get; set; }
    public string Currency { get; set; } = "EUR";
    public bool IsAvailable { get; set; }
    public int? CategoryId { get; set; }
    public Category? Category { get; set; }
    public int Version { get; set; } = 1;
    public DateTimeOffset CreatedAt { get; set; }
    public DateTimeOffset UpdatedAt { get; set; }
    public ICollection<ProductTag> ProductTags { get; } = [];
}

public sealed class Category
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public ICollection<Product> Products { get; } = [];
}

public sealed class Tag
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public ICollection<ProductTag> ProductTags { get; } = [];
}

public sealed class ProductTag
{
    public int ProductId { get; set; }
    public Product Product { get; set; } = default!;
    public int TagId { get; set; }
    public Tag Tag { get; set; } = default!;
}

```

Scalar properties map naturally to columns. `CategoryId` is the nullable foreign key, while `Category` and `Category.Products` are navigation properties that let C# move across the relationship when the related entities are loaded. `ProductTag` is an explicit joining entity for the many-to-many relationship. It currently contains only the two keys, but it can later gain relationship data such as ordering or creation time without changing the conceptual model.

Entities may contain domain behaviour and need not expose public setters everywhere, but the tutorial keeps them straightforward while the EF Core mechanics are introduced. The important boundary is that entity types represent persistence state and relationships. They should not double automatically as API request models or Blazor form models. An incoming client should not be allowed to set database-generated identifiers, audit times, internal relationships, or concurrency fields merely because the entity exposes them. A context derives from `DbContext` and exposes entity sets:

```

public sealed class CatalogDbContext(DbContextOptions<CatalogDbContext> options) :
    DbContext(options)
{
    public DbSet<Product> Products => Set<Product>();
    public DbSet<Category> Categories => Set<Category>();
    public DbSet<Tag> Tags => Set<Tag>();
    public DbSet<ProductTag> ProductTags => Set<ProductTag>();
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.ApplyConfigurationsFromAssembly(typeof(CatalogDbContext).Assembly);
    }
}

```

`DbSet<Product>` is the starting point for product queries and can also mark products for insertion or deletion. It is not a preloaded collection. Calling `db.Products` alone performs no SQL. The context holds metadata, a database connection abstraction, and the change tracker for its current unit of work. A `DbContext` should be short-lived, disposed after the operation, and never shared between concurrent tasks. It is not thread-safe, and an `InvalidOperationException` from EF Core may leave it unusable. In a normal HTTP API, one scoped context per request often matches the unit of work well. A background service, long-running Blazor circuit, or operation that needs several isolated units of work should create contexts on demand through a factory.

The context is not a general application cache. Keeping it alive for a long time causes the change tracker to accumulate entities, increases memory use, creates stale state, and makes it unclear which changes belong to one save. EF Core conventions recognise properties named `Id` or `<TypeName>Id` as primary keys, pair navigation properties with matching foreign keys, and map common CLR types. Conventions keep the model concise, but schema rules that matter should be configured explicitly. Separate configuration classes prevent `OnModelCreating` from becoming one enormous method:

```
public sealed class ProductConfiguration
    : IEntityTypeConfiguration<Product>
{
    public void Configure(EntityTypeBuilder<Product> product)
    {
        product.ToTable("Products");
        product.HasKey(value => value.Id);
        product.Property(value => value.Code).HasMaxLength(40).IsRequired();
        product.HasIndex(value => value.Code).IsUnique();
        product.Property(value => value.Name).HasMaxLength(120).IsRequired();
        product.Property(value => value.Description).HasMaxLength(1000).IsRequired();
        product.Property(value => value.Price).HasPrecision(12, 2);
        product.Property(value =>
value.Currency).HasMaxLength(3).IsFixedLength().IsRequired();
        product.Property(value => value.Version).IsConcurrencyToken();
        product.HasOne(value => value.Category)
            .WithMany(value => value.Products)
            .HasForeignKey(value => value.CategoryId)
            .OnDelete(DeleteBehavior.SetNull);
    }
}
```

The many-to-many join uses a composite key:

```
public sealed class ProductTagConfiguration
    : IEntityTypeConfiguration<ProductTag>
{
    public void Configure(EntityTypeBuilder<ProductTag> link)
    {
        link.ToTable("ProductTags");
        link.HasKey(value => new { value.ProductId, value.TagId });
        link.HasOne(value => value.Product).WithMany(value =>
value.ProductTags).HasForeignKey(value => value.ProductId)
            .OnDelete(DeleteBehavior.Cascade);
        link.HasOne(value => value.Tag).WithMany(value =>
value.ProductTags).HasForeignKey(value => value.TagId)
            .OnDelete(DeleteBehavior.Cascade);
    }
}
```

Configuration expresses the intended database contract: lengths, precision, uniqueness, required values, relationship cardinality, delete behaviour, and concurrency. Data annotations such as `[Required]`, `[MaxLength]`, and `[Timestamp]` can configure part of the model, but fluent configuration is more expressive and keeps persistence rules together. Use one style consistently enough that developers know where to look. Provider-specific configuration is sometimes necessary. An index include list, filtered index, collation, generated column, or row-version column may not translate across databases. Isolate such decisions rather than pretending the provider is irrelevant.

10.3 Registration and Migrations

Place the local connection string in configuration:

```
{
  "ConnectionStrings": {
    "Catalog": "Data Source=catalog.db"
  }
}
```

Register the context before Build:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<CatalogDbContext>(options =>
    options.UseSqlite(
        builder.Configuration.GetConnectionString("Catalog")
        ?? throw new InvalidOperationException("Connection string 'Catalog' is missing.")
    )
);
```

`AddDbContext` registers `CatalogDbContext` as scoped by default. One request can therefore inject the same context into several scoped services and commit one coordinated unit of work. The connection string is configuration, not code, and production credentials should come from a secure environment-specific provider rather than committed source. A context normally opens the physical database connection only when a command needs it and closes it afterwards. Context lifetime and connection lifetime are related but not identical. Keeping a context scoped for one request does not mean holding an open connection during the whole request. For isolated context creation, register a factory:

```
builder.Services.AddDbContextFactory<CatalogDbContext>(options =>
    options.UseSqlite(builder.Configuration.GetConnectionString("Catalog"))
);
```

A service can then create and dispose a context for one operation:

```
await using var db = await factory.CreateDbContextAsync(token);
```

This pattern is especially useful in interactive server rendering, where a scoped service can live for an entire circuit rather than one HTTP request, and in background or parallel operations where each unit of work needs its own context. Once the model and context exist, create the first migration:

```
dotnet ef migrations add InitialCreate
```

EF Core compares the current model with the previous model snapshot and generates a migration class containing `Up` and `Down` methods. `Up` creates or alters schema objects; `Down` describes a possible reversal. The migration is source code and should be reviewed. Verify table names, column types, nullability, foreign keys, delete behaviour, indexes, defaults, and any data movement before applying it. Apply the migration locally:

```
dotnet ef database update
```

EF Core creates a migrations-history table and records which migrations have been applied. Later model changes produce additional migrations:

```
dotnet ef migrations add AddProductSearchName
dotnet ef database update
```

A migration should represent one coherent schema step and should remain committed after it has reached a shared database. Editing or deleting an already-applied migration makes environments disagree about history. Add a new migration to correct it. Automatic application startup is convenient for local development but risky as the default production deployment mechanism. Several instances may race to update the schema, the application account may need excessive DDL permissions, and a long migration can delay or break startup. Production systems commonly generate a reviewed SQL script or migration bundle and apply it as a controlled deployment step:

```
dotnet ef migrations script --idempotent
dotnet ef migrations bundle
```

`EnsureCreated` is not a shortcut for migrations. It creates a schema for a model when no schema exists but does not provide normal incremental migration history. It is appropriate for disposable tests and prototypes, not for a database expected to evolve through production releases.

10.4 LINQ Translation and Projection

A query begins as `IQueryable<T>`:

```
var query = db.Products.Where(product => product.IsAvailable).OrderBy(product =>
product.Name);
```

No database call has occurred. `Where` and `OrderBy` add nodes to an expression tree. EF Core's provider later translates that tree into SQL. Execution begins when the application asks for results through an operation such as `ToListAsync`, `SingleOrDefaultAsync`, `CountAsync`, `AnyAsync`, or asynchronous enumeration:

```
var products = await query.ToListAsync(token);
```

Conceptually:

```
C# query operators
-> expression tree
-> provider translation
-> parameterised SQL
-> database execution
-> returned rows
-> C# objects
```

This distinction explains why moving from `IQueryable<T>` to `IEnumerable<T>` too early can be expensive. The first query filters in the database:

```
var products = await db.Products.Where(product => product.Price <=
maximumPrice).ToListAsync(token);
```

This version loads every product and filters in memory:

```
var products = db.Products.AsEnumerable().Where(product => product.Price <=
maximumPrice).ToList();
```

Ordinary C# methods inside a query may not be translatable. Modern EF Core generally throws when a non-translatable expression appears where database evaluation is required, rather than silently loading the table and evaluating the predicate on the client. Keep database predicates in supported LINQ forms, explicitly materialise only when the remaining data is already bounded, and test provider-specific translations. Loading full entities is useful when they will be changed, but read operations often need only a few values. Projection creates a specific result directly in SQL:

```
public sealed record ProductSummary(
    int Id,
    string Code,
    string Name,
    decimal Price,
    string Currency,
    string? CategoryName,
    int Version
);
var products = await db.Products.AsNoTracking().Where(product =>
product.IsAvailable).OrderBy(product => product.Name)
.Select(product => new ProductSummary(
    product.Id,
    product.Code,
    product.Name,
    product.Price,
    product.Currency,
    product.Category == null
    ? null
```

```

        : product.Category.Name,
        product.Version
    )).ToListAsync(token);

```

The provider selects only the required columns and creates the join needed for `Category.Name`. EF Core does not first create every `Product` and `Category` entity and then run the projection in memory. The projection itself becomes part of the SQL. This is usually the best shape for API responses, list pages, and reports. It reduces transferred columns, change-tracker work, object allocation, and accidental exposure of persistence details. The response model can also combine values from several tables without forcing the UI to understand the entity graph. Apply filtering, ordering, and paging before materialisation:

```

var page = await db.Products.AsNoTracking().Where(product => search == null ||
product.Name.Contains(search))
    .OrderBy(product => product.Name).ThenBy(product => product.Id).Skip((pageNumber - 1) *
pageSize).Take(pageSize)
    .Select(product => new ProductSummary(
        product.Id,
        product.Code,
        product.Name,
        product.Price,
        product.Currency,
        product.Category == null ? null : product.Category.Name,
        product.Version
    )).ToListAsync(token);

```

The secondary unique ordering by `Id` makes paging stable when several products share the same name. Search semantics such as case sensitivity depend on database collation and provider translation; test them against the production database.

10.5 Tracking, Single-Entity Queries, and Related Data

Entity queries are tracking by default. When EF Core materialises an entity, it records its key, current state, and usually original values in the context's change tracker. If the entity is modified, `SaveChanges` can determine what command to issue:

```

var product = await db.Products.SingleOrDefaultAsync(value => value.Id == id, token);
if (product is null)
    return ProductUpdateResult.NotFound();
product.Price = request.Price;
product.UpdatedAt = clock.UtcNow;
product.Version++;
await db.SaveChangesAsync(token);

```

The entity normally moves from `Unchanged` to `Modified`; a new entity added through `Add` is `Added`, while one passed to `Remove` becomes `Deleted`. `SaveChanges` detects changes, orders commands around relationships, executes them, and accepts the resulting states after a successful commit. Read-only queries should normally use `AsNoTracking`:

```

var product = await db.Products.AsNoTracking().SingleOrDefaultAsync(value => value.Id == id,
token);

```

This avoids change-tracker setup and prevents later accidental saving through that context. `AsNoTrackingWithIdentityResolution` can avoid creating multiple instances for the same key within one result while still leaving the context untracked, which is useful for some repeated relationship shapes. Plain projections that contain no entity instances usually need no tracking anyway. Tracking also provides identity resolution: within one context, repeated rows with the same entity key normally refer to the same entity instance. That helps relationship fix-up, but it is another reason not to keep one context indefinitely. Previously loaded state can influence later navigation contents and make a long-lived context behave differently from a fresh query. Several LINQ operators express different expectations:

```

FirstOrDefaultAsync -> zero or more rows are acceptable; take the first
SingleOrDefaultAsync -> zero or one row is expected; more than one is an error

```

FirstAsync -> at least one row is required
SingleAsync -> exactly one row is required

Use `SingleOrDefaultAsync` for a unique key or unique business code when duplicate rows would indicate broken assumptions. Use `FirstOrDefaultAsync` when the ordering deliberately chooses one row from several. `FindAsync` is specialised for primary keys:

```
var product = await db.Products.FindAsync([id], token);
```

It first checks the change tracker and queries the database only if the entity is not already tracked. That is useful inside a unit of work but can surprise code that expects a guaranteed database refresh. `SingleOrDefaultAsync(product => product.Id == id)` always expresses a query and can include projection, `Include`, or no-tracking behaviour. Avoid `ToListAsync` followed by `SingleOrDefault` for one row. It transfers more data and weakens the query's intent. Similarly, use `AnyAsync` for existence and `CountAsync` only when the actual count is needed. The presence of `product.Category` or `product.ProductTags` in C# does not guarantee that the related data was fetched. EF Core supports eager, explicit, and lazy loading. **Eager loading** requests relationships as part of the query:

```
var product = await db.Products.Include(value => value.Category).Include(value =>
value.ProductTags)
    .ThenInclude(link => link.Tag).SingleOrDefaultAsync(value => value.Id == id, token);
```

This is appropriate when the operation needs tracked entities and their relationships. Large collection includes can multiply rows because one product is repeated for each related combination. `AsSplitQuery` can issue several SQL queries instead of one very wide joined result:

```
var product = await db.Products.AsSplitQuery().Include(value =>
value.Category).Include(value => value.ProductTags)
    .ThenInclude(link => link.Tag).SingleOrDefaultAsync(value => value.Id == id, token);
```

Split queries reduce cartesian row multiplication but add commands and may observe changes between those commands unless a suitable transaction isolates them. Choose based on the actual relationship size and consistency requirement. **Explicit loading** loads a relationship later through the tracked entity entry:

```
await db.Entry(product).Collection(value => value.ProductTags).Query()
    .Where(link => link.Tag.Name.StartsWith("Featured")).LoadAsync(token);
```

This is useful when the need is conditional and the context remains alive. **Lazy loading** loads a relationship automatically when code accesses a navigation property, usually through proxies or `ILazyLoader`. It can be convenient but hides database I/O inside normal property access and easily creates N+1 queries during loops or serialization. For web applications, explicit projections and deliberate includes are usually easier to reason about. For read screens, projection often beats all three entity-loading strategies:

```
var details = await db.Products.AsNoTracking().Where(product => product.Id ==
id).Select(product => new ProductDetails(
    product.Id,
    product.Code,
    product.Name,
    product.Description,
    product.Price,
    product.Currency,
    product.Category == null ? null : product.Category.Name,
    product.ProductTags.OrderBy(link => link.Tag.Name).Select(link =>
link.Tag.Name).ToArray(),
    product.Version
)).SingleOrDefaultAsync(token);
```

The query declares exactly what the page needs and avoids exposing a live entity graph to the presentation layer.

10.6 Saving, Disconnected Updates, and Concurrency

Creating an entity begins by marking it as added:

```

var product = new Product
{
    Code = request.Code.Trim().ToUpperInvariant(),
    Name = request.Name.Trim(),
    Description = request.Description.Trim(),
    Price = request.Price,
    Currency = request.Currency.Trim().ToUpperInvariant(),
    IsAvailable = request.IsAvailable,
    CategoryId = request.CategoryId,
    CreatedAt = clock.UtcNow,
    UpdatedAt = clock.UtcNow
};
db.Products.Add(product);
await db.SaveChangesAsync(token);

```

The database generates the key where configured, and EF Core assigns it back to `product.Id`. Updating a tracked entity means loading it, changing allowed fields, and saving. Deleting means loading or attaching the intended row and calling `Remove`:

```

var product = await db.Products.SingleOrDefaultAsync(value => value.Id == id, token);
if (product is null)
    return false;
db.Products.Remove(product);
await db.SaveChangesAsync(token);
return true;

```

For most relational providers, one `SaveChanges` call is transactional when it contains multiple commands: all succeed or the transaction rolls back. EF Core may batch compatible commands, but the generated SQL and batching rules depend on the provider. `SaveChanges` returns the number of state entries written, not a business-level result such as the number of logical products affected. Do not use the returned value as the sole proof that a complex operation achieved its intended meaning. Constraints, concurrency, and application outcomes still need explicit handling. A browser does not hold the tracked `Product` instance. It receives JSON, edits a separate client object, and sends a request later. By then the original `DbContext` has been disposed. The update is disconnected:

```

Request A
  -> context A loads product
  -> response sends DTO
  -> context A is disposed
User edits for thirty seconds
Request B
  -> context B receives update DTO
  -> context B must load or attach stored state

```

The safest ordinary pattern is to load the current entity in context B, copy only permitted fields, and save:

```

var product = await db.Products.SingleOrDefaultAsync(value => value.Id == id, token);
if (product is null)
    return ProductUpdateResult.NotFound();
product.Name = request.Name.Trim();
product.Description = request.Description.Trim();
product.Price = request.Price;
product.Currency = request.Currency.Trim().ToUpperInvariant();
product.IsAvailable = request.IsAvailable;
product.CategoryId = request.CategoryId;
product.UpdatedAt = clock.UtcNow;
product.Version++;
await db.SaveChangesAsync(token);

```

Calling `db.Update(incomingEntity)` marks the whole graph as modified or added according to key information. That can be useful for controlled graphs, but it is dangerous for API input because it may overwrite columns the client never saw, attach unexpected related entities, or allow over-posting of internal values. Request DTOs

plus explicit assignment make the writable contract visible. For high-throughput updates that should avoid the initial read, attach a stub and mark selected properties deliberately, or use `ExecuteUpdate`. Those approaches require explicit validation and concurrency handling because the context is not comparing a fully loaded entity. `Version` is configured as a concurrency token. When EF Core loads a tracked product, it remembers the original version. During save it adds that value to the `WHERE` clause:

```
UPDATE Products
SET Name = @name,
    Price = @price,
    Version = @newVersion
WHERE Id = @id
    AND Version = @originalVersion;
```

If another transaction has changed the product, the original version no longer matches and the command affects zero rows. EF Core throws `DbUpdateConcurrencyException`:

```
try
{
    product.Name = request.Name.Trim();
    product.Version++;
    await db.SaveChangesAsync(token);
}
catch (DbUpdateConcurrencyException)
{
    return ProductUpdateResult.Conflict("The product changed after it was opened.");
}
```

A real conflict handler may load current database values, show both versions, retry a non-overlapping change, or ask the user to review the new state. Blindly retrying the same stale values simply overwrites the other change after another read and may defeat the purpose of concurrency control. SQL Server can use a database-generated `rowversion` byte array with `.IsRowVersion()`. Other providers may use PostgreSQL system columns or application-managed tokens such as an integer, GUID, or timestamp. Application-managed tokens must change whenever a relevant update occurs. The token also needs to cross the client boundary, commonly as a version field or encoded ETag, so the server can verify that the client edited the expected version.

Concurrency exceptions and uniqueness violations are different. A concurrency exception means a conditional update or delete matched no expected row. A unique constraint means the new stored values conflict with another row. Both may become 409 `Conflict`, but they require different messages and recovery.

10.7 Transactions and Set-Based Operations

A single `SaveChanges` call normally supplies the transaction boundary needed for one aggregate update. An explicit transaction is useful when several saves or commands must commit together:

```
await using var transaction =
    await db.Database.BeginTransactionAsync(token);
try
{
    var product = await db.Products.SingleAsync(value => value.Id == id, token);
    product.Price = newPrice;
    product.Version++;
    await db.SaveChangesAsync(token);
    db.ProductPriceAudit.Add(new ProductPriceAudit
    {
        ProductId = id,
        Price = newPrice,
        ChangedAt = clock.UtcNow
    });
    await db.SaveChangesAsync(token);
    await transaction.CommitAsync(token);
}
catch
{
}
```

```

        await transaction.RollbackAsync(token);
        throw;
    }

```

When `SaveChanges` runs inside an existing transaction, EF Core can create a savepoint before writing. If a save fails, it can roll back to that savepoint while leaving the outer transaction available, subject to provider behaviour. Explicit transaction code should remain rare and close to the operation that genuinely needs it. Do not keep the transaction open while waiting for the user, sending email, or calling an unrelated remote API. A database rollback cannot unsend an email or reverse a completed payment call. For reliable cross-system work, save the database change and an outbox record in one transaction, then let a background worker deliver the external message and mark it complete. Chapter 13 returns to that pattern.

Retry strategies and explicit transactions need coordination. A transient-failure execution strategy may need to run the entire transaction delegate so the whole unit can be replayed safely. Never add automatic retries to a state-changing operation without understanding whether repeating it can duplicate external effects. Change tracking is ideal when application logic needs entity state, relationships, validation, and a coordinated save. It is unnecessary when one database expression can update or delete a known set.

```

var affected = await db.Products.Where(product => product.CategoryId == categoryId &&
    product.IsAvailable)
    .ExecuteUpdateAsync(
        setters => setters.SetProperty(product => product.IsAvailable, false)
        .SetProperty(product => product.UpdatedAt, clock.UtcNow),
        token
    );

```

`ExecuteUpdateAsync` sends an immediate set-based `UPDATE`; `ExecuteDeleteAsync` does the equivalent for deletion. They do not load entities, use the change tracker, or wait for `SaveChanges`. They also do not automatically update already tracked instances. Mixing a set-based update with stale tracked entities in the same context can later overwrite the database values when `SaveChanges` runs. These methods do not automatically add a concurrency token condition. Include one in the predicate and inspect the affected-row count when optimistic concurrency matters:

```

var affected = await db.Products.Where(product => product.Id == id && product.Version ==
    expectedVersion)
    .ExecuteUpdateAsync(
        setters => setters.SetProperty(product => product.Price, newPrice)
        .SetProperty(product => product.Version, product => product.Version + 1),
        token
    );
if (affected == 0)
    return ProductUpdateResult.ConflictOrNotFound();

```

Several `ExecuteUpdate` or `ExecuteDelete` calls do not automatically become one transaction merely because they use the same context. Open an explicit transaction when they must commit together.

10.8 Raw SQL and Dapper

LINQ covers most ordinary queries, but raw SQL is appropriate when the database has a feature the provider cannot express well, when a carefully tuned query performs significantly better, or when an existing stored procedure or view must be used. An entity query can begin with parameterised SQL:

```

var products = await db.Products.FromSql(
    $"""
    SELECT *
    FROM Products
    WHERE Price >= {minimumPrice}
    """)
    .AsNoTracking().ToListAsync(token);

```

The interpolated value becomes a database parameter; it is not inserted as executable command text. The `Raw` variants accept ordinary strings and require explicit parameter handling. Never copy user input into raw SQL

syntax. EF Core can compose additional LINQ over composable SQL, but the SQL must satisfy provider rules. Commands that do not return entity rows can use relational execution APIs. Raw SQL should remain localised and tested against the actual provider because it reduces portability and bypasses some model-level guarantees. Dapper is a small mapper that executes SQL and maps result rows with less abstraction:

```
await using var connection = dataSource.CreateConnection();
var products = await connection.QueryAsync<ProductSummary>(
    """
    SELECT Id, Code, Name, Price, Currency
    FROM Products
    WHERE IsAvailable = @isAvailable
    ORDER BY Name
    """,
    new { isAvailable = true }
);
```

Using Dapper for selected reporting or highly tuned queries does not require abandoning EF Core for writes and ordinary access. Both can share the same database and, when necessary, the same ADO.NET transaction. The cost is that SQL, mapping, migrations, and provider details become more explicit. Choose it for a concrete query requirement, not because manually writing every command feels more “architectural.”

10.9 DbContext Lifetime in Interactive Blazor

A conventional HTTP request creates one dependency-injection scope, so a scoped `CatalogDbContext` lives only until the response completes. An interactive server rendering circuit can remain alive for many user interactions, and scoped services are normally shared across that circuit. Injecting a scoped context directly into a long-lived component or circuit service can therefore keep it alive too long and expose it to overlapping operations. Use a factory and create one context per component operation:

```
@inject IDbContextFactory<CatalogDbContext> ContextFactory
@code {
    private async Task LoadAsync()
    {
        await using var db =
            await ContextFactory.CreateDbContextAsync();
        _products = await db.Products
            .AsNoTracking()
            .OrderBy(product => product.Name)
            .Select(product => new ProductSummary(
                product.Id,
                product.Code,
                product.Name,
                product.Price,
                product.Currency,
                product.Category == null
                    ? null
                    : product.Category.Name,
                product.Version
            ))
            .ToListAsync();
    }
}
```

A stronger application boundary is still to inject `IProductCatalog` into the component and let the infrastructure implementation create contexts. That keeps EF Core out of presentation code and allows the same operations to serve Minimal APIs, components, workers, and tests. The factory guidance remains relevant inside that infrastructure service. Interactive WebAssembly components cannot connect directly to a protected server database. They run in the browser and call an HTTP API. Database credentials, provider packages, and server-only entities must remain on the server side.

10.10 Inspecting SQL and the Product Data Service

EF Core integrates with `Microsoft.Extensions.Logging`. During development, enable command logging through category configuration:

```
{
  "Logging": {
    "LogLevel": {
      "Microsoft.EntityFrameworkCore.Database.Command": "Information"
    }
  }
}
```

The logs reveal command text, duration, and parameters, subject to sensitive-data settings. Do not enable sensitive-data logging casually in production because parameter values may contain personal or secret information. `ToQueryString()` shows the SQL shape before execution:

```
var query = db.Products.Where(product => product.IsAvailable).OrderBy(product =>
product.Name).Select(product => new
{
    product.Id,
    product.Name
});
var sql = query.ToQueryString();
```

It is a debugging representation, not a command that should be copied and executed blindly. The actual database logs and execution plan remain the authority. When a page is slow, inspect how many commands were sent, whether each command selected only needed columns, whether filters and paging remained in SQL, whether a collection include multiplied rows, and whether an N+1 pattern appeared. Then inspect indexes and the database plan. EF Core performance is rarely improved by adding abstractions around a query that has not been measured. The in-memory store from Chapter 5 can now be replaced by an EF Core implementation. Read operations project directly to result models, while writes load tracked entities and translate database outcomes into application results:

```
public sealed class EfProductCatalog(
    CatalogDbContext db,
    IClock clock,
    ILogger<EfProductCatalog> logger
) : IProductCatalog
{
    public async Task<IReadOnlyList<ProductSummary>> ListAsync(Cancellation_token token =
default)
    {
        return await db.Products.AsNoTracking().OrderBy(product =>
product.Name).ThenBy(product => product.Id)
        .Select(product => new ProductSummary(
            product.Id,
            product.Code,
            product.Name,
            product.Price,
            product.Currency,
            product.Category == null
                ? null
                : product.Category.Name,
            product.Version
        )).ToListAsync(token);
    }
    public async Task<ProductDetails?> FindAsync(int id, Cancellation_token token = default)
    {
        return await db.Products.AsNoTracking().Where(product => product.Id ==
id).Select(product => new ProductDetails(
            product.Id,
```

```

        product.Code,
        product.Name,
        product.Description,
        product.Price,
        product.Currency,
        product.IsAvailable,
        product.CategoryId,
        product.Category == null
            ? null
            : product.Category.Name,
        product.ProductTags.OrderBy(link => link.Tag.Name).Select(link =>
link.Tag.Name).ToArray(),
        product.Version
    )).SingleOrDefaultAsync(token);
}
public async Task<ProductCreateResult> CreateAsync(CreateProductRequest request,
CancellationTokens token = default)
{
    var validation = ProductValidation.Validate(request);
    if (!validation.IsSuccess)
        return ProductCreateResult.Invalid(validation.Errors);
    var now = clock.UtcNow;
    var product = new Product
    {
        Code = request.Code.Trim().ToUpperInvariant(),
        Name = request.Name.Trim(),
        Description = request.Description.Trim(),
        Price = request.Price,
        Currency = request.Currency.Trim().ToUpperInvariant(),
        IsAvailable = request.IsAvailable,
        CategoryId = request.CategoryId,
        Version = 1,
        CreatedAt = now,
        UpdatedAt = now
    };
    db.Products.Add(product);
    try
    {
        await db.SaveChangesAsync(token);
    }
    catch (DbUpdateException exception)
    {
        logger.LogWarning(
            exception,
            "Product {ProductCode} could not be created",
            product.Code
        );
        return ProductCreateResult.Conflict("A product with this code may already
exist.");
    }
    return ProductCreateResult.Success(
        new ProductSummary(
            product.Id,
            product.Code,
            product.Name,
            product.Price,
            product.Currency,
            null,
            product.Version
        )
    )
}

```

```

    }
    };
}

```

The example catches `DbUpdateException` near the persistence boundary, but a production implementation should inspect provider-specific information before declaring that every update exception is a duplicate code. Foreign-key violations, unavailable databases, and other failures need different handling. A small translation layer can map known constraint names or provider codes to stable application errors and let unexpected failures propagate to central logging. Registration changes from the memory implementation to the database implementation:

```
builder.Services.AddScoped<IProductCatalog, EfProductCatalog>();
```

The Minimal API and Blazor components do not need to know whether products come from a locked list, SQLite, or PostgreSQL. They depend on the operation contract. That is the practical value of the service boundary established earlier: infrastructure can change without moving database code into endpoints or presentation components.

10.11 Common Mistakes and Key Ideas

Use one short-lived `DbContext` per unit of work, never share it across concurrent operations, and do not treat tracked entities as an application cache. Keep read queries as `IQueryable` until filtering, ordering, projection, aggregation, and paging have translated to SQL. Prefer no-tracking projections for reads, tracked entities for deliberate load-change-save operations, and set-based commands when one database statement expresses the change. Large `Include` graphs and lazy-loading loops often hide excess rows or N+1 commands.

Disconnected web input should arrive through request DTOs, not entity graphs passed directly to `Update`. Load current state or attach only the intended fields, include a concurrency token for edits that span requests, and translate known database conflicts without hiding unexpected provider failures. Generated migrations still require review for data loss, defaults, indexes, locking, and provider-specific SQL; `EnsureCreated` is not a migration strategy. Inspect command logs, `ToQueryString`, round-trip counts, and execution plans before optimising. EF Core maps and coordinates relational work, but query shape, context lifetime, constraints, and deployment safety remain application responsibilities.

Part V — Secure, Maintainable Applications

11. Authentication and Authorization

The Product Catalog can now receive requests, render a Blazor interface, and persist products through Entity Framework Core. The remaining problem is trust. A public visitor may be allowed to browse products, while only signed-in editors may change them and only administrators may manage users. The application must establish who is making a request, decide what that identity may do, and continue enforcing those decisions even when a client modifies its own UI or sends HTTP requests directly.

Authentication determines the identity associated with a request. **Authorization** decides whether that identity may access a resource or perform an operation. They are related but separate. A signed-in user can still be forbidden from editing a product, and an anonymous visitor can still be authorised to view a public page. Authentication creates evidence about the caller; authorization evaluates that evidence together with the requested operation and, sometimes, the specific resource.

Request arrives

- > Authentication validates cookie or token
- > ClaimsPrincipal is created
- > Authorization evaluates endpoint policy
- > Endpoint or component operation runs only when allowed

Security is not one middleware call added near the end of development. It affects account storage, cookies, tokens, redirects, API responses, Blazor rendering, database queries, logs, deployment, and every boundary at which untrusted data enters the application. This chapter explains the ASP.NET Core security model from those boundaries inward.

11.1 Identity, Schemes, and Authentication Configuration

An identity is the application's representation of a user, service, or device. Authentication answers, "Which identity does this credential prove?" Authorization answers, "May that identity perform this operation?" Consider three Product Catalog requests:

GET	/api/products/42	-> public
PUT	/api/products/42	-> authenticated editor
DELETE	/api/products/42	-> administrator

The GET endpoint may permit anonymous access. The PUT endpoint first requires a valid identity and then checks an editing policy. The DELETE endpoint requires stronger permission. Signing in does not imply universal access, and hiding the Delete button does not enforce the delete rule. The server endpoint must make the final decision because the browser is controlled by the user. Authentication also does not prove that every claim about the person is true forever. It proves that a configured authentication handler accepted the presented evidence at a particular time. A disabled account, revoked session, changed role, expired token, or removed permission may require revalidation. Security design therefore includes credential lifetime and revocation, not only the first successful login.

ASP.NET Core authentication is organised around named **schemes**. A scheme connects a name with an authentication handler and its options. The handler understands one credential mechanism, such as a protected cookie, JWT bearer token, OpenID Connect response, client certificate, or a custom API key.

Scheme "CatalogCookie"

- > CookieAuthenticationHandler
- > cookie name, paths, lifetime, validation events

Scheme "Bearer"

- > JwtBearerHandler
- > issuer, audience, signing keys, token validation

Authentication services support several related actions:

Authenticate	-> inspect the request and build a principal
Challenge	-> respond when authentication is required
Forbid	-> respond when identity exists but access is denied
SignIn	-> establish a persistent sign-in mechanism
SignOut	-> remove that sign-in mechanism

The default scheme is used when an endpoint or policy does not name another. An application can have separate defaults for authentication, challenge, forbid, sign-in, and sign-out, though one cookie scheme often handles them all in a browser application. Mixed applications may use cookies for interactive pages and bearer tokens for external API clients. A scheme is not a user database. Cookie authentication can accept a `ClaimsPrincipal` produced by custom application code, while ASP.NET Core Identity can manage users and then use a cookie scheme to keep them signed in. OpenID Connect can authenticate through an external provider and still create a local cookie for subsequent requests. A small cookie-based application can register the framework services like this:

```
using Microsoft.AspNetCore.Authentication.Cookies;
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme).
AddCookie(options
=>
{
    options.Cookie.Name = ".ProductCatalog.Auth";
    options.Cookie.HttpOnly = true;
    options.Cookie.SecurePolicy = CookieSecurePolicy.Always;
    options.Cookie.SameSite = SameSiteMode.Lax;
    options.LoginPath = "/account/login";
    options.AccessDeniedPath = "/account/denied";
    options.ExpireTimeSpan = TimeSpan.FromHours(8);
});
builder.Services.AddAuthorization();
var app = builder.Build();
app.UseHttpsRedirection();
app.UseAuthentication();
app.UseAuthorization();
app.MapProductEndpoints();
app.Run();
```

`UseAuthentication` examines the request through the selected authentication handler and populates `HttpContext.User`. `UseAuthorization` evaluates the policy attached to the selected endpoint. Authentication must run first because authorization needs the resulting principal. In Minimal API applications, some middleware can be inserted automatically when the corresponding services are registered, but explicit placement keeps the ordering clear when the pipeline contains other middleware. The cookie is marked `HttpOnly` so ordinary JavaScript cannot read it, `Secure` so it travels only over HTTPS, and `SameSite=Lax` to limit many cross-site requests while preserving common navigation flows. These settings reduce risk but do not remove the need for antiforgery protection on relevant state-changing requests.

11.2 Principals, Claims, Cookies, and Session Outcomes

After successful authentication, ASP.NET Core assigns a `ClaimsPrincipal` to `HttpContext.User`. A principal can contain one or more identities, and each identity contains claims. A claim is a type-value pair describing something asserted about the subject:

```
nameidentifier -> "42"
name           -> "Dominik"
email         -> "dominik@example.com"
role          -> "CatalogEditor"
department    -> "Sales"
permission     -> "products.edit"
```

Claims normally describe what the subject is or what trusted information is known about it. Authorization interprets them to decide what the subject may do. The distinction is subtle but useful: `department=Sales` is a fact, while a policy can translate that fact into access to a sales report. A permission claim such as `products.edit` directly represents an access decision issued by a trusted identity system, but the application still chooses how to evaluate it. Create a principal manually for a simple custom login:

```
using System.Security.Claims;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Authentication.Cookies;
static ClaimsPrincipal CreatePrincipal(UserAccount account)
```

```

{
    Claim[] claims =
    [
        new(ClaimTypes.NameIdentifier, account.Id.ToString()),
        new(ClaimTypes.Name, account.DisplayName),
        new(ClaimTypes.Email, account.Email),
        new(ClaimTypes.Role, account.Role)
    ];
    var identity = new ClaimsIdentity(claims,
    CookieAuthenticationDefaults.AuthenticationScheme);
    return new ClaimsPrincipal(identity);
}

```

`ClaimsIdentity.IsAuthenticated` is true when the identity has a non-empty authentication type. The role checks performed by `IsInRole` use the identity's configured role-claim type, which is why claim mapping from external providers matters. Claim names should form a deliberate internal contract rather than depend accidentally on whatever names one provider returns. After validating credentials, the server can sign in the principal:

```

app.MapPost(
    "/account/login",
    async (LoginRequest request, HttpContext context, IUserAccounts accounts,
    CancellationToken token) =>
    {
        var account = await accounts.ValidateAsync(request.Email, request.Password, token);
        if (account is null)
            return Results.Unauthorized();
        var properties = new AuthenticationProperties
        {
            IsPersistent = request.RememberMe,
            AllowRefresh = true,
            ExpiresUtc = DateTimeOffset.UtcNow.AddHours(8)
        };
        await context.SignInAsync(
            CookieAuthenticationDefaults.AuthenticationScheme,
            CreatePrincipal(account),
            properties
        );
        return Results.NoContent();
    }
);

```

The cookie handler serializes the authentication ticket, protects it against reading and modification, and adds it to the response. On later requests, the browser sends the cookie automatically. The handler unprotects the ticket, reconstructs the principal, and assigns it to `HttpContext.User`.

Successful login

- > Server creates `ClaimsPrincipal`
- > Cookie handler protects authentication ticket
- > Browser stores cookie

Later request

- > Browser sends cookie
- > Cookie handler validates and unprotects ticket
- > `HttpContext.User` is populated

The password is never stored in the cookie. Depending on the implementation, the ticket may contain claims directly or an identifier for server-side ticket storage. Cookie protection relies on ASP.NET Core Data Protection keys. When several server instances share the application, they must share compatible keys and application identity or one instance will be unable to read cookies created by another. Key persistence and rotation therefore become deployment concerns. Signing out removes the browser's sign-in cookie:

```
app.MapPost(
    "/account/logout",
    async (HttpContext context) =>
    {
        await context.SignOutAsync(CookieAuthenticationDefaults.AuthenticationScheme);
        return Results.NoContent();
    }
);
```

Deleting the cookie does not reverse completed actions, and it may not revoke other active sessions. Applications that need central session revocation should maintain session records or validate a user security stamp rather than relying only on the ticket's expiry. A cookie ticket can have an absolute expiry, a sliding expiry, or both. Sliding expiry renews a ticket after sufficient use, extending an active session. That is convenient but can keep a stolen cookie useful for longer. High-risk applications may prefer shorter sessions, reauthentication before sensitive actions, and explicit server-side revocation. "Remember me" should not mean "never expire." It normally creates a persistent cookie that survives browser restart while still having a bounded lifetime. Session cookies without an expiry disappear when the browser session ends, although browser restoration features can blur that boundary.

The server may also need to notice account changes before the ticket expires. ASP.NET Core Identity uses a security stamp that can invalidate old sessions after password or security changes. A custom system can include a session identifier or version in the ticket and validate it periodically against shared storage. Never log raw cookies, bearer tokens, password-reset links, or other credentials. A log system is commonly accessible to more people and retained longer than the application database. Security-sensitive identifiers should be redacted or represented by non-secret correlation values. When an endpoint requires authentication and the caller is anonymous, authorization triggers a **challenge**. When the caller is authenticated but lacks permission, it triggers **forbid**.

Anonymous request to protected endpoint

- > Challenge
- > login redirect for an interactive page
- > 401 for an API endpoint

Authenticated user without permission

- > Forbid
- > access-denied page for an interactive page
- > 403 for an API endpoint

In .NET 10, known API endpoints protected by cookie authentication return 401 or 403 rather than redirecting to HTML login or access-denied pages. Interactive web pages can still redirect. This distinction prevents an API client from receiving a 200 login page after following a redirect when it expected JSON. Endpoints can also produce challenge or forbid explicitly:

```
return Results.Challenge();
return Results.Forbid();
```

Do not return 401 merely because a signed-in user lacks one permission; that user is authenticated, so the correct broad outcome is 403. Conversely, a malformed credential is not 403; the application has not established a valid identity.

11.3 Endpoint, Role, Policy, and Resource Authorization

The simplest endpoint rule requires any authenticated user:

```
app.MapPut("/api/products/{id:int}", UpdateProduct).RequireAuthorization();
```

An entire route group can share the requirement:

```
var administration = app.MapGroup("/api/admin").RequireAuthorization();
administration.MapPost("/products", CreateProduct);
administration.MapDelete("/products/{id:int}", DeleteProduct);
```

Allow a public exception with anonymous metadata:

```
app.MapGet("/api/products", ListProducts).AllowAnonymous();
```

Controllers use [Authorize] and [AllowAnonymous]:

```
[Authorize]
[ApiController]
[Route("api/admin/products")]
public sealed class ProductAdministrationController : ControllerBase
{
    [AllowAnonymous]
    [HttpGet]
    public Task<IReadOnlyList<ProductSummary>> List(Cancellation token) =>
        products.ListAsync(token);
}
```

Authorization metadata protects the endpoint before its handler runs. It is stronger and easier to review than remembering to place `if (!User.Identity.IsAuthenticated)` inside every action. Operation-specific checks may still be necessary after loading a resource, but the broad rule belongs in endpoint metadata or a policy. A fallback policy can require authentication globally, with public endpoints opting out. This is often safer than protecting endpoints one by one because newly added endpoints do not accidentally become public:

```
builder.Services.AddAuthorization(options =>
{
    options.FallbackPolicy = new
AuthorizationPolicyBuilder().RequireAuthenticatedUser().Build();
});
```

Role-based authorization checks whether the principal contains a matching role claim:

```
app.MapDelete("/api/products/{id:int}", DeleteProduct)
    .RequireAuthorization(policy => policy.RequireRole("CatalogAdministrator"));
```

Controllers and routable Razor components can use:

```
[Authorize(Roles = "CatalogAdministrator")]
```

Roles are useful when the organisation already assigns stable groups such as administrator, editor, auditor, or support agent. They become awkward when every operation needs a slightly different combination. Role names also tend to leak organisational structure into application code. A role may grant several permissions:

```
CatalogViewer
-> products.read
CatalogEditor
-> products.read
-> products.create
-> products.edit
CatalogAdministrator
-> all editor permissions
-> products.delete
-> users.manage
```

The application can still receive role claims from the identity system and translate them into policies. Avoid scattering role-name checks throughout domain and presentation code. Central policy names communicate intent more clearly and allow the underlying rules to change without rewriting every endpoint. Policy-based authorization names an access rule:

```
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy(
        "EditProducts",
        policy => policy.RequireAuthenticatedUser().RequireClaim("permission",
"products.edit")
    );
});
```

```

options.AddPolicy(
    "DeleteProducts",
    policy => policy.RequireAuthenticatedUser().RequireRole("CatalogAdministrator")
        .RequireClaim("permission", "products.delete")
);
});

```

Attach the policy to endpoints:

```

productEndpoints.MapPut("/{id:int}", UpdateProduct).RequireAuthorization("EditProducts");
productEndpoints.MapDelete("/{id:int}",
DeleteProduct).RequireAuthorization("DeleteProducts");

```

A policy contains one or more requirements. Every requirement normally needs a successful handler. Built-in requirements cover authenticated users, roles, claims, and assertions. Custom requirements handle rules that deserve their own type:

```

public sealed record MinimumAccountAgeRequirement(TimeSpan MinimumAge) :
IAuthorizationRequirement;

```

```

public sealed class MinimumAccountAgeHandler(
    TimeProvider timeProvider
) : AuthorizationHandler<MinimumAccountAgeRequirement>
{
    protected override Task HandleRequirementAsync(
        AuthorizationHandlerContext context,
        MinimumAccountAgeRequirement requirement
    )
    {
        var createdValue = context.User.FindFirst("account_created")?.Value;
        if (
            DateTimeOffset.TryParse(createdValue, out var created) &&
            timeProvider.GetUtcNow() - created >= requirement.MinimumAge
        )
        {
            context.Succeed(requirement);
        }
        return Task.CompletedTask;
    }
}

```

The policy name expresses the application rule, while the handler owns its evaluation. This is preferable to repeated claim parsing in endpoints. Authorization handlers should avoid modifying application state; they answer whether access is allowed. Expensive database checks may be necessary, but they should be designed carefully because authorization can run frequently. Endpoint metadata is evaluated before the handler has usually loaded product 42. Some rules depend on that product: the user may edit products in their assigned category, modify only records they created, or access data for their tenant. The endpoint must first load the resource and then ask `IAuthorizationService` to evaluate it.

```

public sealed record ProductOwnerRequirement
    : IAuthorizationRequirement;
public sealed class ProductOwnerHandler
    : AuthorizationHandler<ProductOwnerRequirement, Product>
{
    protected override Task HandleRequirementAsync(
        AuthorizationHandlerContext context,
        ProductOwnerRequirement requirement,
        Product resource
    )
    {
        var userId = context.User.FindFirstValue(ClaimTypes.NameIdentifier);

```

```

        if (userId == resource.OwnerId)
            context.Succeed(requirement);
        return Task.CompletedTask;
    }
}

```

The endpoint performs an imperative check:

```

static async Task<IResult> UpdateProduct(
    int id,
    UpdateProductRequest request,
    CatalogDbContext db,
    IAuthorizationService authorization,
    ClaimsPrincipal user,
    CancellationToken token
)
{
    var product = await db.Products.SingleOrDefaultAsync(value => value.Id == id, token);
    if (product is null)
        return Results.NotFound();
    var allowed = await authorization.AuthorizeAsync(user, product, new
    ProductOwnerRequirement());
    if (!allowed.Succeeded)
        return Results.Forbid();
    // Apply and save the update.
    return Results.NoContent();
}

```

The order can reveal information. Returning 404 before authorization confirms that the product exists, while returning 404 for both missing and inaccessible resources hides that fact. Choose deliberately according to the application's privacy requirements. Tenant filtering should also occur in database queries where possible. Loading another tenant's row and rejecting it later increases the chance of accidental disclosure. Authorization checks remain useful, but query boundaries should prevent unrelated data from entering the operation at all.

11.4 ASP.NET Core Identity and Passwords

Cookie authentication by itself does not create users, hash passwords, generate reset tokens, confirm email addresses, manage logout, or support multifactor authentication. ASP.NET Core Identity supplies those account-management features. It commonly stores users, password hashes, roles, claims, login providers, security stamps, and tokens through Entity Framework Core. A custom user can derive from `IdentityUser`:

```

public sealed class CatalogUser : IdentityUser
{
    public string DisplayName { get; set; } = "";
}

```

The Identity context stores the account model:

```

public sealed class CatalogIdentityDbContext(
    DbContextOptions<CatalogIdentityDbContext> options
) : IdentityDbContext<CatalogUser>(options);

```

Typical registration is:

```

builder.Services.AddIdentityCore<CatalogUser>(options =>
{
    options.User.RequireUniqueEmail = true;
    options.SignIn.RequireConfirmedEmail = true;
    options.Lockout.MaxFailedAccessAttempts = 5;
}).AddRoles<IdentityRole>().AddEntityFrameworkStores<CatalogIdentityDbContext>().
AddSignInManager()

```

```
.AddDefaultTokenProviders();
builder.Services.AddAuthentication(IdentityConstants.ApplicationScheme).AddIdentityCookies();
```

Exact registration differs depending on whether the application uses default Identity UI, custom pages, Minimal Identity APIs, external providers, or a separate identity service. The key point is that Identity manages local account lifecycle and integrates it with ASP.NET Core authentication. It is not the same thing as Microsoft Entra ID, OAuth, OpenID Connect, or an identity server. Use Identity rather than inventing password hashing, reset tokens, logout, email confirmation, and security-stamp validation from scratch unless the application has a specific account system it must integrate.

A server should not store plaintext passwords or reversible encrypted passwords. It stores a salted, intentionally expensive password hash. During login, the password hasher applies the same algorithm and parameters to the supplied password and compares the result. A salt prevents equal passwords from producing equal stored values and weakens precomputed attacks. Work factors make large-scale guessing more expensive.

Registration

```
password + random salt
-> password hashing algorithm
-> stored hash and parameters
```

Login

```
supplied password + stored parameters
-> password hashing algorithm
-> constant-time verification
```

ASP.NET Core Identity's `PasswordHasher<TUser>` handles the supported format, salt, work factor, and re-hashing decisions. Do not use a fast general-purpose hash such as SHA-256 directly for passwords, and do not build a new password-storage format with `KeyDerivation.Pbkdf2` when Identity's password hasher fits the application. Password policy should balance resistance to guessing with usability. Length matters more than forcing frequent arbitrary changes or requiring predictable character substitutions. Rate limiting, logout, breached-password detection where appropriate, multifactor authentication, and secure reset flows strengthen the account beyond the composition rule alone. A password-reset token is itself a credential. It must be unpredictable, time-limited, single-purpose, and protected from logs and analytics. Error messages for login and reset should avoid revealing whether a particular email address is registered.

11.5 External Login, OAuth, and OpenID Connect

An application can let a provider such as Google, Microsoft, or an organisational identity system authenticate the user. The Product Catalog redirects the browser to the provider, the provider authenticates the person, and the browser returns through a controlled callback. The application validates the response and establishes its own local session.

```
Browser -> Product Catalog
Product Catalog -> redirect to identity provider
Browser -> identity provider login
Identity provider -> callback with protected response
Product Catalog -> validate response
Product Catalog -> create local cookie
Browser -> signed-in Product Catalog session
```

The application never receives the provider's password. It receives identity information and tokens according to the configured protocol. A local user record may still be needed to store application roles, preferences, tenant membership, audit identity, or a link to the external subject identifier. Account linking must be careful. Matching external accounts only by an unverified email address can connect the wrong identities. Use the provider's stable issuer and subject identifiers and follow verified-email rules deliberately. Keep callback URLs exact and validate correlation and state values, which the authentication middleware normally handles. OAuth 2.0 is primarily an authorization framework for granting a client limited access to an API. OpenID Connect adds an identity layer that lets a client authenticate a user. They often appear together, which is why they are frequently confused.

OAuth access token

```
-> presented to an API
-> grants scoped access
```

OpenID Connect ID token

- > presented to the client application
- > describes the authenticated sign-in

A web application using OpenID Connect commonly follows the authorization-code flow with Proof Key for Code Exchange:

1. Application redirects browser to provider.
2. Provider authenticates user and obtains consent when needed.
3. Provider redirects back with a short-lived authorization code.
4. Application exchanges code through a back-channel request.
5. Application validates ID token and protocol state.
6. Application creates its local authentication cookie.
7. Access token is used only when calling the protected API.

The browser carries the authorization code, not the long-lived application secret. PKCE binds the code exchange to the client that initiated the flow. A confidential server-side web client also authenticates itself with a secret or assertion stored only on the server. Do not treat an access token as a login session merely because it contains readable claims, and do not send an ID token to an API as if it were an access token. Validate issuer, audience, signature, lifetime, nonce, and protocol state through established middleware rather than parsing token strings manually.

11.6 Bearer, Access, and Refresh Tokens

A client calls a protected API by adding a bearer token:

Authorization: Bearer eyJhbGciOi...

“Bearer” means possession is sufficient to use the token. Anyone who obtains it can present it until it expires or is otherwise rejected. Transport, storage, logging, browser script access, and token lifetime therefore matter. JWT is a common token format with three base64url-encoded sections:

header.payload.signature

The payload is encoded, not normally encrypted. Anyone holding the token can often read its claims. The signature lets the API detect modification and verify the issuer; it does not hide the contents. Sensitive data should not be placed in JWT claims merely because the token is signed. Register JWT bearer authentication by validating the trusted issuer, intended audience, signature keys, and lifetime:

```
using Microsoft.AspNetCore.Authentication.JwtBearer;
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme).AddJwtBearer(
    options
=>
    {
        options.Authority = builder.Configuration[
            "Authentication:Authority"
        ];
        options.Audience = "product-catalog-api";
        options.RequireHttpsMetadata = true;
    });
```

The authority publishes metadata and signing keys through standard endpoints. The API should not accept arbitrary issuers, skip audience validation, trust unsigned tokens, or make long clock-skew allowances merely to silence failures. JWTs work well for APIs called by mobile apps, service clients, or separately hosted frontends. A browser application on the same site often benefits from a secure `HttpOnly` cookie because the browser handles it without exposing the credential to JavaScript. Token choice should follow the client and deployment model, not a rule that “modern APIs must use JWT.”

Access tokens should be short-lived because bearer tokens are difficult to revoke instantly in a fully self-contained model. A refresh token can obtain new access tokens without asking the user to sign in again, but the refresh token is more powerful and normally longer-lived. It must be stored more securely, rotated where supported, and revoked when suspicious activity or sign-out occurs.

Short-lived access token

- > sent to API frequently

Longer-lived refresh token

- > sent only to authorization server
- > obtains replacement access token

A server-side web application can keep tokens in protected server storage and expose only its own cookie to the browser, often called a backend-for-frontend approach. This reduces direct token exposure in browser JavaScript. A pure browser client cannot keep a client secret and must use public-client protocol guidance. Token revocation is a trade-off. A self-contained access token can be validated without a database lookup, which improves scalability, but its claims may remain accepted until expiry. Short lifetimes, security-event checks, reference tokens, token introspection, or revocation lists can provide stronger control at additional cost.

11.7 Blazor Authentication State and Protected UI

Server-rendered and interactive server Blazor components use the authenticated ASP.NET Core user. Authentication state is exposed to components through `AuthenticationStateProvider` and cascading authentication state. A component can receive it:

```
@using Microsoft.AspNetCore.Components.Authorization
<CascadingAuthenticationState>
    <Routes />
</CascadingAuthenticationState>

@code {
    [CascadingParameter]
    private Task<AuthenticationState> AuthenticationStateTask
        { get; set; } = default!;
    private async Task<string?> GetUserIdAsync()
    {
        var state = await AuthenticationStateTask;
        return state.User.FindFirst(
            ClaimTypes.NameIdentifier
       )?.Value;
    }
}
```

Templates may configure the cascading state through services rather than explicit markup, depending on application structure. Components should normally use declarative authorization UI rather than repeatedly reading claims in code. Interactive server components execute on the server, so server-side authorization checks can protect operations. Interactive WebAssembly components execute in a browser controlled by the user. Their authentication state is useful for presentation and deciding which API calls to attempt, but it is not a trusted enforcement boundary. Every server API must authenticate and authorize independently.

Blazor UI check

- > improves user experience

Server endpoint check

- > enforces security

Never place database credentials, signing keys, client secrets, or private application rules in a WebAssembly project. Downloaded assemblies and browser memory are available to the user. A routable Razor component can require authorization:

```
@page "/admin/products"
[Authorize(Policy = "EditProducts")]
<h1>Product administration</h1>
```

`[Authorize]` on a Razor component is applied through routing, so it belongs on `@page` components. It does not automatically protect arbitrary child components rendered inside an already selected page. Use `AuthorizeView` to render different content based on authentication or policy:

```
<AuthorizeView Policy="EditProducts">
    <Authorized>
        <a href="/admin/products">Manage products</a>
    </Authorized>
    <NotAuthorized>
        <a href="/account/login">Sign in</a>
    </NotAuthorized>
</AuthorizeView>
```

```

    </NotAuthorized>
</AuthorizeView>

```

It can expose the current principal:

```

<AuthorizeView>
    <Authorized Context="authentication">
        <p>Hello, @authentication.User.Identity?.Name.</p>
    </Authorized>
</AuthorizeView>

```

`AuthorizeView` hides or displays UI; it does not secure the event handler or endpoint behind that UI. A determined user can call a public method through another path or send the HTTP request directly. The application service and server endpoint still need authorization. Component authorization can also change while a long-lived circuit remains active. A disabled account or changed claim may not appear immediately without revalidation or a new authentication state. High-risk operations should validate current server state at execution time.

11.8 CSRF, XSS, Injection, Files, and Secrets

Cross-site request forgery targets browser applications whose credentials are attached automatically, especially cookies. A user is signed in to `catalog.example.com` and then visits a malicious site. That site causes the browser to submit a state-changing request to the catalog. The browser may attach the catalog cookie even though the request originated elsewhere.

```

User is signed in to catalog.example.com
-> Malicious page submits POST to catalog.example.com
-> Browser automatically attaches catalog cookie
-> Server sees an authenticated request

```

The attacker usually cannot read the response because of same-origin rules, but it may not need to. Changing an email address, deleting a record, or initiating a transfer can still cause damage. Antiforgery protection binds the request to a value that the attacker's page cannot supply correctly. The synchronizer-token pattern commonly uses a protected cookie plus a request token included in a form field or header. ASP.NET Core form and Razor infrastructure can generate and validate these tokens. Minimal API endpoints that accept form data can require antiforgery validation, while JSON APIs need an intentional strategy based on their credential mechanism and client.

`SameSite` cookies reduce many cross-site cases but should not be the only defence for sensitive cookie-authenticated operations. `GET` endpoints must not change application state, because browsers, crawlers, prefetchers, and external links can issue them without antiforgery intent. CORS is not a substitute for CSRF protection: CORS controls whether browser code can read certain responses, while a forged request may still be sent. Bearer tokens placed explicitly in an `Authorization` header are not normally attached by the browser to a malicious origin, so they change the CSRF threat. They increase other risks when stored where injected scripts can read them. Security choices move risk; they rarely remove it completely.

Cross-site scripting occurs when untrusted content becomes executable browser code. Suppose a product description contains `<script>...</script>` and the application inserts it as raw HTML. Every user who views the product may execute the attacker's script in the Product Catalog origin. That script can perform actions as the user, read non-`HttpOnly` browser storage, alter forms, and send data elsewhere. Razor and Blazor encode ordinary string values when rendering them:

```

<p>@product.Description</p>

```

A description containing `<script>` appears as text rather than executable markup. Problems arise when code deliberately bypasses encoding:

```

@((MarkupString)product.Description)

```

Only render trusted or properly sanitised HTML through `MarkupString`. Sanitisation is different from encoding: encoding displays the characters safely, while sanitisation attempts to preserve allowed HTML and remove dangerous elements, attributes, URLs, and CSS. Use a maintained HTML sanitiser for user-authored rich text instead of writing a few string replacements. JavaScript contexts need JavaScript-safe encoding, URL contexts need validated URLs, and HTML attributes need attribute-safe handling. Avoid constructing executable script strings from untrusted data. A Content Security Policy can reduce the impact of some XSS failures, but it complements rather than replaces correct encoding and DOM ownership.

XSS can defeat many other protections because the malicious script runs in the trusted origin. `HttpOnly` cookies prevent direct cookie reading, yet the script may still send authenticated same-origin requests. Preventing script injection is therefore central to browser security. Authentication does not make input trustworthy. A signed-in user may be malicious or compromised. Continue parameterising database values, validating dynamic query structure, and avoiding shell-command construction. EF Core parameterises ordinary LINQ values, while raw SQL must use parameter-aware APIs as explained in Chapter 10.

File uploads combine several risks. The original filename is untrusted and must not become a server path directly. Limit size and accepted type, generate server-side storage names, keep uploaded files outside executable application directories, scan when the risk warrants it, and return content with safe headers. A claimed MIME type and extension are hints, not proof of content. Redirect targets also need validation. A login endpoint that accepts `returnUrl` should allow only local destinations unless external redirects are explicitly intended. Otherwise, an attacker can use the trusted domain as an open redirect into a phishing site.

Connection strings, external-provider secrets, encryption keys, signing credentials, and email-provider credentials are operational secrets. Do not place them in committed `appsettings.json`, source code, browser JavaScript, or a Blazor WebAssembly assembly. Local development can use user secrets or environment-specific protected tooling. Production should use the hosting platform's secret store, environment injection, managed identity, or another controlled provider. A secret has a lifecycle: creation, restricted access, rotation, revocation, auditing, and eventual removal. Merely hiding it from Git does not solve rotation or operator access. Prefer short-lived credentials and workload identity where infrastructure supports them. Public identifiers such as an OAuth client ID are not secrets. A server-side client secret is. The distinction depends on whether possession grants authority, not on whether the string looks random.

HTTPS protects credentials and content in transit between endpoints. It does not protect secrets embedded in downloaded client code, exposed through logs, returned in error pages, or stored with excessive permissions.

11.9 The Product Catalog Security Boundary

A practical first Product Catalog security model can remain small:

Anonymous

-> list and view products

CatalogEditor

-> create and update products

CatalogAdministrator

-> editor permissions

-> delete products

-> manage accounts

Express operation rules as policies:

```
public static class CatalogPolicies
{
    public const string EditProducts = nameof(EditProducts);
    public const string DeleteProducts = nameof(DeleteProducts);
}
```

```
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy(
        CatalogPolicies.EditProducts,
        policy => policy.RequireAuthenticatedUser().RequireClaim("permission",
"products.edit")
    );
    options.AddPolicy(
        CatalogPolicies.DeleteProducts,
        policy => policy.RequireAuthenticatedUser().RequireClaim("permission",
"products.delete")
    );
});
```

Apply them near the routes:

```

var products = app.MapGroup("/api/products");
products.MapGet("/", ListProducts).AllowAnonymous();
products.MapGet("/{id:int}", FindProduct).AllowAnonymous();
products.MapPost("/", CreateProduct).RequireAuthorization(CatalogPolicies.EditProducts);
products.MapPut("/{id:int}",
UpdateProduct).RequireAuthorization(CatalogPolicies.EditProducts);
products.MapDelete("/{id:int}",
DeleteProduct).RequireAuthorization(CatalogPolicies.DeleteProducts);

```

The Blazor UI uses the same policy names to show management actions, but the API remains authoritative:

```

<AuthorizeView Policy="@CatalogPolicies.EditProducts">
    <a href="/admin/products/new">Create product</a>
</AuthorizeView>

```

Audit security-sensitive changes with the authenticated user identifier, product identifier, operation, outcome, and trace identifier. Avoid storing full credentials or unnecessary personal data in audit entries. The authenticated user ID should come from a trusted claim, not from a request field the client can change. Assume an editor changes product 42. The browser already has a protected authentication cookie from an earlier sign-in.

Blazor form submits update

```

-> Browser sends PUT /api/products/42 with cookie
-> Cookie handler validates ticket
-> HttpContext.User receives claims
-> Authorization evaluates EditProducts
-> Endpoint binds UpdateProductRequest
-> Application service validates operation
-> EF Core updates row with concurrency token
-> Audit entry records user and product
-> Endpoint returns 200 or 204
-> Blazor updates displayed state

```

Several failures have distinct meanings:

No valid cookie	-> 401
Valid user without policy	-> 403
Product does not exist	-> 404
Input is invalid	-> 400
Product changed meanwhile	-> 409
Unexpected infrastructure	-> 500

The UI may redirect an anonymous user to sign in, display an access-denied message, preserve form input after a validation error, or reload after a conflict. Those are presentation choices. The server's status and policy enforcement remain consistent regardless of which client called it.

11.10 Testing Security

Security tests should exercise the actual boundary rather than only unit-test one claim predicate. Integration tests can send requests as anonymous users, editors, and administrators and verify the response:

Anonymous GET product	-> 200
Anonymous PUT product	-> 401
Viewer PUT product	-> 403
Editor PUT valid product	-> 200
Editor DELETE product	-> 403
Administrator DELETE product	-> 204
Expired or invalid token	-> 401
Forged antiforgery request	-> rejected

Resource-based rules need cases for owned, unowned, and missing resources. Authentication tests should cover ticket expiry, logout, revoked sessions, and claim changes where relevant. External-login tests should verify correlation failure and account-linking behaviour without relying only on a live provider. Do not disable HTTPS validation, token validation, antiforgery, or authorization in tests merely to make requests easy. Test helpers

can create valid principals or issue test credentials while preserving the production pipeline. A system that is difficult to test securely often has identity concerns spread across too many layers.

11.11 Common Mistakes and Key Ideas

Client-side visibility is not enforcement. Hidden buttons, route guards, `AuthorizeView`, and `WebAssembly` checks improve the interface, while endpoint and resource authorization protect the operation. Use policies named for operations, centralise claim mapping, and distinguish authentication failure, forbidden access, and missing resources deliberately. Do not invent password hashing or protocol processing, log credentials, place tokens in URLs, or accept weak issuer, audience, signature, or lifetime validation. Cookies, bearer access tokens, ID tokens, and refresh tokens have different purposes. CORS, authentication, authorization, antiforgery, and content encoding also solve different problems: none substitutes for the others.

Authentication never makes request data trusted. Keep untrusted strings encoded or sanitised, parameterise database commands, generate server-side file names, validate redirect targets, and keep secrets outside source control and browser code. ASP.NET Core Identity, cookie and bearer handlers, OpenID Connect middleware, claims, roles, policies, and resource handlers form a security pipeline that should be exercised through integration tests rather than trusted as scattered conditionals.

12. Application Structure

The Product Catalog now has every major technical layer needed for an ordinary business application: an ASP.NET Core host, HTTP endpoints, Blazor components, application services, Entity Framework Core persistence, authentication, and authorization. The remaining challenge is not adding another framework. It is arranging the existing code so that changes stay local, tests remain useful, and one technical concern does not quietly spread through the whole system.

Application structure is often discussed through large diagrams and rigid labels, but the practical question is simpler: **which code should know about which other code?** A product rule should not depend on `HttpContext`. A Blazor form should not construct SQL. A database entity should not automatically become an API request. An endpoint should not decide inventory policy. At the same time, a five-file feature does not need six projects and twenty interfaces merely to satisfy an architecture name.

This chapter develops a structure for the Product Catalog that is strong enough for a real application but restrained enough to remain understandable. It separates domain rules, application operations, infrastructure, HTTP, and Blazor presentation; then it connects DTOs, validation, errors, pagination, filtering, file handling, and testing to those boundaries. The goal is not to produce the maximum number of layers. It is to make dependencies point in a useful direction.

12.1 Dependency Direction and Solution Layout

A maintainable web application normally has code near the centre that expresses business meaning and code near the outside that talks to frameworks and external systems.

Browser and HTTP

- > Presentation adapters
- > Application operations
- > Domain rules

Infrastructure

- > Database, files, email, external APIs
- > Implements contracts required by application code

The centre should not need to know whether the caller was a Minimal API endpoint, controller, Blazor component, scheduled job, or integration test. It should also not need to know whether products are stored in SQLite, PostgreSQL, or memory. Outer code is allowed to depend inward because it adapts external mechanisms to application meaning. Inner code should not depend outward because that would make every business change sensitive to hosting and persistence details.

This does not require a framework. It can be achieved with ordinary projects, namespaces, interfaces at real boundaries, and constructor injection. The structure becomes valuable when it produces concrete effects: a product rule can be tested without starting a web server, an EF Core implementation can be replaced without changing endpoints, and a Blazor component can consume a stable application result rather than catch provider-specific exceptions. One reasonable solution layout is:

ProductCatalog.sln

```
src/  
  ProductCatalog.Domain/  
  ProductCatalog.Application/  
  ProductCatalog.Infrastructure/  
  ProductCatalog.Web/  
tests/  
  ProductCatalog.Domain.Tests/  
  ProductCatalog.Application.Tests/  
  ProductCatalog.Infrastructure.Tests/  
  ProductCatalog.Web.Tests/
```

`ProductCatalog.Domain` contains business concepts and rules that should remain independent of ASP.NET Core, EF Core, Blazor, and deployment. `ProductCatalog.Application` contains use cases, operation contracts, DTOs used between the application and its adapters, validation that belongs to those operations, and abstractions for required infrastructure. `ProductCatalog.Infrastructure` implements database, file, email, and external-service contracts. `ProductCatalog.Web` is the executable host and contains Minimal API mappings, authentication setup, Blazor components, HTTP-specific models where needed, and the composition root. The project references point inward:

```

Domain
    -> no project references
Application
    -> Domain
Infrastructure
    -> Application
    -> Domain
Web
    -> Application
    -> Infrastructure

```

The web project references infrastructure so that `Program.cs` can register concrete implementations. Application code does not reference infrastructure. That inversion is the important part: the application defines what it needs, and infrastructure implements it. A smaller application can combine `Domain` and `Application`, or keep everything in one project with namespaces and folders. The principles still apply. Projects create compile-time boundaries, but they also add ceremony, build work, and navigation cost. Use a separate project when it protects a meaningful dependency boundary or supports reuse and testing, not merely because a diagram contains a box.

12.2 Domain, Application, Infrastructure, and Web

The domain layer contains rules that are meaningful even without HTTP, a database, or a UI. For the Product Catalog, a product may have rules around code format, price, currency, availability, category assignment, and allowed status transitions. A domain type can enforce some of its own invariants:

```

public sealed class Product
{
    private Product()
    {
    }
    public Product(ProductCode code, string name, Money price, CategoryId? categoryId)
    {
        Code = code;
        Rename(name);
        Price = price;
        CategoryId = categoryId;
        IsAvailable = true;
    }
    public ProductId Id { get; private set; }
    public ProductCode Code { get; private set; }
    public string Name { get; private set; } = "";
    public Money Price { get; private set; }
    public CategoryId? CategoryId { get; private set; }
    public bool IsAvailable { get; private set; }
    public void Rename(string name)
    {
        var normalized = name.Trim();
        if (normalized.Length is < 1 or > 120)
            throw new DomainRuleException("Product name must contain between 1 and 120
            characters.");
        Name = normalized;
    }
    public void ChangePrice(Money price)
    {
        Price = price;
    }
    public void MarkUnavailable()
    {
        IsAvailable = false;
    }
}

```

Value objects can make invalid states harder to create:

```
public readonly record struct ProductCode
{
    public ProductCode(string value)
    {
        var normalized = value.Trim().ToUpperInvariant();
        if (
            normalized.Length is < 3 or > 40 ||
            normalized.Any(character => !char.IsLetterOrDigit(character) && character is not
'-' and not '_'')
        )
        {
            throw new DomainRuleException("Product code contains unsupported characters.");
        }
        Value = normalized;
    }
    public string Value { get; }
    public override string ToString() => Value;
}
```

This style is useful when the rule appears throughout the application and should never be bypassed. It is unnecessary for every scalar value. Turning every `string` and `int` into a type can make ordinary code difficult to read without adding real protection. Introduce a domain type when it carries meaning, validation, comparison rules, or behaviour that would otherwise be duplicated. The domain should not return `IResult`, `ProblemDetails`, `ValidationMessageStore`, or `DbUpdateException`. Those belong to outer layers. A domain exception can signal an invariant violation, but expected user outcomes are often clearer as application results, especially when several validation errors should be returned together. The application layer coordinates one meaningful operation. Examples include:

- ListProducts
- GetProductDetails
- CreateProduct
- UpdateProduct
- DeleteProduct
- UploadProductImage
- AssignProductTags

An operation loads needed state through abstractions, applies domain rules, saves changes, and returns an application-level outcome. It does not decide whether success becomes 200, 201, a Blazor navigation, or a background notification. A command model describes the input:

```
public sealed record UpdateProductCommand(
    int ProductId,
    string Name,
    decimal Price,
    string Currency,
    int? CategoryId,
    int Version
);
```

The result describes the operation outcome:

```
public enum UpdateProductStatus
{
    Success,
    NotFound,
    Invalid,
    Conflict,
    Forbidden
}

public sealed record UpdateProductResult(
    UpdateProductStatus Status,
```

```

        ProductDetails? Product = null,
        IReadOnlyDictionary<string, string[]>? Errors = null
    );

```

The service coordinates the use case:

```

public sealed class UpdateProductHandler(
    IProductRepository products,
    ICurrentPrincipal principal,
    IUnitOfWork unitOfWork,
    TimeProvider timeProvider
)
{
    public async Task<UpdateProductResult> HandleAsync(UpdateProductCommand command,
        CancellationToken token = default)
    {
        if (!principal.HasPermission("products.edit"))
            return new(UpdateProductStatus.Forbidden);
        var product = await products.FindAsync(new ProductId(command.ProductId), token);
        if (product is null)
            return new(UpdateProductStatus.NotFound);
        if (product.Version != command.Version)
            return new(UpdateProductStatus.Conflict);
        var errors = ProductInputValidation.Validate(command.Name, command.Price,
            command.Currency);
        if (errors.Count > 0)
            return new(UpdateProductStatus.Invalid, Errors: errors);
        product.Rename(command.Name);
        product.ChangePrice(new Money(command.Price, command.Currency));
        product.AssignCategory(command.CategoryId is null ? null : new
        CategoryId(command.CategoryId.Value));
        product.MarkUpdated(timeProvider.GetUtcNow());
        await unitOfWork.SaveChangesAsync(token);
        return new(UpdateProductStatus.Success, ProductMappings.ToDetails(product));
    }
}

```

This handler is independent of HTTP and Blazor. It depends on application abstractions and domain types. An endpoint can translate the status to HTTP, while a component can translate it to messages and navigation. The operation is also easy to test with fake repositories or a controlled test database. Do not create a handler class for every private method merely to imitate a pattern. An application service with several cohesive product operations may be clearer in a modest system. Split when operations gain different dependencies, authorization rules, transactions, or complexity. Infrastructure code talks to systems outside the application's core: relational databases, file systems, object storage, email providers, payment APIs, search services, and clocks supplied by the host. The application defines the contract it needs:

```

public interface IProductRepository
{
    Task<Product?> FindAsync(ProductId id, CancellationToken token = default);
    Task<PagedResult<ProductListItem>> ListAsync(ProductQuery query, CancellationToken token
= default);
    void Add(Product product);
    void Remove(Product product);
}

```

Infrastructure implements it with EF Core:

```

public sealed class EfProductRepository(CatalogDbContext db) : IProductRepository
{
    public Task<Product?> FindAsync(ProductId id, CancellationToken token = default)
    {

```

```

        return db.Products.SingleOrDefaultAsync(product => product.Id == id, token);
    }
    public async Task<PagedResult<ProductListItem>> ListAsync(ProductQuery query,
        CancellationToken token = default)
    {
        var products = db.Products.AsNoTracking().Apply(query);
        var totalCount = await products.CountAsync(token);
        var items = await products.OrderBy(product => product.Name).ThenBy(product =>
            product.Id).Skip(query.Offset)
            .Take(query.PageSize).Select(ProductProjections.ListItem).ToListAsync(token);
        return new(items, totalCount);
    }
    public void Add(Product product)
    {
        db.Products.Add(product);
    }
    public void Remove(Product product)
    {
        db.Products.Remove(product);
    }
}

```

Repositories are useful when they express domain or application operations and hide provider-specific persistence behaviour. They are less useful when they merely reproduce every `DbSet` method with names such as `GetAll`, `GetById`, `Insert`, `Update`, and `Delete`. EF Core already provides a unit-of-work and repository-like model. Add a repository when it protects query shape, aggregate boundaries, persistence details, or test seams; do not wrap EF Core solely to claim that it has been abstracted. Read-heavy operations may query `CatalogDbContext` directly from an application query service and project results without reconstructing a rich domain entity. That is still a valid inward-facing application boundary when the query remains separated from HTTP and presentation. The web project owns framework-specific concerns:

- Program.cs and DI registration
- Middleware and endpoint mapping
- Authentication and authorization configuration
- HTTP request and response contracts
- Problem Details mapping
- Blazor pages and components
- Static assets
- Browser integration

A Minimal API handler should remain a translation layer:

```

static async Task<
    Results<
        Ok<ProductDetailsResponse>,
        BadRequest<HttpValidationProblemDetails>,
        NotFound,
        Conflict,
        ForbidHttpResult
    >
> UpdateProduct(int id, UpdateProductRequest request, UpdateProductHandler handler,
    CancellationToken token)
{
    var result = await handler.HandleAsync(
        new UpdateProductCommand(
            id,
            request.Name,
            request.Price,
            request.Currency,
            request.CategoryId,
            request.Version

```

```

        ),
        token
    );
    return result.Status switch
    {
        UpdateProductStatus.Success =>
            TypedResults.Ok(ProductHttpMappings.ToResponse(result.Product!)),
        UpdateProductStatus.Invalid =>
            TypedResults.ValidationProblem(result.Errors!),
        UpdateProductStatus.NotFound =>
            TypedResults.NotFound(),
        UpdateProductStatus.Conflict =>
            TypedResults.Conflict(),
        UpdateProductStatus.Forbidden =>
            TypedResults.Forbid(),
        _ => throw new UnreachableException()
    };
}

```

This code knows HTTP status types and transport models. It does not know how the product is stored or which domain method changes its price. The handler knows application outcomes but not HTTP status codes. A Blazor component can call the same operation through an injected application facade when running on the server, or call the HTTP API when running in WebAssembly. Its job is to maintain UI state and display the result:

Application result

- > Success: update displayed product
- > Invalid: show field errors
- > Conflict: ask user to reload
- > Forbidden: show access message
- > NotFound: navigate to not-found page

The mapping is different from HTTP, which is exactly why the application result should not itself be an `IResult`.

12.3 DTOs and Mapping

A data transfer object is a shape used to cross a boundary. The word is often used broadly, but the important point is that the type exists for transfer rather than as the complete internal model. The Product Catalog may have several related shapes:

```

public sealed record CreateProductRequest(string Code, string Name, decimal Price, string
Currency, int? CategoryId);
public sealed record UpdateProductRequest(string Name, decimal Price, string Currency, int?
CategoryId, int Version);
public sealed record ProductListItemResponse(
    int Id,
    string Code,
    string Name,
    decimal Price,
    string Currency,
    string? CategoryName,
    bool IsAvailable
);
public sealed record ProductDetailsResponse(
    int Id,
    string Code,
    string Name,
    string Description,
    decimal Price,
    string Currency,
    int? CategoryId,
    string? CategoryName,

```

```

    string[] Tags,
    int Version
);

```

The create request has no identifier because the server creates it. The update request carries a concurrency version. The list response omits large descriptions and tag details. The details response contains the complete view needed by one screen. These shapes are intentionally different. Reusing one entity everywhere creates hidden coupling. Adding an EF Core navigation property can affect JSON serialization. Renaming a database column can break a client. A form can submit fields that the server never intended to accept. A cyclical entity graph can cause serialization failure. Separate contracts make each boundary explicit.

Do not duplicate types reflexively when two layers genuinely share a stable contract. An application-level `ProductDetails` can sometimes be returned directly from a server-rendered component because no network serialization boundary exists. The decision is about ownership and stability, not the belief that every layer must rename the same four properties. Mapping converts one boundary shape into another:

```

public static class ProductHttpMappings
{
    public static UpdateProductCommand ToCommand(int id, UpdateProductRequest request)
    {
        return new(id, request.Name, request.Price, request.Currency, request.CategoryId,
            request.Version);
    }
    public static ProductDetailsResponse ToResponse(ProductDetails product)
    {
        return new(
            product.Id,
            product.Code,
            product.Name,
            product.Description,
            product.Price,
            product.Currency,
            product.CategoryId,
            product.CategoryName,
            product.Tags.ToArray(),
            product.Version
        );
    }
}

```

Manual mapping is often preferable for important contracts because every transferred field is visible. Mapping libraries can reduce repetitive property assignment, but they also make missing or unintended mappings less obvious and add configuration that must be tested. Use one when it removes significant real repetition without hiding security-sensitive or meaning-changing transformations. Mapping is a good place for transport conversion, not business rules. Currency normalization, permission checks, and category validity belong in the application or domain. Renaming `CategoryName` to `category` for a response belongs in the adapter.

12.4 Validation, Errors, and Problem Details

Validation is not one universal method. Different layers answer different questions:

HTTP binding

-> Can the request be parsed?

Presentation validation

-> Can the user receive immediate feedback?

Application validation

-> Is the requested operation complete and meaningful?

Domain validation

-> Would the resulting state violate an invariant?

Database constraints

-> Can invalid state be stored despite another writer or bug?

A request model can use data annotations for obvious transport rules:

```
public sealed record CreateProductRequest(
    [property: Required, StringLength(40)] string Code,
    [property: Required, StringLength(120)] string Name,
    [property: Range(0, 1_000_000)] decimal Price,
    [property: Required, StringLength(3, MinimumLength = 3)]
    string Currency,
    int? CategoryId
);
```

Blazor can use the same annotations for immediate form feedback. The application must still validate independently because clients can bypass the UI, and not every operation enters through model binding. Domain objects should reject states that must never exist. Database constraints protect the final stored state. Cross-field and state-dependent rules rarely fit simple annotations. “A discontinued product cannot be marked available,” “the category must exist,” and “the product code must be unique” need application or domain logic plus persistence checks. Return validation errors in a stable field-oriented form:

```
public sealed class ValidationErrors
{
    private readonly Dictionary<string, List<string>> _errors = [];
    public bool IsValid => _errors.Count == 0;
    public void Add(string field, string message)
    {
        if (!_errors.TryGetValue(field, out var messages))
        {
            messages = [];
            _errors[field] = messages;
        }
        messages.Add(message);
    }
    public IReadOnlyDictionary<string, string[]> ToDictionary()
    {
        return _errors.ToDictionary(pair => pair.Key, pair => pair.Value.ToArray());
    }
}
```

Field keys form part of the client contract. Use names the client can understand rather than persistence paths such as `Product.Category.Navigation.Name`. Avoid validating the same rule in five handwritten places. Shared operation validation can be reused by the HTTP adapter and interactive server components when appropriate, but the authoritative check remains on the server and domain boundary. Applications become difficult to maintain when every method invents its own failure style: `null`, `false`, exceptions, tuples, strings, and status codes. A small vocabulary makes outcomes predictable. One approach is a generic result:

```
public enum ErrorType
{
    Validation,
    NotFound,
    Conflict,
    Forbidden,
    Unavailable,
    Unexpected
}
public sealed record Error(
    string Code,
    string Message,
    ErrorType Type,
    IReadOnlyDictionary<string, string[]>? Details = null
);
public sealed record Result<T>(T? Value, Error? Error)
{
    public bool IsSuccess => Error is null;
```

```

    public static Result<T> Success(T value) => new(value, null);
    public static Result<T> Failure(Error error) => new(default, error);
}

```

The application returns errors meaningful to callers:

```

product.not-found
product.code-conflict
product.version-conflict
product.invalid
catalog.unavailable

```

The HTTP layer maps them:

```

Validation -> 400
Forbidden -> 403
NotFound -> 404
Conflict -> 409
Unavailable -> 503
Unexpected -> 500

```

The Blazor layer maps the same errors to field messages, alerts, reload prompts, or navigation. Stable codes are more useful than matching exact English text. User-facing messages can be localised or improved without changing client logic. Exceptions remain appropriate for unexpected failures, broken invariants, and infrastructure problems that cannot be handled meaningfully at the current layer. Expected “product not found” or “code already exists” outcomes should not require exception control flow through the entire stack. Infrastructure may throw `DbUpdateException`, but its adapter should translate known constraint failures into application errors and let unknown failures propagate to central exception handling. ASP.NET Core supports RFC-style Problem Details for API errors. Register it:

```
builder.Services.AddProblemDetails();
```

Unexpected exceptions can be converted through exception handling middleware, while expected errors can return deliberate problem responses:

```

return TypedResults.Problem(
    statusCode: StatusCodes.Status409Conflict,
    title: "Product update conflict",
    detail: "The product changed after it was opened.",
    type: "https://example.com/problems/product-version-conflict",
    extensions: new Dictionary<string, object?>
    {
        ["code"] = "product.version-conflict",
        ["traceId"] = Activity.Current?.Id
    }
);

```

Validation problems use a field dictionary:

```
return TypedResults.ValidationProblem(errors, title: "Product validation failed");
```

Do not expose stack traces, SQL, connection strings, filesystem paths, or provider messages in the problem body. The full exception belongs in server logs, correlated through a trace identifier. The client needs a stable code, broad category, safe explanation, and enough context to recover. A central mapper avoids writing one large switch in every endpoint:

```

public static IActionResult ToHttpResult<T>(this Result<T> result, Func<T, IActionResult> success)
{
    if (result.IsSuccess)
        return success(result.Value!);
    return result.Error!.Type switch
    {
        ErrorType.Validation =>
            TypedResults.ValidationProblem(result.Error.Details!),
    }
}

```

```

        ErrorType.NotFound =>
            TypedResults.NotFound(),
        ErrorType.Conflict =>
            TypedResults.Conflict(
                new { result.Error.Code, result.Error.Message }
            ),
        ErrorType.Forbidden =>
            TypedResults.Forbid(),
        ErrorType.Unavailable =>
            TypedResults.Problem(statusCode: 503, title: result.Error.Message),
        _ => TypedResults.Problem(statusCode: 500)
    };
}

```

Keep the mapping explicit enough that endpoint contracts remain understandable and OpenAPI metadata remains accurate.

12.5 Pagination, Filtering, and Sorting

An unbounded GET `/api/products` works with ten rows and becomes dangerous with one million. Pagination limits database work, object materialisation, serialization, transfer, browser memory, and DOM size. A simple query model can validate limits:

```

public sealed record ProductQuery(
    int Page = 1,
    int PageSize = 20,
    string? Search = null,
    int? CategoryId = null,
    bool? IsAvailable = null,
    string Sort = "name",
    string Direction = "asc"
)
{
    public int Offset => (Page - 1) * PageSize;
}

```

The application should enforce bounds:

```

public static Result<ProductQuery> Normalize(ProductQuery query)
{
    if (query.Page < 1)
        return Result<ProductQuery>.Failure(Errors.InvalidPage);
    if (query.PageSize is < 1 or > 100)
        return Result<ProductQuery>.Failure(Errors.InvalidPageSize);
    return Result<ProductQuery>.Success(
        query with
        {
            Search = query.Search?.Trim(),
            Sort = query.Sort.Trim().ToLowerInvariant(),
            Direction = query.Direction.Trim().ToLowerInvariant()
        }
    );
}

```

A paged response includes items and navigation information:

```

public sealed record PagedResponse<T>(IEnumerable<T> Items, int Page, int PageSize, int
TotalCount)
{
    public int TotalPages =>
        (int)Math.Ceiling((double)TotalCount / PageSize);
}

```

Counting every row can be expensive for very large or complex queries. Some interfaces need an exact count; others work better with “has more” or a continuation token. Choose based on the user experience rather than automatically returning `TotalCount`. Offset pagination supports direct page numbers but becomes expensive at deep offsets and can shift when rows are inserted or deleted. Keyset pagination uses the last ordered key:

GET /api/products?afterName=Mouse&afterId=42&pageSize=20

The query requests values after the last (Name, Id) pair. It scales better for forward scrolling but does not naturally jump to page 500. The stable ordering must include a unique tiebreaker. Filters should be applied in the database before materialisation:

```
public static IQueryable<Product> Apply(this IQueryable<Product> products, ProductQuery query)
{
    if (!string.IsNullOrEmpty(query.Search))
    {
        products = products.Where(product => product.Name.Contains(query.Search));
    }
    if (query.CategoryId is not null)
    {
        products = products.Where(product => product.CategoryId == query.CategoryId);
    }
    if (query.IsAvailable is not null)
    {
        products = products.Where(product => product.IsAvailable == query.IsAvailable);
    }
    return products;
}
```

Sorting should not insert a client-supplied column name into raw SQL. Map accepted values to expressions:

```
public static IOrderedQueryable<Product> Order(this IQueryable<Product> products, ProductQuery query)
{
    var descending = query.Direction == "desc";
    return query.Sort switch
    {
        "name" => descending
            ? products.OrderByDescending(product => product.Name)
            : products.OrderBy(product => product.Name),
        "price" => descending
            ? products.OrderByDescending(product => product.Price)
            : products.OrderBy(product => product.Price),
        "updated" => descending
            ? products.OrderByDescending(product => product.UpdatedAt)
            : products.OrderBy(product => product.UpdatedAt),
        _ => products.OrderBy(product => product.Name)
    };
}
```

Always add a unique secondary order before paging:

```
var ordered = queryable.Order(query).ThenBy(product => product.Id);
```

Search behaviour depends on collation and provider. `Contains` may be case-sensitive or insensitive depending on the database. Leading-wildcard text search may not use a normal index. For larger catalogues, full-text search or a dedicated search system may become appropriate. Begin with the real requirement, inspect generated SQL and query plans, and avoid pretending that every search problem is solved by loading rows and using `StringComparison` in memory.

12.6 File Uploads and Replacement

A product image upload begins as an HTTP multipart form, becomes a stream, passes application validation, is stored by infrastructure, and produces a reference saved with the product.

Browser InputFile

- > multipart/form-data
- > HTTP endpoint
- > application upload operation
- > file storage abstraction
- > database metadata

The application contract should describe the content without depending on `IFormFile`:

```
public sealed record UploadProductImageCommand(  
    int ProductId,  
    string OriginalFileName,  
    string ContentType,  
    long Length,  
    Stream Content  
);
```

The HTTP adapter converts `IFormFile` into that command:

```
static async Task<IResult> UploadProductImage(  
    int id,  
    IFormFile file,  
    UploadProductImageHandler handler,  
    CancellationToken token  
)  
{  
    const long maxFileSize = 5 * 1024 * 1024;  
    if (file.Length > maxFileSize)  
        return TypedResults.BadRequest(new { error = "The file exceeds the 5 MB limit." });  
  
    await using var stream = file.OpenReadStream();  
    var result = await handler.HandleAsync(new(id, file.FileName, file.ContentType,  
        file.Length, stream), token);  
    return result.ToHttpResult(image => TypedResults.Ok(image));  
}
```

The application validates size, allowed media types, product existence, and authorization. Infrastructure generates a storage key and writes the stream:

```
public interface IFileStorage  
{  
    Task<StoredFile> SaveAsync(Stream content, string extension, CancellationToken token =  
default);  
    Task<Stream?> OpenReadAsync(string storageKey, CancellationToken token = default);  
    Task DeleteAsync(string storageKey, CancellationToken token = default);  
}
```

Do not trust the original filename as a path. It may contain traversal sequences, invalid characters, misleading extensions, or names that overwrite existing files. Generate a server-side key and store the original name only as display metadata. Do not trust the client-provided content type alone; inspect signatures where relevant and consider image decoding or malware scanning according to risk. Store uploads outside the executable web root unless direct public serving is intentional. Private files should be returned through an authorised endpoint or signed temporary URL. Public product images may live in object storage or a controlled static-content location, but users should not be able to upload executable scripts into the application's origin.

File metadata belongs in the database; large file bytes often belong in filesystem or object storage. Database storage can still be appropriate for small, transactional files, but it increases backup size and database load. Choose according to consistency, scale, operational tooling, and access patterns. A database transaction cannot

automatically roll back an object-storage write. Replacing a product image across two systems needs a deliberate order. A safe approach is:

1. Store new file under a new key.
2. Save database reference to new key.
3. Commit database change.
4. Delete old file after success.

If step 2 or 3 fails, delete the unused new file through cleanup logic. If step 4 fails, the product still points to the correct new file, while an orphan remains for later cleanup. The opposite order—deleting the old file before committing the new reference—can leave the product with no valid image. For highly reliable workflows, record cleanup or delivery work in an outbox table inside the database transaction and let a background worker perform it. The same pattern appears with email and notifications in Chapter 13.

12.7 Composition in Program.cs

As the application grows, Program.cs should remain readable:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddApplication().AddInfrastructure(builder.Configuration).
AddProductCatalogWeb();
var app = builder.Build();
app.UseProductCatalogPipeline();
app.MapProductCatalogEndpoints();
app.MapRazorComponents<App>().AddInteractiveServerRenderMode();
app.Run();
```

Extension methods group related registration:

```
public static class ApplicationRegistration
{
    public static IServiceCollection AddApplication(this IServiceCollection services)
    {
        services.AddScoped<CreateProductHandler>();
        services.AddScoped<UpdateProductHandler>();
        services.AddScoped<DeleteProductHandler>();
        services.AddScoped<ListProductsHandler>();
        return services;
    }
}
```

Infrastructure registration selects implementations:

```
public static IServiceCollection AddInfrastructure(this IServiceCollection services,
IConfiguration configuration)
{
    services.AddDbContext<CatalogDbContext>(options =>
options.UseSqlite(configuration.GetConnectionString("Catalog")));
    services.AddScoped<IProductRepository, EfProductRepository>();
    services.AddScoped<IUnitOfWork>(provider =>
provider.GetRequiredService<CatalogDbContext>());
    services.AddSingleton<IFileStorage, LocalFileStorage>();
    return services;
}
```

Do not hide the entire application behind dozens of tiny registration methods. Group by meaningful subsystem, and keep it possible to discover which implementations are selected. Composition code should be concise, not mysterious.

12.8 Testing Domain and Application Boundaries

Different tests answer different questions. A domain test verifies rules without frameworks. An application test verifies operation coordination. An infrastructure test verifies mapping and provider behaviour. A web test verifies the actual HTTP or component boundary.

Domain test
 -> Product rejects negative price
 Application test
 -> UpdateProduct returns conflict for stale version
 Infrastructure test
 -> EF query filters and orders correctly in SQLite
 Web integration test
 -> PUT /api/products/42 returns 403 for viewer
 Blazor component test
 -> validation errors appear beside fields

A test pyramid is not a fixed ratio. The important idea is to place each assertion at the cheapest boundary that can prove it. Do not start Kestrel and a database to test a pure price rule. Do not use a mocked repository to prove that an EF Core query translates correctly. Do not unit-test a route string by invoking a private handler and then assume the HTTP endpoint is mapped correctly. A domain rule test is fast and direct:

```
public sealed class ProductTests
{
    [Fact]
    public void RenameRejectsBlankName()
    {
        var product = ProductFactory.Create();
        var action = () => product.Rename(" ");
        action.Should().Throw<DomainRuleException>().WithMessage("*name*");
    }
    [Fact]
    public void MarkUnavailableChangesAvailability()
    {
        var product = ProductFactory.Create();
        product.MarkUnavailable();
        product.IsAvailable.Should().BeFalse();
    }
}
```

These tests need no dependency injection container, HTTP client, browser, or database. They document invariants and behaviour. Avoid testing automatic property assignments that contain no meaningful logic merely to increase coverage. An application handler can be tested with fakes that preserve useful behaviour:

```
[Fact]
public async Task UpdateReturnsConflictForStaleVersion()
{
    var product = ProductFactory.Create(version: 4);
    var repository = new FakeProductRepository(product);
    var handler = new UpdateProductHandler(
        repository,
        new FakeCurrentPrincipal("products.edit"),
        new FakeUnitOfWork(),
        new FakeTimeProvider()
    );
    var result = await handler.HandleAsync(new(product.Id.Value, "Keyboard", 119m, "EUR",
        null, 3));
    result.Status.Should().Be(UpdateProductStatus.Conflict);
}
```

Use a fake or mock to control an external boundary and observe a meaningful interaction. Avoid mocking every concrete class in a chain. If the test needs ten mocks to construct one handler, the operation may have too many responsibilities or the test may be coupled to implementation details. Application tests should verify outcomes, changed domain state, and important side effects such as one save or one notification request. They should not assert every internal method call.

12.9 Testing Infrastructure, HTTP, Authentication, and Components

EF Core's in-memory provider does not behave like a relational database. It does not enforce all relational constraints, translate SQL, or reproduce provider-specific behaviour. Use the actual production provider when practical, or SQLite when the production-independent relational behaviour is what matters. A SQLite test can create a temporary database:

```
public sealed class CatalogDbTest : IAsyncLifetime
{
    private readonly SQLiteConnection _connection =
        new("Data Source=:memory:");
    public CatalogDbContext Db { get; private set; } = default!;
    public async Task InitializeAsync()
    {
        await _connection.OpenAsync();
        var options =
            new DbContextOptionsBuilder<CatalogDbContext>().UseSqlite(_connection).Options;
        Db = new CatalogDbContext(options);
        await Db.Database.EnsureCreatedAsync();
    }
    public async Task DisposeAsync()
    {
        await Db.DisposeAsync();
        await _connection.DisposeAsync();
    }
}
```

The connection must remain open for an in-memory SQLite database to survive across contexts. For PostgreSQL or SQL Server-specific behaviour, use the real engine through a disposable test database or container. Test migrations, constraints, concurrency, collation, and generated queries where those details matter. Infrastructure tests should verify more than repository return values. They can prove that unique constraints reject duplicates, delete behaviour matches the model, projections translate, and concurrency tokens produce conflicts. `WebApplicationFactory<TEnterpoint>` can host the application in memory and provide an `HttpClient` that sends requests through the ASP.NET Core pipeline. The web project exposes its generated `Program` type to the test assembly:

```
public partial class Program;
```

A test can then send a real request:

```
public sealed class ProductEndpointsTests(
    WebApplicationFactory<Program> factory
) : IClassFixture<WebApplicationFactory<Program>>
{
    [Fact]
    public async Task UnknownProductReturnsNotFound()
    {
        using var client = factory.CreateClient();
        var response = await client.GetAsync("/api/products/999999");
        response.StatusCode.Should().Be(HttpStatusCode.NotFound);
    }
}
```

The test covers routing, model binding, middleware, serialization, authentication configuration, and endpoint mapping. Replace infrastructure registrations for deterministic tests:

```
var factory = new WebApplicationFactory<Program>().WithWebHostBuilder(builder =>
{
    builder.ConfigureServices(services =>
    {
        services.RemoveAll<IProductRepository>();
        services.AddSingleton<
            IProductRepository,
```

```

        FakeProductRepository
    >();
    });
});

```

For database integration, replace the connection with a temporary relational database and apply migrations. Do not remove authorization globally for tests that are supposed to prove authorization behaviour. Instead, register a test authentication scheme that creates principals with controlled claims. A test handler can authenticate requests from a simple header:

```

public sealed class TestAuthenticationHandler(
    IOptionsMonitor<AuthenticationSchemeOptions> options,
    ILoggerFactory logger,
    UrlEncoder encoder
) : AuthenticationHandler<AuthenticationSchemeOptions>(options, logger, encoder)
{
    protected override Task<AuthenticateResult> HandleAuthenticateAsync()
    {
        if (!Request.Headers.TryGetValue("X-Test-User", out var userId))
        {
            return Task.FromResult(AuthenticateResult.NoResult());
        }
        var claims = new List<Claim>
        {
            new(ClaimTypes.NameIdentifier, userId.ToString())
        };
        foreach (var permission in Request.Headers["X-Test-Permission"])
        {
            claims.Add(new("permission", permission!));
        }
        var identity = new ClaimsIdentity(claims, Scheme.Name);
        var principal = new ClaimsPrincipal(identity);
        var ticket = new AuthenticationTicket(principal, Scheme.Name);
        return Task.FromResult(AuthenticateResult.Success(ticket));
    }
}

```

Tests can now assert 401, 403, and success while keeping authorization middleware and policies active. The test scheme belongs only in the test host. Blazor components can be tested with a component testing library such as bUnit. A component test renders the component with controlled services and interacts with its DOM.

```

[Fact]
public void ProductEditorShowsNameValidation()
{
    using var context = new TestContext();
    var component = context.RenderComponent<ProductEditor>();
    component.Find("form").Submit();
    component.Markup.Should().Contain("A product name is required.");
}

```

Useful component tests cover conditional rendering, event callbacks, validation messages, loading states, and policy-dependent UI. They should not attempt to prove browser layout, CSS rendering, or JavaScript library behaviour that requires a real browser. End-to-end tests with Playwright are better for critical workflows involving actual navigation, focus, JavaScript, and browser security. Use end-to-end tests selectively. They are slower and more fragile than lower-level tests, but one realistic path such as login, create product, edit product, and verify update can catch integration failures no isolated test sees.

12.10 Feature Organisation Without Ceremony

A project containing hundreds of files becomes difficult to navigate when everything is grouped only by technical type:

```
Controllers/  
Services/  
Repositories/  
Models/  
Validators/  
Mappings/
```

A feature-oriented structure keeps related code closer:

```
ProductCatalog.Application/  
  Products/  
    CreateProduct/  
      CreateProductCommand.cs  
      CreateProductHandler.cs  
      CreateProductResult.cs  
    UpdateProduct/  
      UpdateProductCommand.cs  
      UpdateProductHandler.cs  
    ListProducts/  
      ProductQuery.cs  
      ListProductsHandler.cs  
      ProductListItem.cs
```

The web project can mirror the feature:

```
ProductCatalog.Web/  
  Features/  
    Products/  
      ProductEndpoints.cs  
      ProductHttpMappings.cs  
      ProductContracts.cs  
    Components/  
      Products.razor  
      ProductEditor.razor  
      ProductCard.razor
```

Do not create a folder for every single file. Group when several files form one feature or responsibility. A compact feature with one handler and one result can remain together until splitting improves navigation. Feature organisation and layered projects solve different problems. Projects protect dependency direction; feature folders reduce the distance between related code. Several warning signs indicate that structure has become ceremony:

An interface has exactly one trivial implementation and no boundary value.
A repository wraps DbSet with identical methods.
Every operation passes through five classes that only forward arguments.
Mappings copy the same shape repeatedly without ownership differences.
One feature requires editing ten projects for a small field.
Tests mock every layer but never prove the actual system works.

Abstraction should remove volatility, protect a boundary, name a concept, or improve testing. It should not merely make the project resemble a diagram. The opposite extreme is also costly: one web project containing endpoint lambdas, EF Core entities, queries, domain rules, file access, and component code with no internal boundaries. That design is easy for the first week and expensive after several features. A useful rule is to introduce the smallest structure that makes current dependencies correct and current code understandable, while leaving room to extract a boundary when a real second implementation, source of complexity, or testing need appears.

12.11 The Complete Structure

An update request now crosses clear boundaries:

```
PUT /api/products/42  
-> Product endpoint binds UpdateProductRequest  
-> HTTP mapping creates UpdateProductCommand  
-> UpdateProductHandler checks permission and loads product  
-> Product domain methods enforce invariants
```

- > Repository tracks persistence entity
- > Unit of work calls SaveChanges
- > EF Core sends conditional UPDATE
- > Application returns UpdateProductResult
- > Endpoint maps result to HTTP
- > Browser receives response
- > Blazor updates UI state

No layer is unnecessary in this path. The endpoint owns HTTP. The application handler owns the operation. The domain owns product invariants. Infrastructure owns EF Core. Blazor owns presentation state. Each can change for reasons local to its responsibility. An interactive server component may call `UpdateProductHandler` directly rather than going through HTTP:

Blazor event

- > UpdateProductHandler
- > Domain
- > Infrastructure
- > Application result
- > Component state

A WebAssembly component must cross the HTTP boundary:

Blazor WebAssembly

- > HttpClient
- > Product endpoint
- > Application handler
- > Domain
- > Infrastructure

The application operation remains the same. Only the adapter path differs. Do not let framework types leak inward without a reason. `HttpContext`, `IFormFile`, `NavigationManager`, `EditContext`, `ProblemDetails`, and EF Core exceptions belong near their framework boundaries. Translate them into application models and results. Do not use database entities as universal models. Request, response, form, query, and persistence shapes often differ because they have different owners and security requirements. Map deliberately and avoid allowing clients to set internal fields through over-posting. Do not confuse validation with domain correctness or database integrity. Validate at the appropriate boundaries, preserve domain invariants, and keep database constraints. Return stable error codes and structured details rather than raw provider messages or arbitrary strings.

Do not load unbounded collections, apply filters after materialisation, or accept arbitrary sort expressions. Bound page size, use stable ordering, and map accepted filter and sort values to database queries. Do not trust filenames, extensions, content types, or file paths supplied by the browser. Stream large files, generate storage keys, keep private files outside public roots, and design replacement across database and file storage as a multi-system workflow. Do not test every concern through one test style. Pure rules deserve pure tests, provider translation needs a relational database, endpoint contracts need the actual ASP.NET Core pipeline, and browser interaction needs component or end-to-end tests.

Finally, do not let architecture become an end in itself. A structure is successful when a developer can locate the code for one feature, understand its dependencies, change one concern without unrelated edits, and verify the result with an appropriate test.

Maintainable structure follows dependency direction. Domain code expresses business meaning, application code coordinates use cases, infrastructure implements databases and external systems, and the web project adapts HTTP and Blazor while composing the concrete application. DTOs protect boundaries, mapping remains explicit, and validation, domain invariants, and database constraints answer different questions. Stable application errors can become Problem Details or UI states without exposing provider details. Pagination, filtering, sorting, and uploads must keep work bounded and secure. Tests should run at the cheapest boundary that proves the behaviour: pure rules, application coordination, real provider behaviour, the HTTP pipeline, component rendering, or a small number of browser journeys. Architecture is useful only when it makes ownership, change, and verification clearer.

Part VI — Running a Real Application

13. Features Beyond Request and Response

Most examples so far have followed one clean path: a client sends a request, the server performs an operation, and the response returns before the request ends. That model covers a large part of web development, but real applications also need work that outlives one request. A product change may need to notify connected browsers, invalidate caches, send email, generate thumbnails, write audit exports, retry a temporarily unavailable provider, or continue after the original user closes the tab. Some work should happen immediately but outside the caller's response time; other work must be durable enough to survive process restarts and server failures.

These features are often grouped under words such as background jobs, real-time communication, notifications, and caching. They solve different problems. SignalR delivers messages to currently connected clients. A background worker executes work independently of the request that scheduled it. A durable queue or outbox prevents important work from disappearing when a process stops. A cache avoids repeating expensive reads but must never become the only copy of business data. Email reaches users who are not connected, but delivery is slow, external, and not guaranteed merely because an API accepted the message.

This chapter connects those mechanisms through the Product Catalog. When an editor changes product 42, the database update should commit once, connected viewers should learn about the change, cached product data should become stale no longer, and selected users may receive email. The chapter begins with hosted services and work queues, then builds reliable delivery through an outbox, introduces SignalR and notification channels, and finishes with caching and operational concerns.

13.1 The Request Boundary and Fire-and-Forget Work

A request should normally remain open until the operation that determines its outcome has completed. If updating a product requires a database transaction, the endpoint should not return success before that transaction commits. The client needs to know whether the update succeeded, failed validation, encountered a conflict, or could not reach the database. Other effects do not need to delay the response. Sending three hundred notification emails, generating several image variants, or refreshing a search index may take seconds or minutes. Holding the HTTP request open for that work increases latency, consumes resources, and creates ambiguous failures when the client disconnects midway. A useful division is:

Inside the request

- > validate command
- > authorize operation
- > change authoritative state
- > commit transaction
- > return stable result

Outside the request

- > send email
- > broadcast update
- > generate derived files
- > retry external calls
- > rebuild expensive projections

The distinction is not simply “fast” versus “slow.” The deciding question is whether the response would be dishonest if the work had not completed. A product update should not return 200 OK before the product exists in durable storage. An email notification may legitimately be queued and delivered later, so the update can succeed even if the mail provider is temporarily unavailable. This code looks convenient:

```
app.MapPost(
    "/api/products/{id:int}/publish",
    async (int id, PublishProductHandler handler, IEmailSender email, CancellationToken
    token) =>
    {
        var result = await handler.HandleAsync(id, token);
        _ = email.SendProductPublishedAsync(id, token);
        return result.IsSuccess
            ? Results.NoContent()
            : Results.BadRequest();
    }
);
```

```
    }
};
```

The email task is started and then ignored. It still uses the request cancellation token, so the client disconnecting can cancel it. Its scoped dependencies may be disposed when the request ends. An exception has no clear owner and may be unobserved. The process can stop before delivery completes. The caller receives success even though the application has not recorded that email still needs to be sent.

`Task.Run` does not solve those problems. It moves work to the thread pool but does not give it durable ownership, retries, lifecycle management, or a valid dependency injection scope. Fire-and-forget work is acceptable only for genuinely disposable effects where loss and failure are explicitly harmless, such as an optional in-process metric that has another source. Business work should be handed to a queue or recorded durably before the request ends.

13.2 Hosted Services, Scopes, and In-Process Queues

ASP.NET Core can run background work through hosted services. A hosted service implements `IHostedService`, while `BackgroundService` supplies a simpler base class for a long-running asynchronous loop.

```
public sealed class CatalogMaintenanceWorker(ILogger<CatalogMaintenanceWorker> logger) :
    BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            logger.LogInformation(
                "Catalog maintenance started at {Time}",
                DateTimeOffset.UtcNow
            );
            await Task.Delay(TimeSpan.FromMinutes(10), stoppingToken);
        }
    }
}
```

Register it with the host:

```
builder.Services.AddHostedService<CatalogMaintenanceWorker>();
```

The host starts the service with the application and signals cancellation during shutdown. The worker should observe that token, stop accepting new work, and finish or abandon current work according to a deliberate policy. Exceptions escaping `ExecuteAsync` can stop the host depending on configuration, which is often preferable to silently losing a critical worker. Expected per-item failures should be handled inside the loop so that one bad message does not terminate all processing. Hosted services are registered as singletons. No request scope is created for them automatically. A worker must not inject a scoped `CatalogDbContext` directly and keep it for its whole lifetime. A background worker often needs scoped services such as an EF Core context or application handler. It should create a new scope for one batch or one message:

```
public sealed class OutboxWorker(IServiceScopeFactory scopeFactory, ILogger<OutboxWorker>
    logger) : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            await using var scope =
                scopeFactory.CreateAsyncScope();
            var dispatcher = scope.ServiceProvider.GetRequiredService<OutboxDispatcher>();
            var processed = await dispatcher.DispatchBatchAsync(stoppingToken);
            if (processed == 0)
            {
                // ...
            }
        }
    }
}
```

```

        await Task.Delay(TimeSpan.FromSeconds(2), stoppingToken);
    }
}
}
}

```

The scope owns the context and other scoped services. Disposing it releases those resources before the next iteration. Creating one context per item is simple but may add overhead; creating one per bounded batch is often a good compromise. Do not keep one context for the entire worker lifetime because tracked entities accumulate, database state becomes stale, and a failed context may remain unusable. The worker is infrastructure. The scoped `OutboxDispatcher` owns the operation. This keeps the long-running loop small and allows its batch logic to be tested independently. For temporary work that may be lost on restart, an in-process queue is enough. `Channel<T>` provides an asynchronous producer-consumer queue with optional bounds and backpressure.

```

public sealed record CatalogWorkItem(string Kind, int ProductId);
public interface ICatalogWorkQueue
{
    ValueTask EnqueueAsync(CatalogWorkItem item, CancellationToken token = default);
    IAsyncEnumerable<CatalogWorkItem> ReadAllAsync(CancellationToken token = default);
}

```

```

public sealed class CatalogWorkQueue : ICatalogWorkQueue
{
    private readonly Channel<CatalogWorkItem> _channel =
        Channel.CreateBounded<CatalogWorkItem>(
            new BoundedChannelOptions(500)
            {
                FullMode = BoundedChannelFullMode.Wait,
                SingleReader = true,
                SingleWriter = false
            }
        );
    public ValueTask EnqueueAsync(CatalogWorkItem item, CancellationToken token = default)
    {
        return _channel.Writer.WriteAsync(item, token);
    }
    public IAsyncEnumerable<CatalogWorkItem> ReadAllAsync(CancellationToken token = default)
    {
        return _channel.Reader.ReadAllAsync(token);
    }
}

```

A worker consumes the queue:

```

public sealed class CatalogWorkQueueWorker(
    ICatalogWorkQueue queue,
    IServiceScopeFactory scopeFactory,
    ILogger<CatalogWorkQueueWorker> logger
) : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        await foreach (var item in queue.ReadAllAsync(stoppingToken))
        {
            try
            {
                await using var scope =
                    scopeFactory.CreateAsyncScope();
                var processor =
                    scope.ServiceProvider.GetRequiredService<CatalogWorkProcessor>();
                await processor.ProcessAsync(item, stoppingToken);
            }
            catch { }
        }
    }
}

```

```

    }
    catch (OperationCanceledException)
        when (stoppingToken.IsCancellationRequested)
    {
        break;
    }
    catch (Exception exception)
    {
        logger.LogError(
            exception,
            "Catalog work item {Kind} for product {ProductId} failed",
            item.Kind,
            item.ProductId
        );
    }
}
}
}
}

```

Register the queue as one singleton and the worker separately:

```

builder.Services.AddSingleton<
    ICatalogWorkQueue,
    CatalogWorkQueue
>();
builder.Services.AddHostedService<CatalogWorkQueueWorker>();

```

A bounded queue prevents unlimited memory growth. When it is full, producers wait, which applies backpressure rather than quietly allocating until the process fails. Other policies can drop old or new items, but loss must be an explicit decision. This queue is process-local. Restarting the application removes pending items, and another server instance has a different queue. It suits cache warming, optional thumbnail regeneration, and other reconstructible work. It is not sufficient for billing, email confirmation, audit export, or any action the application promises to complete.

13.3 Durable Outbox Delivery

Important background work needs durable ownership. A database-backed job table, external message broker, or cloud queue can hold work independently of one process. The Product Catalog already has a relational database, so an **outbox** is a natural first step. The outbox stores messages in the same database transaction as the business change:

```

Transaction begins
  -> Update Product row
  -> Insert OutboxMessage row
Transaction commits
Later
  -> Worker reads OutboxMessage
  -> Delivers notification
  -> Marks message complete

```

If the transaction rolls back, neither the product update nor the message remains. If it commits and the process immediately crashes, the outbox row still exists for another worker run. This closes the dangerous gap between “database changed” and “notification was queued.” A compact entity is:

```

public sealed class OutboxMessage
{
    public Guid Id { get; set; }
    public string Type { get; set; } = "";
    public string Payload { get; set; } = "";
    public DateTimeOffset CreatedAt { get; set; }
    public DateTimeOffset? ProcessedAt { get; set; }
    public int AttemptCount { get; set; }
}

```

```

    public DateTimeOffset? NextAttemptAt { get; set; }
    public string? LastError { get; set; }
}

```

The application records an event while updating the product:

```

product.ChangePrice(newPrice);
db.OutboxMessages.Add(new OutboxMessage
{
    Id = Guid.NewGuid(),
    Type = "product.changed",
    Payload = JsonSerializer.Serialize(new ProductChangedMessage(product.Id.Value,
product.Version + 1)),
    CreatedAt = timeProvider.GetUtcNow()
});
await db.SaveChangesAsync(token);

```

One `SaveChanges` transaction commits both changes. The application has not delivered the message yet, but it has durably accepted responsibility for it. A worker can fail after delivering a message but before marking the outbox row complete:

```

Worker sends email successfully
-> process stops
-> ProcessedAt was never saved
-> worker retries same message

```

This means practical background delivery is commonly **at least once**. A message may be delivered more than once. Exactly-once delivery across independent systems is difficult and usually replaced by idempotency. Every message receives a stable identifier. A downstream handler records processed identifiers or performs an operation whose final state is unchanged when repeated. Email itself is not naturally idempotent, so the notification system may keep one delivery row per (`OutboxMessageId`, `Recipient`, `Channel`) and ensure only one successful delivery is recorded. A duplicate can still occur in the narrow failure window after provider acceptance and before local confirmation, which is why user-facing email content should tolerate occasional duplication.

Retries should target transient failures such as timeouts, rate limits, and temporary provider unavailability. Validation errors, invalid addresses, and forbidden operations are not repaired by waiting. Exponential backoff with jitter prevents many failed messages from retrying at the same instant:

```

Attempt 1 -> wait about 5 seconds
Attempt 2 -> wait about 15 seconds
Attempt 3 -> wait about 45 seconds
Attempt 4 -> wait about 2 minutes

```

After a bounded number of attempts, move the message to a failed state or dead-letter queue for investigation. Infinite rapid retries can overload both the application and the dependency it is trying to reach. A single worker can query pending rows and process them in order:

```

var messages = await db.OutboxMessages
    .Where(message => message.ProcessedAt == null && (message.NextAttemptAt == null ||
message.NextAttemptAt <= now))
    .OrderBy(message => message.CreatedAt).Take(50).ToListAsync(token);

```

With several application instances, two workers can read the same rows. Production implementations need an atomic claim, provider-specific locking, a lease column, or a queue system designed for competing consumers. A claim may set `LockedUntil` and `LockedBy` in one conditional update. If a worker stops, the lease eventually expires so another worker can retry. Do not hide this concurrency problem behind a repository method named `GetPending`. The database operation must guarantee that only the intended workers own a message at one time, while still accepting that a claimed message may be retried after uncertain failure. For modest applications, a well-designed database outbox is enough. Larger systems may publish outbox messages to a broker and let separate worker processes consume them. The application contract does not need to change; only the infrastructure does.

13.4 Scheduled Work

Queued work has a specific trigger: a product changed, a file was uploaded, or a user requested an export. Scheduled work runs because time reached a condition: expire old drafts every hour, generate a daily report, or refresh an exchange-rate table at 02:00. A simple periodic worker can use `PeriodicTimer`:

```
public sealed class ExpiredDraftCleanupWorker(IServiceScopeFactory scopeFactory) :
    BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        using var timer = new PeriodicTimer(TimeSpan.FromHours(1));

        try
        {
            while (await timer.WaitForNextTickAsync(stoppingToken))
            {
                await using var scope = scopeFactory.CreateAsyncScope();
                var cleanup =
                    scope.ServiceProvider.GetRequiredService<ExpiredDraftCleanup>();
                await cleanup.RunAsync(stoppingToken);
            }
        }
        catch (OperationCanceledException) when (stoppingToken.IsCancellationRequested)
        {
        }
    }
}
```

This is suitable when approximate intervals are enough and one running instance may execute the work. It does not guarantee “exactly at 02:00,” persist missed runs, coordinate multiple servers, or provide an administration UI. Those requirements justify a dedicated job scheduler or external platform service. Scheduled jobs should also be idempotent. A daily cleanup may run twice during failover or not at all while the application is stopped. Persist the last successful run or select work by current state rather than assuming the timer fired exactly once.

13.5 SignalR, Groups, Publishing, and Reconnection

HTTP requests are initiated by the client. SignalR adds a long-lived connection through which the server can send messages to connected clients. It chooses an appropriate transport, manages connection state, serializes method arguments, and presents a hub-based programming model. Register and map a hub:

```
builder.Services.AddSignalR();
var app = builder.Build();
app.MapHub<CatalogHub>("/hubs/catalog");
```

A strongly typed hub defines the methods the server can invoke on clients:

```
public interface ICatalogClient
{
    Task ProductChanged(ProductChangedNotification notification);
    Task NotificationReceived(UserNotification notification);
}

public sealed class CatalogHub
    : Hub<ICatalogClient>
{
}
```

A hub is transient. A new hub instance handles each invocation, so fields on the hub are not durable connection or application state. Use `Context` for current connection information, `Groups` for connection grouping, and `IHubContext` to send messages from services outside the hub. SignalR is not a database and not a guaranteed delivery queue. A disconnected browser does not receive a message sent only to its old connection. Important notifications should also be stored so the client can load missed items after reconnecting. SignalR can address all clients, one connection, one authenticated user, or a group.

```

Clients.All
    -> every connected client
Clients.Client(connectionId)
    -> one connection
Clients.User(userId)
    -> all connections associated with one user
Clients.Group(groupName)
    -> connections currently in one group

```

A user may have several tabs and devices, so `Clients.User` can reach several connections. By default, the user identifier is normally derived from the authenticated principal's name-identifier claim. The identifier must be stable and unique. Groups are useful for subscriptions such as "viewers of product 42" or "members of tenant 7." A client can ask to join a group:

```

public sealed class CatalogHub(IAuthorizationService authorization) : Hub<ICatalogClient>
{
    public async Task WatchProduct(int productId)
    {
        var allowed = await authorization.AuthorizeAsync(Context.User!, productId,
            "ViewProduct");
        if (!allowed.Succeeded)
            throw new HubException("The product cannot be watched.");
        await Groups.AddToGroupAsync(
            Context.ConnectionId,
            $"product:{productId}"
        );
    }
}

```

Group membership is connection-oriented, not a durable record of user access. Reconnecting clients may need to rejoin. A group name is not authorization; the server must verify permission before adding a connection and must continue protecting the underlying API. Do not let a caller join arbitrary tenant or user groups merely by supplying a string. Product updates happen in application handlers, not hub methods. A publisher can receive a typed hub context:

```

public interface IProductRealtimePublisher
{
    Task PublishChangedAsync(ProductChangedNotification notification, CancellationToken
token = default);
}
public sealed class SignalRProductRealtimePublisher(
    IHubContext<CatalogHub, ICatalogClient> hub
) : IProductRealtimePublisher
{
    public Task PublishChangedAsync(ProductChangedNotification notification,
CancellationToken token = default)
    {
        return
            hub.Clients.Group($"product:{notification.ProductId}").ProductChanged(notification);
    }
}

```

The Product Catalog should normally call this publisher from the outbox dispatcher, after the product transaction commits. Calling SignalR inside the database transaction keeps locks open while waiting on network work and still cannot roll the broadcast back if the transaction later fails. The outbox dispatcher can route message types:

```

public sealed class OutboxMessageRouter(IProductRealtimePublisher realtime,
IEmailNotificationSender email)
{
    public async Task DispatchAsync(OutboxMessage message, CancellationToken token =
default)

```

```

{
    switch (message.Type)
    {
        case "product.changed":
            var changed =
                JsonSerializer.Deserialize<
                    ProductChangedNotification
                >(message.Payload)!;
            await realtime.PublishChangedAsync(changed, token);
            break;
        case "product.low-stock":
            var lowStock =
                JsonSerializer.Deserialize<
                    LowStockNotification
                >(message.Payload)!;
            await email.SendLowStockAsync(lowStock, token);
            break;
        default:
            throw new InvalidOperationException(
                $"Unknown outbox message type '{message.Type}'."
            );
    }
}
}

```

A real implementation should version message contracts and avoid renaming types in a way that makes old persisted payloads unreadable. A SignalR client registers handlers before starting the connection. A Blazor WebAssembly client service can use the .NET SignalR client package:

```

public sealed class CatalogLiveClient(NavigationManager navigation) : IAsyncDisposable
{
    private readonly HubConnection _connection =
        new HubConnectionBuilder().WithUrl(navigation.ToAbsoluteUri("/hubs/catalog")).
        WithAutomaticReconnect().Build();
    public event Action<ProductChangedNotification>?
        ProductChanged;
    public async Task StartAsync(CancellationToken token = default)
    {
        _connection.On<ProductChangedNotification>("ProductChanged", message =>
ProductChanged?.Invoke(message));
        await _connection.StartAsync(token);
    }
    public Task WatchProductAsync(int productId, CancellationToken token = default)
    {
        return _connection.InvokeAsync("WatchProduct", productId, token);
    }
    public async ValueTask DisposeAsync()
    {
        await _connection.DisposeAsync();
    }
}

```

Automatic reconnect retries the transport connection; it does not guarantee that the client received every event or retained every group subscription. After reconnecting, the client should rejoin needed groups and reload authoritative state when missing an event would matter. For an interactive server-rendered Blazor component, component logic already runs on the server over its Blazor circuit. It can update its own state immediately after its own command. Receiving independent cross-session events still requires a deliberate bridge, such as a browser SignalR client, a circuit-aware state service, or a different application design. Do not assume that the internal Blazor circuit automatically subscribes a component to custom application events.

13.6 Persistent Notifications and Email

A notification is more than a SignalR message. A useful model separates the user-visible notification from delivery attempts.

Notification

- > recipient
- > type
- > title and body
- > related resource
- > created time
- > read time

NotificationDelivery

- > notification
- > channel: in-app, email, push
- > status
- > attempt count
- > last error
- > delivered time

Persisting the notification lets users open a notification centre and retrieve items missed while offline. SignalR can announce that a new item exists. Email can provide an external copy. Marking an in-app notification as read should update the durable record rather than only remove one rendered component. A service can create the notification and outbox message in one transaction:

```
var notification = new Notification
{
    Id = Guid.NewGuid(),
    RecipientId = recipientId,
    Type = "product.price-changed",
    Title = "Product price changed",
    Body = $"{product.Name} now costs {product.Price}.",
    ResourceId = product.Id.Value.ToString(),
    CreatedAt = now
};
db.Notifications.Add(notification);
db.OutboxMessages.Add(new OutboxMessage
{
    Id = Guid.NewGuid(),
    Type = "notification.created",
    Payload = JsonSerializer.Serialize(new NotificationCreatedMessage(notification.Id,
notification.RecipientId)),
    CreatedAt = now
});
await db.SaveChangesAsync(token);
```

The worker sends a SignalR message to `Clients.User(recipientId)` and optionally queues email according to user preferences. The browser can then request the durable notification by ID or refresh its unread list. Sending email is network I/O to another system. The provider may accept a message and deliver it later, reject it synchronously, throttle requests, or accept it and eventually receive a bounce. “The SDK call succeeded” normally means the provider accepted responsibility, not that the recipient read or even received the message. Keep email behind an application-facing abstraction:

```
public interface IEmailSender
{
    Task<EmailSendResult> SendAsync(EmailMessage message, CancellationToken token =
default);
}
public sealed record EmailMessage(string Recipient, string Subject, string TextBody, string?
HtmlBody = null);
```

Infrastructure can use a provider SDK or maintained SMTP library. The legacy `System.Net.Mail.SmtpClient` type remains available but is not recommended for new development. The application layer should not depend

on provider response types or construct provider-specific templates. Email content needs the same security care as HTML pages. Encode untrusted values before inserting them into HTML templates. Do not include secrets, password-reset tokens, or sensitive medical and financial details in logs. Reset and invitation links are credentials and should expire, be single-purpose, and avoid analytics systems that capture full URLs. The worker should distinguish temporary provider failures from permanent address or template failures. Record provider message identifiers when available, but do not treat them as proof of final delivery. Webhooks for bounce and complaint events can update delivery status later.

13.7 Cache Fundamentals, Memory Cache, and HybridCache

A cache holds data that can be reconstructed from an authoritative source. If deleting the cache destroys the only copy, it was not a cache. The Product Catalog database remains the truth; caches reduce repeated database and serialization work. Caching has several levels:

Browser cache

-> reused by one browser

Output cache

-> stores complete HTTP responses

Application data cache

-> stores objects or query results

Distributed cache

-> shared by several application instances

Database cache

-> internal pages and execution plans managed by database

The same item can exist at several levels. A product details endpoint may use an application cache to avoid a database query, output caching to avoid serialization and endpoint execution, and browser caching to avoid the network entirely. More layers can improve performance but make invalidation harder. Add the narrowest cache that solves a measured problem. Before caching, ask how expensive the source operation is, how stale the result may be, how large the entry is, how it is invalidated, whether results vary by user or tenant, and what happens during a cache outage. Register `IMemoryCache`:

```
builder.Services.AddMemoryCache();
```

A query service can cache one product:

```
public sealed class CachedProductQueries(IMemoryCache cache, CatalogDbContext db)
{
    public Task<ProductDetails?> FindAsync(int id, CancellationToken token = default)
    {
        return cache.GetOrCreateAsync(
            $"product:{id}",
            async entry =>
            {
                entry.AbsoluteExpirationRelativeToNow =
                    TimeSpan.FromMinutes(5);
                return await db.Products
                    .AsNoTracking()
                    .Where(product => product.Id == id)
                    .Select(ProductProjections.Details)
                    .SingleOrDefaultAsync(token);
            }
        );
    }
}
```

Memory cache is extremely fast but belongs only to the current process. Two server instances can hold different values. Restarting the process clears it. Entries consume managed memory, so unbounded user-generated keys can become a denial-of-service problem. Cache keys should use trusted, normalized identifiers, not arbitrary long query strings. Caching null results can reduce repeated lookups for missing products, but a long negative-cache lifetime can hide a newly created item. Use shorter expiration or explicit invalidation. `IMemoryCache` does not automatically prevent several simultaneous misses from running the same expensive factory. For high-contention entries, add coordination or use a cache abstraction with stampede protection.

HybridCache provides one API over a fast local cache and an optional distributed secondary cache. It also coordinates concurrent misses for the same key so one factory call loads the value while the other callers wait. Install and register it:

```
dotnet add package Microsoft.Extensions.Caching.Hybrid
```

```
builder.Services.AddHybridCache();
```

Use it in a query service:

```
public sealed class CachedProductQueries(HybridCache cache,
IDbContextFactory<CatalogDbContext> contextFactory)
{
    public async Task<ProductDetails?> FindAsync(int id, CancellationToken token = default)
    {
        return await cache.GetOrCreateAsync(
            $"product:{id}",
            async cancellationToken =>
            {
                await using var db = await
contextFactory.CreateDbContextAsync(cancellationToken);
                return await db.Products
                    .AsNoTracking()
                    .Where(product => product.Id == id)
                    .Select(ProductProjections.Details)
                    .SingleOrDefaultAsync(cancellationToken);
            },
            cancellationToken: token
        );
    }
}
```

With only default registration, the primary local cache is used. Adding a supported distributed-cache backend lets instances share serialized entries while retaining local speed. Distributed caching is not automatically faster than the database for every query; it adds network calls, serialization, configuration, and failure modes. It is valuable when the source is significantly more expensive, values are reused often, and several instances need a shared copy. Use structured key schemes such as `tenant:7:product:42`. Include every trusted dimension that changes the result, including tenant, culture, currency, or permission-sensitive variant. Omitting one can leak data between users or return the wrong representation.

13.8 Invalidation, Output Caching, Stampedes, and Failure

Expiration limits how long stale data can survive if explicit invalidation fails. Invalidation removes or supersedes entries immediately when authoritative data changes. A product update can invalidate its details entry:

```
await cache.RemoveAsync(
    $"product:{productId}",
    token
);
```

Collection queries create many possible keys: page, search, category, sort, and availability. Removing each exact key is difficult. HybridCache supports tags, allowing related entries to be invalidated logically:

```
return await cache.GetOrCreateAsync(
    $"products:{query.CacheKey}",
    load,
    new HybridCacheEntryOptions
    {
        Expiration = TimeSpan.FromMinutes(5),
        LocalCacheExpiration = TimeSpan.FromMinutes(1)
    },
    tags: ["products"],
```

```

    cancellationToken: token
);

```

After a product change:

```
await cache.RemoveByTagAsync("products", token);
```

Another strategy is versioned keys. Store a catalog version and include it in list keys:

```
products:v41:category:3:page:1
```

Incrementing the version makes old entries unreachable; they expire naturally later. Versioning avoids enumerating keys but leaves old values consuming cache space until expiration. Invalidation should happen after the database commit. Removing the cache before a transaction that later fails causes unnecessary misses. Publishing invalidation through the outbox lets all instances observe the committed change. Even then, keep a bounded expiration because messages and cache operations can fail. Application caching stores data used by code. Output caching stores the completed HTTP response so later matching requests can bypass most of the pipeline and endpoint execution. Register the service and middleware:

```

builder.Services.AddOutputCache(options =>
{
    options.AddPolicy(
        "PublicProducts",
        policy => policy.Expire(TimeSpan.FromSeconds(30)).SetVaryByQuery("page", "pageSize",
"search", "category")
        .Tag("products")
    );
});
var app = builder.Build();
app.UseAuthentication();
app.UseAuthorization();
app.UseOutputCache();

```

Apply the policy:

```
app.MapGet("/api/products", ListProducts).CacheOutput("PublicProducts");
```

The cache key must vary by every request value that changes the response. Failing to vary by culture, tenant, query, or custom header can return another variant. Authenticated and user-specific responses should not be cached under a shared public key. Default output-cache policies avoid many unsafe cases, but custom policies need careful review. After a product change, evict tagged output:

```
await outputCache.EvictByTagAsync("products", token);
```

Output caching does not replace browser cache headers. It is server-side reuse. Browser and intermediary caching use HTTP response headers and conditional requests. The two can cooperate but solve different transfer boundaries. Suppose a popular product entry expires while one thousand requests arrive. Without coordination, all one thousand can query the database simultaneously. This is a cache stampede.

Entry expires

- > 1000 cache misses
- > 1000 database queries
- > database overload

Stampede protection lets one caller refresh while others wait or temporarily receive stale data. **HybridCache** coordinates concurrent factory calls for the same key, and output caching uses resource locking by default. A custom cache implementation should address the same problem instead of only storing values. Expiration should also include some randomness for large groups of independently cached entries so they do not all expire at one exact second. Prewarming may help for a small number of predictable hot values, but loading the entire database into cache at startup usually delays readiness and creates another large synchronization problem.

A distributed cache is another network dependency. If product data can still be loaded safely from the database, a cache outage should normally reduce performance rather than make the catalog unavailable. Log and measure the failure, then use the authoritative source. Do not retry cache calls aggressively inside every request. A failed shared cache can cause every server to spend more time retrying than performing useful database work.

Timeouts should be short relative to the source query, and circuit-breaking or temporary bypass can prevent repeated delay.

Write-through caching updates the cache as part of each write path. Cache-aside removes the old entry and lets the next read repopulate it. Cache-aside is simpler and avoids treating the cache as part of the authoritative transaction, though the first reader after a write pays the reload cost. Choose deliberately and preserve correctness when cache writes fail.

13.9 File Jobs and Background Observability

Chapter 12 introduced safe file upload and storage. Once an image is stored, the application may need thumbnails, format conversion, virus scanning, metadata extraction, or CDN publication. Those derived operations belong naturally in the outbox or a durable job queue.

Upload request

- > store original file
- > save file metadata and outbox job
- > commit
- > return accepted product state

Background worker

- > scan or decode original
- > generate derived sizes
- > store derivatives
- > update metadata
- > invalidate image response cache
- > notify connected client

Do not return a public image URL before the application knows whether the file is safe to expose. The product can show a processing state until the background job completes. Public immutable files work well with long browser and CDN cache lifetimes when their URL contains a content hash or version:

`/images/products/42/photo-a93f84c2.webp`

Replacing the image creates a new URL, so old cached bytes remain correct for the old address and no mass invalidation is required. Mutable URLs such as `/products/42/image` need shorter caching or explicit revalidation. A request failure is visible to the caller. Background failure can remain invisible unless the application records it. Monitor at least:

Queue depth

Oldest pending item age

Processing rate

Success and failure counts

Retry count

Dead-letter count

Current worker health

Provider latency and error rate

Cache hit and miss rate

SignalR connection count

Logs should include message ID, message type, attempt, related product or user ID, and trace or correlation information. Do not log complete email bodies, tokens, secrets, or unbounded payloads. A health check should distinguish readiness from liveness. A worker process may be alive but unable to reach the database or message provider. Whether that should mark the whole web application unready depends on the feature. Public product reads may remain healthy while email delivery is degraded. Report component status without unnecessarily removing the entire site from service. Administrative tooling should show failed jobs and permit safe retry after the underlying problem is corrected. Manual retry must retain the original idempotency identifier rather than create an unrelated duplicate operation.

13.10 One Complete Product Change

The Product Catalog can now process one update without coupling every effect to the request:

Editor saves product 42

- > Blazor sends PUT `/api/products/42`
- > Endpoint authenticates, authorizes, and binds input

- > UpdateProductHandler loads product
- > Domain validates price change
- > EF Core updates Product
- > EF Core inserts product.changed outbox row
- > Transaction commits
- > Endpoint returns updated product
- > Blazor updates editor immediately

Outbox worker later claims message

- > Invalidates product and list caches
- > Evicts product output-cache tag
- > Sends SignalR ProductChanged to product:42 group
- > Creates or delivers user notifications
- > Sends email where preferences require it
- > Marks outbox message processed

A viewer connected through SignalR receives the event and reloads product 42. A disconnected viewer receives nothing in real time but sees current data on the next request. A user notification remains in the database and appears after sign-in. Email may arrive later. If the mail provider is unavailable, the product update remains committed and the delivery retries independently. This design accepts that not all effects become visible at the same instant. The authoritative state changes atomically; derived views converge shortly afterward. The system is eventually consistent outside the product transaction, but it does not lose responsibility for promised work.

13.11 Common Mistakes and Key Ideas

Do not start untracked tasks from endpoints or capture scoped services after a request ends. Process-local queues are appropriate only when lost work is acceptable; promised work belongs in durable storage such as an outbox or broker. Workers create a scope per item or bounded batch, stop claiming work during shutdown, use stable message identifiers, tolerate at-least-once delivery, bound retries, and preserve failed items for investigation.

SignalR hubs are transient and live messages can be missed. Persist notifications that must survive disconnection, re-authorize group membership, reload authoritative state after reconnect, and treat provider acceptance of email as a delivery attempt rather than proof of arrival. A cache is similarly non-authoritative: use bounded and normalized keys, include tenant or user dimensions, invalidate after commit, retain expiration as a safety net, and plan for stampedes, outages, and several application instances. Choose the simplest mechanism that satisfies durability and timeliness. Not every effect needs real-time delivery, but every effect that outlives the request needs an explicit owner, failure policy, and observable queue or delivery state.

14. Deployment and Operation

A web application is not finished when it works under `dotnet run`. Development runs with local configuration, a developer certificate, source files, a writable project directory, one process, and a person watching the terminal. Production must start after a reboot, survive ordinary failures, receive traffic through a real domain, protect secrets, preserve authentication keys, apply database changes safely, expose useful health information, and produce enough telemetry for someone to understand failures without attaching a debugger. Deployment is therefore not a final copy step. It is the process of turning source code into a repeatable release and placing that release inside an operational environment. Operation begins where deployment ends: keeping the process available, observing its behaviour, rotating credentials, restoring backups, scaling when needed, and replacing one version with another without corrupting data or losing users unnecessarily.

This chapter deploys the Product Catalog first as a published .NET application on Linux behind Nginx, then shows the equivalent container model. The technologies can change—a cloud platform may replace `systemd`, a managed ingress may replace Nginx, and a secret manager may replace environment variables—but the responsibilities remain the same.

14.1 Production Layers, Publishing, and Release Identity

The application process is only one part of a production service. A typical Linux deployment has several layers:

Browser

- > DNS
- > Public IP or load balancer
- > TLS and reverse proxy
- > Kestrel
- > ASP.NET Core application
- > Database, cache, file storage, email, and other dependencies

A process manager starts the application after boot, restarts it after failure, supplies environment variables, controls its operating-system identity, and captures its output. The reverse proxy accepts public traffic, owns the domain and certificate, and forwards requests to a private Kestrel address. Monitoring systems call health endpoints and collect logs, metrics, and traces. A deployment mechanism replaces the application files or container image. Database migrations and backups are managed independently of the web process. These layers should have explicit ownership. Kestrel runs the .NET application; it should not be expected to renew public certificates, manage service startup, retain logs forever, or perform database backups. Conversely, Nginx should not contain business routing rules that belong in ASP.NET Core. Production becomes understandable when each layer does a small, visible job.

`dotnet build` compiles a project for development and testing. Its output may assume the project structure, include files useful to debugging, and omit content needed for a standalone deployment. `dotnet publish` creates the deployment artifact:

```
dotnet publish src/ProductCatalog.Web/ProductCatalog.Web.csproj \
  --configuration Release \
  --output artifacts/publish/ProductCatalog
```

The publish directory contains the compiled assemblies, dependency descriptions, runtime configuration, static assets, configuration files included by the project, and other files required to run the application. Deploy the publish output rather than copying `bin/Release` manually. Publishing does not start the application or apply its database migrations. It creates a versioned artifact that another step can test and deploy. A reliable pipeline builds once and promotes the same artifact through environments:

Commit

- > Restore
- > Build
- > Test
- > Publish
- > Package immutable artifact
- > Deploy to staging
- > Verify
- > Promote same artifact to production

Rebuilding separately for production can introduce different dependencies or source content after staging was tested. Environment-specific behaviour should normally come from runtime configuration, not from compiling a

different application for every server. A framework-dependent deployment contains the application and expects a compatible .NET runtime on the server:

```
dotnet publish -c Release -o artifacts/publish/ProductCatalog
```

It runs with:

```
dotnet ProductCatalog.Web.dll
```

The artifact is smaller, and the server runtime can be patched centrally. The deployment must ensure that the required runtime is installed. A self-contained deployment includes the selected runtime for one operating-system and processor combination:

```
dotnet publish -c Release -r linux-x64 --self-contained true -o artifacts/publish/ProductCatalog
```

It runs through the generated executable:

```
./ProductCatalog.Web
```

The artifact is larger, but the application does not depend on a machine-wide .NET installation. Runtime security updates arrive only when the application is republished and redeployed, so self-contained does not remove patch management; it moves runtime ownership into the application release. Choose deliberately. Framework-dependent publishing is simple when a team controls a small group of servers. Self-contained publishing is useful on machines where installing a shared runtime is undesirable or when the release must carry an exact runtime. Containers commonly use framework-dependent application output on top of a versioned ASP.NET runtime image. Single-file and native ahead-of-time publishing can simplify distribution or improve selected startup and memory characteristics, but compatibility and reflection constraints vary. They should solve a measured deployment requirement rather than be enabled automatically for an ordinary web application.

Every deployed artifact should identify the source and version from which it came. Useful values include:

- Application version
- Git commit identifier
- Build number
- Build time
- Environment
- Database migration version

Expose non-sensitive release information through logs and perhaps an authenticated diagnostics endpoint:

```
app.MapGet("/internal/version", (IHostEnvironment environment) => TypedResults.Ok(new
{
    Version = typeof(Program).Assembly.GetName().Version?.ToString(),
    Environment = environment.EnvironmentName
})).RequireAuthorization("Operations");
```

Do not expose connection strings, server paths, loaded secrets, complete environment variables, or framework internals through a public diagnostics page. The purpose is to answer “Which release is running?” rather than to dump the process. A version also improves incident response. Logs from three instances can be grouped by release, a failed deployment can be correlated with one commit, and a rollback target can be selected without guessing which files were copied last Tuesday.

14.2 Linux Directories and systemd

A simple server layout separates releases, shared writable state, and the currently active version:

```
/opt/product-catalog/
  releases/
    2026-07-13.1/
    2026-07-20.1/
  current -> releases/2026-07-20.1/
/var/lib/product-catalog/
  uploads/
  data-protection-keys/
```

```
/etc/product-catalog/  
product-catalog.env
```

Published files under `/opt` are treated as release content and should not be modified by the running process. Writable data belongs under a separate directory with explicit backup and retention rules. Configuration and secret references belong outside the release. A symbolic link such as `current` allows deployment to place a complete new release beside the old one and switch the active path atomically. Run the process under a dedicated operating-system account:

```
sudo useradd --system --home /var/lib/product-catalog --shell /usr/sbin/nologin  
productcatalog  
sudo mkdir -p /opt/product-catalog/releases /var/lib/product-catalog/uploads  
sudo chown -R productcatalog:productcatalog /var/lib/product-catalog
```

The account should have only the permissions the application needs. It normally does not need a login shell, root access, write access to published binaries, or permission to read unrelated application secrets. Operating-system permissions are another security boundary, not merely deployment housekeeping. On many Linux distributions, `systemd` can start and supervise the application. A service unit might be stored at `/etc/systemd/system/product-catalog.service`:

```
[Unit]  
Description=Product Catalog ASP.NET Core application  
After=network-online.target  
Wants=network-online.target  
[Service]  
WorkingDirectory=/opt/product-catalog/current  
ExecStart=/usr/bin/dotnet /opt/product-catalog/current/ProductCatalog.Web.dll  
User=productcatalog  
Group=productcatalog  
Environment=ASPNETCORE_ENVIRONMENT=Production  
Environment=ASPNETCORE_URLS=http://127.0.0.1:5080  
EnvironmentFile=-/etc/product-catalog/product-catalog.env  
Restart=always  
RestartSec=5  
KillSignal=SIGINT  
SyslogIdentifier=product-catalog  
NoNewPrivileges=true  
PrivateTmp=true  
[Install]  
WantedBy=multi-user.target
```

After creating or changing the unit:

```
sudo systemctl daemon-reload  
sudo systemctl enable --now product-catalog  
sudo systemctl status product-catalog
```

Logs written to standard output and standard error are available through the journal:

```
journalctl -u product-catalog --since "30 minutes ago"  
journalctl -u product-catalog -f
```

`Restart=always` recovers from an unexpected process exit, but repeated restart loops still require alerting and investigation. A process manager is not a substitute for correct error handling. It ensures that a transient crash does not leave the service permanently stopped. The service binds Kestrel to `127.0.0.1:5080`, so it is reachable only from the same machine. Nginx will own public ports 80 and 443. This keeps Kestrel's internal endpoint out of the public network surface.

14.3 Reverse Proxies, Forwarded Headers, and HTTPS

A reverse proxy accepts the public connection and forwards the request to Kestrel:

```
Client HTTPS request  
-> Nginx on :443
```

```
-> local HTTP request to 127.0.0.1:5080
-> Kestrel
```

This arrangement provides a place for certificate management, domain routing, request-size limits, static response rules, buffering, and hosting several applications on one machine. Kestrel remains the application server and executes ASP.NET Core middleware and endpoints. A compact Nginx site configuration is:

```
server {
    listen 80;
    server_name catalog.example.com;
    return 301 https://$host$request_uri;
}
server {
    listen 443 ssl;
    http2 on;
    server_name catalog.example.com;
    ssl_certificate /etc/letsencrypt/live/catalog.example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/catalog.example.com/privkey.pem;
    client_max_body_size 10m;
    location / {
        proxy_pass http://127.0.0.1:5080;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Forwarded-Host $host;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection $connection_upgrade;
    }
}
```

SignalR and interactive server Blazor need connection upgrade support for WebSockets. Some Nginx configurations define `$connection_upgrade` through a map in the `http` block:

```
map $http_upgrade $connection_upgrade {
    default upgrade;
    '' close;
}
```

Validate before reloading:

```
sudo nginx -t
sudo systemctl reload nginx
```

A reload should preserve existing connections while applying the new configuration. Certificate paths and distribution-specific Nginx layout vary, but the forwarding responsibilities remain the same. Nginx terminates HTTPS and forwards an ordinary HTTP request to Kestrel. Without forwarded-header processing, ASP.NET Core sees:

```
Scheme      -> http
Remote IP   -> 127.0.0.1
Host        -> possibly the internal host
```

The proxy adds headers describing the original request. ASP.NET Core must process them before middleware that depends on scheme, host, or client address:

```
using Microsoft.AspNetCore.HttpOverrides;
using System.Net;
builder.Services.Configure<ForwardedHeadersOptions>(options =>
{
    options.ForwardedHeaders =
        ForwardedHeaders.XForwardedFor |
        ForwardedHeaders.XForwardedProto |
        ForwardedHeaders.XForwardedHost;
    options.KnownProxies.Add(IPAddress.Loopback);
});
```

```
});
var app = builder.Build();
app.UseForwardedHeaders();
app.UseHttpsRedirection();
app.UseAuthentication();
app.UseAuthorization();
```

Forwarded headers are trustworthy only when they come from a known proxy. An internet client can invent `X-Forwarded-For` or `X-Forwarded-Proto`; accepting those values from arbitrary sources allows address spoofing and incorrect security decisions. Configure known proxies or networks according to the deployment topology. In recent ASP.NET Core releases, forwarded headers from unknown proxies are ignored by default, so a deployment that omits trust configuration may observe wrong schemes, redirect loops, failed external login callbacks, or incorrect client addresses rather than silently trusting unverified headers. If several proxies or load balancers are chained, configure the expected network path and forwarding limit rather than allowing all sources. Application authorization should still not depend solely on a client IP unless the operational environment genuinely guarantees that boundary.

The public connection should use HTTPS. Nginx or a managed load balancer may terminate TLS, but ASP.NET Core still needs to understand that the original request was secure through forwarded headers. This affects generated redirect URLs, secure cookies, OpenID Connect callbacks, and request scheme checks. Use HTTP Strict Transport Security in production:

```
if (!app.Environment.IsDevelopment())
    app.UseHsts();
app.UseHttpsRedirection();
```

HSTS tells supporting browsers to use HTTPS for the domain for a period of time. Enable it only after HTTPS works reliably for the domain and subdomain policy is understood, because a bad HSTS configuration can make an HTTP-only recovery path unavailable to browsers. Certificates expire. A production setup needs automated issuance or renewal and a monitored reload path. A certificate existing today is not an operational plan. Alert before expiry and verify renewal from outside the server. Traffic between the reverse proxy and Kestrel can remain local HTTP when both processes share a trusted machine. If that hop crosses an untrusted network, protect it with TLS or another private network boundary. “HTTPS at the browser” does not automatically protect every internal hop.

14.4 Configuration and Data Protection

The published application should contain safe defaults, while environment-specific values arrive at runtime. Environment variables map double underscores to configuration sections:

```
ConnectionStrings__Catalog="Host=db.internal;Database=catalog;Username=app;Password=..."
Catalog__PublicBaseUrl="https://catalog.example.com"
Email__Provider="Postmark"
```

The service unit can load a protected environment file, but plain environment variables are not an ideal secret store in every platform. Process inspectors, support bundles, deployment tooling, and accidental diagnostics may expose them. Managed hosting commonly offers a secret manager or workload identity that avoids long-lived passwords. Prefer identity-based access for databases and cloud services when available. Validate required options at startup:

```
builder.Services
    .AddOptions<EmailOptions>()
    .BindConfiguration("Email")
    .ValidateDataAnnotations()
    .Validate(options => Uri.TryCreate(options.BaseUrl, UriKind.Absolute, out _), "Email
BaseUrl is invalid.")
    .ValidateOnStart();
```

Failing startup clearly is better than accepting traffic and discovering the missing secret only when the first customer requests a password reset. Do not mount development `appsettings.Development.json`, user-secrets storage, or local certificates into production. `ASPNETCORE_ENVIRONMENT=Production` should be explicit, and development exception pages must remain disabled. ASP.NET Core Data Protection protects authentication

cookies, antiforgery tokens, and other application data. If its key ring exists only inside an ephemeral container or temporary user profile, restarting or replacing the application can invalidate every protected cookie and sign users out. If several instances have different key rings, a cookie created by one instance may fail on another. Persist the keys in storage shared by all instances of the same application and protect them at rest:

```
builder.Services
    .AddDataProtection()
    .SetApplicationName("ProductCatalog")
    .PersistKeysToFileSystem(
        new DirectoryInfo("/var/lib/product-catalog/data-protection-keys")
    );
```

The directory must remain private to the service account and backed up according to the application's session-recovery needs. On a multi-machine deployment, use an appropriate shared provider such as database, distributed cache, object storage, or a platform integration. `SetApplicationName` isolates compatible applications sharing the same key storage. Changing the application name, losing the key ring, or swapping to an environment with unrelated keys invalidates existing protected values. Treat key-ring deployment as part of release design, especially with blue-green environments and deployment slots.

14.5 Migrations and Schema Compatibility

Production schema changes should not normally be an uncontrolled side effect of every application instance starting. Several instances may race, the application account may not have schema-modification permission, and a long migration may leave the service unavailable without a clear operator. Generate and review an idempotent SQL script:

```
dotnet ef migrations script \
    --idempotent \
    --project src/ProductCatalog.Infrastructure \
    --startup-project src/ProductCatalog.Web \
    --output artifacts/migrations/ProductCatalog.sql
```

Or build a migration bundle:

```
dotnet ef migrations bundle \
    --project src/ProductCatalog.Infrastructure \
    --startup-project src/ProductCatalog.Web \
    --output artifacts/migrations/product-catalog-migrate
```

The release pipeline can back up or snapshot the database where appropriate, apply the reviewed migration with a privileged deployment identity, verify it, and only then activate application instances that require the new schema. The runtime application account can retain narrower read and write permissions. Migrations need realistic testing against production-like data volume. Adding a nullable column is usually cheap; rebuilding a large table, creating an index, or backfilling millions of rows may lock data or run for a long time. The generated migration expresses the intended structure, not its operational safety under production load. A zero- or low-downtime deployment briefly runs old and new application versions against the same database. A migration that immediately renames or removes a column can break the old version before traffic has left it. Use expand-and-contract changes:

```
Release A
    -> add new nullable column
    -> old application still works
Release B
    -> application writes old and new forms
    -> backfill existing rows
Release C
    -> application reads new form
    -> old column remains temporarily
Release D
    -> remove old column after no old instance remains
```

The same principle applies to outbox payloads, cache keys, SignalR messages, and API contracts. A new worker may read messages written by the previous release. A browser can keep old JavaScript or WebAssembly as-

sets during deployment. Persisted and distributed contracts need an overlap period. Rollback also becomes easier when the database remains backward compatible. Reverting application binaries cannot undo a destructive schema migration automatically. Forward-compatible schema design is more reliable than assuming every deployment can be reversed by copying old files.

14.6 Release Switching, Rollback, and Shutdown

Copy each release into a new directory rather than overwriting files used by the running process:

```
release=/opt/product-catalog/releases/2026-07-20.1
sudo mkdir -p "$release"
sudo tar -xzf ProductCatalog-2026-07-20.1.tar.gz -C "$release"
sudo chown -R root:root "$release"
sudo ln -sf "$release" /opt/product-catalog/current
sudo systemctl restart product-catalog
```

Before switching, verify checksums, configuration availability, required directories, and migration compatibility. Keep at least one known-good release for rollback. A rollback switches the link back and restarts the service:

```
sudo ln -sf /opt/product-catalog/releases/2026-07-13.1 /opt/product-catalog/current
sudo systemctl restart product-catalog
```

This is simple replacement deployment and causes a short interruption. For many small applications, a brief controlled restart is acceptable and safer than a complicated zero-downtime system. Requirements should drive deployment complexity. For lower downtime, run two instances on different internal ports, verify the new instance's readiness, move proxy traffic, drain the old instance, and then stop it. This blue-green pattern consumes more resources but makes rollback fast because the old instance remains available until confidence is established. During deployment, the process manager sends a termination signal. ASP.NET Core stops accepting new work and signals application lifetime cancellation. Active HTTP requests, SignalR connections, and background workers need a bounded opportunity to finish.

A worker should stop claiming new messages when shutdown begins and either complete or release its current lease. A long upload may need to fail cleanly and retry. A database transaction should finish quickly rather than remain open during the entire shutdown period. The proxy or load balancer should stop routing new traffic before the process is killed. Interactive server Blazor users lose their circuits when the hosting process stops. The browser can reconnect, but component state held only in memory may disappear. Important draft or workflow state should be persisted independently rather than relying on one process surviving every deployment. Graceful shutdown reduces disruption; it cannot make arbitrary in-memory state durable. Design state ownership first, then choose a shutdown timeout long enough for normal requests without allowing a broken process to block deployment indefinitely.

14.7 Containers, SDK Publishing, and Persistent State

A container image contains the application, runtime, operating-system libraries, and startup command in an immutable package. The container runtime supplies configuration, networking, writable volumes, resource limits, and process supervision.

Container image

- > read-only application layers
- > ProductCatalog.Web
- > ASP.NET Core runtime

Container instance

- > environment configuration
- > network endpoint
- > temporary writable layer
- > mounted persistent volumes where required

Containers improve repeatability and portability, but they do not eliminate deployment responsibilities. Certificates, secrets, database migrations, backups, health checks, logs, data-protection keys, and rolling compatibility still need owners. A multi-stage Dockerfile keeps the SDK out of the final image:

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
```

```

COPY ProductCatalog.sln ./
COPY src/ProductCatalog.Web/ProductCatalog.Web.csproj src/ProductCatalog.Web/
COPY src/ProductCatalog.Application/ProductCatalog.Application.csproj
src/ProductCatalog.Application/
COPY src/ProductCatalog.Domain/ProductCatalog.Domain.csproj src/ProductCatalog.Domain/
COPY src/ProductCatalog.Infrastructure/ProductCatalog.Infrastructure.csproj
src/ProductCatalog.Infrastructure/

RUN dotnet restore src/ProductCatalog.Web/ProductCatalog.Web.csproj

COPY . .
RUN dotnet publish src/ProductCatalog.Web/ProductCatalog.Web.csproj \
    --configuration Release \
    --output /app/publish \
    --no-restore

FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS final
WORKDIR /app
COPY --from=build /app/publish .
ENV ASPNETCORE_HTTP_PORTS=8080
EXPOSE 8080
ENTRYPOINT ["dotnet", "ProductCatalog.Web.dll"]

```

Copy project files before the remaining source so Docker can reuse the restore layer when dependencies have not changed. Build contexts should exclude `bin`, `obj`, test results, local secrets, databases, and other irrelevant files through `.dockerignore`. Use pinned and regularly updated base-image versions according to the organisation's patch policy. Rebuilding the same source later can otherwise pick up a different mutable image. The .NET SDK can create a container image directly:

```

dotnet publish src/ProductCatalog.Web/ProductCatalog.Web.csproj \
    --configuration Release \
    --os linux \
    --arch x64 \
    /t:PublishContainer \
    -p:ContainerRepository=product-catalog \
    -p:ContainerImageTag=2026.07.20.1

```

SDK container publishing is useful when the image needs a straightforward .NET layout and no custom operating-system build steps. A Dockerfile remains useful when additional native packages, files, build tools, or carefully controlled layers are required. Whichever method is used, the image should be built once, scanned, tagged immutably, and promoted by digest or unique version. Avoid deploying only `latest`, because it does not identify which image is running and may point to different content over time. Writing an uploaded image or SQLite database into the container's ordinary writable layer does not make it durable. Replacing the container can remove it. Persistent data belongs in a mounted volume or external service:

```

services:
  catalog:
    image: registry.example.com/product-catalog:2026.07.20.1
    environment:
      ASPNETCORE_ENVIRONMENT: Production
      ASPNETCORE_HTTP_PORTS: 8080
    volumes:
      - data-protection:/var/lib/product-catalog/data-protection-keys
    ports:
      - "127.0.0.1:5080:8080"
    restart: unless-stopped
volumes:
  data-protection:

```

For multiple instances, local volumes are rarely enough for shared uploads or keys. Use object storage, a shared key provider, and a database reachable by all instances. Do not use a local SQLite file as a casually shared

multi-instance production database through a network volume; its locking and operational model differ from a server database. The container should run as a non-root user where the selected image and hosting platform allow it. Grant write access only to mounted directories that need it. The application directory itself should remain read-only.

14.8 Health, Telemetry, and Alerts

Health checks are HTTP endpoints intended for load balancers, orchestrators, and monitoring systems. Register and map a basic check:

```
builder.Services.AddHealthChecks();
var app = builder.Build();
app.MapHealthChecks("/health/live");
```

A liveness check asks whether the process is responsive enough to remain running. It should not fail merely because one optional dependency is unavailable, or an orchestrator may restart a healthy process repeatedly without fixing the dependency. A readiness check asks whether this instance should receive normal traffic. Tag dependency checks and filter by tag:

```
builder.Services
    .AddHealthChecks()
    .AddDbContextCheck<CatalogDbContext>("catalog-database", tags: ["ready"]);
var app = builder.Build();
app.MapHealthChecks("/health/live", new HealthCheckOptions
{
    Predicate = _ => false
});
app.MapHealthChecks("/health/ready", new HealthCheckOptions
{
    Predicate = check => check.Tags.Contains("ready")
});
```

The load balancer can remove an unready instance while leaving the process alive for recovery and diagnostics. Protect detailed health output; a public endpoint should not reveal database server names, exception messages, or internal topology. The status and a small stable body are enough for external probes. Health checks should complete quickly with strict timeouts. A probe that waits thirty seconds for every dependency can consume resources and cause overlapping probe storms. Logs record discrete events with context. Metrics record numerical behaviour over time. Traces connect operations across process and network boundaries.

```
Log
  -> Product 42 update failed with version conflict
Metric
  -> 37 conflicts in the last five minutes
Trace
  -> Browser request -> ASP.NET Core -> SQL -> email provider
```

Use structured logging placeholders rather than interpolated text:

```
logger.LogInformation(
    "Updated product {ProductId} to version {ProductVersion} in {ElapsedMilliseconds} ms",
    product.Id,
    product.Version,
    elapsed.TotalMilliseconds
);
```

The logging provider can preserve `ProductId`, `ProductVersion`, and duration as fields. Avoid logging secrets, cookies, authorization headers, password-reset links, or entire request bodies. Personal data should be logged only when operationally necessary and retained according to policy. In a Linux service, console logs can flow to `journald`. In containers, write to standard output and let the platform collect and ship logs. Writing rotating files inside each container complicates retention and retrieval. OpenTelemetry provides a vendor-neutral path for exporting logs, metrics, and traces. ASP.NET Core and .NET already emit useful instrumentation for requests, Kestrel, `HttpClient`, and runtime behaviour. Add application spans and metrics around meaningful operations

rather than every private method. A production dashboard should show request rate, latency percentiles, error rate, active connections, database duration, queue age, cache behaviour, memory, CPU, and deployment version.

One failed request may be ordinary invalid input. One hundred 500 responses per minute or an outbox whose oldest item is two hours old is an incident. Alerts should use sustained rates, latency, saturation, queue age, certificate expiry, backup failure, and health-state changes rather than sending an email for every exception. A useful minimum set for the Product Catalog is:

- Public availability and TLS expiry
- HTTP 5xx rate
- Request latency
- CPU and memory saturation
- Database connection and query failures
- Oldest outbox message age
- Failed notification count
- Disk or volume space
- Backup age and restore verification
- Current deployment version

Every alert needs an owner and a response. A dashboard nobody watches and an alert nobody understands do not improve reliability. Link alerts to concise runbooks that explain where to inspect logs, how to identify the current release, how to disable a failing integration, and how to roll back safely.

14.9 Scaling and Resource Limits

Adding a second application instance changes several earlier assumptions:

- Instance A memory $\neq$ Instance B memory
- Instance A queue $\neq$ Instance B queue
- Instance A local files $\neq$ Instance B local files
- Instance A Data Protection keys must be compatible with Instance B

Authoritative state belongs in shared durable systems. Distributed caching can improve repeated reads, while the database remains the truth. The outbox must coordinate competing workers. Uploads need shared object storage. Authentication cookies require a shared Data Protection key ring. Ordinary HTTP APIs can be load balanced across instances without remembering which server handled the previous request. Interactive server Blazor and SignalR maintain long-lived connections. An interactive server circuit lives in the memory of one server, so a web farm requires session affinity for that connection path unless a supported external SignalR arrangement changes the topology. A reconnect to another process cannot reconstruct arbitrary in-memory component state automatically.

Custom SignalR messages across multiple application instances also need a scale-out mechanism so a message sent through instance A can reach clients connected to instance B. Managed SignalR services or a supported backplane can provide that coordination. Do not assume `Clients.All` inside one process means all clients in the deployment. Scale only after measuring the bottleneck. A single well-sized instance with a reliable database may serve a substantial business application and is operationally simpler than a premature cluster. Production traffic is not always polite. Limit request body sizes, upload sizes, concurrency, timeouts, queue capacity, cache size, and expensive query page sizes. Validate limits at both proxy and application boundaries so the behaviour remains clear. Rate limiting can protect selected endpoints:

```
builder.Services.AddRateLimiter(options =>
{
    options.AddFixedWindowLimiter("login", limiter =>
    {
        limiter.PermitLimit = 10;
        limiter.Window = TimeSpan.FromMinutes(1);
        limiter.QueueLimit = 0;
    });
});
var app = builder.Build();
app.UseRateLimiter();
app.MapPost("/account/login", Login).RequireRateLimiting("login");
```

An in-process limiter protects one instance. A distributed attack or multi-instance deployment may require limits at the load balancer, API gateway, CDN, or shared store. Rate limiting also does not replace authentication lockout, DDoS protection, input validation, or database query limits. Timeouts should be shorter than upstream proxy timeouts and should cancel downstream database or HTTP work where safe. A timeout that returns to the client while the server continues the same expensive work provides little protection.

14.10 Backups and the Deployment Sequence

The database, uploaded files, object storage metadata, and perhaps Data Protection keys need backup policies. A backup plan defines frequency, retention, encryption, off-site location, access control, and acceptable data-loss and recovery times. Two operational targets are useful:

Recovery Point Objective

-> How much recent data may be lost?

Recovery Time Objective

-> How long may restoration take?

A nightly database backup has an RPO of up to roughly one day unless transaction-log or continuous recovery is added. Keeping backups for thirty days does not prove that any of them can be restored. Automated restore tests should regularly create an isolated environment, restore data, run integrity checks, and verify that the application can start and read representative records. Do not place backups on the same disk and account as the primary data only. A disk failure, mistaken deletion, or compromised credential can remove both. Protect backup credentials separately and record restoration steps before an incident occurs. A controlled Product Catalog release can follow this sequence:

1. Build and test one immutable release.
2. Publish application and migration artifacts.
3. Scan dependencies and container or package content.
4. Deploy release files or image without activating them.
5. Verify required configuration, secrets, keys, and writable storage.
6. Back up or snapshot data where the migration risk requires it.
7. Apply reviewed backward-compatible migrations.
8. Start the new instance on an unused internal endpoint.
9. Wait for readiness and run smoke tests.
10. Route traffic to the new instance.
11. Drain and stop the previous instance.
12. Observe errors, latency, queues, and business operations.
13. Keep the previous release available for rollback.

The smoke test should use the real public path where possible: DNS, HTTPS, reverse proxy, authentication, database, and static assets. Testing only `curl localhost:5080` proves Kestrel but not the certificate, forwarded headers, proxy upgrade handling, or public routing. A rollback decision should be based on predetermined signals such as elevated errors, failed migrations, broken login, or key business operations. During an incident, “wait a little longer” is not a deployment strategy.

14.11 Production Request Path and Key Risks

The request path now includes operational infrastructure:

Browser requests `https://catalog.example.com/products/42`

- > DNS resolves the public address
- > Load balancer or Nginx accepts TLS
- > Reverse proxy adds trusted forwarded headers
- > Kestrel receives local HTTP request
- > Forwarded Headers restores original scheme and address
- > ASP.NET Core authenticates and authorizes
- > Blazor or API endpoint invokes application operation
- > EF Core queries the production database
- > Response returns through Kestrel and proxy
- > Structured logs, metrics, and trace describe the path

A product update additionally records an outbox message in the database transaction. A background worker later invalidates caches, publishes SignalR updates, and sends notification email. Health probes decide whether the instance should receive traffic. The process manager or container platform restarts it after failure. Data

Protection keys preserve authentication across replacement. Backups and migrations protect the durable state. The application code from earlier chapters still runs, but production adds systems that make its assumptions real. Do not deploy `bin` output, source directories, or a locally built Debug configuration by copying whatever happens to exist on a development machine. Publish an immutable Release artifact and record its source version.

Do not run the application as root, expose Kestrel's private port unnecessarily, or give the service write access to its binaries. Separate release files, writable state, and configuration. Do not trust forwarded headers from arbitrary clients. Configure the known proxy path and run forwarded-header processing before HTTPS redirection, authentication, and any code that reads scheme or client address. Do not assume TLS renewal, process restart, log retention, or certificate monitoring happens automatically merely because Nginx or a cloud platform is present. Verify the complete lifecycle. Do not place production secrets in Git, container images, browser code, or diagnostics output. Validate configuration on startup and use managed identity or a secret manager where practical.

Do not lose Data Protection keys during restart or give every instance a separate key ring. Authentication cookies and antiforgery data depend on those keys. Do not let every web instance apply migrations at startup. Review and test migration artifacts, use a controlled deployment identity, and keep schema changes compatible with overlapping versions. Do not write important files, SQLite databases, queues, or keys into ephemeral container storage. Externalize durable state and back it up. Do not make liveness depend on every optional service, expose detailed health exceptions publicly, or let probes wait indefinitely. Separate liveness and readiness according to routing and restart decisions. Do not scale out while retaining process-local assumptions. Memory caches, in-process queues, SignalR clients, Blazor circuits, uploads, and key rings all need an intentional multi-instance design.

Finally, do not call a backup successful until a restore has been tested. Availability is not only keeping the current process alive; it is the ability to reconstruct the service after data loss, failed deployment, or infrastructure replacement.

Production surrounds the ASP.NET Core process with DNS, TLS, a reverse proxy or load balancer, process supervision, configuration, secrets, durable storage, telemetry, backups, and a deployment mechanism. `dotnet publish` creates the release artifact; Linux services or containers run it under a restricted identity. Forwarded headers must be accepted only from trusted proxies, Data Protection keys must survive replacement, and migrations should be controlled deployment artifacts with backward-compatible schema changes. Health checks guide restart and routing, while logs, metrics, and traces explain behaviour. Scaling requires removing process-local assumptions around caches, queues, files, keys, SignalR, and interactive server circuits. Backups count only when restoration is tested, and rollback works only when release and schema compatibility were designed before the incident.

15. The Complete Request Revisited

The tutorial began with one user action: entering a URL and waiting for a page to appear. At that point, the path from address bar to pixels contained many unfamiliar systems. DNS, TLS, HTTP, Kestrel, middleware, routing, components, databases, authentication, caches, background workers, and production infrastructure seemed like separate subjects. They are now parts of one path.

This final chapter does not introduce another framework. It follows the Product Catalog through two complete interactions. The first is a read: an anonymous visitor opens the product page. The second is a write: an authenticated editor changes a price. The read shows how a document reaches the browser and becomes interactive. The write shows how browser state becomes an HTTP request, crosses authorization and application boundaries, changes the database, produces a response, rerenders the UI, and schedules effects that continue after the request ends. The purpose is not to memorise every individual step. It is to learn where to look when something goes wrong and where each responsibility belongs when the application changes.

15.1 From the URL to the Public Edge

The deployed Product Catalog can be pictured as five connected regions:

Browser

-> HTML, CSS, DOM, events, Blazor, browser storage

Public network edge

-> DNS, HTTPS, reverse proxy, forwarded headers

ASP.NET Core host

-> Kestrel, middleware, routing, authentication, authorization, endpoints

Application and infrastructure

-> use cases, domain rules, EF Core, database, files, external services

Work beyond the request

-> outbox, workers, SignalR, notifications, caching, telemetry

A request does not enter every subsystem on every journey. A cached public image may be served before application code runs. A health probe may stop at one small endpoint. A read may query the database but produce no background work. A product update may use nearly the entire path. The structure is still useful because it shows the available boundaries and their direction.

Outer mechanism

-> translates into application meaning

-> application operation

-> translates into infrastructure work

The browser does not call EF Core. The database does not return a Blazor component. Middleware does not decide product pricing. Each boundary converts one representation into another. Assume a visitor enters:

`https://catalog.example.com/products/42`

The browser separates the address into scheme, host, default port, path, and no query or fragment:

Scheme -> `https`

Host -> `catalog.example.com`

Port -> `443`

Path -> `/products/42`

The path identifies the requested application location. It does not reveal whether the server will return static HTML, server-rendered Razor output, a Blazor page, a redirect, or an error. That decision belongs to the server. The browser first needs a network address. It checks its own DNS cache, the operating-system cache, and then a resolver if necessary. The resolver eventually returns an IPv4 or IPv6 address associated with the deployment's public edge. The address may belong directly to one server, but it often belongs to a load balancer, reverse proxy, or managed hosting service.

`catalog.example.com`

-> DNS

-> public network address

DNS establishes where traffic should go. It does not prove the identity of the server or encrypt the connection. That happens next. The browser connects to port 443 and begins a TLS handshake. The public server presents a certificate valid for `catalog.example.com`. The browser verifies the certificate chain, host name, validity period, and other security properties, then negotiates encrypted session keys.

Browser

```
-> Connect to catalog.example.com:443
<- Certificate and TLS parameters
-> Validate identity
<-> Establish encrypted channel
```

The application has not received an HTTP request yet. A certificate failure, unsupported protocol, firewall block, or unreachable address prevents the request from entering ASP.NET Core at all. This distinction matters when debugging. A browser certificate warning or connection refusal is not an application 500; no application response exists. After TLS succeeds, the browser sends the HTTP request through the encrypted connection:

```
GET /products/42 HTTP/1.1
Host: catalog.example.com
Accept: text/html
Cookie: .ProductCatalog.Auth=...
```

The cookie may be absent for an anonymous visitor. Other headers describe accepted content, browser capabilities, caching state, language, compression, and connection details. Nginx or another public edge receives the HTTPS request. It may reject an oversized body, redirect plain HTTP, apply connection limits, or route several domains to different applications. For this request, it forwards traffic to Kestrel on a private address:

```
Public request
  https://catalog.example.com/products/42
Internal request
  http://127.0.0.1:5080/products/42
```

The proxy adds trusted forwarding headers that describe the original request:

```
X-Forwarded-For: 198.51.100.24
X-Forwarded-Proto: https
X-Forwarded-Host: catalog.example.com
```

Kestrel sees the internal connection from the proxy. Forwarded Headers middleware later restores the external scheme, host, and client address, but only because the proxy is configured as trusted. Without that restoration, ASP.NET Core may think the request used HTTP and generate incorrect redirects, cookies, or external-login callback URLs. The reverse proxy does not normally know which product is being requested in the business sense. It sees an HTTP path and forwards it. Product routing remains inside ASP.NET Core.

15.2 From Kestrel to the Selected Endpoint

Kestrel reads the incoming HTTP data and creates the features from which ASP.NET Core constructs an `HttpContext`. The context belongs to this request and contains:

```
Request method, path, headers, cookies, and body
Response status, headers, and body
Current ClaimsPrincipal
Request service scope
Cancellation token
Trace identifier
Selected endpoint metadata
```

The host creates one dependency injection scope for the request. Scoped services such as `CatalogDbContext`, application handlers, and request-oriented services can be resolved from it. A singleton remains shared by all requests, while transients are created when requested.

```
Application process
  -> shared singleton services
Request /products/42
  -> one request scope
  -> scoped services for this request
```

The `HttpContext` must not be stored for later background work. The request will end, its scope will be disposed, and its cancellation token may already be cancelled. Work that must continue later receives copied identifiers or a durable message rather than the live context. The context moves through middleware in registration order. A production pipeline might contain:

```

app.UseForwardedHeaders();
app.UseExceptionHandler();
app.UseHsts();
app.UseHttpsRedirection();
app.UseAuthentication();
app.UseAuthorization();
app.UseOutputCache();
app.MapRazorComponents<App>().AddInteractiveServerRenderMode();
app.MapProductEndpoints();
app.MapHub<CatalogHub>("/hubs/catalog");
app.MapHealthChecks("/health/live");

```

The exact order depends on the application, but the responsibilities remain distinct. Forwarded headers restore the external request before code examines scheme or address. Exception handling wraps later components. HTTPS and HSTS enforce the public transport policy. Authentication reconstructs the current principal. Authorization evaluates endpoint requirements. Output caching may return a stored response without executing the endpoint. Routing selects the endpoint that matches the method and path. Middleware can perform work on both sides of the endpoint:

Request

```

-> Middleware A before
    -> Middleware B before
        -> Endpoint
    <- Middleware B after
<- Middleware A after

```

Response

If middleware short-circuits, later middleware and the endpoint do not run. A cached public response, rejected rate limit, authentication challenge, or static asset can all end the pipeline early. Routing compares GET /products/42 with registered endpoints. In a Blazor application, the server may first select the Razor components endpoint, after which the Blazor router selects a component whose @page template matches the route:

```

@page "/products/{Id:int}"
@code {
    [Parameter]
    public int Id { get; set; }
}

```

The route parameter is converted from the path segment "42" into the integer 42. Successful conversion does not prove that the product exists; it only proves that the URL has the expected shape. For a Minimal API request such as GET /api/products/42, endpoint routing selects a handler directly:

```

products.MapGet("/{id:int}", GetProduct);

```

Both models rely on route templates and endpoint metadata. The difference is what executes after the match: a component route or an API handler. Metadata may declare that the product page is public, that an administration page requires a policy, that an API response can be cached, or that an endpoint participates in OpenAPI. Routing therefore selects both executable code and the information other middleware needs to handle it. If the browser sent an authentication cookie, the configured cookie handler unprotects its ticket and reconstructs a ClaimsPrincipal. The principal may contain claims such as:

```

nameidentifier -> 42
name           -> Ana
role           -> CatalogEditor
permission     -> products.edit

```

If no valid cookie exists, the principal remains anonymous. The public product-details page allows anonymous access, so the request can continue. An administration page might require authentication and trigger a challenge instead. Authentication does not load every user property or automatically approve every operation. It answers which accepted identity is associated with the request. Authorization then evaluates that identity against the selected endpoint policy and, when necessary, the specific product. An invalid or expired credential normally produces 401 when authentication is required. A valid identity without the required permission produces 403. A public page does not need either failure merely because the visitor is anonymous.

15.3 The Read Path Through Application Code and EF Core

A server-rendered Blazor component can inject an application query service and load the product during its lifecycle:

```
@page "/products/{Id:int}"
@inject IProductQueries Products
@if (_product is null)
{
    <p>Product not found.</p>
}
else
{
    <ProductDetailsView Product="_product" />
}
@code {
    [Parameter]
    public int Id { get; set; }
    private ProductDetails? _product;
    protected override async Task OnParametersSetAsync()
    {
        _product = await Products.FindAsync(Id);
    }
}
```

A WebAssembly component cannot call the server's application service directly. It uses `HttpClient` to call the API:

```
_product = await Http.GetFromJsonAsync<ProductDetailsResponse>($"/api/products/{Id}",
token);
```

The two hosting paths differ at the outer adapter:

```
Interactive server component
    -> application service directly
WebAssembly component
    -> HTTP API
    -> application service
```

The application query itself can remain the same. It should not depend on `NavigationManager`, `HttpContext`, component state, or JSON serialization. It receives an identifier and returns an application result. A product query service coordinates the read operation:

```
public sealed class ProductQueries(IDbContextFactory<CatalogDbContext> contextFactory,
HybridCache cache)
    : IProductQueries
{
    public async Task<ProductDetails?> FindAsync(int id, CancellationToken token = default)
    {
        return await cache.GetOrCreateAsync(
            $"product:{id}",
            async cancellationToken =>
            {
                await using var db = await
contextFactory.CreateDbContextAsync(cancellationToken);
                return await db.Products
                    .AsNoTracking()
                    .Where(product => product.Id == id)
                    .Select(ProductProjections.Details)
                    .SingleOrDefaultAsync(cancellationToken);
            },
            cancellationToken: token
        );
    }
}
```

```

    }
}

```

The query owns application meaning: find the details needed by the product screen. Infrastructure details remain behind EF Core and the cache. The query does not return an HTTP result or a render fragment. A missing product becomes `null` or a stable not-found result, which the outer adapter translates appropriately. The cache is consulted first. A valid entry can avoid a database query. A miss invokes the factory. The cache remains a reusable copy; the database is authoritative. If the cache fails and the implementation permits fallback, the application still loads from the database. The EF Core query remains an expression tree until `SingleOrDefaultAsync` executes it. The provider translates the relevant operations:

```

Where
    -> SQL WHERE
Select
    -> selected columns and joins
SingleOrDefaultAsync
    -> execute query and expect zero or one result

```

Conceptually, the SQL may resemble:

```

SELECT p.Id, p.Code, p.Name, p.Description, p.Price, p.Currency,
       c.Name AS CategoryName, p.Version
FROM Products AS p
LEFT JOIN Categories AS c ON p.CategoryId = c.Id
WHERE p.Id = @id;

```

The parameter value is sent separately rather than concatenated into SQL text. The database engine parses and plans the command, uses indexes where useful, reads matching rows, and returns the selected values. EF Core materialises `ProductDetails` directly from the projection. `AsNoTracking` is appropriate because the read does not intend to modify the entity. Projection avoids loading every product column and relationship. The application receives one ordinary C# result without needing to know how the provider represented the join. If the database is unavailable, the provider throws an infrastructure exception. The exception handler logs the full error and returns a safe production response. The component or API client sees an application failure, not a connection string or SQL stack trace.

15.4 From Server Output to Browser Pixels

For server rendering, the component uses the returned data to produce a render tree. Blazor compares component output and produces HTML for the initial response. The server may include markers and boot resources needed for later interactivity. For an API call, the endpoint maps the application result to a typed HTTP result:

```

static async Task<Results<Ok<ProductDetailsResponse>, NotFound>> GetProduct(
    int id,
    IProductQueries products,
    CancellationToken token
)
{
    var product = await products.FindAsync(id, token);
    return product is null
        ? TypedResults.NotFound()
        : TypedResults.Ok(ProductHttpMappings.ToResponse(product));
}

```

ASP.NET Core serializes the response model as JSON, sets the status and content type, and writes the response body. The same application result therefore reaches two possible outer representations:

```

ProductDetails
    -> Blazor render tree and HTML
ProductDetails
    -> ProductDetailsResponse and JSON

```

Neither representation should require changing the domain or persistence model. After the endpoint completes, control returns through middleware in reverse order. Middleware may add headers, compress the body, record duration, update metrics, or store a cacheable response. Kestrel writes the response to the reverse proxy, which returns it through the encrypted public connection. A successful document response might contain:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Encoding: br
Cache-Control: no-cache
```

A successful API response might contain:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
```

The browser interprets the status, headers, and body. A status code is not merely logging metadata. It controls client behaviour, caching, redirects, error handling, and browser tools. At this point, the server-side request scope is disposed. Scoped services are released, including the `DbContext` if one was created directly in the request scope. The rendered response may continue living in the browser, but the original server request no longer exists. For an HTML response, the browser parses the document and constructs the DOM. It discovers links to CSS, images, fonts, JavaScript, and Blazor boot resources and requests them separately. CSS rules are matched to elements, layout calculates positions and dimensions, painting produces visual layers, and compositing produces the final pixels.

```
HTML
  -> DOM
CSS
  -> style rules
DOM + styles
  -> layout
  -> paint
  -> composite
  -> pixels
```

If the page uses interactive server rendering, the browser establishes the Blazor connection after initial rendering. Component code continues running on the server, while events and render updates cross the circuit. If it uses WebAssembly, the browser downloads the .NET runtime and required assemblies and runs component code locally. In either case, the browser remains responsible for the DOM, CSS, input, accessibility tree, and final rendering. The page may look complete before every resource finishes. A product image can arrive after the product name. A cached stylesheet may require no transfer. A failed JavaScript module can break one interactive feature while static content remains visible.

15.5 From User Input to an Authorized Update

The visitor now sees product 42. An authorised editor opens the edit form and changes the price from 129.00 to 119.00. The text box contains browser-side state. It is not yet database state. In Blazor, `@bind` updates the form model when the configured binding event occurs. `EditForm` and validation components can display immediate feedback. That feedback improves the user experience but does not establish trust. A caller can bypass the component and send an HTTP request directly. When the editor selects Save, the event handler runs:

```
private async Task SaveAsync()
{
    _isSaving = true;
    _message = null;
    try
    {
        var result = await Products.UpdateAsync(_model, _saveCancellation.Token);
        Handle(result);
    }
    finally
    {
        _isSaving = false;
    }
}
```

```

    }
}

```

Changing `_isSaving` and later handling the result changes component state. Blazor schedules rendering after the event callback completes or after awaited asynchronous work yields and completes according to the component lifecycle. The component describes the UI it wants; it does not manually edit individual DOM nodes. In WebAssembly, `Products.UpdateAsync` normally sends an HTTP request. In an interactive server component, it may call the application handler directly. We will follow the HTTP path because it crosses every major boundary. The client constructs a request:

```

PUT /api/products/42 HTTP/1.1
Host: catalog.example.com
Content-Type: application/json
Cookie: .ProductCatalog.Auth=...
If-Match: "7"

```

```

{
  "name": "Mechanical Keyboard",
  "price": 119.00,
  "currency": "EUR",
  "categoryId": 3,
  "version": 7
}

```

The exact concurrency contract may use a body version, an ETag in `If-Match`, or another token. The essential meaning is the same: “Apply this change only if I edited the expected version.” The browser automatically attaches the matching authentication cookie. Because the credential is sent automatically, the server also needs antiforgery protection for relevant cookie-authenticated state-changing operations. The form or client supplies the expected antiforgery token according to the chosen ASP.NET Core pattern. The request travels through DNS, TLS, reverse proxy, Kestrel, forwarded headers, authentication, authorization, routing, and binding as before. The differences are method, body, current identity, authorization requirements, and the fact that the operation changes state. ASP.NET Core binds route value 42 and deserializes the JSON body into an update request:

```

public sealed record UpdateProductRequest(
    string Name,
    decimal Price,
    string Currency,
    int? CategoryId,
    int Version
);

```

The DTO contains only fields the client may submit. It is not the EF Core entity. The client cannot set `CreatedAt`, internal owner information, audit fields, database-generated keys, or arbitrary navigation properties merely because those properties exist in persistence code. Binding answers whether the transport can be converted to the declared .NET shape. Validation then checks rules such as required name, currency format, and non-negative price. Authorization checks whether the current principal may edit products. Domain logic checks whether the requested transition creates a valid product state. The database finally enforces storage constraints.

Binding

-> Can the bytes become the expected input type?

Input validation

-> Are submitted values complete and well formed?

Authorization

-> May this principal request the operation?

Domain rules

-> Is the resulting state allowed?

Database constraints

-> Can invalid state be stored despite another writer or bug?

Passing one stage does not bypass the others. The update endpoint requires the `EditProducts` policy:

```
products.MapPut("/{id:int}",
UpdateProduct).RequireAuthorization(CatalogPolicies.EditProducts);
```

Authorization evaluates the principal created by authentication. The policy may require an authenticated user and a `products.edit` permission claim. If the product belongs to a tenant or owner, the handler may also perform resource-based authorization after loading it. The Blazor UI may hide the Save button from unauthorized users, but the endpoint remains authoritative. Browser markup, WebAssembly code, route guards, and `AuthorizeView` are visible or controllable by the user. Security decisions are enforced on the server operation. An anonymous caller receives 401. A signed-in caller without permission receives 403. Neither reaches the product update logic.

15.6 From Application Handler to Rerender

The endpoint maps transport input into an application command:

```
var command = new UpdateProductCommand(
    id,
    request.Name,
    request.Price,
    request.Currency,
    request.CategoryId,
    request.Version
);
var result = await handler.HandleAsync(command, token);
```

The application handler loads the product, checks operation-level rules, applies domain methods, and saves one unit of work. It does not return `IResult`, navigate a component, or write JSON.

```
UpdateProductHandler
-> Load product
-> Verify expected version
-> Apply domain methods
-> Save database changes
-> Record outbox event
-> Return application result
```

Expected outcomes remain explicit:

```
Success
NotFound
Invalid
Conflict
Forbidden
```

Unexpected infrastructure failures can propagate to the exception boundary. This distinction keeps ordinary user outcomes out of exception control flow while preserving full error information for genuine failures. EF Core loads the product as a tracked entity. The context records its key and original concurrency value. Domain methods change the allowed properties:

```
product.Rename(command.Name);
product.ChangePrice(new Money(command.Price, command.Currency));
product.AssignCategory(command.CategoryId);
product.MarkUpdated(timeProvider.GetUtcNow());
```

An outbox message is added to the same context:

```
db.OutboxMessages.Add(OutboxMessages.ProductChanged(product));
await db.SaveChangesAsync(token);
```

`SaveChangesAsync` generates an UPDATE whose WHERE clause contains both the product key and original concurrency token:

```
UPDATE Products
SET Name = @name,
```

```

        Price = @price,
        Currency = @currency,
        CategoryId = @categoryId,
        Version = @newVersion,
        UpdatedAt = @updatedAt
WHERE Id = @id
AND Version = @originalVersion;

```

It also inserts the outbox row. Both commands execute in one database transaction. Either the product change and message are committed together, or neither is committed. If another editor already changed the product, the update affects zero rows and EF Core throws `DbUpdateConcurrencyException`. The application translates that into a conflict result rather than pretending the update succeeded. The HTTP adapter converts the application result:

```

Success    -> 200 OK with updated product
Invalid    -> 400 Bad Request with validation details
Forbidden  -> 403 Forbidden
NotFound   -> 404 Not Found
Conflict   -> 409 Conflict

```

For success:

```

HTTP/1.1 200 OK
Content-Type: application/json

```

```

{
  "id": 42,
  "name": "Mechanical Keyboard",
  "price": 119.00,
  "currency": "EUR",
  "version": 8
}

```

For a stale edit:

```

HTTP/1.1 409 Conflict
Content-Type: application/problem+json

```

```

{
  "type": "https://example.com/problems/product-version-conflict",
  "title": "Product update conflict",
  "status": 409,
  "detail": "The product changed after it was opened.",
  "code": "product.version-conflict"
}

```

The response body contains safe client information. Server logs contain the trace identifier and deeper diagnostic context. Raw provider errors, SQL, stack traces, and connection details do not cross the production boundary. The client receives the response. On success, it replaces the displayed product and clears editing state. On validation failure, it maps field errors into the form. On conflict, it can offer Reload or Compare. On 401, it can start a sign-in flow. On 403, it shows an access message. On unexpected failure, it preserves the user's input and provides a retry path where safe. The component changes ordinary fields:

```

_product = response;
_isEditing = false;
_message = "Product updated.";

```

Blazor renders the component again, compares the new render tree with the previous one, and sends or applies only the required UI changes. The browser updates the DOM and repaints the changed price and message.

```

HTTP response
-> Client result model
-> Component state changes
-> New render tree

```

- > Render-tree diff
- > DOM update
- > Browser repaint

This is the answer to the early question “Who requests rerendering?” The framework schedules rendering after recognised component events and lifecycle work, while component code changes the state that makes the next render different. External events may need `InvokeAsync(StateHasChanged)` or another framework-aware notification path because Blazor cannot infer arbitrary state changes outside its event flow.

15.7 Background Work, SignalR, and Cache Invalidation

The update response does not wait for every secondary effect. The database transaction already recorded an outbox message, so the application has durably accepted responsibility for later work.

Request completed

- > Product row is committed
- > Outbox row is committed
- > Client received updated product

A background worker claims the message later. It creates a dependency injection scope, loads the outbox item, and performs effects such as:

- Remove product details from data cache
- Invalidate product-list cache tag
- Evict output-cache entries
- Publish SignalR ProductChanged message
- Create persistent user notifications
- Send email where preferences require it
- Mark outbox message processed

These effects are outside the product transaction and may become visible slightly later. If the worker stops, the durable message remains. If delivery succeeds but marking completion fails, the message may run again, so handlers need idempotency. The product update is immediately authoritative in the database. Caches, connected clients, and external email converge afterward. This is eventual consistency around one atomic core change. A viewer currently watching product 42 may have joined a SignalR group such as `product:42`. The outbox worker publishes:

ProductChanged

- > ProductId = 42
- > Version = 8

The event is a notification that authoritative state changed, not a replacement database. The client can reload product details through the normal query path. That approach avoids embedding every product field in real-time messages and handles missed intermediate changes. A disconnected browser receives no live message. When it later navigates or reconnects, it reads current state from the API or server-rendered page. A persistent notification centre can show durable user notifications that were created while the browser was offline. SignalR groups do not enforce permission. The server authorised group membership and still protects the product API. A malicious client cannot gain product access merely by guessing a group name. The read path may contain several caches:

- Browser cache
- Output cache
- Hybrid or distributed data cache
- Database internal cache

The product update invalidates application-controlled copies after commit. A details key such as `product:42` is removed. Product-list entries may be invalidated through a tag or versioned key. Output-cache entries tagged `products` are evicted. Browser caching should use response validators or short freshness according to how current the screen must be. Invalidation can fail, so expiration remains a safety net. The database is still authoritative. A cache outage should normally reduce performance rather than lose product data. Immutable static assets follow a different strategy. A new CSS or image file receives a versioned URL, so old cached content remains correct for its old address and no immediate global invalidation is needed.

15.8 Telemetry, Failure, and Boundary-First Debugging

The production request produces several forms of telemetry. A trace connects the reverse-proxied HTTP request, application operation, EF Core command, and perhaps later background work through propagated identifiers. Structured logs record meaningful events:

```
Request started
User 17 attempted product update
Product 42 changed from version 7 to 8
Outbox message 9f... created
Request completed with 200 in 84 ms
```

Metrics aggregate behaviour:

```
Request rate
Latency percentiles
HTTP 4xx and 5xx counts
Database command duration
Concurrency conflict rate
Cache hit rate
Outbox queue age
SignalR connection count
Email delivery failures
```

A single 409 may be ordinary concurrent editing. A rising conflict rate after one release may reveal a UI or token bug. One slow query may be noise; sustained high database latency may need an index, query change, or capacity response. Telemetry should reveal boundaries without exposing credentials or unnecessary personal data. Cookies, bearer tokens, password-reset URLs, request bodies, and secrets do not belong in logs. The complete path is useful because failures have different shapes depending on where they occur.

```
DNS failure
-> browser cannot locate host
TLS failure
-> secure connection cannot be established
Proxy failure
-> gateway error or no connection to Kestrel
Kestrel or startup failure
-> connection refused or service unavailable
Routing failure
-> 404 or 405
Binding or validation failure
-> 400
Authentication failure
-> 401
Authorization failure
-> 403
Missing product
-> 404
Concurrency conflict
-> 409
Rate limit
-> 429
Dependency unavailable
-> 503 or controlled failure
Unexpected exception
-> 500 with correlated server log
Blazor state or rendering problem
-> request may succeed while UI remains wrong
CSS problem
-> DOM may be correct while pixels look wrong
```

These outcomes should not be debugged with the same tool. Browser Network and Console panels diagnose the client boundary. Reverse-proxy and service-manager logs diagnose process reachability. ASP.NET Core logs diagnose routing, binding, security, and endpoint execution. SQL logging and execution plans diagnose

database work. Component inspection and DOM tools diagnose rendering. When something fails, begin from the visible symptom and move one boundary at a time rather than changing code in several layers. For a page that does not open:

Does DNS resolve?

- > Can TLS connect?
- > Does the proxy return a response?
- > Is Kestrel listening?
- > Is the process healthy?
- > Did routing match?

For an API call:

Was the request sent?

- > Correct method and URL?
- > Correct headers and body?
- > Which status returned?
- > Did authentication and authorization succeed?
- > Did the endpoint execute?
- > Which application result was produced?
- > Which database command ran?

For a Blazor UI that does not update:

Did the event fire?

- > Did the handler complete?
- > Did component state actually change?
- > Was a render scheduled?
- > Did the render tree change?
- > Did the DOM change?
- > Did CSS hide or reposition it?

For stale data:

Is the database current?

- > Did the query load current state?
- > Is a tracked DbContext stale?
- > Did the data cache invalidate?
- > Did output or browser cache reuse an old response?
- > Did the client miss a real-time event and fail to reload?

Each question either identifies the failing boundary or proves that the next boundary should be inspected. This method is faster than assuming that every visible UI problem is a Blazor bug or every failed request is a database bug.

15.9 Responsibilities and Final Application Shape

The complete path contains many libraries, but maintainability comes from placing responsibilities correctly:

Browser and Blazor component

- > display state and handle interaction

HTTP endpoint

- > translate transport input and output

Application operation

- > coordinate one use case

Domain model

- > enforce business invariants

Infrastructure

- > implement database, file, cache, email, and provider access

Database

- > preserve relational integrity and durable state

Background system

- > own work that outlives the request

Deployment environment

- > run, secure, observe, and replace the process

A framework type crossing inward is not automatically wrong, but it should have a reason. `HttpContext`, `IFormFile`, `ProblemDetails`, `NavigationManager`, EF Core exceptions, provider SDK types, and SignalR hubs naturally belong near their adapters. Application and domain code should speak in product operations and stable results. The same rule prevents unnecessary architecture. A small product rule does not need a distributed message bus. A local application may not need five projects. A cache is not required before measurements show repeated expensive work. The right design is the smallest one that keeps ownership clear and current requirements correct. A practical final solution may look like:

`ProductCatalog.Domain`

-> `Product`, `ProductCode`, `Money`, domain rules

`ProductCatalog.Application`

-> commands, queries, handlers, results, infrastructure contracts

`ProductCatalog.Infrastructure`

-> EF Core, database mappings, file storage, email, cache, outbox delivery

`ProductCatalog.Web`

-> `Program.cs`, middleware, Minimal APIs, Blazor components, authentication

`ProductCatalog.Tests`

-> domain, application, infrastructure, HTTP, component, and end-to-end tests

The running system adds:

Nginx or managed edge

systemd or container platform

Relational database

Shared cache when needed

Object or file storage

Data Protection key storage

Logs, metrics, traces, health checks, and alerts

Backup and restore process

Not every application needs every item on day one. The structure describes where a requirement belongs when it arrives.

15.10 The End-to-End Mental Model

A complete web interaction is a sequence of boundaries rather than one local call. A URL becomes DNS and TLS traffic, passes through a proxy and Kestrel, moves through middleware and routing, acquires an authenticated principal, invokes an application operation, becomes SQL through EF Core, and returns as HTML or JSON that the browser renders into pixels. A write adds binding, validation, authorization, domain rules, concurrency, a database transaction, and a component rerender; an outbox can then carry cache invalidation, SignalR, notification, and email work beyond the request. Debugging follows the same path one boundary at a time. The durable mental model is ownership: browser, HTTP adapter, application, domain, infrastructure, database, background system, and deployment environment each solve a different part of the operation.

Appendices

Appendix A. Practical Command and Diagnostic Reference

The main chapters explain how the web stack works and why its boundaries matter. This appendix is a compact working reference for the commands and diagnostic paths used while building the application. It is not meant to be memorised. Its purpose is to reduce the time between observing a problem and reaching the layer that can explain it. The most useful habit is to identify the boundary before selecting a tool. The .NET CLI answers project, build, test, package, and publish questions. Browser developer tools answer request, DOM, CSS, storage, and client-runtime questions. ASP.NET Core logs answer host, middleware, routing, authentication, and endpoint questions. EF Core logging and database tools answer query, transaction, constraint, and migration questions. Process managers, reverse proxies, health checks, and telemetry answer production questions.

Project does not compile

-> .NET CLI and compiler output

Browser cannot connect

-> DNS, TLS, process, port, proxy, and Kestrel

Request returns wrong result

-> Network panel, ASP.NET Core logs, endpoint and application result

UI does not update

-> component state, render scheduling, DOM, and CSS

Data is wrong or slow

-> generated SQL, database constraints, indexes, and execution plan

Production instance is unhealthy

-> health checks, service logs, metrics, traces, and deployment version

A.1 SDKs and Runtimes

Begin by confirming which `dotnet` executable and SDK the shell is actually using:

```
which dotnet
dotnet --version
dotnet --info
dotnet --list-sdks
dotnet --list-runtimes
```

`dotnet --version` shows the selected SDK. `dotnet --info` adds operating-system, architecture, SDK, runtime, and base-path information. `--list-sdks` and `--list-runtimes` reveal every installed version. These commands are especially useful when an editor, terminal, CI job, or service uses a different installation than expected. A `global.json` file can pin or constrain SDK selection:

```
{
  "sdk": {
    "version": "10.0.300",
    "rollForward": "latestPatch"
  }
}
```

Run `dotnet --version` from the solution directory after adding or changing the file. The selected SDK depends on the current directory and parent directories because the CLI searches upward for `global.json`. An editor launched from another folder may therefore select a different SDK unless its workspace and environment are configured consistently. A target framework such as `net10.0` is not the same as the selected SDK or installed runtime. The SDK compiles the project, the target framework describes the API surface and runtime family for which it is compiled, and the runtime executes a framework-dependent deployment.

A.2 Projects and Solutions

List installed templates:

```
dotnet new list
```

Search for a template:

```
dotnet new search blazor
```

Inspect template options before creating a project:

```
dotnet new blazor --help
dotnet new webapi --help
dotnet new classlib --help
dotnet new xunit --help
```

Create common project types:

```
dotnet new blazor -n ProductCatalog.Web
dotnet new classlib -n ProductCatalog.Domain
dotnet new classlib -n ProductCatalog.Application
dotnet new classlib -n ProductCatalog.Infrastructure
dotnet new xunit -n ProductCatalog.Application.Tests
```

Template options evolve, so read `--help` from the installed SDK rather than copying an old command blindly. The generated project is a starting point, not an architectural requirement. Remove sample code that does not belong to the application, but first understand which registrations and files support the selected Blazor render modes. Inspect project and solution contents with ordinary filesystem tools:

```
find . -maxdepth 3 -type f | sort
find . -type d \( -name bin -o -name obj \) -prune -o -type f -print | sort
```

The second command excludes generated `bin` and `obj` directories. When a build appears to use stale generated files, clean them through the CLI first rather than deleting arbitrary source content. Create a solution and add projects:

```
dotnet new sln -n ProductCatalog
dotnet sln ProductCatalog.sln add src/ProductCatalog.Domain/ProductCatalog.Domain.csproj
dotnet sln ProductCatalog.sln add
src/ProductCatalog.Application/ProductCatalog.Application.csproj
dotnet sln ProductCatalog.sln add
src/ProductCatalog.Infrastructure/ProductCatalog.Infrastructure.csproj
dotnet sln ProductCatalog.sln add src/ProductCatalog.Web/ProductCatalog.Web.csproj
dotnet sln ProductCatalog.sln add
tests/ProductCatalog.Application.Tests/ProductCatalog.Application.Tests.csproj
```

List the projects already included:

```
dotnet sln ProductCatalog.sln list
```

Add project references where dependencies point inward:

```
dotnet add src/ProductCatalog.Application reference src/ProductCatalog.Domain
dotnet add src/ProductCatalog.Infrastructure reference src/ProductCatalog.Application
dotnet add src/ProductCatalog.Infrastructure reference src/ProductCatalog.Domain
dotnet add src/ProductCatalog.Web reference src/ProductCatalog.Application
dotnet add src/ProductCatalog.Web reference src/ProductCatalog.Infrastructure
dotnet add tests/ProductCatalog.Application.Tests reference src/ProductCatalog.Application
```

Inspect references:

```
dotnet list src/ProductCatalog.Web reference
dotnet list src/ProductCatalog.Infrastructure reference
```

A build can enforce only project-level direction. It cannot prevent a namespace inside `Application` from taking on web responsibilities if both live in one project. Compile-time project boundaries and disciplined type ownership work together.

A.3 Build, Watch, and Packages

The normal development cycle is:

```
dotnet restore
dotnet build
dotnet test
dotnet run --project src/ProductCatalog.Web
```

`dotnet build` performs restore automatically unless `--no-restore` is supplied. CI pipelines often separate restore so later stages can use `--no-restore` and prove that dependency resolution happened once:

```
dotnet restore ProductCatalog.sln
dotnet build ProductCatalog.sln --configuration Release --no-restore
dotnet test ProductCatalog.sln --configuration Release --no-build
```

Build one project:

```
dotnet build src/ProductCatalog.Web/ProductCatalog.Web.csproj
```

Treat warnings as errors when the solution is ready for that standard:

```
dotnet build ProductCatalog.sln -warnaserror
```

Clean generated build output:

```
dotnet clean ProductCatalog.sln
```

When generated state is genuinely corrupted, remove `bin` and `obj` directories after stopping processes that may use them:

```
find . -type d \( -name bin -o -name obj \) -prune -exec rm -rf {} +
```

Use this deliberately. Repeatedly deleting build output can hide an SDK, package, editor, or project-configuration problem rather than solving it. Run with file watching:

```
dotnet watch --project src/ProductCatalog.Web
```

Hot reload applies supported changes while the process remains active and restarts when required. If behaviour differs between `dotnet watch`, `dotnet run`, and the editor debugger, compare the working directory, selected SDK, launch profile, environment variables, and listening URLs. Add a package:

```
dotnet add src/ProductCatalog.Infrastructure package Microsoft.EntityFrameworkCore.Sqlite
dotnet add src/ProductCatalog.Infrastructure package Microsoft.EntityFrameworkCore.Design
```

Specify a version when the solution requires deliberate version control:

```
dotnet add src/ProductCatalog.Infrastructure package Microsoft.EntityFrameworkCore.Sqlite
--version 10.0.0
```

List direct and transitive dependencies:

```
dotnet list ProductCatalog.sln package
dotnet list ProductCatalog.sln package --include-transitive
```

Check for outdated packages:

```
dotnet list ProductCatalog.sln package --outdated
```

Check for known vulnerable packages where supported by the selected SDK:

```
dotnet list ProductCatalog.sln package --vulnerable --include-transitive
```

A package update is a code change. Read release notes, rebuild, run tests, and inspect behaviour that depends on provider translation, authentication, serialization, or UI rendering. Do not update production dependencies only because a command reports a newer version, and do not ignore security updates merely because the application still compiles. Central package management can keep versions in one `Directory.Packages.props` file when a solution has many projects:

```
<Project>
  <PropertyGroup>
    <ManagePackageVersionsCentrally>true</ManagePackageVersionsCentrally>
  </PropertyGroup>
  <ItemGroup>
    <PackageVersion Include="Microsoft.EntityFrameworkCore.Sqlite" Version="10.0.0" />
    <PackageVersion Include="Microsoft.EntityFrameworkCore.Design" Version="10.0.0" />
  </ItemGroup>
</Project>
```

Projects then reference packages without repeating their versions.

A.4 Tests

Run every test:

```
dotnet test ProductCatalog.sln
```

Run one project:

```
dotnet test tests/ProductCatalog.Application.Tests
```

Run Release tests without rebuilding:

```
dotnet test ProductCatalog.sln --configuration Release --no-build
```

Filter by fully qualified name or trait:

```
dotnet test --filter "FullyQualifiedName~UpdateProduct"
dotnet test --filter "Category=Integration"
dotnet test --filter "Category!=EndToEnd"
```

Increase console detail when investigating a failure:

```
dotnet test --logger "console;verbosity=detailed"
```

Collect test results:

```
dotnet test --logger "trx;LogFileName=ProductCatalog.trx" --results-directory
artifacts/test-results
```

A passing unit-test suite does not prove HTTP routing, EF Core provider translation, browser behaviour, or production configuration. Match the test to the boundary: ordinary C# tests for domain rules, controlled collaborators for application operations, a relational provider for EF Core behaviour, `WebApplicationFactory` for HTTP, bUnit for component state and markup, and Playwright for a small number of critical browser journeys.

A.5 EF Core Migrations

Install or update the EF Core tool:

```
dotnet tool install --global dotnet-ef
dotnet tool update --global dotnet-ef
dotnet ef --version
```

A local tool manifest can make the tool version part of the repository:

```
dotnet new tool-manifest
dotnet tool install dotnet-ef
dotnet tool restore
dotnet tool run dotnet-ef --version
```

Create a migration when the context and startup project live in different projects:

```
dotnet ef migrations add InitialCreate \
  --project src/ProductCatalog.Infrastructure \
  --startup-project src/ProductCatalog.Web \
  --context CatalogDbContext
```

List migrations:

```
dotnet ef migrations list \
  --project src/ProductCatalog.Infrastructure \
  --startup-project src/ProductCatalog.Web
```

Apply migrations locally:

```
dotnet ef database update \
  --project src/ProductCatalog.Infrastructure \
  --startup-project src/ProductCatalog.Web
```

Remove only the latest unapplied migration when it was generated incorrectly:

```
dotnet ef migrations remove \
  --project src/ProductCatalog.Infrastructure \
  --startup-project src/ProductCatalog.Web
```

Generate a reviewed production script:

```
dotnet ef migrations script --idempotent \
  --project src/ProductCatalog.Infrastructure \
  --startup-project src/ProductCatalog.Web \
  --output artifacts/migrations/ProductCatalog.sql
```

Generate a migration bundle:

```
dotnet ef migrations bundle \
  --project src/ProductCatalog.Infrastructure \
  --startup-project src/ProductCatalog.Web \
  --output artifacts/migrations/product-catalog-migrate
```

Before applying a migration, inspect the generated operations and SQL. Confirm nullability, defaults, foreign keys, delete behaviour, indexes, data conversion, table rebuilds, and possible locks. Test it against production-like data volume. A migration that completes instantly on an empty development database may be disruptive on millions of rows. When the tools cannot create the context, read the design-time error carefully. Common causes are a missing connection string, the wrong startup project, several contexts without `--context`, startup code that requires unavailable services, or a design-time factory that no longer matches the context constructor.

A.6 Running and Configuring the Application

Run a project:

```
dotnet run --project src/ProductCatalog.Web
```

Select a launch profile:

```
dotnet run --project src/ProductCatalog.Web --launch-profile https
```

Ignore launch profiles and supply values explicitly:

```
ASPNETCORE_ENVIRONMENT=Development \
ASPNETCORE_URLS=http://127.0.0.1:5080 \
dotnet run --project src/ProductCatalog.Web --no-launch-profile
```

Override hierarchical configuration through environment variables:

```
Catalog__DefaultCurrency=EUR \
Catalog__PageSize=50 \
dotnet run --project src/ProductCatalog.Web
```

Pass command-line configuration after `--`:

```
dotnet run --project src/ProductCatalog.Web -- --Catalog:PageSize=50
```

The effective value depends on provider precedence. When configuration is wrong, identify every source capable of setting it: base JSON, environment JSON, user secrets, environment variables, command-line arguments, launch profile, service unit, container configuration, and platform secret injection. Do not print the complete configuration tree in production. It can contain connection strings, provider credentials, signing material, or personal data. Log the presence and safe identity of required configuration rather than its secret value.

A.7 Ports, HTTP, and Browser Tools

When the browser reports that it cannot connect, verify that the process is running and listening:

```
ps aux | grep ProductCatalog
ss -ltnp
ss -ltnp | grep 5080
```

Test the local Kestrel endpoint:

```
curl -i http://127.0.0.1:5080/health/live
```

Test the public route separately:

```
curl -i https://catalog.example.com/health/live
```

If the local request works but the public request fails, inspect DNS, firewall, certificate, reverse proxy, and forwarded-host configuration. If neither works, inspect process startup and Kestrel binding. If the public endpoint works but a browser fails, inspect browser certificate state, proxy settings, CORS, service workers, and client-side code. On a `systemd` host:

```
systemctl status product-catalog
journalctl -u product-catalog --since "30 minutes ago"
journalctl -u product-catalog -f
```

A restart loop often appears clearly in the service status and journal. Find the first startup exception rather than focusing only on the repeated final exit code. Show response headers and body:

```
curl -i https://catalog.example.com/api/products
```

Show connection, TLS, request, and response detail:

```
curl -v https://catalog.example.com/api/products
```

Send JSON:

```
curl -i \
  -X POST \
  -H "Content-Type: application/json" \
  -d '{"code":"KB-01","name":"Mechanical Keyboard","price":129,"currency":"EUR"}' \
  https://catalog.example.com/api/products
```

Send an authorization header:

```
curl -i \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  https://catalog.example.com/api/admin/products
```

Save response headers and body separately:

```
curl -D artifacts/headers.txt \
  -o artifacts/body.json \
  https://catalog.example.com/api/products/42
```

Follow redirects when that is the behaviour being tested:

```
curl -i -L https://catalog.example.com/account
```

Do not add `-k` or `--insecure` as a permanent way to make certificate failures disappear. It disables server-certificate verification and hides the exact property HTTPS is meant to provide. Use it only in a controlled diagnostic experiment and then fix the certificate trust or host mismatch. `curl` is not constrained by browser CORS. When `curl` succeeds and browser JavaScript fails, inspect the browser Console and Network panels for origin, preflight, credential, mixed-content, or certificate restrictions. The Network panel answers whether the browser sent a request, which URL and method it used, which headers and cookies accompanied it, how long each phase took, whether a redirect occurred, which status returned, and what body arrived. Preserve the log while navigating when a redirect or document reload would otherwise clear it.

The Elements panel answers whether the expected DOM exists and which CSS rules win. When content is present but invisible, inspect `display`, `visibility`, `opacity`, dimensions, overflow, positioning, stacking context, and inherited colour rather than assuming rendering failed. The Console reveals JavaScript exceptions, failed module loads, Blazor boot failures, interop errors, mixed-content warnings, and browser security messages. The Application panel shows cookies, browser storage, IndexedDB, cache storage, and service workers. The Performance panel is useful after correctness has been established; it can reveal long scripting tasks, repeated layout, excessive rendering, and slow interactions. A productive order for a UI failure is:

```
Network
-> Did the required data or document arrive?
Console
-> Did client startup or interop fail?
Component state
-> Did the application store the expected result?
Elements
-> Did the DOM change?
Styles
-> Did CSS make the changed DOM visible?
```

A.8 ASP.NET Core and Blazor Diagnosis

Enable useful framework categories temporarily through configuration:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Information",
      "Microsoft.AspNetCore.Routing": "Debug",
      "Microsoft.AspNetCore.Authorization": "Debug"
    }
  }
}
```

Avoid leaving broad debug logging enabled indefinitely in a busy production application. It can add volume, cost, and sensitive context. Interpret common outcomes by boundary:

Symptom	Likely first boundary
Connection refused	Process, binding, port, firewall
TLS warning	Certificate, host, trust, expiry
404	Proxy path, routing, or deliberate not-found result
405	Correct path with wrong HTTP method
400	Binding, malformed body, conversion, or validation
401	Missing, invalid, or expired authentication
403	Authenticated principal lacks authorization
409	Concurrency, duplicate value, or state conflict
415	Request <code>Content-Type</code> is unsupported
429	Rate-limiting policy
500	Unexpected application or infrastructure failure

Symptom	Likely first boundary
503	Dependency or instance is temporarily unavailable

Use the response trace identifier to find the corresponding server log. Log structured operation identifiers such as product ID, user ID, message ID, and release version, but never raw cookies, bearer tokens, reset links, or complete secrets. When an event appears not to work, verify that the rendered element is interactive under the chosen render mode. Static server-side rendering can produce HTML without retaining event handlers after the response. An interactive component requires an interactive render mode and successful client boot. Then follow the component path:

Browser event occurred

- > Blazor dispatched the event
- > Handler ran
- > Awaited operation completed
- > Component field or parameter changed
- > Render was scheduled
- > Render tree became different
- > DOM update arrived or was applied

Set breakpoints or temporary logs in the event handler and lifecycle methods. Check whether a parent replaces a child parameter after the child changes it. Check whether a long-lived service changed state without notifying the component. For events originating outside Blazor's recognised flow, marshal back through `InvokeAsync` and request rendering deliberately. For server interactivity, inspect circuit disconnections, SignalR transport failures, proxy WebSocket configuration, and server logs. For WebAssembly, inspect downloaded assemblies, API requests, CORS, browser storage, and client-side exceptions. A server-side scoped service can live for the circuit, while a WebAssembly scoped service effectively lives for the browser application. Do not assume request-scoped lifetime in either case.

Avoid using `@ref` to make components call each other as a general state architecture. Prefer parameters, `EventCallback`, cascading values, and explicit state services. Component references are useful for focused imperative operations such as placing focus or invoking a specialised control API.

A.9 EF Core and Dependency Injection Diagnosis

Enable command logging during development:

```
{
  "Logging": {
    "LogLevel": {
      "Microsoft.EntityFrameworkCore.Database.Command": "Information"
    }
  }
}
```

Inspect a query without executing it:

```
var sql = db.Products
    .Where(product => product.IsAvailable)
    .OrderBy(product => product.Name)
    .Select(product => new { product.Id, product.Name })
    .ToString();
```

When a query is slow, answer these questions in order:

- How many SQL commands ran?
- Did an N+1 pattern appear?
- Were filtering and paging translated into SQL?
- Were only needed columns selected?
- Did Include multiply rows?
- Did the query use stable ordering?
- Which index can support its filter, join, and order?
- What does the database execution plan show?

Do not conclude that EF Core is slow because a query created by the application is inefficient. The ORM translates the requested expression; it cannot invent missing constraints, bounded result sizes, or the right index without a model and workload. When **SaveChanges** fails, distinguish the category. **DbUpdateConcurrencyException** usually means a conditional update or delete matched no expected row. **DbUpdateException** wraps broader provider failures such as unique, foreign-key, check, type, or connection errors. Inspect provider codes or constraint names at the infrastructure boundary and translate known cases into stable application results. A test using EF Core's in-memory provider cannot prove relational constraints or SQL translation. Use SQLite for portable relational behaviour and the real production database for provider-specific types, collations, migrations, indexes, locking, and concurrency.

A dependency injection startup error normally identifies the service that could not be created and the dependency that was missing or invalid. Read the complete inner-exception chain. Common causes include an unregistered interface, wrong constructor, singleton depending on scoped state, circular dependency, options validation failure, and a service registered in a project extension that was never called. Keep registrations discoverable:

```
builder.Services
    .AddApplication()
    .AddInfrastructure(builder.Configuration)
    .AddProductCatalogWeb();
```

Then inspect the relevant extension rather than searching the whole repository for one service. Use **ValidateOnStart** for options whose absence should prevent the application from accepting traffic. In Development, scope validation can reveal captive scoped dependencies before they become stale-state or disposal failures. When a service resolves in an HTTP request but fails in a background worker, remember that hosted services are singletons and no request scope exists. Create a scope for one message or batch. When a **DbContext** behaves unpredictably in interactive server Blazor, replace a long-lived injected context with **IdbContextFactory<TContext>** and one context per operation.

A.10 Authentication and Authorization Diagnosis

Separate the questions:

```
Did the request present a credential?
Did the configured handler accept it?
Which ClaimsPrincipal was created?
Which policy was selected?
Which requirement failed?
Was the response a challenge or forbid?
```

Inspect cookies in the browser Application panel and request headers in the Network panel. A cookie can exist but fail decryption because Data Protection keys changed, the application name differs, the cookie expired, the host or path no longer matches, or secure-cookie requirements are not satisfied. For bearer tokens, inspect only non-secret metadata through trusted tooling. Validate issuer, audience, signature, and lifetime. Do not paste production tokens into public decoding sites or logs. A JWT payload being readable does not mean the signature is valid. Use a test authentication scheme in integration tests rather than disabling authorization. Test anonymous, authenticated-but-forbidden, and authorised cases through the actual pipeline. When UI visibility differs from endpoint access, remember that **AuthorizeView** and hidden buttons are presentation controls; server policies are enforcement.

A.11 Production Diagnosis

Identify the running release first:

```
Application version
Commit or build identifier
Container image digest
Environment
Database migration version
Instance name
```

Then determine whether the failure affects one instance, all instances, one dependency, or one route. Check readiness, liveness, service logs, reverse-proxy logs, current error rate, latency, CPU, memory, disk, database connectivity, outbox age, and certificate expiry. A useful production sequence is:

1. Confirm user impact and start time.
2. Identify the current and previous releases.
3. Check public health and one representative business operation.
4. Compare affected and healthy instances.
5. Inspect errors and traces around the first failure.
6. Decide whether rollback is safe for the current schema.
7. Stop further rollout or shift traffic when necessary.
8. Restore service before performing a complete root-cause analysis.
9. Preserve evidence and document the timeline.

Do not restart repeatedly without first capturing the startup error and current state. A restart can restore service after a transient failure, but it can also erase useful process evidence, repeat a destructive startup action, or hide a resource leak until it grows again. Health checks are signals for infrastructure, not a replacement for business monitoring. An application can respond to `/health/live` while login, checkout, or product updates are broken. Combine technical probes with a small number of representative synthetic journeys.

A.12 Publishing and Inspecting Releases

Create a Release artifact:

```
dotnet publish src/ProductCatalog.Web/ProductCatalog.Web.csproj \
  --configuration Release \
  --output artifacts/publish/ProductCatalog
```

Inspect its contents:

```
find artifacts/publish/ProductCatalog -maxdepth 2 -type f | sort
```

Run the published framework-dependent application:

```
ASPNETCORE_ENVIRONMENT=Production \
ASPNETCORE_URLS=http://127.0.0.1:5080 \
dotnet artifacts/publish/ProductCatalog/ProductCatalog.Web.dll
```

Publish self-contained for Linux x64:

```
dotnet publish src/ProductCatalog.Web/ProductCatalog.Web.csproj \
  --configuration Release \
  --runtime linux-x64 \
  --self-contained true \
  --output artifacts/publish/ProductCatalog-linux-x64
```

The published directory is the deployment input. Do not deploy the project directory, source tree, `obj`, or arbitrary contents from `bin`. Package and checksum the artifact so the server receives the exact output that was tested. After deployment, test both the private Kestrel route and public HTTPS route, then verify authentication, static assets, database connectivity, migration compatibility, Data Protection keys, SignalR or Blazor connections, and one state-changing operation in a safe staging or smoke-test context.

A.13 Boundary Checklist

When time is limited, use this condensed map:

Build fails

-> SDK, target framework, package restore, project references, compiler output

Application does not start

-> configuration validation, DI construction, port, certificate, database startup dependency

No browser connection

-> DNS, TLS, firewall, proxy, process, Kestrel binding

Wrong HTTP response

-> method, path, headers, body, middleware order, routing, binding, policy, endpoint result

Wrong UI

-> client request, component state, render mode, lifecycle, rerender, DOM, CSS

Wrong data

-> request DTO, application mapping, domain rule, tracking, concurrency, SQL, constraint

Slow request

-> trace timing, repeated calls, query shape, indexes, cache behaviour, external dependency

Lost background work

-> fire-and-forget task, non-durable queue, outbox claim, retry, idempotency, dead-letter state

Works on one instance only

-> process-local cache, queue, files, Data Protection keys, SignalR scale-out, Blazor affinity

Deployment regression

-> release identity, configuration difference, schema compatibility, key persistence, proxy change

Cannot recover data

-> backup age, restore procedure, external file storage, key ownership, tested recovery objective